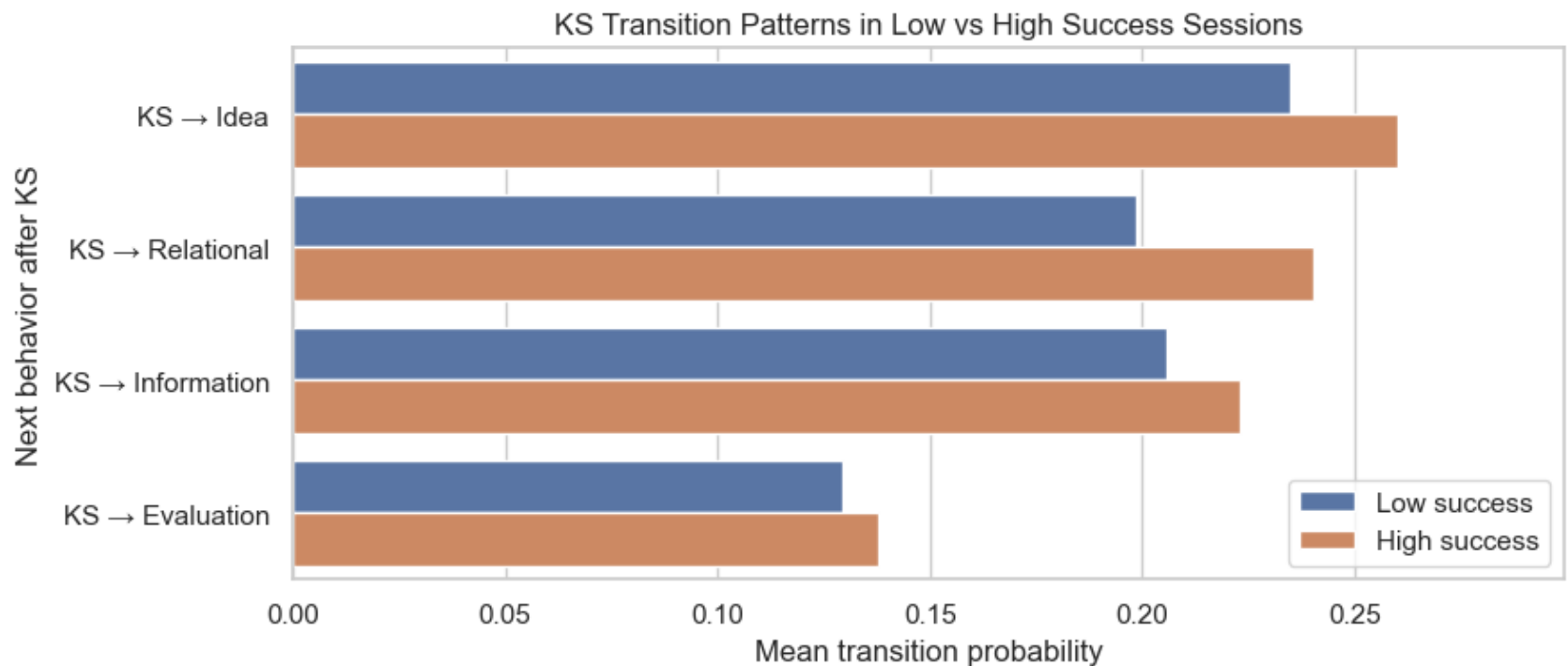




01.07.2026 Implementation

```
In [9]: # =====  
# Linkography-style "code function by temporal position + neighbors"  
# Works across ALL session JSONs in: ../data/<conf>/session_data/*.json  
# Handles 10-minute timer resets by unwrapping times.  
# =====  
  
import os  
import json  
import glob  
import re  
from collections import Counter, defaultdict  
  
import numpy as np  
import pandas as pd
```

```

# ----- CONFIG -----
ROOT = "/Users/maxchalekson/Desktop/gemini_data_analysis/data" # <-- change if needed
GLOB_PATTERN = os.path.join(ROOT, "*", "session_data", "*.json")

SCORE_THRESHOLD = 1          # include annotation code if score >= this
WRAP_SECONDS = 10 * 60       # timer resets every 10 minutes
WRAP_TOL = 30                # seconds tolerance to detect wrap

FOCAL_CODE = "Knowledge Sharing" # code you want the context analysis for
TOPK = 8                      # how many neighbor codes to show per third

# ----- HELPERS -----
def _parse_mmss(s: str):
    """Parse 'MM:SS' or 'HH:MM:SS' into seconds. Returns None if invalid."""
    if s is None:
        return None
    s = str(s).strip()
    if not s:
        return None

    # allow 'M:SS', 'MM:SS', 'H:MM:SS'
    parts = s.split(":")
    if len(parts) == 2:
        m, sec = parts
        if not (m.isdigit() and sec.isdigit()):
            return None
        return int(m) * 60 + int(sec)
    if len(parts) == 3:
        h, m, sec = parts
        if not (h.isdigit() and m.isdigit() and sec.isdigit()):
            return None
        return int(h) * 3600 + int(m) * 60 + int(sec)

    return None

def _extract_start_end_seconds(item: dict):

```

```

"""
Extract start/end seconds from one utterance dict.
Supports:
    - start_time/end_time (e.g., '09:39')
    - timestamp 'MM:SS-MM:SS'
Returns (start_sec, end_sec) possibly None.
"""

st = None
en = None

if "start_time" in item and "end_time" in item:
    st = _parse_mmss(item.get("start_time"))
    en = _parse_mmss(item.get("end_time"))

if (st is None or en is None) and isinstance(item.get("timestamp"), str):
    # e.g., '00:00-01:34'
    ts = item["timestamp"].strip()
    m = re.match(r"^s*([0-9:]+)\s*-\s*([0-9:]+)\s*$", ts)
    if m:
        st2 = _parse_mmss(m.group(1))
        en2 = _parse_mmss(m.group(2))
        st = st if st is not None else st2
        en = en if en is not None else en2

return st, en

def _unwrap_times(st_list, en_list, wrap_seconds=WRAP_SECONDS, tol=WRAP_TOL):
    """
    Unwrap a sequence of start/end times that reset every wrap_seconds.
    Strategy:
        - iterate in order; whenever start time drops "a lot", add wrap_seconds to offset
        - also fix end < start within utterance by adding wrap_seconds
    """
    offset = 0
    last_st = None
    out_st = []
    out_en = []

```

```

for st, en in zip(st_list, en_list):
    if st is None:
        out_st.append(None)
        out_en.append(None if en is None else en + offset)
        continue

    if last_st is not None and st + tol < last_st:
        offset += wrap_seconds

    st_u = st + offset

    if en is None:
        en_u = None
    else:
        en_u = en + offset
        # if end appears to be after reset relative to start, lift it
        if en_u + tol < st_u:
            en_u += wrap_seconds

    out_st.append(st_u)
    out_en.append(en_u)
    last_st = st

return out_st, out_en

def _third_from_time(t, tmax):
    if t is None or tmax is None or tmax <= 0:
        return None
    frac = t / tmax
    if frac <= 1/3:
        return "beginning"
    elif frac <= 2/3:
        return "middle"
    else:
        return "end"

def _extract_codes(item: dict, score_threshold=SCORE_THRESHOLD):

```

```

"""
Pull codes from item['annotations'] which looks like:
{'Knowledge Sharing': {'score': 2, ...}, ...}
"""
ann = item.get("annotations", {})
codes = []
if isinstance(ann, dict):
    for code, payload in ann.items():
        if isinstance(payload, dict):
            sc = payload.get("score", None)
            if sc is not None and sc >= score_threshold:
                codes.append(code)
return codes

# ----- LOAD ALL SESSIONS -----
paths = sorted(glob.glob(GLOB_PATTERN))
print(f"Found {len(paths)} session JSONs under: {GLOB_PATTERN}")

rows = []
skipped = 0

for fp in paths:
    try:
        with open(fp, "r") as f:
            obj = json.load(f)
    except Exception as e:
        print(f"[WARN] Failed reading {fp}: {e}")
        skipped += 1
        continue

    all_data = obj.get("all_data", None)
    if not isinstance(all_data, list) or len(all_data) == 0:
        print(f"[WARN] No all_data list in {fp}; skipping.")
        skipped += 1
        continue

    # extract raw start/end in sequence order
    st_raw = []

```

```

en_raw = []
for item in all_data:
    st, en = _extract_start_end_seconds(item)
    st_raw.append(st)
    en_raw.append(en)

# If absolutely none of the utterances have parsable times, still proceed using 'when' (if pres
has_any_time = any(x is not None for x in st_raw)
if has_any_time:
    st_abs, en_abs = _unwrap_times(st_raw, en_raw)
    tmax = max([t for t in st_abs if t is not None] + [0])
else:
    st_abs = [None] * len(all_data)
    en_abs = [None] * len(all_data)
    tmax = None

# session id from filename
session_id = os.path.splitext(os.path.basename(fp))[0]

for i, item in enumerate(all_data):
    codes = _extract_codes(item)

    # third: prefer explicit 'when' if present, else infer from time
    third = item.get("when", None)
    if third is None:
        third = _third_from_time(st_abs[i], tmax)

    rows.append({
        "session_id": session_id,
        "path": fp,
        "idx": i,
        "t_abs_start": st_abs[i],
        "t_abs_end": en_abs[i],
        "third": third,
        "speaker": item.get("speaker", None),
        "codes": codes,
        "transcript": item.get("transcript", None),
    })

```

```

df = pd.DataFrame(rows)
print(f"Built utterance-level df: {df.shape[0]} rows from {len(df['session_id'].unique())} sessions

# basic sanity
print(df[['third']].value_counts(dropna=False).head(10))

# ----- CONTEXT / FUNCTION ANALYSIS -----
def compute_code_contexts(df, focal_code=FOCAL_CODE):
    """
    For each occurrence of focal_code, look at immediate previous and next utterance codes.
    Split by temporal third.
    Returns dict: third -> {'prev': Counter, 'next': Counter, 'n': int}
    """
    out = {t: {"prev": Counter(), "next": Counter(), "n": 0} for t in ["beginning", "middle", "end"]}

    # process session-by-session to respect adjacency
    for sid, g in df.sort_values(["session_id", "idx"]).groupby("session_id"):
        g = g.reset_index(drop=True)
        for i in range(len(g)):
            codes_i = g.at[i, "codes"] or []
            if focal_code not in codes_i:
                continue

            third = g.at[i, "third"]
            out.setdefault(third, {"prev": Counter(), "next": Counter(), "n": 0})
            out[third]["n"] += 1

            # previous
            if i - 1 >= 0:
                prev_codes = g.at[i - 1, "codes"] or []
                out[third]["prev"].update(prev_codes)

            # next
            if i + 1 < len(g):
                next_codes = g.at[i + 1, "codes"] or []
                out[third]["next"].update(next_codes)

    return out

```

```

def pretty_top(counter: Counter, k=TOPK):
    return pd.DataFrame(counter.most_common(k), columns=["code", "count"])

contexts = compute_code_contexts(df, focal_code=FOCAL_CODE)

print("\n" + "="*90)
print(f"CONTEXTS FOR: {FOCAL_CODE}")
print("="*90)

for third in ["beginning", "middle", "end"]:
    block = contexts.get(third, {"prev": Counter(), "next": Counter(), "n": 0})
    n = block["n"]
    print(f"\n--- {third.upper()} (n={n} occurrences of '{FOCAL_CODE}') ---")

    print("\nTop PREVIOUS-utterance codes:")
    display(pretty_top(block["prev"], TOPK))

    print("\nTop NEXT-utterance codes:")
    display(pretty_top(block["next"], TOPK))

# ----- OPTIONAL: TEMPORAL ROLE PATTERN SUMMARY (dominant codes by third) -----
def dominant_codes_by_third(df):
    agg = defaultdict(Counter)
    for _, r in df.iterrows():
        third = r["third"]
        codes = r["codes"] or []
        agg[third].update(codes)
    return {k: v for k, v in agg.items()}

dom = dominant_codes_by_third(df)

print("\n" + "="*90)
print("DOMINANT CODES BY TEMPORAL THIRD (utterance-level frequency)")
print("="*90)
for third in ["beginning", "middle", "end"]:

```



```
print(f"\n--- {third.upper()} ---")
display(pretty_top(dom.get(third, Counter()), TOPK))
```

Found 157 session JSONs under: /Users/maxchalekson/Desktop/gemini_data_analysis/data/*/session_data/*.json

Built utterance-level df: 30558 rows from 157 sessions. Skipped files: 0

third

end 11675

beginning 10044

middle 8839

Name: count, dtype: int64

```
=====
CONTEXTS FOR: Knowledge Sharing
=====
```

--- BEGINNING (n=3986 occurrences of 'Knowledge Sharing') ---

Top PREVIOUS-utterance codes:

	code	count
0	Knowledge Sharing	1541
1	Participation Dynamics	1049
2	Relational Climate	1028
3	Information Seeking	830
4	Coordination and Decision Practices	800
5	Idea Management	691
6	Evaluation Practices	283
7	Integration Practices	88

Top NEXT-utterance codes:

	code	count
0	Knowledge Sharing	1586
1	Relational Climate	1134
2	Participation Dynamics	812
3	Idea Management	748
4	Information Seeking	747
5	Coordination and Decision Practices	715
6	Evaluation Practices	374
7	Integration Practices	117

--- MIDDLE (n=3918 occurrences of 'Knowledge Sharing') ---

Top PREVIOUS-utterance codes:

	code	count
0	Knowledge Sharing	1845
1	Idea Management	1098
2	Information Seeking	1057
3	Relational Climate	614
4	Evaluation Practices	608
5	Participation Dynamics	356
6	Coordination and Decision Practices	232
7	Integration Practices	199

Top NEXT-utterance codes:

	code	count
0	Knowledge Sharing	1843
1	Idea Management	1143
2	Information Seeking	919
3	Evaluation Practices	661
4	Relational Climate	643
5	Participation Dynamics	269
6	Coordination and Decision Practices	237
7	Integration Practices	215

--- END (n=3868 occurrences of 'Knowledge Sharing') ---

Top PREVIOUS-utterance codes:

	code	count
0	Knowledge Sharing	1577
1	Information Seeking	983
2	Idea Management	934
3	Relational Climate	631
4	Coordination and Decision Practices	509
5	Evaluation Practices	477
6	Participation Dynamics	321
7	Integration Practices	191

Top NEXT-utterance codes:

	code	count
0	Knowledge Sharing	1534
1	Idea Management	912
2	Information Seeking	767
3	Relational Climate	714
4	Coordination and Decision Practices	556
5	Evaluation Practices	505
6	Participation Dynamics	255
7	Integration Practices	213

=====

DOMINANT CODES BY TEMPORAL THIRD (utterance-level frequency)

=====

---- BEGINNING ----

	code	count
0	Knowledge Sharing	3986
1	Relational Climate	2593
2	Coordination and Decision Practices	2257
3	Information Seeking	1674
4	Idea Management	1661
5	Participation Dynamics	1648
6	Evaluation Practices	685
7	Integration Practices	220

---- MIDDLE ----

	code	count
0	Knowledge Sharing	3918
1	Idea Management	2417
2	Information Seeking	1885
3	Relational Climate	1413
4	Evaluation Practices	1319
5	Participation Dynamics	673
6	Coordination and Decision Practices	654
7	Integration Practices	419

--- END ---

	code	count
0	Knowledge Sharing	3868
1	Idea Management	2448
2	Relational Climate	2383
3	Coordination and Decision Practices	2288
4	Information Seeking	2100
5	Evaluation Practices	1242
6	Participation Dynamics	799
7	Integration Practices	688

```
In [10]: from collections import Counter
import pandas as pd

FOCAL_CODES = [
    "Knowledge Sharing",
```

```

    "Information Seeking",
    "Evaluation Practices",
    "Integration Practices",
    "Coordination and Decision Practices",
]

# You can tweak these sets to match your codebook names exactly
CLOSURE_CODES = {"Integration Practices", "Coordination and Decision Practices"}
CRITIQUE_CODES = {"Evaluation Practices"}
SEED_CODES = {"Information Seeking", "Knowledge Sharing"}

def counter_to_pct_df(cntr: Counter, topk=10):
    total = sum(cntr.values()) or 1
    dfp = pd.DataFrame(cntr.most_common(topk), columns=["code", "count"])
    dfp["pct"] = dfp["count"] / total
    return dfp

def role_signature(context_block):
    """
    Compute a few interpretable conditional indices from prev/next counters.
    These are *examples* that map to your narrative.
    """
    prev = context_block["prev"]
    nxt = context_block["next"]
    n_prev = sum(prev.values()) or 1
    n_next = sum(nxt.values()) or 1

    def pct_in(counter, keys, denom):
        return sum(counter[k] for k in keys if k in counter) / denom

    return {
        "Prev critique (Evaluation)": pct_in(prev, CRITIQUE_CODES, n_prev),
        "Next closure (Integration/Decision)": pct_in(nxt, CLOSURE_CODES, n_next),
        "Prev seed (InfoSeek/KS)": pct_in(prev, SEED_CODES, n_prev),
    }

# contexts was computed in your previous run; reuse it by recomputing per focal code:
def compute_code_contexts(df, focal_code):
    out = {t: {"prev": Counter(), "next": Counter(), "n": 0} for t in ["beginning", "middle", "end"]}

```

```

for sid, g in df.sort_values(["session_id", "idx"]).groupby("session_id"):
    g = g.reset_index(drop=True)
    for i in range(len(g)):
        codes_i = g.at[i, "codes"] or []
        if focal_code not in codes_i:
            continue

        third = g.at[i, "third"]
        if third not in out:
            continue

        out[third]["n"] += 1

        if i > 0:
            out[third]["prev"].update(g.at[i-1, "codes"] or [])
        if i < len(g)-1:
            out[third]["next"].update(g.at[i+1, "codes"] or [])

    return out

# ---- RUN AND PRINT SLIDE-READY OUTPUT ----
for focal in FOCAL_CODES:
    ctx = compute_code_contexts(df, focal)

    print("\n" + "="*100)
    print(f"FOCAL CODE: {focal}")
    print("="*100)

    for third in ["beginning", "middle", "end"]:
        block = ctx[third]
        print(f"\n--- {third.upper()} (n={block['n']} utterances containing '{focal}') ---")

        prev_df = counter_to_pct_df(block["prev"], topk=8)
        next_df = counter_to_pct_df(block["next"], topk=8)

        sig = role_signature(block)

        print("\nTop PREVIOUS codes (with %):")

```

```

display(prev_df)

print("\nTop NEXT codes (with %):")
display(next_df)

print("\nRole-signature indices (conditional):")
print({k: round(v, 3) for k, v in sig.items()})

```

```

=====
=
FOCAL CODE: Knowledge Sharing
=====
=

```

--- BEGINNING (n=3986 utterances containing 'Knowledge Sharing') ---

Top PREVIOUS codes (with %):

	code	count	pct
0	Knowledge Sharing	1541	0.244216
1	Participation Dynamics	1049	0.166244
2	Relational Climate	1028	0.162916
3	Information Seeking	830	0.131537
4	Coordination and Decision Practices	800	0.126783
5	Idea Management	691	0.109509
6	Evaluation Practices	283	0.044849
7	Integration Practices	88	0.013946

Top NEXT codes (with %):

	code	count	pct
0	Knowledge Sharing	1586	0.254452
1	Relational Climate	1134	0.181935
2	Participation Dynamics	812	0.130274
3	Idea Management	748	0.120006
4	Information Seeking	747	0.119846
5	Coordination and Decision Practices	715	0.114712
6	Evaluation Practices	374	0.060003
7	Integration Practices	117	0.018771

Role-signature indices (conditional):

```
{'Prev critique (Evaluation)': 0.045, 'Next closure (Integration/Decision)': 0.133, 'Prev seed (InfoSeek/KS)': 0.376}
```

--- MIDDLE (n=3918 utterances containing 'Knowledge Sharing') ---

Top PREVIOUS codes (with %):

	code	count	pct
0	Knowledge Sharing	1845	0.307039
1	Idea Management	1098	0.182726
2	Information Seeking	1057	0.175903
3	Relational Climate	614	0.102180
4	Evaluation Practices	608	0.101182
5	Participation Dynamics	356	0.059244
6	Coordination and Decision Practices	232	0.038609
7	Integration Practices	199	0.033117

Top NEXT codes (with %):

	code	count	pct
0	Knowledge Sharing	1843	0.310793
1	Idea Management	1143	0.192749
2	Information Seeking	919	0.154975
3	Evaluation Practices	661	0.111467
4	Relational Climate	643	0.108432
5	Participation Dynamics	269	0.045363
6	Coordination and Decision Practices	237	0.039966
7	Integration Practices	215	0.036256

Role-signature indices (conditional):

{'Prev critique (Evaluation)': 0.101, 'Next closure (Integration/Decision)': 0.076, 'Prev seed (InfoSeek/KS)': 0.483}

--- END (n=3868 utterances containing 'Knowledge Sharing') ---

Top PREVIOUS codes (with %):

	code	count	pct
0	Knowledge Sharing	1577	0.280455
1	Information Seeking	983	0.174818
2	Idea Management	934	0.166104
3	Relational Climate	631	0.112218
4	Coordination and Decision Practices	509	0.090521
5	Evaluation Practices	477	0.084830
6	Participation Dynamics	321	0.057087
7	Integration Practices	191	0.033968

Top NEXT codes (with %):

	code	count	pct
0	Knowledge Sharing	1534	0.281158
1	Idea Management	912	0.167155
2	Information Seeking	767	0.140579
3	Relational Climate	714	0.130865
4	Coordination and Decision Practices	556	0.101906
5	Evaluation Practices	505	0.092559
6	Participation Dynamics	255	0.046738
7	Integration Practices	213	0.039040

Role-signature indices (conditional):
{'Prev critique (Evaluation)': 0.085, 'Next closure (Integration/Decision)': 0.141, 'Prev seed (InfoSeek/KS)': 0.455}

=====

=

FOCAL CODE: Information Seeking

=====

=

--- BEGINNING (n=1674 utterances containing 'Information Seeking') ---

Top PREVIOUS codes (with %):

	code	count	pct
0	Knowledge Sharing	723	0.274487
1	Idea Management	405	0.153759
2	Information Seeking	380	0.144267
3	Relational Climate	356	0.135156
4	Coordination and Decision Practices	336	0.127563
5	Participation Dynamics	206	0.078208
6	Evaluation Practices	173	0.065680
7	Integration Practices	55	0.020881

Top NEXT codes (with %):

	code	count	pct
0	Knowledge Sharing	857	0.339675
1	Information Seeking	391	0.154974
2	Idea Management	372	0.147444
3	Relational Climate	292	0.115735
4	Coordination and Decision Practices	240	0.095125
5	Evaluation Practices	172	0.068173
6	Participation Dynamics	157	0.062228
7	Integration Practices	42	0.016647

Role-signature indices (conditional):

```
{'Prev critique (Evaluation)': 0.066, 'Next closure (Integration/Decision)': 0.112, 'Prev seed (InfoSeek/KS)': 0.419}
```

--- MIDDLE (n=1885 utterances containing 'Information Seeking') ---

Top PREVIOUS codes (with %):

	code	count	pct
0	Knowledge Sharing	920	0.311548
1	Idea Management	565	0.191331
2	Information Seeking	434	0.146969
3	Relational Climate	341	0.115476
4	Evaluation Practices	312	0.105655
5	Participation Dynamics	149	0.050457
6	Coordination and Decision Practices	138	0.046732
7	Integration Practices	94	0.031832

Top NEXT codes (with %):

	code	count	pct
0	Knowledge Sharing	1059	0.375798
1	Idea Management	490	0.173882
2	Information Seeking	429	0.152236
3	Relational Climate	288	0.102200
4	Evaluation Practices	286	0.101490
5	Participation Dynamics	109	0.038680
6	Coordination and Decision Practices	91	0.032292
7	Integration Practices	66	0.023421

Role-signature indices (conditional):

{'Prev critique (Evaluation)': 0.106, 'Next closure (Integration/Decision)': 0.056, 'Prev seed (InfoSeek/KS)': 0.459}

--- END (n=2100 utterances containing 'Information Seeking') ---

Top PREVIOUS codes (with %):

	code	count	pct
0	Knowledge Sharing	790	0.266712
1	Idea Management	534	0.180284
2	Information Seeking	413	0.139433
3	Relational Climate	380	0.128292
4	Coordination and Decision Practices	358	0.120864
5	Evaluation Practices	229	0.077313
6	Participation Dynamics	132	0.044564
7	Integration Practices	126	0.042539

Top NEXT codes (with %):

	code	count	pct
0	Knowledge Sharing	954	0.332057
1	Idea Management	460	0.160111
2	Information Seeking	407	0.141664
3	Relational Climate	318	0.110686
4	Coordination and Decision Practices	305	0.106161
5	Evaluation Practices	243	0.084581
6	Participation Dynamics	103	0.035851
7	Integration Practices	83	0.028890

Role-signature indices (conditional):

{'Prev critique (Evaluation)': 0.077, 'Next closure (Integration/Decision)': 0.135, 'Prev seed (InfoSeek/KS)': 0.406}

```
=====
=
FOCAL CODE: Evaluation Practices
=====
=
```

--- BEGINNING (n=685 utterances containing 'Evaluation Practices') ---

Top PREVIOUS codes (with %):

	code	count	pct
0	Knowledge Sharing	360	0.298013
1	Idea Management	268	0.221854
2	Information Seeking	166	0.137417
3	Evaluation Practices	130	0.107616
4	Relational Climate	122	0.100993
5	Participation Dynamics	87	0.072020
6	Coordination and Decision Practices	57	0.047185
7	Integration Practices	18	0.014901

Top NEXT codes (with %):

	code	count	pct
0	Knowledge Sharing	296	0.258290
1	Idea Management	207	0.180628
2	Information Seeking	179	0.156195
3	Relational Climate	146	0.127400
4	Evaluation Practices	134	0.116928
5	Participation Dynamics	79	0.068935
6	Coordination and Decision Practices	68	0.059337
7	Integration Practices	37	0.032286

Role-signature indices (conditional):

```
{'Prev critique (Evaluation)': 0.108, 'Next closure (Integration/Decision)': 0.092, 'Prev seed (InfoSeek/KS)': 0.435}
```

--- MIDDLE (n=1319 utterances containing 'Evaluation Practices') ---

Top PREVIOUS codes (with %):

	code	count	pct
0	Knowledge Sharing	655	0.304085
1	Idea Management	458	0.212628
2	Evaluation Practices	298	0.138347
3	Information Seeking	282	0.130919
4	Relational Climate	193	0.089601
5	Participation Dynamics	120	0.055710
6	Coordination and Decision Practices	80	0.037140
7	Integration Practices	68	0.031569

Top NEXT codes (with %):

	code	count	pct
0	Knowledge Sharing	612	0.280734
1	Idea Management	459	0.210550
2	Information Seeking	311	0.142661
3	Evaluation Practices	301	0.138073
4	Relational Climate	215	0.098624
5	Participation Dynamics	101	0.046330
6	Coordination and Decision Practices	94	0.043119
7	Integration Practices	87	0.039908

Role-signature indices (conditional):

{'Prev critique (Evaluation)': 0.138, 'Next closure (Integration/Decision)': 0.083, 'Prev seed (InfoSeek/KS)': 0.435}

--- END (n=1242 utterances containing 'Evaluation Practices') ---

Top PREVIOUS codes (with %):

	code	count	pct
0	Knowledge Sharing	525	0.269646
1	Idea Management	422	0.216744
2	Information Seeking	253	0.129944
3	Evaluation Practices	245	0.125835
4	Relational Climate	200	0.102722
5	Coordination and Decision Practices	138	0.070878
6	Participation Dynamics	93	0.047766
7	Integration Practices	71	0.036466

Top NEXT codes (with %):

	code	count	pct
0	Knowledge Sharing	460	0.242488
1	Idea Management	352	0.185556
2	Relational Climate	275	0.144966
3	Evaluation Practices	238	0.125461
4	Information Seeking	224	0.118081
5	Coordination and Decision Practices	173	0.091197
6	Participation Dynamics	95	0.050079
7	Integration Practices	80	0.042172

Role-signature indices (conditional):
{'Prev critique (Evaluation)': 0.126, 'Next closure (Integration/Decision)': 0.133, 'Prev seed (InfoSeek/KS)': 0.4}

=====

=

FOCAL CODE: Integration Practices

=====

=

--- BEGINNING (n=220 utterances containing 'Integration Practices') ---

Top PREVIOUS codes (with %):

	code	count	pct
0	Knowledge Sharing	108	0.284960
1	Idea Management	77	0.203166
2	Relational Climate	45	0.118734
3	Information Seeking	39	0.102902
4	Evaluation Practices	33	0.087071
5	Participation Dynamics	31	0.081794
6	Integration Practices	23	0.060686
7	Coordination and Decision Practices	23	0.060686

Top NEXT codes (with %):

	code	count	pct
0	Knowledge Sharing	91	0.260000
1	Idea Management	70	0.200000
2	Information Seeking	56	0.160000
3	Relational Climate	47	0.134286
4	Participation Dynamics	26	0.074286
5	Integration Practices	23	0.065714
6	Coordination and Decision Practices	19	0.054286
7	Evaluation Practices	18	0.051429

Role-signature indices (conditional):

{'Prev critique (Evaluation)': 0.087, 'Next closure (Integration/Decision)': 0.12, 'Prev seed (Info Seek/KS)': 0.388}

--- MIDDLE (n=419 utterances containing 'Integration Practices') ---

Top PREVIOUS codes (with %):

	code	count	pct
0	Knowledge Sharing	220	0.305132
1	Idea Management	156	0.216366
2	Evaluation Practices	90	0.124827
3	Relational Climate	82	0.113731
4	Information Seeking	66	0.091540
5	Coordination and Decision Practices	40	0.055479
6	Integration Practices	34	0.047157
7	Participation Dynamics	33	0.045770

Top NEXT codes (with %):

	code	count	pct
0	Knowledge Sharing	202	0.281729
1	Idea Management	169	0.235704
2	Information Seeking	96	0.133891
3	Evaluation Practices	69	0.096234
4	Relational Climate	68	0.094840
5	Coordination and Decision Practices	44	0.061367
6	Integration Practices	35	0.048815
7	Participation Dynamics	34	0.047420

Role-signature indices (conditional):

{'Prev critique (Evaluation)': 0.125, 'Next closure (Integration/Decision)': 0.11, 'Prev seed (Info Seek/KS)': 0.397}

--- END (n=688 utterances containing 'Integration Practices') ---

Top PREVIOUS codes (with %):

	code	count	pct
0	Knowledge Sharing	217	0.210884
1	Idea Management	181	0.175899
2	Coordination and Decision Practices	180	0.174927
3	Integration Practices	121	0.117590
4	Relational Climate	105	0.102041
5	Information Seeking	86	0.083576
6	Evaluation Practices	81	0.078717
7	Participation Dynamics	58	0.056365

Top NEXT codes (with %):

	code	count	pct
0	Knowledge Sharing	185	0.190134
1	Idea Management	176	0.180884
2	Coordination and Decision Practices	136	0.139774
3	Information Seeking	123	0.126413
4	Relational Climate	123	0.126413
5	Integration Practices	120	0.123330
6	Evaluation Practices	70	0.071942
7	Participation Dynamics	40	0.041110

Role-signature indices (conditional):
{'Prev critique (Evaluation)': 0.079, 'Next closure (Integration/Decision)': 0.263, 'Prev seed (InfoSeek/KS)': 0.294}

=====

=

FOCAL CODE: Coordination and Decision Practices

=====

=

--- BEGINNING (n=2257 utterances containing 'Coordination and Decision Practices') ---

Top PREVIOUS codes (with %):

	code	count	pct
0	Coordination and Decision Practices	980	0.314708
1	Knowledge Sharing	709	0.227681
2	Relational Climate	674	0.216442
3	Participation Dynamics	247	0.079319
4	Information Seeking	238	0.076429
5	Idea Management	182	0.058446
6	Evaluation Practices	66	0.021195
7	Integration Practices	18	0.005780

Top NEXT codes (with %):

	code	count	pct
0	Coordination and Decision Practices	985	0.291765
1	Knowledge Sharing	809	0.239633
2	Relational Climate	577	0.170912
3	Participation Dynamics	366	0.108412
4	Information Seeking	337	0.099822
5	Idea Management	219	0.064870
6	Evaluation Practices	59	0.017476
7	Integration Practices	24	0.007109

Role-signature indices (conditional):

{'Prev critique (Evaluation)': 0.021, 'Next closure (Integration/Decision)': 0.299, 'Prev seed (InfoSeek/KS)': 0.304}

--- MIDDLE (n=654 utterances containing 'Coordination and Decision Practices') ---

Top PREVIOUS codes (with %):

	code	count	pct
0	Knowledge Sharing	238	0.238477
1	Coordination and Decision Practices	198	0.198397
2	Relational Climate	143	0.143287
3	Idea Management	141	0.141283
4	Evaluation Practices	93	0.093186
5	Information Seeking	92	0.092184
6	Participation Dynamics	49	0.049098
7	Integration Practices	44	0.044088

Top NEXT codes (with %):

	code	count	pct
0	Knowledge Sharing	229	0.215428
1	Coordination and Decision Practices	198	0.186265
2	Idea Management	180	0.169332
3	Information Seeking	142	0.133584
4	Relational Climate	123	0.115710
5	Evaluation Practices	80	0.075259
6	Participation Dynamics	69	0.064911
7	Integration Practices	42	0.039511

Role-signature indices (conditional):

{'Prev critique (Evaluation)': 0.093, 'Next closure (Integration/Decision)': 0.226, 'Prev seed (InfoSeek/KS)': 0.331}

--- END (n=2288 utterances containing 'Coordination and Decision Practices') ---

Top PREVIOUS codes (with %):

	code	count	pct
0	Coordination and Decision Practices	961	0.307323
1	Knowledge Sharing	561	0.179405
2	Relational Climate	494	0.157979
3	Idea Management	312	0.099776
4	Information Seeking	306	0.097857
5	Participation Dynamics	180	0.057563
6	Evaluation Practices	176	0.056284
7	Integration Practices	137	0.043812

Top NEXT codes (with %):

	code	count	pct
0	Coordination and Decision Practices	956	0.307890
1	Relational Climate	511	0.164573
2	Knowledge Sharing	503	0.161997
3	Information Seeking	353	0.113688
4	Idea Management	296	0.095330
5	Integration Practices	177	0.057005
6	Participation Dynamics	173	0.055717
7	Evaluation Practices	136	0.043800

Role-signature indices (conditional):

```
{'Prev critique (Evaluation)': 0.056, 'Next closure (Integration/Decision)': 0.365, 'Prev seed (InfoSeek/KS)': 0.277}
```

```
In [ ]: import matplotlib.pyplot as plt
import pandas as pd
```

```
"""
```

Do not use the first graph generated from this plot

```
"""
```

```
FOCAL = "Knowledge Sharing"
```

```
TOPK = 6 # number of neighbor codes to show per PREV/NEXT
```

```
ctx = compute_code_contexts(df, FOCAL)
```

```
# ----- GRAPH 1: Role-signature indices across thirds -----
```

```
thirds = ["beginning", "middle", "end"]
```

```
sig_rows = []
```

```
for th in thirds:
```

```
    sig = role_signature(ctx[th])
```

```
    sig_rows.append({"third": th, **sig})
```

```
sig_df = pd.DataFrame(sig_rows)
```

```
plt.figure(figsize=(9, 4))
```

```
for col in ["Prev seed (InfoSeek/KS)", "Prev critique (Evaluation)", "Next closure (Integration/Deco"]
```

```
    plt.plot(sig_df["third"], sig_df[col], marker="o", label=col)
```

```
plt.title(f"Functional Role of '{FOCAL}' by Meeting Phase (Conditional Indices)")
```

```
plt.xlabel("Temporal third")
```

```
plt.ylabel("Conditional proportion")
```

```
plt.ylim(0, max(0.05, sig_df.drop(columns=["third"]).to_numpy().max() * 1.15))
```

```
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
# ----- GRAPH 2: Top neighbors (PREV and NEXT) per third -----
```

```
# Build dataframes of top neighbors for plotting
```

```
plot_rows_prev = []
```

```
plot_rows_next = []
```

```
for th in thirds:
```

```

prev_df = counter_to_pct_df(ctx[th]["prev"], topk=TOPK)
next_df = counter_to_pct_df(ctx[th]["next"], topk=TOPK)

for _, r in prev_df.iterrows():
    plot_rows_prev.append({"third": th, "neighbor_code": r["code"], "pct": r["pct"]})
for _, r in next_df.iterrows():
    plot_rows_next.append({"third": th, "neighbor_code": r["code"], "pct": r["pct"]})

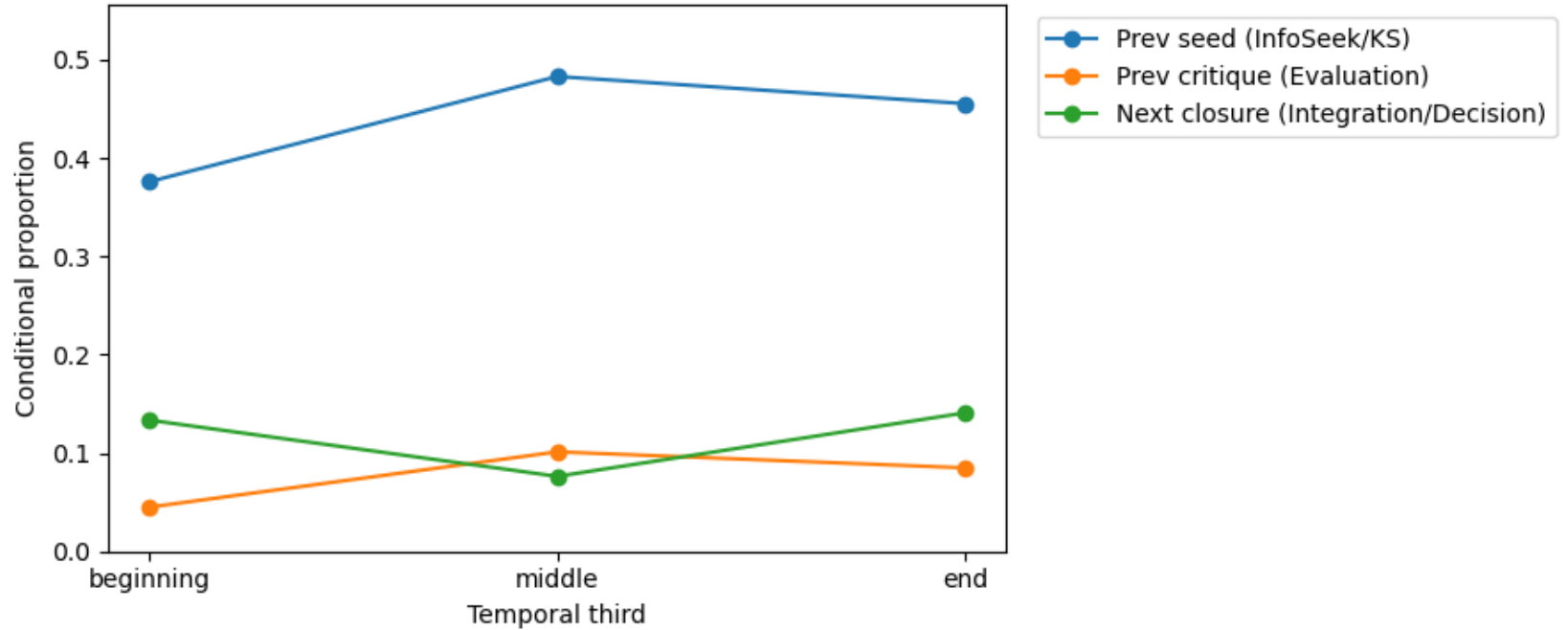
prev_plot = pd.DataFrame(plot_rows_prev)
next_plot = pd.DataFrame(plot_rows_next)

def bar_panel(plot_df, title):
    fig, axes = plt.subplots(1, 3, figsize=(14, 4), sharey=True)
    for ax, th in zip(axes, thirds):
        sub = plot_df[plot_df["third"] == th].sort_values("pct", ascending=True)
        ax.barh(sub["neighbor_code"], sub["pct"])
        ax.set_title(th.capitalize())
        ax.set_xlabel("Pct")
    axes[0].set_ylabel("Neighbor code")
    fig.suptitle(title, y=1.02)
    plt.tight_layout()
    plt.show()

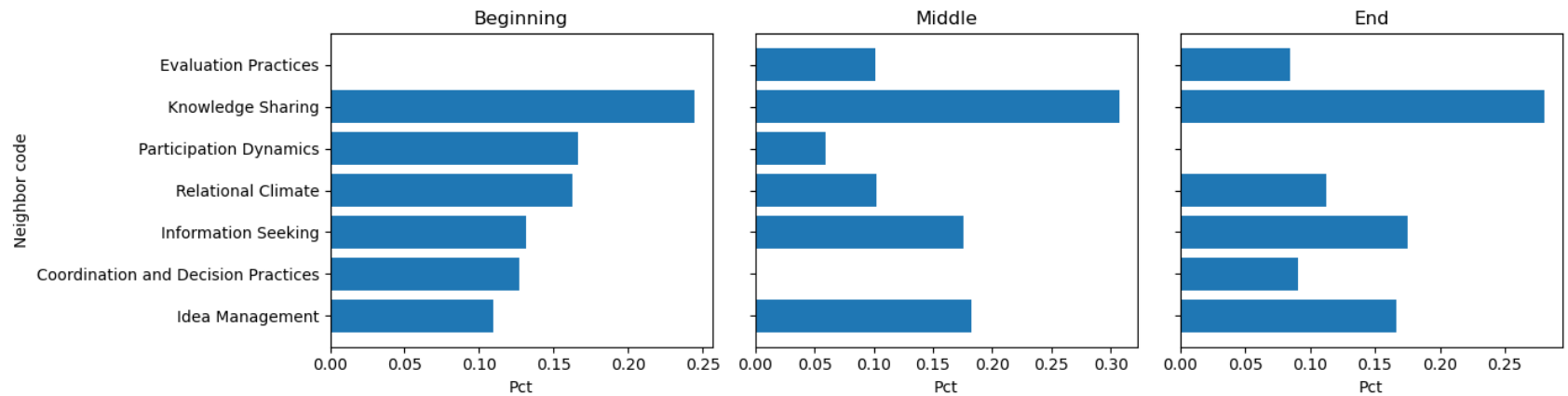
bar_panel(prev_plot, f"Most Common PREVIOUS Codes Around '{FOCAL}' (conditional on '{FOCAL}', by ph
bar_panel(next_plot, f"Most Common NEXT Codes Around '{FOCAL}' (conditional on '{FOCAL}', by phase)

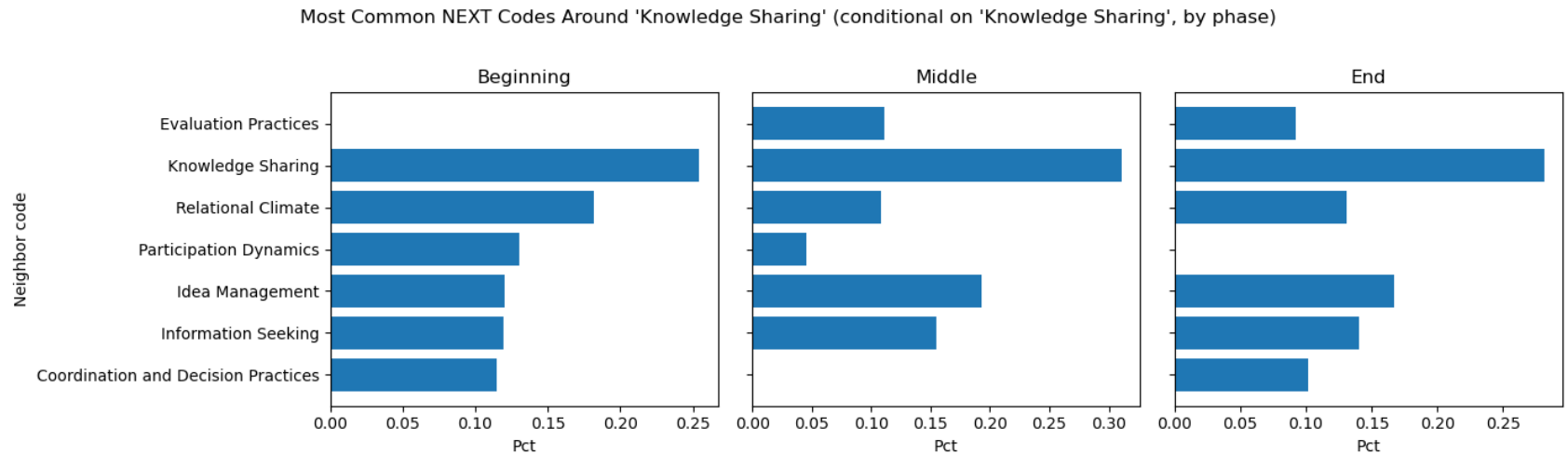
```

Functional Role of 'Knowledge Sharing' by Meeting Phase (Conditional Indices)



Most Common PREVIOUS Codes Around 'Knowledge Sharing' (conditional on 'Knowledge Sharing', by phase)





01/08/2026 work

refer to chatgpt code chunk

also need to adjust this into (or find the data when i do the initial implementation when i get to the lab)

8. HOOK FOR OUTCOMES (proposal success) WITHOUT FAKE PRETENDING If you later have session-level outcomes (team formed, funded, etc.):

- make a df_outcomes with columns: session_id, any_team, any_funded, facilitator_present, etc.
- aggregate df → session features, then merge.

```
In [6]: import json, glob, re
from collections import Counter, defaultdict
import pandas as pd
import numpy as np
from IPython.display import display
```

```
# =====
# CONFIG
# =====
```

```

DATA_GLOB = "/Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data

PHASES = ["beginning", "middle", "end"]

FOCAL_CODES = [
    "Knowledge Sharing",
    "Information Seeking",
    "Evaluation Practices",
    "Integration Practices",
    "Coordination and Decision Practices",
    "Participation Dynamics",
    "Relational Climate",
    "Idea Management",
]

SEED = {"Information Seeking", "Knowledge Sharing"}
CRITIQUE = {"Evaluation Practices"}
SYNTHESIS = {"Integration Practices"}
CLOSURE = {"Coordination and Decision Practices"}
IDEA_MGMT = {"Idea Management"}

FACILITATOR_LABELS = {"Discussion Facilitator", "Facilitator", "discussion facilitator"}

TOPK = 12
EXAMPLE_WINDOW = 1
N_EXAMPLES_PER_GROUP = 2
EXAMPLES_ONLY_FOR_SLICE = "overall"

# =====
# HELPERS
# =====

_ts_pat = re.compile(r"^s*(\d{1,2}:\d{2})(?::\d{2})?s*$")

def mmss_to_sec(s: str):
    """Parse 'MM:SS' or 'HH:MM:SS' into seconds."""
    if not isinstance(s, str):
        return None
    s = s.strip()

```



```

if not s:
    return None
parts = s.split(":")
try:
    if len(parts) == 2:
        m, sec = int(parts[0]), int(parts[1])
        return 60*m + sec
    if len(parts) == 3:
        h, m, sec = int(parts[0]), int(parts[1]), int(parts[2])
        return 3600*h + 60*m + sec
except:
    return None
return None

def parse_timestamp_range(ts: str):
    """
    Parse 'MM:SS-MM:SS' (or HH:MM:SS variants) -> (start_sec, end_sec)
    """
    if not isinstance(ts, str):
        return (None, None)
    ts = ts.strip()
    if "-" not in ts:
        return (None, None)
    a, b = ts.split("-", 1)
    return (mmss_to_sec(a), mmss_to_sec(b))

def safe_get_text(utt: dict):
    for k in ["transcript", "utterance", "text", "content", "sentence"]:
        v = utt.get(k)
        if isinstance(v, str) and v.strip():
            return v.strip()
    return None

def extract_utterances(loader_json):
    if isinstance(loader_json, list):
        return loader_json
    if isinstance(loader_json, dict):
        if "all_data" in loader_json and isinstance(loader_json["all_data"], list):
            return loader_json["all_data"]

```

```

        if "data" in loaded_json and isinstance(loaded_json["data"], list):
            return loaded_json["data"]
    return []

def role_bucket(role_str):
    if role_str is None:
        return "non-facilitator"
    return "facilitator" if role_str in FACILITATOR_LABELS else "non-facilitator"

def infer_state_from_codes(codes):
    S = set(codes)
    if S & CLOSURE:
        return "Commitment/Closure"
    if S & SYNTHESIS:
        return "Synthesis"
    if S & CRITIQUE:
        return "Stress-testing"
    if S & SEED:
        return "Seeding"
    if S & IDEA_MGMT:
        return "Idea Development"
    if "Participation Dynamics" in S:
        return "Participation Structuring"
    if "Relational Climate" in S:
        return "Relational Support"
    return "Other"

def jensen_shannon(p, q, eps=1e-12):
    keys = set(p.keys()) | set(q.keys())
    P = np.array([p.get(k, 0.0) for k in keys], dtype=float) + eps
    Q = np.array([q.get(k, 0.0) for k in keys], dtype=float) + eps
    P /= P.sum()
    Q /= Q.sum()
    M = 0.5 * (P + Q)

    def kl(a, b):
        return float(np.sum(a * np.log(a / b)))

    return 0.5 * kl(P, M) + 0.5 * kl(Q, M)

```

```

def counter_to_prob(counter: Counter):
    tot = sum(counter.values())
    if tot == 0:
        return {}
    return {k: v / tot for k, v in counter.items()}

def assign_third_from_pos(pos01: float):
    if pos01 < 1/3:
        return "beginning"
    if pos01 < 2/3:
        return "middle"
    return "end"

# =====
# 1) LOAD ALL SESSIONS → UTTERANCE-LEVEL DF (timestamp-derived thirds, reset-safe)
# =====

rows = []
paths = sorted(glob.glob(DATA_GLOB))
if not paths:
    raise FileNotFoundError(f"No JSON files matched: {DATA_GLOB}")

bad_files = 0
skipped_no_times = 0

for path in paths:
    with open(path, "r") as f:
        loaded = json.load(f)

        utterances = extract_utterances(loaded)
        if not utterances:
            bad_files += 1
            continue

        session_id = path.split("/")[-1].replace(".json", "")

        # First pass: get raw start/end seconds for each utterance
        raw = []

```

```

for idx, utt in enumerate(utterances):
    if not isinstance(utt, dict):
        continue

    ann = utt.get("annotations", {}) or {}
    if not isinstance(ann, dict) or not ann:
        continue

    # Prefer explicit start_time/end_time; else parse "timestamp"
    st = mmss_to_sec(utt.get("start_time")) if utt.get("start_time") else None
    en = mmss_to_sec(utt.get("end_time")) if utt.get("end_time") else None
    if st is None or en is None:
        st2, en2 = parse_timestamp_range(utt.get("timestamp"))
        st = st if st is not None else st2
        en = en if en is not None else en2

    raw.append((idx, utt, st, en))

# If we genuinely have no usable times, fall back to existing "when" fields
have_any_times = any(st is not None and en is not None for _, _, st, en in raw)

if not have_any_times:
    # fallback: use the provided "when" if present
    skipped_no_times += 1
    for idx, utt, st, en in raw:
        ann = utt.get("annotations", {}) or {}
        codes = list(ann.keys())
        if not codes:
            continue

        third = utt.get("when")
        if third not in PHASES:
            any_meta = next(iter(ann.values()), {})
            if isinstance(any_meta, dict):
                third = any_meta.get("when", None)
        if third not in PHASES:
            third = "unknown"

    rows.append({

```

```

        "session_id": session_id,
        "idx": idx,
        "speaker": utt.get("speaker"),
        "role_raw": utt.get("role"),
        "role": role_bucket(utt.get("role")),
        "third": third,
        "codes": codes,
        "text": safe_get_text(utt),
        "t_start": None,
        "t_end": None,
        "pos01": None,
    })
    continue

```

```

# Second pass: make timeline monotonic per session (handles timer resets)
# We assume utterances are in order already, but we still sort by idx.
raw = sorted(raw, key=lambda x: x[0])

```

```

offset = 0
prev_end = None
fixed = []

```

```

for idx, utt, st, en in raw:
    ann = utt.get("annotations", {}) or {}
    codes = list(ann.keys())
    if not codes:
        continue

    if st is None or en is None:
        # if missing, just skip the timing but keep the row; we'll later mark third unknown
        fixed.append((idx, utt, None, None, codes))
        continue

```

```

# If time goes backwards a lot, assume a reset and bump offset by 10 minutes (600s).
# If your resets are a different size, change 600.
if prev_end is not None and st + offset < prev_end - 30:
    offset += 600

```

```

st_fix = st + offset

```

```

en_fix = en + offset

# if end < start after offset, force end >= start
if en_fix < st_fix:
    en_fix = st_fix

prev_end = en_fix
fixed.append((idx, utt, st_fix, en_fix, codes))

# Define session duration from max end
ends = [en for _, _, en, _ in fixed if en is not None]
session_end = max(ends) if ends else None
if session_end is None or session_end <= 0:
    # shouldn't happen if have_any_times True, but guard anyway
    for idx, utt, st_fix, en_fix, codes in fixed:
        rows.append({
            "session_id": session_id,
            "idx": idx,
            "speaker": utt.get("speaker"),
            "role_raw": utt.get("role"),
            "role": role_bucket(utt.get("role")),
            "third": "unknown",
            "codes": codes,
            "text": safe_get_text(utt),
            "t_start": st_fix,
            "t_end": en_fix,
            "pos01": None,
        })
    continue

for idx, utt, st_fix, en_fix, codes in fixed:
    if st_fix is None:
        third = "unknown"
        pos01 = None
    else:
        pos01 = st_fix / session_end
        third = assign_third_from_pos(pos01)

    rows.append({

```

```

        "session_id": session_id,
        "idx": idx,
        "speaker": utt.get("speaker"),
        "role_raw": utt.get("role"),
        "role": role_bucket(utt.get("role")),
        "third": third,
        "codes": codes,
        "text": safe_get_text(utt),
        "t_start": st_fix,
        "t_end": en_fix,
        "pos01": pos01,
    })

df = pd.DataFrame(rows).sort_values(["session_id", "idx"]).reset_index(drop=True)

print(f"Matched {len(paths)} session JSONs.")
print(f"Skipped {bad_files} files with unexpected structure (no utterance list found).")
print(f"Built utterance-level df: {len(df)} rows from {df.session_id.nunique()} sessions.")
print(f"Fallback-to-when used for sessions with no valid times: {skipped_no_times}")
print(df["third"].value_counts())

# =====
# 2) IDEA STATES
# =====

df["idea_state"] = df["codes"].apply(infer_state_from_codes)

print("\nIdea-state distribution (overall):")
print(df["idea_state"].value_counts().head(15))

# =====
# 3) CONTEXT: prev/next codes around a focal code (multi-label aware)
# =====

def compute_contexts(df_in, focal_code, phase=None, role=None):
    prev_all, next_all = Counter(), Counter()
    per_s_prev = defaultdict(Counter)
    per_s_next = defaultdict(Counter)
    n = 0

```

```

sub = df_in
if phase is not None:
    sub = sub[sub["third"] == phase]
if role is not None:
    sub = sub[sub["role"] == role]

for sid, g in sub.sort_values(["session_id", "idx"]).groupby("session_id"):
    g = g.reset_index(drop=True)
    for i in range(len(g)):
        if focal_code not in g.at[i, "codes"]:
            continue
        n += 1
        if i > 0:
            prev_codes = g.at[i-1, "codes"] or []
            prev_all.update(prev_codes)
            per_s_prev[sid].update(prev_codes)
        if i < len(g)-1:
            next_codes = g.at[i+1, "codes"] or []
            next_all.update(next_codes)
            per_s_next[sid].update(next_codes)

    return {
        "prev": prev_all, "next": next_all, "n": n,
        "per_session_prev": per_s_prev, "per_session_next": per_s_next
    }

def stability_report(per_session_counters):
    pooled = Counter()
    for c in per_session_counters.values():
        pooled.update(c)
    pooled_p = counter_to_prob(pooled)

    jsds = []
    for sid, c in per_session_counters.items():
        if sum(c.values()) < 5:
            continue
        p = counter_to_prob(c)
        jsds.append(jensen_shannon(pooled_p, p))

```



```

    return {
        "n_sessions_used": len(jsds),
        "mean_jsd": float(np.mean(jsds)) if jsds else np.nan,
        "median_jsd": float(np.median(jsds)) if jsds else np.nan,
    }

# =====
# 4) BASELINE PROB + LIFT
# =====

baseline_counts = Counter()
for codes in df["codes"]:
    baseline_counts.update(codes)
baseline_prob = counter_to_prob(baseline_counts)

def lift_rows(neighbor_counter: Counter, baseline_prob: dict, focal_code: str, slice_name: str, direction: str):
    tot = sum(neighbor_counter.values()) or 1
    out = []
    for code, c in neighbor_counter.items():
        p_neigh = c / tot
        p_base = baseline_prob.get(code, 1e-12)
        lift = p_neigh / p_base if p_base > 0 else np.inf
        out.append({
            "focal_code": focal_code,
            "slice": slice_name,
            "direction": direction,
            "neighbor_code": code,
            "neighbor_count": c,
            "neighbor_total": tot,
            "p_neighbor": p_neigh,
            "p_base": p_base,
            "lift": lift,
        })
    return out

# =====
# 5) STABILITY TABLE (overall + phase + phase×role)
# =====

```

```

df_phased = df[df["third"].isin(PHASES)].copy()

slice_specs = [("overall", dict())]
slice_specs += [(f"phase={ph}", dict(phase=ph)) for ph in PHASES]
slice_specs += [(f"phase={ph},role={rl}", dict(phase=ph, role=rl)) for ph in PHASES for rl in ["fac

stability_rows = []
for focal in FOCAL_CODES:
    for slice_name, kwargs in slice_specs:
        ctx = compute_contexts(df_phased, focal, **kwargs)
        stab_prev = stability_report(ctx["per_session_prev"])
        stab_next = stability_report(ctx["per_session_next"])
        stability_rows.append({
            "focal_code": focal,
            "slice": slice_name,
            "n_focal": ctx["n"],
            "mean_jsd_prev": stab_prev["mean_jsd"],
            "mean_jsd_next": stab_next["mean_jsd"],
        })

stability_df = pd.DataFrame(stability_rows).sort_values(["focal_code", "slice"]).reset_index(drop=True)

print("\nSTABILITY SUMMARY (lower mean_jsd = more stable across sessions):")
display(stability_df)

# =====
# 6) NEIGHBORS + LIFT TABLE (Top-K)
# =====

all_neighbor_rows = []
for focal in FOCAL_CODES:
    for slice_name, kwargs in slice_specs:
        ctx = compute_contexts(df_phased, focal, **kwargs)
        all_neighbor_rows += lift_rows(ctx["prev"], baseline_prob, focal, slice_name, "prev")
        all_neighbor_rows += lift_rows(ctx["next"], baseline_prob, focal, slice_name, "next")

neighbors_lift_df = pd.DataFrame(all_neighbor_rows)
neighbors_lift_df = neighbors_lift_df[neighbors_lift_df["neighbor_code"] != neighbors_lift_df["focal

```

```

neighbors_topk_df = (
    neighbors_lift_df
    .groupby(["focal_code", "slice", "direction"], as_index=False, group_keys=False)
    .apply(lambda g: g.sort_values(["lift", "neighbor_count"], ascending=[False, False]).head(TOPK))
    .reset_index(drop=True)
    .sort_values(["slice", "focal_code", "direction", "lift", "neighbor_count"],
                  ascending=[True, True, True, False, False])
    .reset_index(drop=True)
)

print("\nCONDENSED TOP-K NEIGHBOR+LIFT TABLE:")
display(neighbors_topk_df)

# =====
# 7) EXAMPLES TABLE (overall only)
# =====

def window_string(g, focal_i, window=1):
    lo = max(0, focal_i - window)
    hi = min(len(g) - 1, focal_i + window)
    parts = []
    for j in range(lo, hi + 1):
        mark = " <-- FOCAL" if j == focal_i else ""
        txt = g.at[j, "text"] if isinstance(g.at[j, "text"], str) and g.at[j, "text"].strip() else ""
        parts.append(f"[{j}] {g.at[j, 'speaker']} ({g.at[j, 'role']},{g.at[j, 'third']}): {g.at[j, 'code']}{mark}")
    return "\n".join(parts)

def find_examples(df_in, focal, neighbor_code, direction="next", n_examples=2, window=1):
    examples = []
    df_sorted = df_in.sort_values(["session_id", "idx"])

    for sid, g in df_sorted.groupby("session_id"):
        g = g.reset_index(drop=True)
        if len(g) < 2:
            continue

        for i in range(len(g)):
            if focal not in g.at[i, "codes"]:

```

```

        continue

    if direction == "next":
        if i >= len(g) - 1:
            continue
        hit = neighbor_code in g.at[i + 1, "codes"]
    else: # "prev"
        if i == 0:
            continue
        hit = neighbor_code in g.at[i - 1, "codes"]

    if not hit:
        continue

    examples.append({
        "example_session_id": sid,
        "example_focal_idx": i,
        "example_window": window_string(g, i, window=window),
    })
    if len(examples) >= n_examples:
        return examples
return examples

```

```

overall_top = neighbors_topk_df[neighbors_topk_df["slice"] == EXAMPLES_ONLY_FOR_SLICE].copy()

```

```

example_rows = []
for _, r in overall_top.iterrows():
    ex = find_examples(
        df_phased,
        focal=r["focal_code"],
        neighbor_code=r["neighbor_code"],
        direction=r["direction"],
        n_examples=N_EXAMPLES_PER_GROUP,
        window=EXAMPLE_WINDOW
    )
    if not ex:
        example_rows.append(**r.to_dict(), "example_session_id": None, "example_focal_idx": None,
    else:
        for e in ex:

```

```

        example_rows.append(**r.to_dict(), **e})

examples_df = pd.DataFrame(example_rows)

print(f"\nEXAMPLES TABLE (slice == '{EXAMPLES_ONLY_FOR_SLICE}', up to {N_EXAMPLES_PER_GROUP} per gr
display(examples_df.head(30))

print("\nDone. You now have:")
print("1) stability_df      -> stability across sessions (overall + phase + phase×role)")
print("2) neighbors_topk_df -> top neighbor transitions w/ lift (overall + phase + phase×role)")
print("3) examples_df       -> transcript windows for high-lift transitions (overall only)")

```

Matched 157 session JSONs.

Skipped 0 files with unexpected structure (no utterance list found).

Built utterance-level df: 30558 rows from 157 sessions.

Fallback-to-when used for sessions with no valid times: 0

third

end 11584

beginning 10080

middle 8894

Name: count, dtype: int64

Idea-state distribution (overall):

idea_state

Seeding 12016

Commitment/Closure 5319

Relational Support 4292

Stress-testing 3369

Other 2076

Participation Structuring 1357

Synthesis 1132

Idea Development 997

Name: count, dtype: int64

STABILITY SUMMARY (lower mean_jsd = more stable across sessions):

	focal_code		slice	n_focal	mean_jsd_prev	mean_jsd_next
0	Coordination and Decision Practices		overall	5319	0.033371	0.031457
1	Coordination and Decision Practices		phase=beginning	2290	0.065696	0.060467
2	Coordination and Decision Practices		phase=beginning,role=facilitator	1421	0.067772	0.058996
3	Coordination and Decision Practices		phase=beginning,role=non-facilitator	869	0.118650	0.112262
4	Coordination and Decision Practices		phase=end	2341	0.068528	0.067776
...	...		...	...	...	...
75	Relational Climate		phase=end,role=facilitator	903	0.114950	0.112951
76	Relational Climate		phase=end,role=non-facilitator	2484	0.070365	0.080417
77	Relational Climate		phase=middle	2139	0.081478	0.084334
78	Relational Climate		phase=middle,role=facilitator	546	0.134414	0.140805
79	Relational Climate		phase=middle,role=non-facilitator	1593	0.097508	0.086988

80 rows × 5 columns

CONDENSED TOP-K NEIGHBOR+LIFT TABLE:

```

/var/folders/wp/cn7__9416yj3c5_vcykys8hc0000gn/T/ipykernel_60716/517433897.py:444: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.
      .apply(lambda g: g.sort_values(["lift", "neighbor_count"], ascending=[False, False]).head(TOPK))

```

	focal_code	slice	direction	neighbor_code	neighbor_count	neighbor_total	p_neighbor
0	Coordination and	overall	next	Participation	684	8497	0.080499

Decision Practices			Dynamics				
1	Coordination and Decision Practices	overall	next	Integration Practices	252	8497	0.029658
2	Coordination and Decision Practices	overall	next	Relational Climate	1576	8497	0.185477
3	Coordination and Decision Practices	overall	next	Information Seeking	862	8497	0.101448
4	Coordination and Decision Practices	overall	next	Knowledge Sharing	1577	8497	0.185595
...	...	...	...	...	...	...	...
1275	Relational Climate	phase=middle,role=non-facilitator	prev	Information Seeking	287	2479	0.115772
1276	Relational Climate	phase=middle,role=non-facilitator	prev	None	100	2479	0.040339
1277	Relational Climate	phase=middle,role=non-facilitator	prev	Integration Practices	65	2479	0.026220
1278	Relational Climate	phase=middle,role=non-facilitator	prev	Participation Dynamics	78	2479	0.031464
1279	Relational Climate	phase=middle,role=non-facilitator	prev	Coordination and Decision Practices	80	2479	0.032271

1280 rows × 9 columns

EXAMPLES TABLE (slice == 'overall', up to 2 per group):

	focal_code	slice	direction	neighbor_code	neighbor_count	neighbor_total	p_neighbor	p_base	li
0	Coordination and Decision Practices	overall	next	Participation Dynamics	684	8497	0.080499	0.070651	1.13938
1	Coordination and Decision Practices	overall	next	Participation Dynamics	684	8497	0.080499	0.070651	1.13938
2	Coordination and Decision Practices	overall	next	Integration Practices	252	8497	0.029658	0.027742	1.06904
3	Coordination and Decision Practices	overall	next	Integration Practices	252	8497	0.029658	0.027742	1.06904
4	Coordination and Decision Practices	overall	next	Relational Climate	1576	8497	0.185477	0.180334	1.02852
5	Coordination and Decision Practices	overall	next	Relational Climate	1576	8497	0.185477	0.180334	1.02852
6	Coordination and Decision Practices	overall	next	Information Seeking	862	8497	0.101448	0.116766	0.86887
7	Coordination and Decision Practices	overall	next	Information Seeking	862	8497	0.101448	0.116766	0.86887

8	Coordination and Decision Practices	overall	next	Knowledge Sharing	1577	8497	0.185595	0.242595	0.76504
9	Coordination and Decision Practices	overall	next	Knowledge Sharing	1577	8497	0.185595	0.242595	0.76504
10	Coordination and Decision Practices	overall	next	None	252	8497	0.029658	0.042379	0.69982
11	Coordination and Decision Practices	overall	next	None	252	8497	0.029658	0.042379	0.69982
12	Coordination and Decision Practices	overall	next	Idea Management	728	8497	0.085677	0.135648	0.63167
13	Coordination and Decision Practices	overall	next	Idea Management	728	8497	0.085677	0.135648	0.63167
14	Coordination and Decision Practices	overall	next	Evaluation Practices	347	8497	0.040838	0.075306	0.54229
15	Coordination and Decision Practices	overall	next	Evaluation Practices	347	8497	0.040838	0.075306	0.54229
16	Coordination and Decision	overall	prev	Relational Climate	1737	8285	0.209656	0.180334	1.16261

25	and Decision Practices	overall	prev	Knowledge Sharing	1550	8285	0.187085	0.242595	0.77118
26	Coordination and Decision Practices	overall	prev	Information Seeking	658	8285	0.079421	0.116766	0.68017
27	Coordination and Decision Practices	overall	prev	Information Seeking	658	8285	0.079421	0.116766	0.68017
28	Coordination and Decision Practices	overall	prev	Evaluation Practices	404	8285	0.048763	0.075306	0.64753
29	Coordination and Decision Practices	overall	prev	Evaluation Practices	404	8285	0.048763	0.075306	0.64753

Done. You now have:

- 1) stability_df -> stability across sessions (overall + phase + phase×role)
- 2) neighbors_topk_df -> top neighbor transitions w/ lift (overall + phase + phase×role)
- 3) examples_df -> transcript windows for high-lift transitions (overall only)

next code chunk

```
In [8]: from collections import Counter
import pandas as pd
import numpy as np

# Build explicit adjacent transitions (multi-label aware):
# For each adjacent pair (i, i+1), we count all (a in codes_i, b in codes_{i+1}) pairs.
def build_transition_features(df_phased, allowed_codes=None):
    feats = []
```

```

for sid, g in df_phased.sort_values(["session_id", "idx"]).groupby("session_id"):
    g = g.reset_index(drop=True)

    # session-level counters by phase and direction
    ctr = Counter()
    denom = Counter() # denominators for rates (e.g., how many transitions exist per phase)

    for i in range(len(g) - 1):
        phase = g.at[i, "third"]
        if phase not in PHASES:
            continue

        A = g.at[i, "codes"] or []
        B = g.at[i + 1, "codes"] or []

        if allowed_codes is not None:
            A = [x for x in A if x in allowed_codes]
            B = [x for x in B if x in allowed_codes]

        if not A or not B:
            continue

        # denominator: number of adjacent transitions in this phase
        denom[(phase, "next_total")] += 1

        # count all label-pairs
        for a in A:
            for b in B:
                if a == b:
                    continue
                ctr[(phase, "next", a, b)] += 1

    # flatten to a row
    row = {"session_id": sid}
    for (ph, key), d in denom.items():
        row[f"{ph}__{key}"] = d

    for (ph, direction, a, b), c in ctr.items():
        row[f"{ph}__{direction}__{a}__{b}__count"] = c

```

```

    feats.append(row)

feat_df = pd.DataFrame(feats).fillna(0)

# Ensure denominators exist for all phases
for ph in PHASES:
    total_col = f"{ph}__next_total"
    if total_col not in feat_df.columns:
        feat_df[total_col] = 0.0

# Build ALL rate columns in one go (prevents fragmentation)
rate_blocks = []
for ph in PHASES:
    total_col = f"{ph}__next_total"
    total = feat_df[total_col].replace(0, np.nan)

    trans_cols = [
        c for c in feat_df.columns
        if c.startswith(f"{ph}__next__") and c.endswith("__count")
    ]
    if not trans_cols:
        continue

    rates = feat_df[trans_cols].div(total, axis=0).fillna(0.0)
    rates.columns = [c.replace("__count", "__rate") for c in rates.columns]
    rate_blocks.append(rates)

if rate_blocks:
    feat_df = pd.concat([feat_df] + rate_blocks, axis=1)

# Optional: defragment for faster downstream ops
feat_df = feat_df.copy()

return feat_df

```

```

allowed = set(FOCAL_CODES) # or set(baseline_prob.keys()) if you want all codes
session_transition_features = build_transition_features(df_phased, allowed_codes=allowed)

```

```
print(session_transition_features.shape)
display(session_transition_features.head())
```

(157, 340)

	session_id	beginning__next_total	middle__next_total	end__next_total	beginning__next_Idea Management__Knowledge Sharing__count	Ma
0	2020_11_05_NES_S1	33.0	23.0	21.0	5.0	
1	2020_11_05_NES_S2	26.0	26.0	12.0	2.0	
2	2020_11_05_NES_S3	11.0	17.0	14.0	2.0	
3	2020_11_05_NES_S4	17.0	6.0	13.0	1.0	
4	2020_11_05_NES_S5	10.0	11.0	6.0	3.0	

5 rows × 340 columns

```
In [9]: from collections import Counter
import numpy as np
import pandas as pd

PHASES = ["beginning", "middle", "end"]

# =====
# 8) COLLAPSE TO SESSION-LEVEL "MEASUREMENT MODEL" FEATURES
# (interpretable: flow + progression + transition stability)
# =====

# ----- A) Idea-state flow proportions per (session, phase) -----

def build_idea_state_flow(df_in, phases=PHASES):
    """
    Returns wide df:
    session_id
```

```

        beginning__Seeding, beginning__Stress-testing, ...
        middle__...
        end__...
    """
    # counts by (session, phase, idea_state)
    tmp = (
        df_in[df_in["third"].isin(phases)]
        .groupby(["session_id", "third", "idea_state"])
        .size()
        .rename("n")
        .reset_index()
    )

    # totals per (session, phase)
    totals = (
        df_in[df_in["third"].isin(phases)]
        .groupby(["session_id", "third"])
        .size()
        .rename("total")
        .reset_index()
    )

    tmp = tmp.merge(totals, on=["session_id", "third"], how="left")
    tmp["prop"] = tmp["n"] / tmp["total"]

    # pivot to wide
    wide = (
        tmp.pivot_table(index="session_id", columns=["third", "idea_state"], values="prop", fill_val=0)
    )
    wide.columns = [f"{ph}__{st}" for (ph, st) in wide.columns]
    wide = wide.reset_index()

    return wide

# ----- B) Add progression / "shape of meeting" features -----

def add_progression_features(idea_flow_df):
    """

```

```

Adds a few interpretable deltas. Safe: missing columns become 0.
"""
df = idea_flow_df.copy()

def col(name):
    return df[name] if name in df.columns else 0.0

# Core "narrative" deltas
df["delta_synthesis"] = col("end__Synthesis") - col("beginning__Synthesis")
df["delta_closure"] = col("end__Commitment/Closure") - col("beginning__Commitment/Closure")
df["delta_stress_mid"] = col("middle__Stress-testing") - col("beginning__Stress-testing")

# Early signals
df["early_commitment"] = col("beginning__Commitment/Closure")
df["early_seeding"] = col("beginning__Seeding")

# End-state "did they land the plane"
df["late_closure"] = col("end__Commitment/Closure")
df["late_synthesis"] = col("end__Synthesis")

return df

# ----- C) Session-level transition stability proxy (JSD to global) -----
# Your stability_df is across sessions (per focal code). Useful, but not session-level.
# For modeling later, you want at least one stability measure per session.

def jensen_shannon_from_arrays(P, Q, eps=1e-12):
    P = np.asarray(P, dtype=float) + eps
    Q = np.asarray(Q, dtype=float) + eps
    P /= P.sum()
    Q /= Q.sum()
    M = 0.5 * (P + Q)
    return 0.5 * np.sum(P * np.log(P / M)) + 0.5 * np.sum(Q * np.log(Q / M))

def session_transition_jsd(session_transition_features, phases=PHASES):
    """
    Computes per-session JSD distance between that session's transition distribution

```


and the global transition distribution (overall), per phase and overall.

Returns:

```
    session_id,  
    jsd_next_overall,  
    jsd_next_beginning, jsd_next_middle, jsd_next_end  
    """
```

```
df = session_transition_features.copy()
```

```
# all transition count columns (phase-specific)
```

```
count_cols = [c for c in df.columns if c.endswith("__count") and "__next__" in c]
```

```
# global distribution over all sessions
```

```
global_counts = df[count_cols].sum(axis=0).values
```

```
if global_counts.sum() == 0:
```

```
    # degenerate
```

```
    out = df[["session_id"]].copy()
```

```
    out["jsd_next_overall"] = 0.0
```

```
    for ph in phases:
```

```
        out[f"jsd_next_{ph}"] = 0.0
```

```
    return out
```

```
global_vec = global_counts
```

```
# overall session jsd
```

```
jsd_overall = []
```

```
for _, r in df.iterrows():
```

```
    vec = r[count_cols].values
```

```
    if vec.sum() == 0:
```

```
        jsd_overall.append(0.0)
```

```
    else:
```

```
        jsd_overall.append(float(jensen_shannon_from_arrays(vec, global_vec)))
```

```
out = pd.DataFrame({"session_id": df["session_id"], "jsd_next_overall": jsd_overall})
```

```
# per-phase jsd (only columns from that phase)
```

```
for ph in phases:
```

```
    ph_cols = [c for c in count_cols if c.startswith(f"{ph}__next__")]
```

```
    if not ph_cols:
```

```
        out[f"jsd_next_{ph}"] = 0.0
```

```

        continue

    global_ph = df[ph_cols].sum(axis=0).values
    vals = []
    for _, r in df.iterrows():
        vec = r[ph_cols].values
        if vec.sum() == 0 or global_ph.sum() == 0:
            vals.append(0.0)
        else:
            vals.append(float(jensen_shannon_from_arrays(vec, global_ph)))
    out[f"jsd_next_{ph}"] = vals

    return out

# ----- D) Optional: light session-level role composition -----
# (Sometimes you want "how facilitator-heavy is this meeting")

def session_role_mix(df_in, phases=PHASES):
    tmp = df_in[df_in["third"].isin(phases)].copy()
    # role is already "facilitator"/"non-facilitator" in your df
    role_counts = (
        tmp.groupby(["session_id", "role"])
        .size()
        .rename("n")
        .reset_index()
    )
    totals = tmp.groupby("session_id").size().rename("total").reset_index()
    role_counts = role_counts.merge(totals, on="session_id", how="left")
    role_counts["prop"] = role_counts["n"] / role_counts["total"]

    wide = role_counts.pivot_table(index="session_id", columns="role", values="prop", fill_value=0.)
    wide = wide.rename(columns={
        "facilitator": "prop_facilitator_utterances",
        "non-facilitator": "prop_nonfacilitator_utterances"
    }).reset_index()
    return wide

```

```

# =====
# RUN: build the measurement table
# =====

# 1) Flow proportions
idea_flow_df = build_idea_state_flow(df_phased, phases=PHASES)
idea_flow_df = add_progression_features(idea_flow_df)

# 2) Transition stability per session (requires session_transition_features from your chunk)
# If you haven't computed session_transition_features in this notebook cell yet, run that first.
stability_session_df = session_transition_jsd(session_transition_features, phases=PHASES)

# 3) Role mix (optional but usually helpful)
role_mix_df = session_role_mix(df_phased, phases=PHASES)

# 4) Final measurement table
measurement_df = (
    idea_flow_df
    .merge(stability_session_df, on="session_id", how="left")
    .merge(role_mix_df, on="session_id", how="left")
    .fillna(0.0)
)

print("measurement_df shape:", measurement_df.shape)
display(measurement_df.head())

```

measurement_df shape: (157, 38)

	session_id	beginning__Commitment/Closure	beginning__Idea Development	beginning__Other	beginning__Participa Structu
0	2020_11_05_NES_S1	0.090909	0.0	0.000000	0.000
1	2020_11_05_NES_S2	0.178571	0.0	0.035714	0.107
2	2020_11_05_NES_S3	0.090909	0.0	0.000000	0.090
3	2020_11_05_NES_S4	0.352941	0.0	0.000000	0.117
4	2020_11_05_NES_S5	0.200000	0.2	0.000000	0.200

5 rows x 38 columns

```
In [11]: import numpy as np
import pandas as pd

# =====
# 9) SANITY CHECKS + SUMMARY (ALL SESSIONS)
# =====

print("measurement_df shape:", measurement_df.shape) # should be (157, ~something)

# 1) session coverage
n_sessions = measurement_df["session_id"].nunique()
print("Unique sessions:", n_sessions)

# 2) duplicates
dup = measurement_df["session_id"].duplicated().sum()
print("Duplicate session_id rows:", dup)

# 3) missingness
na_cols = measurement_df.columns[measurement_df.isna().any()].tolist()
print("Columns with NA:", len(na_cols))
if na_cols:
    print("NA columns:", na_cols[:20], "...")
    print("Total NA cells:", int(measurement_df.isna().sum().sum()))
```

```

# 4) quick descriptive stats for the core derived metrics
core_cols = [
    "delta_synthesis", "delta_closure", "delta_stress_mid",
    "early_commitment", "early_seeding", "late_closure", "late_synthesis",
    "jsd_next_overall", "jsd_next_beginning", "jsd_next_middle", "jsd_next_end",
    "prop_facilitator_utterances"
]

core_cols = [c for c in core_cols if c in measurement_df.columns]

summary = measurement_df[core_cols].describe(percentiles=[0.05,0.25,0.5,0.75,0.95]).T
display(summary)

# 5) find "interesting" sessions to inspect (extremes)
def top_sessions(col, k=5, ascending=False):
    if col not in measurement_df.columns:
        return None
    return measurement_df[["session_id", col]].sort_values(col, ascending=ascending).head(k)

display(top_sessions("delta_closure", k=8, ascending=False))
display(top_sessions("delta_synthesis", k=8, ascending=False))
display(top_sessions("jsd_next_overall", k=8, ascending=False))

# 6) build a plotting-friendly "long" view for phase trajectories (optional but super useful)
# pick a small set of states to visualize
states_to_plot = ["Seeding", "Stress-testing", "Synthesis", "Commitment/Closure"]

traj_rows = []
for _, r in measurement_df.iterrows():
    sid = r["session_id"]
    for ph in ["beginning", "middle", "end"]:
        row = {"session_id": sid, "phase": ph}
        for st in states_to_plot:
            col = f"{ph}__{st}"
            row[st] = float(r[col]) if col in measurement_df.columns else 0.0
        traj_rows.append(row)

traj_df = pd.DataFrame(traj_rows)

```

```
display(traj_df.head(9))
print("traj_df shape:", traj_df.shape)  # should be 157 * 3 rows
```

measurement_df shape: (157, 38)
 Unique sessions: 157
 Duplicate session_id rows: 0
 Columns with NA: 0

	count	mean	std	min	5%	25%	50%	75%	
delta_synthesis	157.0	0.024601	0.046712	-0.190007	-0.051105	0.000000	0.024096	0.053908	0.
delta_closure	157.0	-0.017536	0.164644	-0.583333	-0.229459	-0.102076	-0.025000	0.070726	0.
delta_stress_mid	157.0	0.085357	0.112382	-0.400000	-0.089524	0.023262	0.094308	0.159091	0.
early_commitment	157.0	0.225404	0.120101	0.000000	0.016327	0.152941	0.220183	0.296296	0.
early_seeding	157.0	0.419412	0.123025	0.000000	0.230746	0.347222	0.408163	0.481481	0.
late_closure	157.0	0.207867	0.117220	0.000000	0.064113	0.131313	0.192308	0.263158	0.
late_synthesis	157.0	0.047796	0.034881	0.000000	0.000000	0.023256	0.041667	0.072727	0.
jsd_next_overall	157.0	0.097646	0.074077	0.000000	0.051351	0.063435	0.075518	0.110035	0.
jsd_next_beginning	157.0	0.095049	0.078744	0.000000	0.034923	0.051499	0.069947	0.108277	0.
jsd_next_middle	157.0	0.098760	0.081733	0.000000	0.040249	0.061756	0.080734	0.102172	0.
jsd_next_end	157.0	0.088974	0.070607	0.000000	0.038018	0.054532	0.070751	0.103929	0.
prop_facilitator_utterances	157.0	0.257867	0.174852	0.000000	0.000000	0.109453	0.281915	0.392694	0.

	session_id	delta_closure
147	2022_04_08_MND_S12	0.750000
120	2021_11_04_NES_S1	0.600281
4	2020_11_05_NES_S5	0.514286
2	2020_11_05_NES_S3	0.509091
29	2021_04_23_MND_S14	0.237037
21	2021_04_22_MND_S6	0.222737
18	2021_04_22_MND_S3	0.220000
22	2021_04_22_MND_S7	0.218391

	session_id	delta_synthesis
18	2021_04_22_MND_S3	0.123051
100	2021_10_07_CMC_S2	0.117647
19	2021_04_22_MND_S4	0.111651
88	2021_09_30_MZT_S6	0.111111
25	2021_04_23_MND_S10	0.110599
130	2021_11_05_NES_S2	0.106667
131	2021_11_05_NES_S3	0.105240
116	2021_10_08_CMC_S6	0.100000

	session_id	jsd_next_overall
152	2022_04_08_MND_S6	0.605322
147	2022_04_08_MND_S12	0.572547
88	2021_09_30_MZT_S6	0.350777
4	2020_11_05_NES_S5	0.331620
11	2020_11_06_NES_S3	0.279502
5	2020_11_05_NES_S6	0.246054
156	output-2021-05-20-ABI-S1	0.228304
3	2020_11_05_NES_S4	0.185960

	session_id	phase	Seeding	Stress-testing	Synthesis	Commitment/Closure
0	2020_11_05_NES_S1	beginning	0.696970	0.212121	0.000000	0.090909
1	2020_11_05_NES_S1	middle	0.360000	0.200000	0.120000	0.200000
2	2020_11_05_NES_S1	end	0.416667	0.166667	0.000000	0.083333
3	2020_11_05_NES_S2	beginning	0.428571	0.142857	0.000000	0.178571
4	2020_11_05_NES_S2	middle	0.615385	0.115385	0.038462	0.000000
5	2020_11_05_NES_S2	end	0.230769	0.615385	0.076923	0.000000
6	2020_11_05_NES_S3	beginning	0.454545	0.181818	0.090909	0.090909
7	2020_11_05_NES_S3	middle	0.263158	0.421053	0.052632	0.052632
8	2020_11_05_NES_S3	end	0.266667	0.133333	0.000000	0.600000

traj_df shape: (471, 6)

```
In [12]: # =====
# NEXT-STAGE ANALYSIS: STRUCTURE BEFORE PREDICTION
# =====
```



```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

# -----
# 1) AVERAGE IDEA-STATE TRAJECTORY (SANITY CHECK FIGURE)
# -----

avg_traj = (
    traj_df
    .groupby("phase")["Seeding", "Stress-testing", "Synthesis", "Commitment/Closure"]
    .mean()
    .loc[["beginning", "middle", "end"]]
)

plt.figure(figsize=(7,5))
for col in avg_traj.columns:
    plt.plot(avg_traj.index, avg_traj[col], marker="o", label=col)

plt.title("Average Idea-State Trajectory Across Sessions")
plt.ylabel("Proportion of Utterances")
plt.xlabel("Session Phase")
plt.legend()
plt.tight_layout()
plt.show()

# -----
# 2) DISTRIBUTIONS OF KEY PROGRESSION METRICS
# -----

plt.figure(figsize=(12,4))

plt.subplot(1,3,1)
plt.hist(measurement_df["delta_synthesis"], bins=20)
plt.title("Δ Synthesis (End - Beginning)")

```

```

plt.subplot(1,3,2)
plt.hist(measurement_df["delta_closure"], bins=20)
plt.title("Δ Closure (End - Beginning)")

plt.subplot(1,3,3)
plt.hist(measurement_df["jsd_next_overall"], bins=20)
plt.title("Transition Stability (JSD)")

plt.tight_layout()
plt.show()

# -----
# 3) UNSUPERVISED PROCESS CLUSTERING (SESSION TYPES)
# -----

# select numeric feature columns only
feature_cols = [
    c for c in measurement_df.columns
    if c not in ["session_id"]
]

X = measurement_df[feature_cols].values
X = StandardScaler().fit_transform(X)

kmeans = KMeans(
    n_clusters=4,      # adjust later if needed
    random_state=42,
    n_init=20
)

measurement_df["process_cluster"] = kmeans.fit_predict(X)

print("\nCluster sizes:")
print(measurement_df["process_cluster"].value_counts())

# -----

```

```
# 4) CLUSTER INTERPRETATION TABLE (THIS IS THE GOLD)
```

```
# -----
```

```
cluster_summary = (  
    measurement_df  
    .groupby("process_cluster")[feature_cols]  
    .mean()  
    .T  
)
```

```
display(cluster_summary)
```

```
# -----
```

```
# 5) OPTIONAL: MEAN TRAJECTORY PER CLUSTER (VERY NICE FIGURE)
```

```
# -----
```

```
traj_with_cluster = traj_df.merge(  
    measurement_df[["session_id", "process_cluster"]],  
    on="session_id",  
    how="left"  
)
```

```
for cl in sorted(traj_with_cluster["process_cluster"].unique()):  
    sub = traj_with_cluster[traj_with_cluster["process_cluster"] == cl]  
    avg = (  
        sub.groupby("phase")[["Seeding", "Stress-testing", "Synthesis", "Commitment/Closure"]]  
        .mean()  
        .loc[["beginning", "middle", "end"]]  
    )
```

```
plt.figure(figsize=(6,4))
```

```
for col in avg.columns:  
    plt.plot(avg.index, avg[col], marker="o", label=col)
```

```
plt.title(f"Idea-State Trajectory – Cluster {cl}")
```

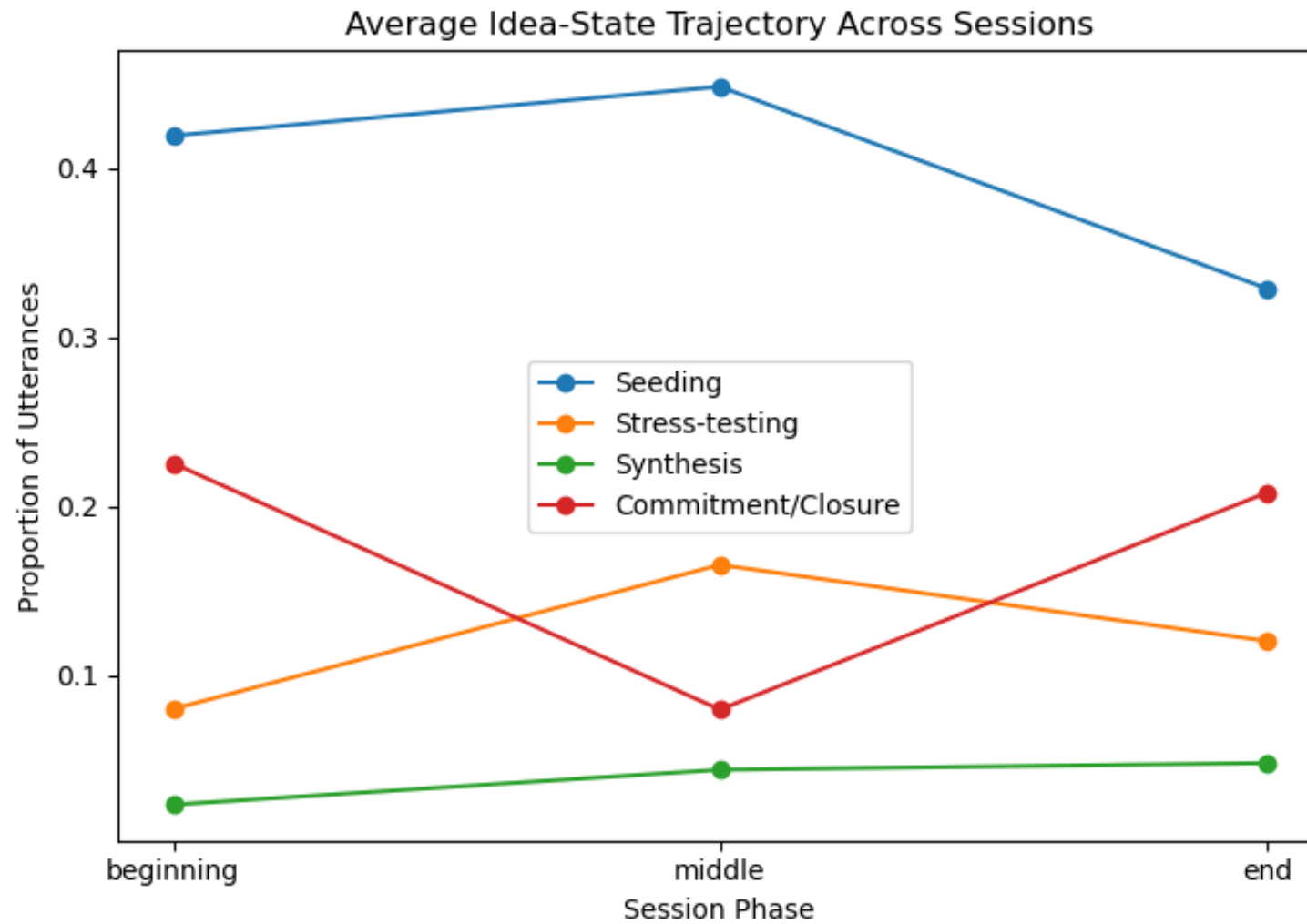
```
plt.ylabel("Proportion")
```

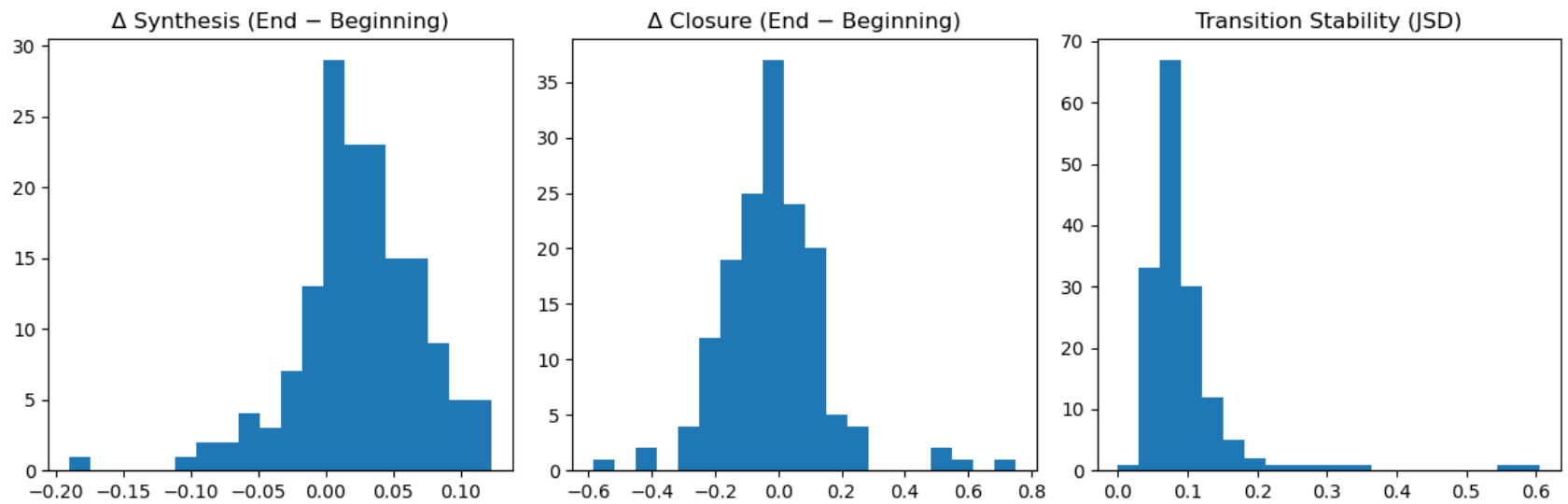
```
plt.xlabel("Phase")
```

```
plt.legend()
```

```
plt.tight_layout()
plt.show()
```

```
# =====
# END: YOU NOW HAVE STRUCTURE, NOT NOISE
# =====
```





Cluster sizes:
process_cluster

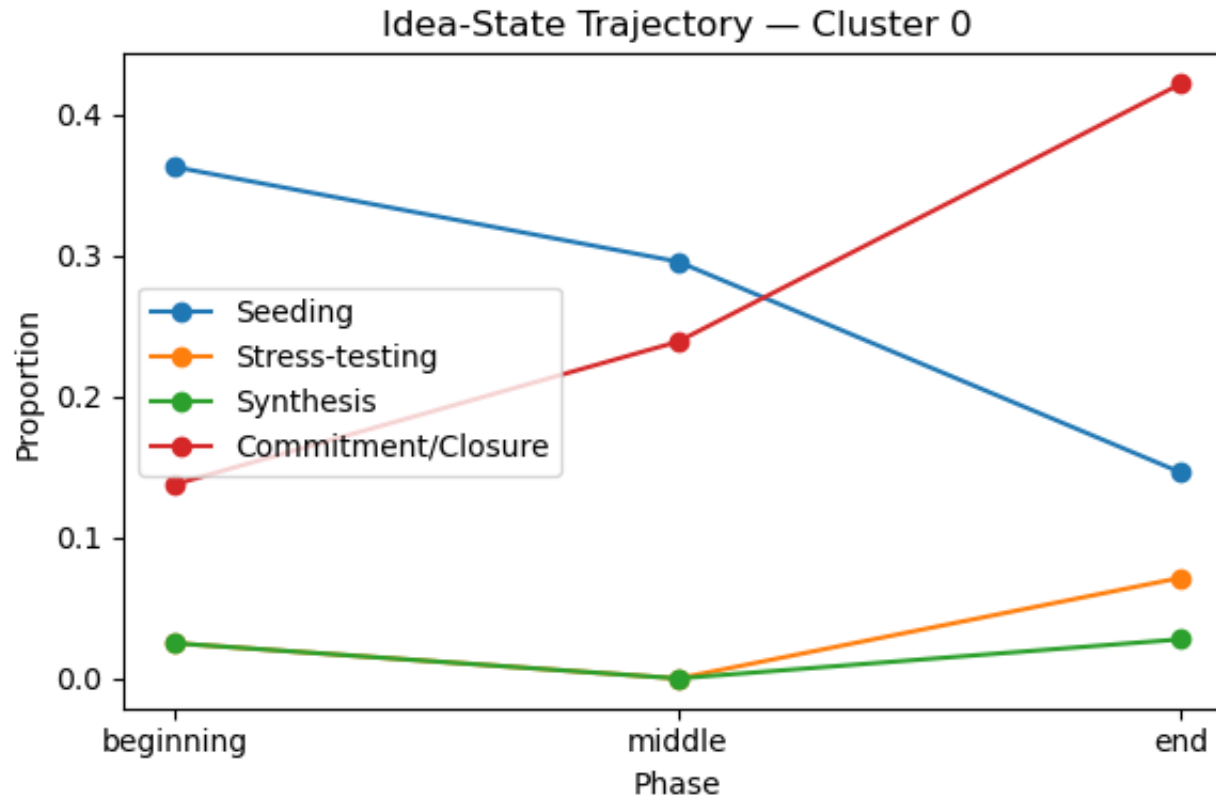
```
1    70
2    60
3    23
0     4
```

Name: count, dtype: int64

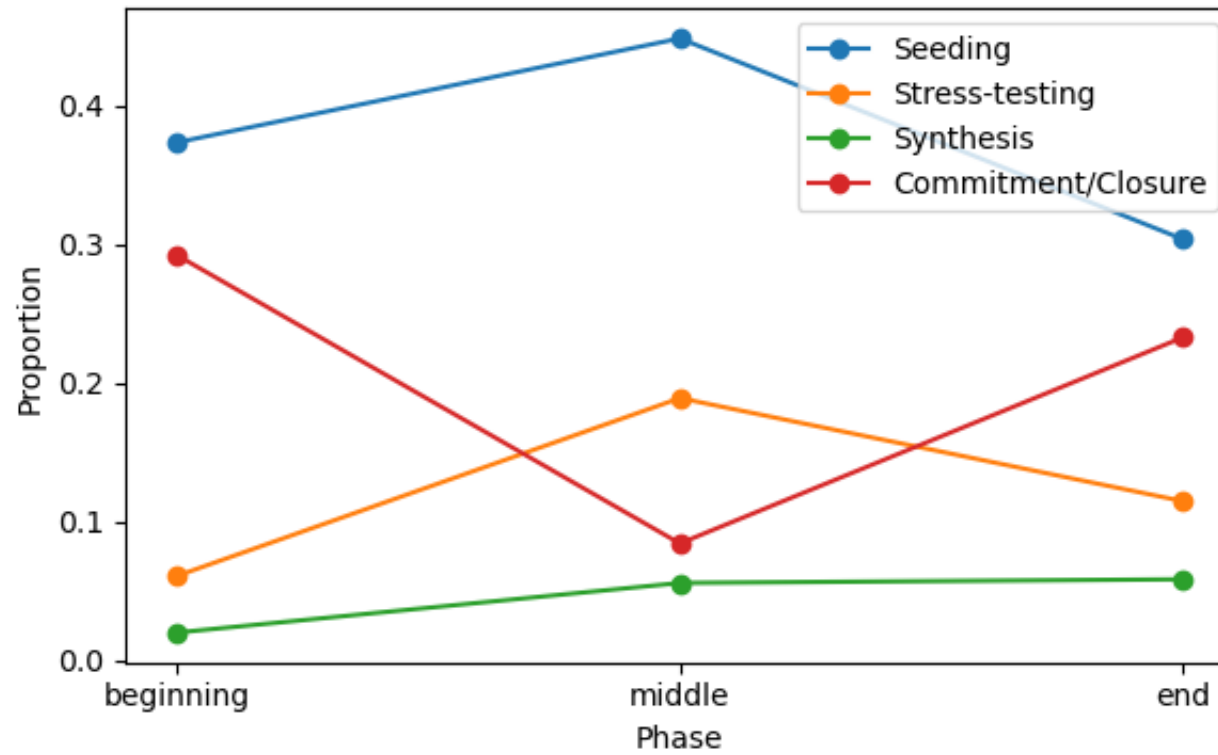
	process_cluster	0	1	2	3
beginning__Commitment/Closure		0.137500	0.292048	0.212303	0.072038
beginning__Idea Development		0.050000	0.015841	0.014744	0.029824
beginning__Other		0.000000	0.031619	0.056012	0.030551
beginning__Participation Structuring		0.075000	0.075977	0.067643	0.035700
beginning__Relational Support		0.325000	0.131110	0.144828	0.033233
beginning__Seeding		0.362500	0.373075	0.429124	0.545000
beginning__Stress-testing		0.025000	0.060529	0.061348	0.196452
beginning__Synthesis		0.025000	0.019802	0.013998	0.057203
end__Commitment/Closure		0.421627	0.232740	0.163998	0.209435

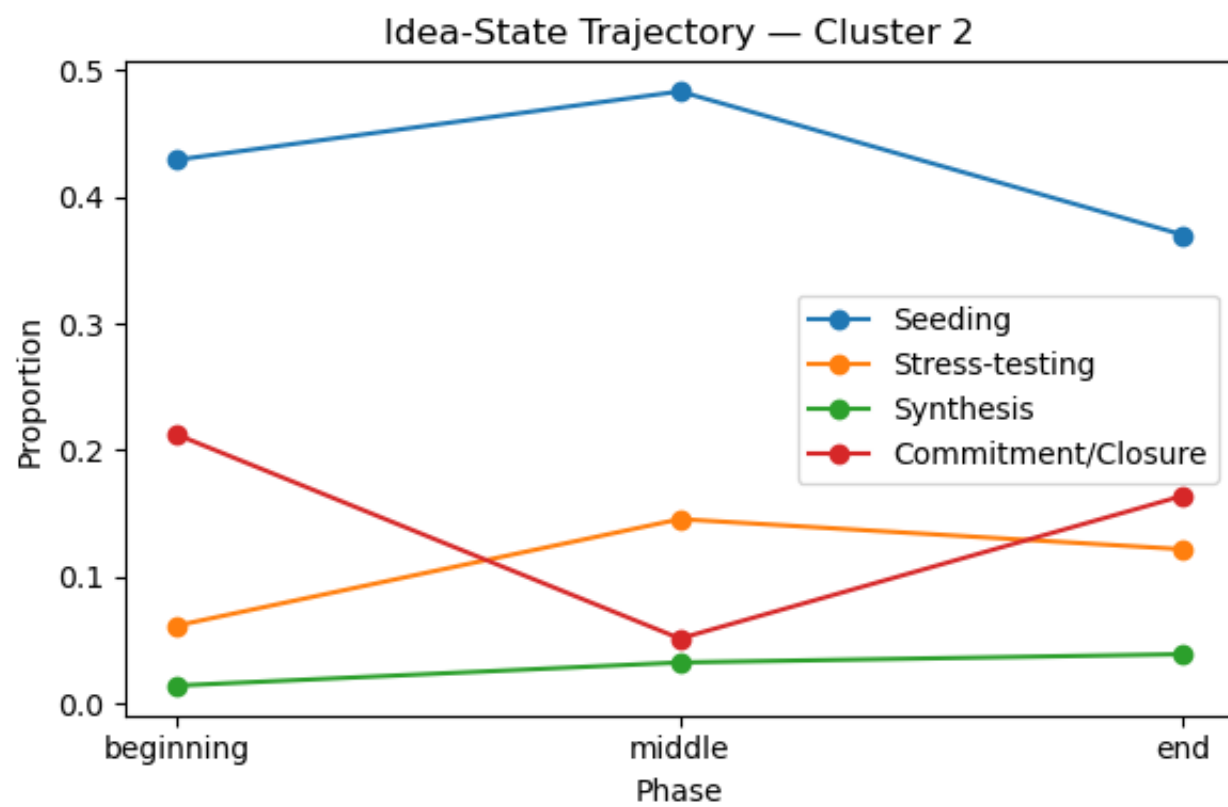
end__Idea Development	0.027778	0.042029	0.042672	0.044869
end__Other	0.055556	0.057070	0.082827	0.049202
end__Participation Structuring	0.000000	0.030658	0.029023	0.026501
end__Relational Support	0.000000	0.161499	0.151193	0.109589
end__Seeding	0.145833	0.303257	0.369797	0.332339
end__Stress-testing	0.071429	0.114505	0.121657	0.141720
end__Synthesis	0.027778	0.058242	0.038833	0.042867
middle__Commitment/Closure	0.238636	0.084074	0.051170	0.111636
middle__Idea Development	0.000000	0.034119	0.030618	0.061461
middle__Other	0.000000	0.058911	0.102308	0.058863
middle__Participation Structuring	0.000000	0.049697	0.032597	0.028968
middle__Relational Support	0.465909	0.080645	0.122411	0.090826
middle__Seeding	0.295455	0.448123	0.483164	0.385493
middle__Stress-testing	0.000000	0.188835	0.145452	0.173557
middle__Synthesis	0.000000	0.055597	0.032279	0.045717
delta_synthesis	0.002778	0.038441	0.024835	-0.014336
delta_closure	0.284127	-0.059308	-0.048305	0.137397
delta_stress_mid	-0.025000	0.128306	0.084104	-0.022895
early_commitment	0.137500	0.292048	0.212303	0.072038
early_seeding	0.362500	0.373075	0.429124	0.545000
late_closure	0.421627	0.232740	0.163998	0.209435
late_synthesis	0.027778	0.058242	0.038833	0.042867
jsd_next_overall	0.465067	0.079437	0.079414	0.136725

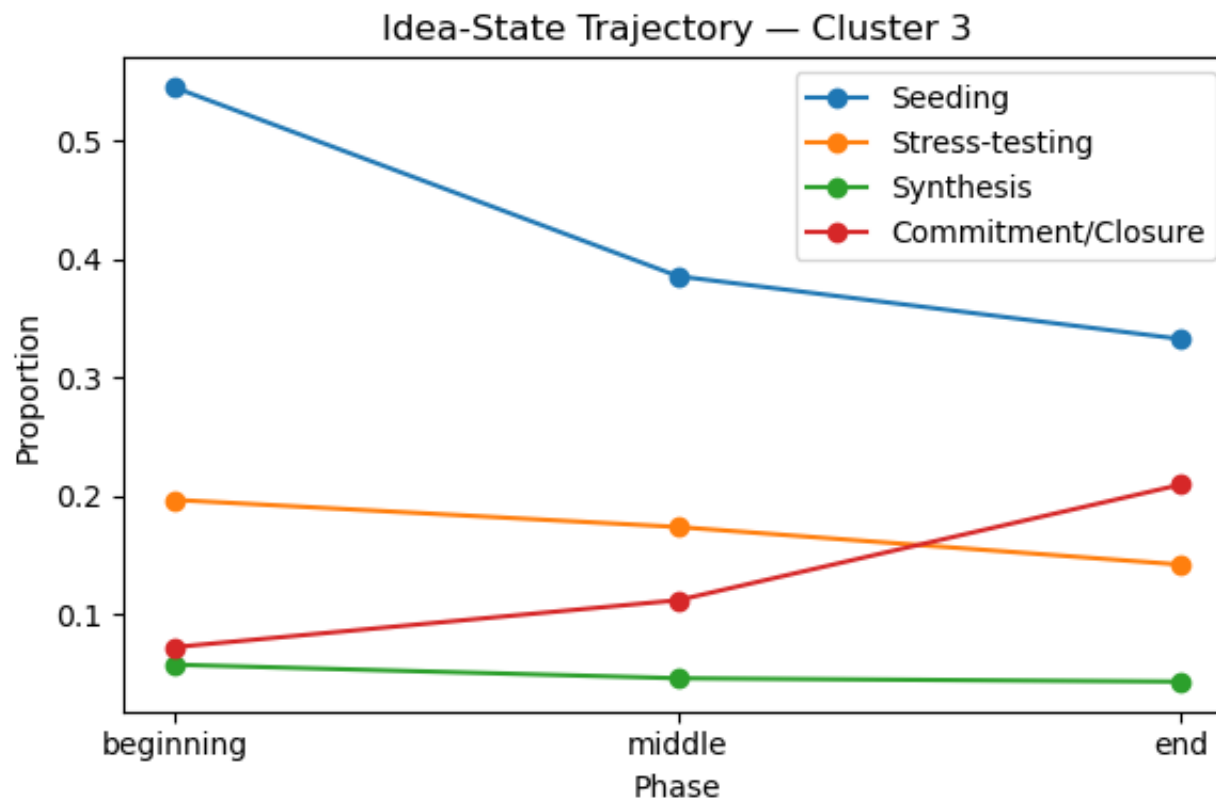
jsd_next_beginning	0.409524	0.076118	0.069638	0.164258
jsd_next_middle	0.474995	0.079648	0.088487	0.118294
jsd_next_end	0.357149	0.076194	0.073800	0.120812
prop_facilitator_utterances	0.256410	0.389948	0.163893	0.101281
prop_nonfacilitator_utterances	0.743590	0.610052	0.836107	0.898719



Idea-State Trajectory — Cluster 1







```
In [14]: from collections import Counter
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# -----
# ASSUMES YOU ALREADY HAVE:
#   df_phased with columns: session_id, idx, third, codes, idea_state, role
#   PHASES = ["beginning", "middle", "end"]
# -----

PHASES = ["beginning", "middle", "end"]
IDEA_STATES_MAIN = ["Seeding", "Stress-testing", "Synthesis", "Commitment/Closure"] # what you want
```

```

# -----
# Jensen-Shannon (stable)
# -----
def jensen_shannon_from_counters(c1: Counter, c2: Counter, eps=1e-12):
    keys = set(c1.keys()) | set(c2.keys())
    P = np.array([c1.get(k, 0.0) for k in keys], dtype=float) + eps
    Q = np.array([c2.get(k, 0.0) for k in keys], dtype=float) + eps
    P /= P.sum()
    Q /= Q.sum()
    M = 0.5 * (P + Q)

    def kl(a, b):
        return float(np.sum(a * np.log(a / b)))

    return 0.5 * kl(P, M) + 0.5 * kl(Q, M)

# -----
# A) Idea-state flow (session × phase proportions)
# -----
def build_idea_state_flow(df_in):
    rows = []
    for sid, g in df_in.groupby("session_id"):
        row = {"session_id": sid}
        for ph in PHASES:
            gph = g[g["third"] == ph]
            total = len(gph)
            if total == 0:
                continue
            vc = gph["idea_state"].value_counts(normalize=True)
            for state, pct in vc.items():
                row[f"{ph}__{state}"] = float(pct)
        rows.append(row)
    return pd.DataFrame(rows).fillna(0.0)

idea_flow_df = build_idea_state_flow(df_phased)

# -----
# B) Progression metrics (human-readable deltas)

```

```

# -----
def add_progression_features(flow_df):
    df = flow_df.copy()

    # convenience accessors (missing columns -> 0)
    def col(ph, state):
        c = f"{ph}__{state}"
        if c not in df.columns:
            df[c] = 0.0
        return c

    # core deltas
    df["delta_synthesis"] = df[col("end", "Synthesis")] - df[col("beginning", "Synthesis")]
    df["delta_closure"] = df[col("end", "Commitment/Closure")] - df[col("beginning", "Commitment/Closure")]
    df["delta_stress_mid"] = df[col("middle", "Stress-testing")] - df[col("beginning", "Stress-testing")]

    # readable "levels"
    df["early_commitment"] = df[col("beginning", "Commitment/Closure")]
    df["early_seeding"] = df[col("beginning", "Seeding")]
    df["late_closure"] = df[col("end", "Commitment/Closure")]
    df["late_synthesis"] = df[col("end", "Synthesis")]

    return df

idea_flow_df = add_progression_features(idea_flow_df)

# -----
# C) Transition stability (JSD) per session (overall + by phase)
# We build next-step distributions (multi-label aware) and compare each session
# to the pooled distribution.
# -----
def session_next_counter(g_session: pd.DataFrame, phase: str | None = None) -> Counter:
    g = g_session.sort_values(["idx"]).reset_index(drop=True)
    ctr = Counter()
    for i in range(len(g) - 1):
        if phase is not None and g.at[i, "third"] != phase:
            continue
        A = g.at[i, "codes"] or []
        B = g.at[i+1, "codes"] or []

```

```

    if not A or not B:
        continue
    for a in A:
        for b in B:
            if a == b:
                continue
            ctr[(a, b)] += 1
    return ctr

def build_jsd_features(df_in):
    # pooled distributions (overall + per phase)
    pooled_overall = Counter()
    pooled_by_phase = {ph: Counter() for ph in PHASES}

    grouped = {sid: g for sid, g in df_in.groupby("session_id")}
    for sid, g in grouped.items():
        c_all = session_next_counter(g, phase=None)
        pooled_overall.update(c_all)
        for ph in PHASES:
            pooled_by_phase[ph].update(session_next_counter(g, phase=ph))

    rows = []
    for sid, g in grouped.items():
        c_all = session_next_counter(g, phase=None)
        row = {"session_id": sid}

        row["jsd_next_overall"] = jensen_shannon_from_counters(pooled_overall, c_all) if sum(c_all.values()) > 0 else 0

        for ph in PHASES:
            c_ph = session_next_counter(g, phase=ph)
            pooled_ph = pooled_by_phase[ph]
            row[f"jsd_next_{ph}"] = jensen_shannon_from_counters(pooled_ph, c_ph) if sum(c_ph.values()) > 0 else 0

        rows.append(row)

    out = pd.DataFrame(rows)
    # if a session has no transitions in a phase, fill with overall (or 0). choose one:
    for ph in PHASES:
        out[f"jsd_next_{ph}"] = out[f"jsd_next_{ph}"].fillna(out["jsd_next_overall"])

```

```

    out["jsd_next_overall"] = out["jsd_next_overall"].fillna(0.0)
    return out

jsd_df = build_jsd_features(df_phased)

# -----
# D) Role proportions per session
# -----
role_props = (
    df_phased
    .groupby(["session_id", "role"])
    .size()
    .unstack(fill_value=0)
    .reset_index()
)
for col in ["facilitator", "non-facilitator"]:
    if col not in role_props.columns:
        role_props[col] = 0
role_props["total_utterances"] = role_props["facilitator"] + role_props["non-facilitator"]
role_props["prop_facilitator_utterances"] = np.where(role_props["total_utterances"] > 0, role_props["facilitator"] / role_props["total_utterances"], 0)
role_props["prop_nonfacilitator_utterances"] = np.where(role_props["total_utterances"] > 0, role_props["non-facilitator"] / role_props["total_utterances"], 0)
role_props = role_props[["session_id", "prop_facilitator_utterances", "prop_nonfacilitator_utterances"]]

# -----
# E) Measurement DF (the "clean" table you actually model later)
# -----
measurement_df = (
    idea_flow_df
    .merge(jsd_df, on="session_id", how="left")
    .merge(role_props, on="session_id", how="left")
    .fillna(0.0)
)

print(f"measurement_df shape: {measurement_df.shape}")
print(f"Unique sessions: {measurement_df['session_id'].nunique()}")
print(f"Duplicate session_id rows: {measurement_df.duplicated('session_id').sum()}")
print(f"Columns with NA: {measurement_df.isna().sum().sum()}")

# -----

```

```

# F) Trajectory DF (long format) for plotting
# -----
def build_traj_df(meas_df, states=IDEA_STATES_MAIN):
    rows = []
    for _, r in meas_df.iterrows():
        sid = r["session_id"]
        for ph in PHASES:
            row = {"session_id": sid, "phase": ph}
            for st in states:
                col = f"{ph}_{st}"
                row[st] = float(r[col]) if col in meas_df.columns else 0.0
            rows.append(row)
    return pd.DataFrame(rows)

traj_df = build_traj_df(measurement_df, states=IDEA_STATES_MAIN)
print(f"traj_df shape: {traj_df.shape}")

# -----
# PLOTS (with labels + units)
# -----

# 1) Average trajectory across sessions
avg_traj = traj_df.groupby("phase")[IDEA_STATES_MAIN].mean().reindex(PHASES)

plt.figure(figsize=(8,5))
for st in IDEA_STATES_MAIN:
    plt.plot(PHASES, avg_traj[st].values, marker="o", label=st)
plt.title("Average Idea-State Trajectory Across Sessions")
plt.xlabel("Session Phase")
plt.ylabel("Proportion of Utterances (0-1)")
plt.ylim(0, 1)
plt.grid(True, alpha=0.3)
plt.legend(title="Idea State")
plt.tight_layout()
plt.show()

# 2) Histograms: delta synthesis, delta closure
plt.figure(figsize=(12,4))

```

```

plt.subplot(1,3,1)
plt.hist(measurement_df["delta_synthesis"].values, bins=20)
plt.title("Δ Synthesis (End – Beginning)")
plt.xlabel("Change in Proportion (0–1 units)")
plt.ylabel("Number of Sessions")

plt.subplot(1,3,2)
plt.hist(measurement_df["delta_closure"].values, bins=20)
plt.title("Δ Commitment/Closure (End – Beginning)")
plt.xlabel("Change in Proportion (0–1 units)")
plt.ylabel("Number of Sessions")

plt.subplot(1,3,3)
plt.hist(measurement_df["jsd_next_overall"].values, bins=20)
plt.title("Transition Stability (JSD)")
plt.xlabel("Mean Jensen–Shannon Divergence (0 = stable)")
plt.ylabel("Number of Sessions")

plt.tight_layout()
plt.show()

# -----
# CLUSTERING (this is what fixes your KeyError: we create 'cluster' AND merge it)
# -----

# Build session-level vector: concatenated (phase × state) proportions for the main states
cluster_cols = []
for ph in PHASES:
    for st in IDEA_STATES_MAIN:
        c = f"{ph}__{st}"
        if c not in measurement_df.columns:
            measurement_df[c] = 0.0
            cluster_cols.append(c)

X = measurement_df[cluster_cols].values
X = StandardScaler().fit_transform(X)

K = 4 # pick 3–6; 4 is a sane default for “types of sessions”
km = KMeans(n_clusters=K, n_init=25, random_state=42)

```



```

measurement_df["cluster"] = km.fit_predict(X)

# IMPORTANT: merge cluster labels into traj_df so traj_df has the 'cluster' column
traj_df = traj_df.merge(measurement_df[["session_id", "cluster"]], on="session_id", how="left")

# Clustered trajectories plot
for k in sorted(traj_df["cluster"].dropna().unique()):
    g = traj_df[traj_df["cluster"] == k]
    avg = g.groupby("phase")[IDEA_STATES_MAIN].mean().reindex(PHASES)

    plt.figure(figsize=(7,4))
    for st in IDEA_STATES_MAIN:
        plt.plot(PHASES, avg[st].values, marker="o", label=st)
    plt.title(f"Idea-State Trajectory – Cluster {int(k)}")
    plt.xlabel("Session Phase")
    plt.ylabel("Proportion of Utterances (0–1)")
    plt.ylim(0, 1)
    plt.grid(True, alpha=0.3)
    plt.legend(title="Idea State")
    plt.tight_layout()
    plt.show()

print("Done.")
print("You now have:")
print("  – measurement_df: one row per session with flow + deltas + JSD + role props + cluster")
print("  – traj_df: long format for plotting (includes cluster)")

```

measurement_df shape: (157, 38)

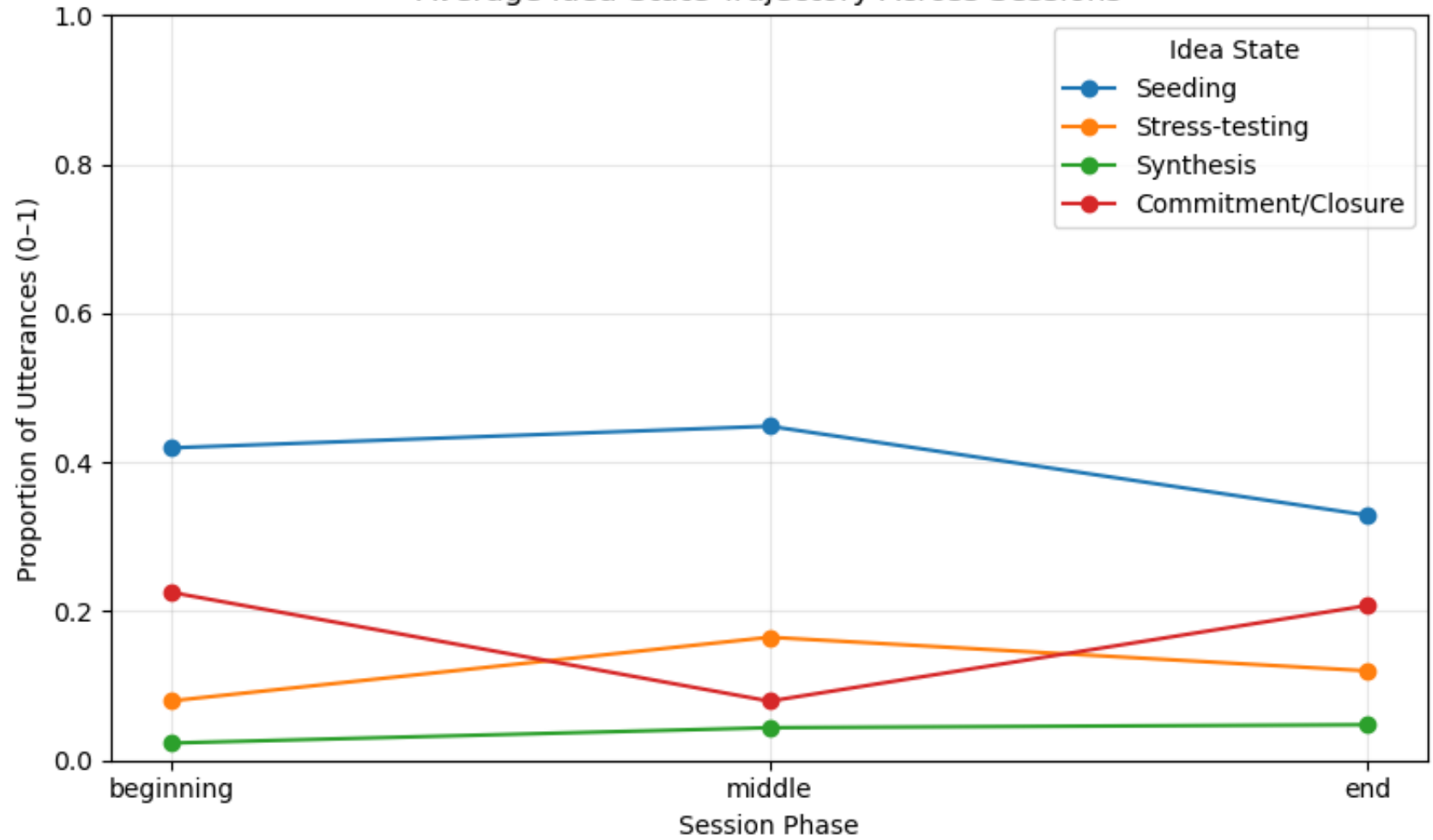
Unique sessions: 157

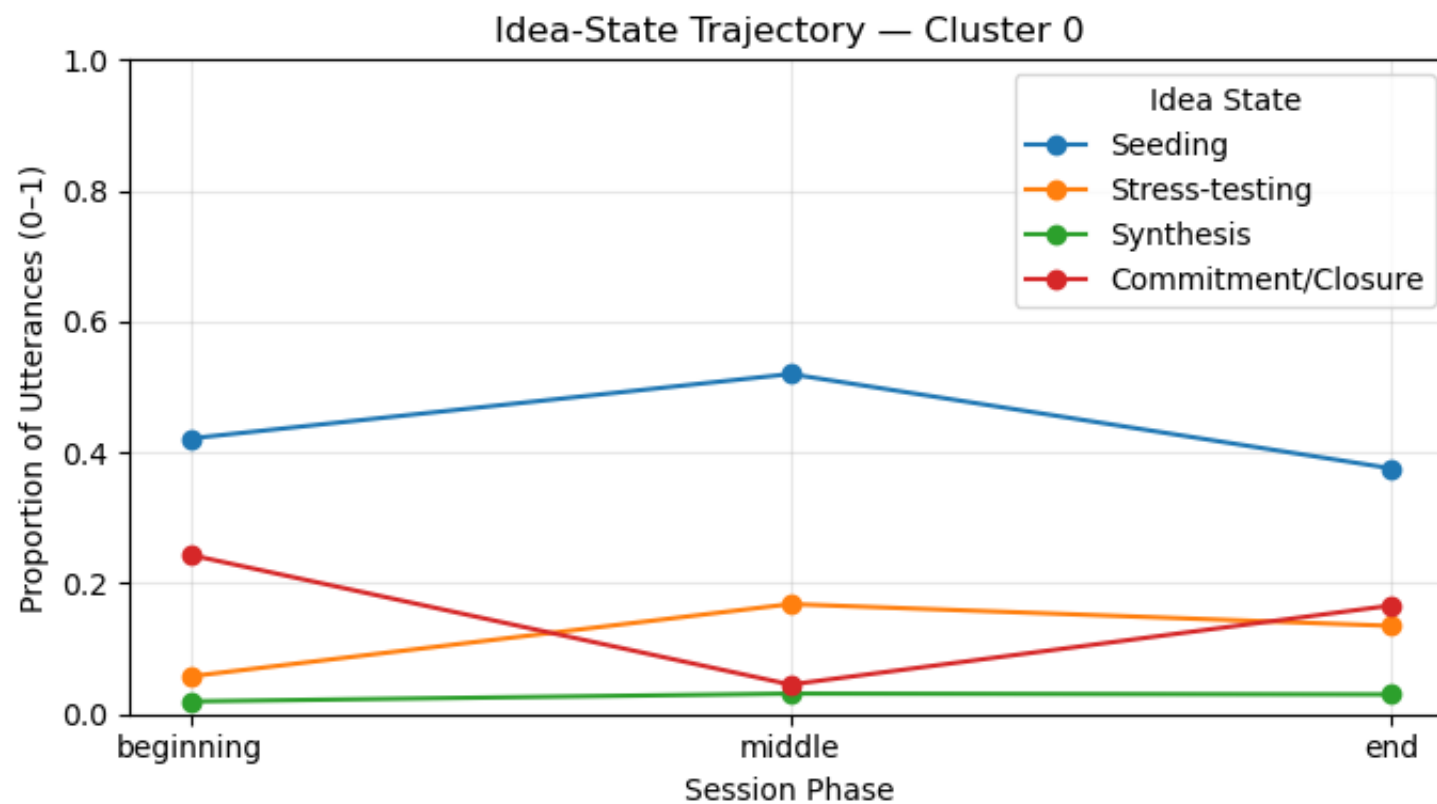
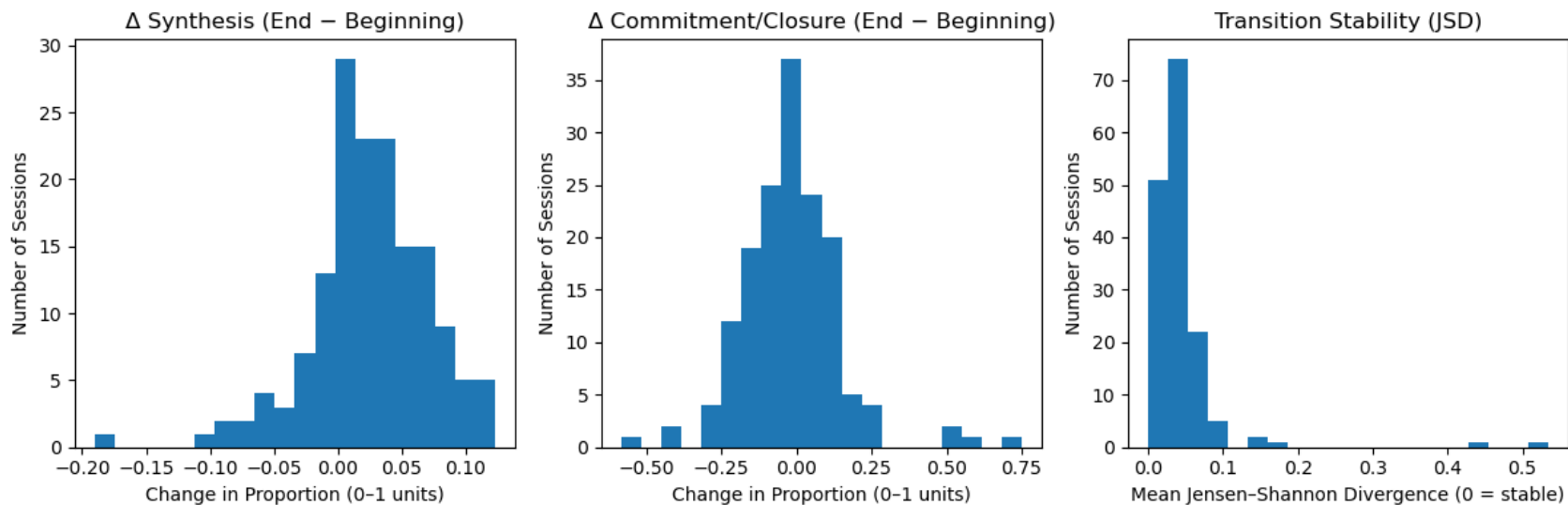
Duplicate session_id rows: 0

Columns with NA: 0

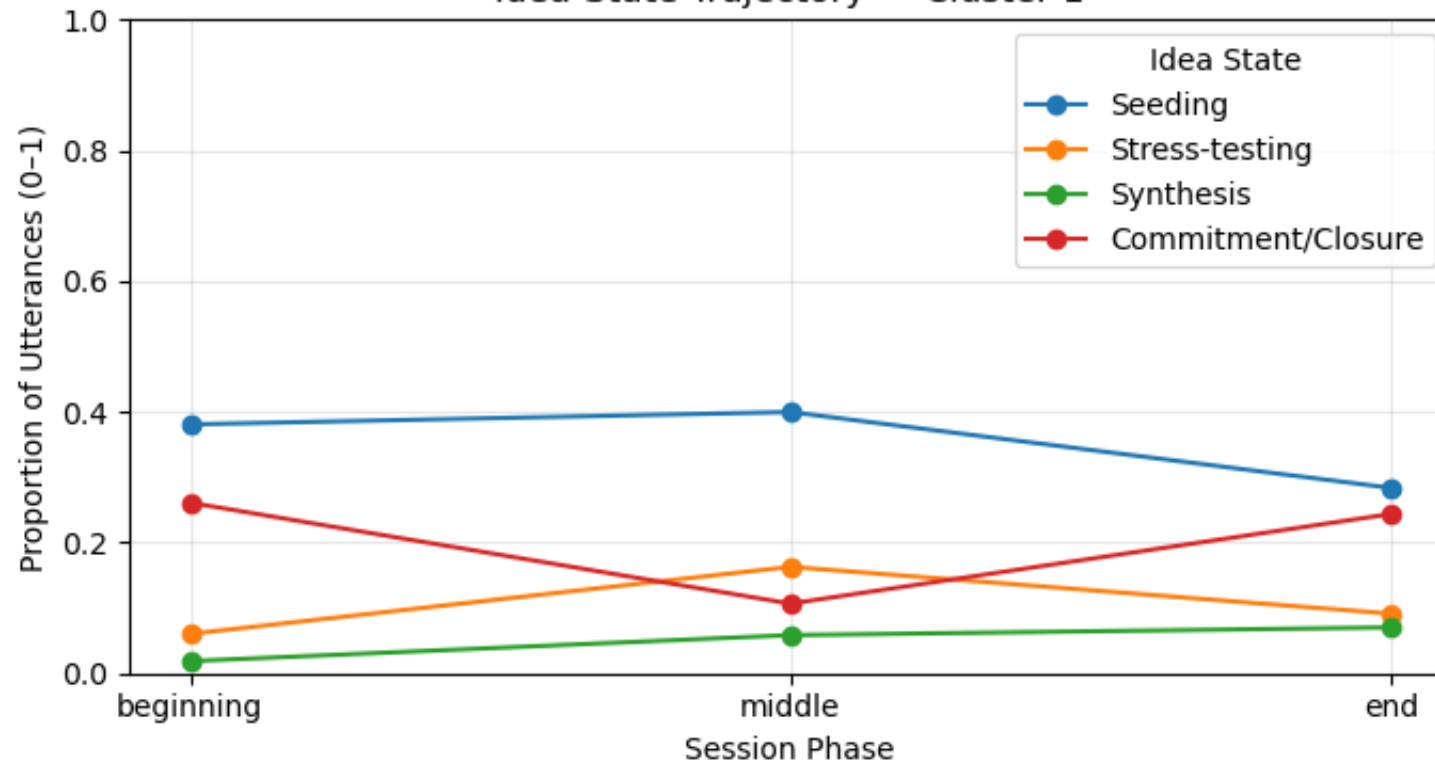
traj_df shape: (471, 6)

Average Idea-State Trajectory Across Sessions

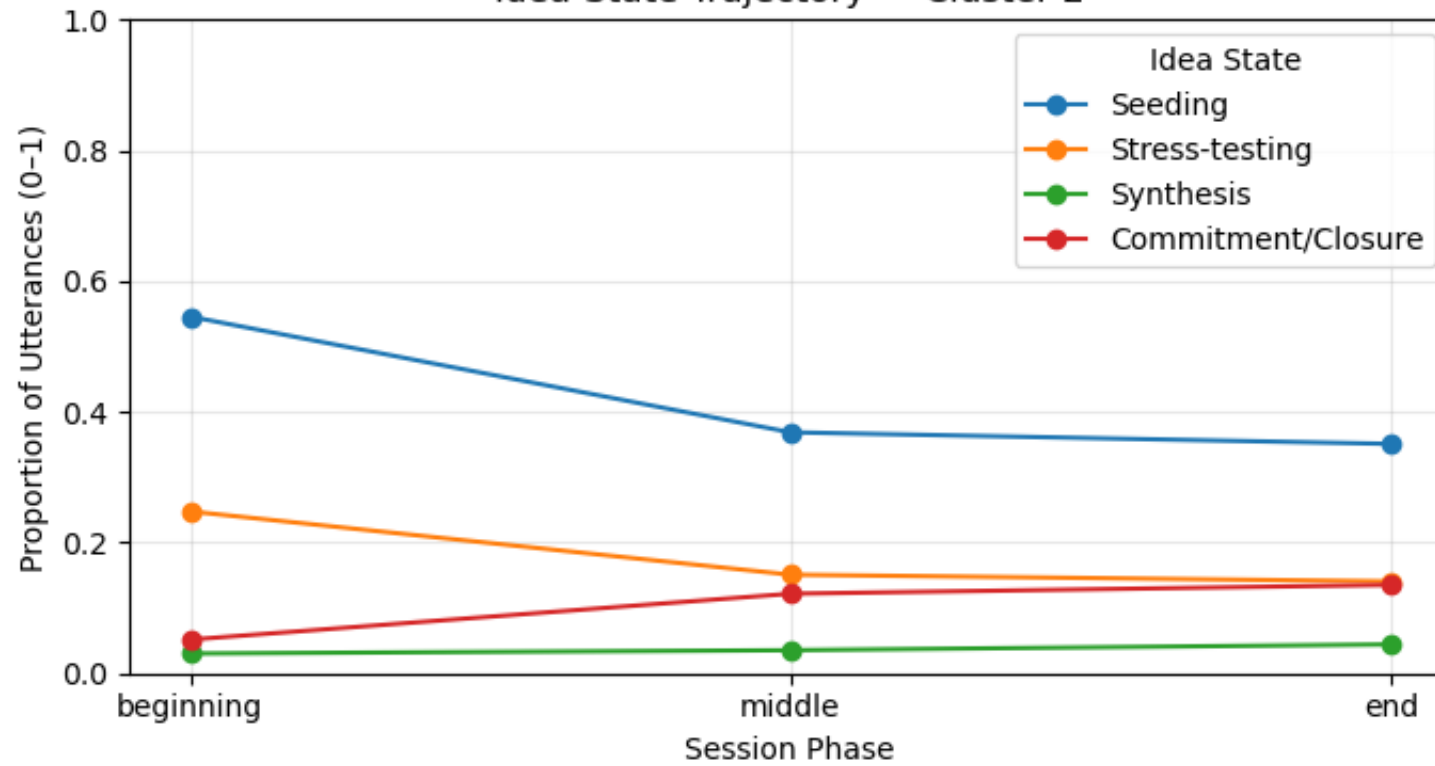


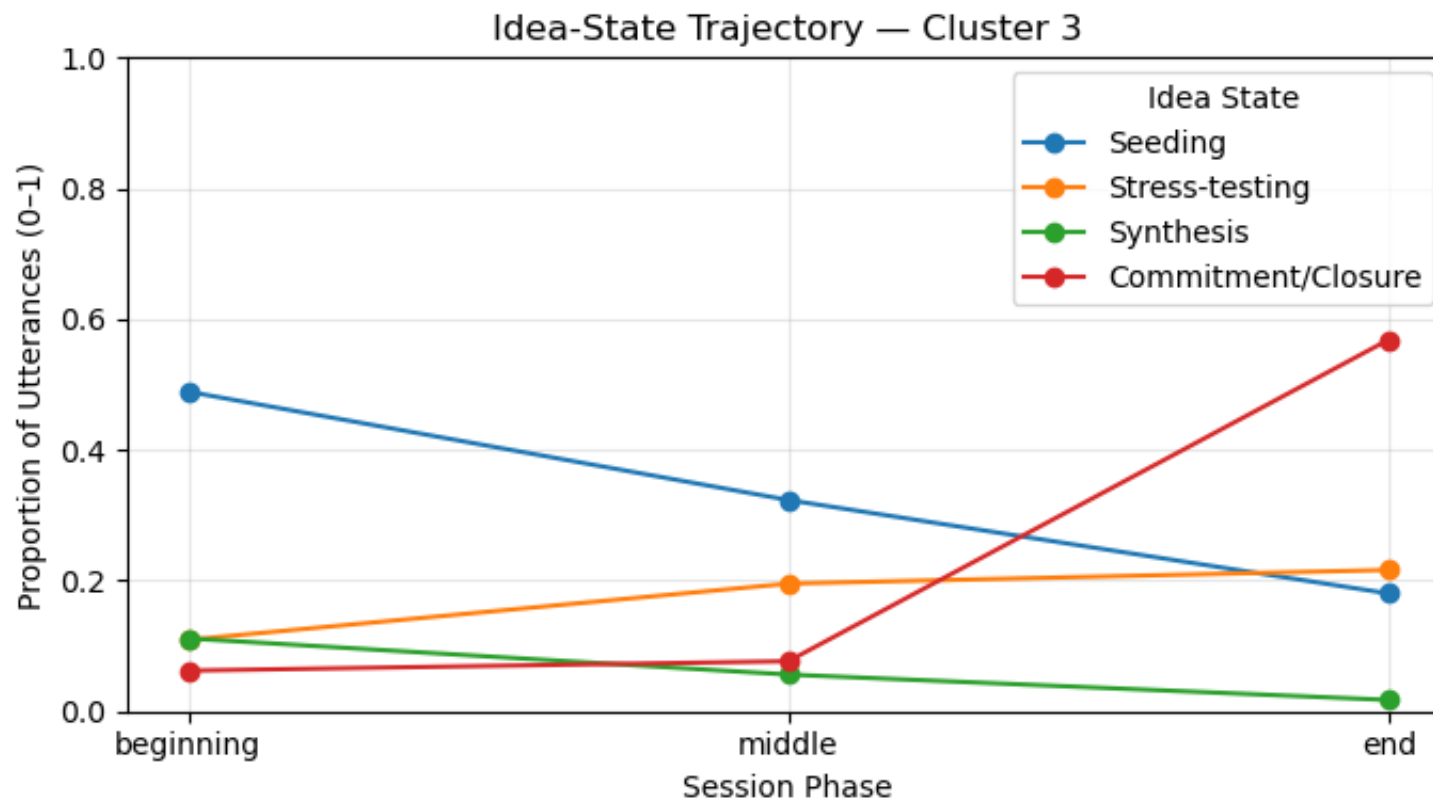


Idea-State Trajectory — Cluster 1



Idea-State Trajectory — Cluster 2





Done.

You now have:

- measurement_df: one row per session with flow + deltas + JSD + role props + cluster
- traj_df: long format for plotting (includes cluster)

```
In [15]: import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler

# -----
# CLUSTER PROFILING
# -----

# 1) Cluster sizes
cluster_sizes = (
    measurement_df.groupby("cluster")["session_id"]
```

```

        .nunique()
        .sort_index()
        .rename("n_sessions")
        .reset_index()
    )
display(cluster_sizes)

# 2) Choose a compact set of interpretable features to describe clusters
PROFILE_FEATURES = [
    # flow levels (early + late)
    "early_seeding", "early_commitment", "late_synthesis", "late_closure",
    # progression
    "delta_synthesis", "delta_closure", "delta_stress_mid",
    # stability
    "jsd_next_overall", "jsd_next_beginning", "jsd_next_middle", "jsd_next_end",
    # roles
    "prop_facilitator_utterances"
]

# keep only columns that exist (avoid KeyError if one is missing)
PROFILE_FEATURES = [c for c in PROFILE_FEATURES if c in measurement_df.columns]

cluster_means = (
    measurement_df.groupby("cluster")[PROFILE_FEATURES]
    .mean()
    .sort_index()
)
display(cluster_means)

# 3) Cluster "signature" features (z-scored vs global mean)
global_mean = measurement_df[PROFILE_FEATURES].mean()
global_std = measurement_df[PROFILE_FEATURES].std(ddof=0).replace(0, np.nan)

cluster_signature_z = (cluster_means - global_mean) / global_std
display(cluster_signature_z)

# show top +/- features per cluster
TOPN = 5
sig_rows = []

```

```

for k in sorted(measurement_df["cluster"].unique()):
    z = cluster_signature_z.loc[k].dropna().sort_values()
    low = z.head(TOPN)
    high = z.tail(TOPN)[::-1]
    sig_rows.append({
        "cluster": int(k),
        "n_sessions": int(cluster_sizes.loc[cluster_sizes["cluster"]==k, "n_sessions"].iloc[0]),
        "most_low_features": ", ".join([f"{idx} ({val:+.2f})" for idx, val in low.items()]),
        "most_high_features": ", ".join([f"{idx} ({val:+.2f})" for idx, val in high.items()]),
    })

cluster_signatures = pd.DataFrame(sig_rows).sort_values("cluster")
display(cluster_signatures)

# -----
# REPRESENTATIVE SESSIONS PER CLUSTER
# (closest to centroid in the clustering feature space)
# -----
PHASES = ["beginning", "middle", "end"]
IDEA_STATES_MAIN = ["Seeding", "Stress-testing", "Synthesis", "Commitment/Closure"]

cluster_cols = [f"{ph}__{st}" for ph in PHASES for st in IDEA_STATES_MAIN]
cluster_cols = [c for c in cluster_cols if c in measurement_df.columns]

X = measurement_df[cluster_cols].values
scaler = StandardScaler()
Xz = scaler.fit_transform(X)

# compute centroids in z-space
centroids = (
    pd.DataFrame(Xz, index=measurement_df.index)
    .groupby(measurement_df["cluster"])
    .mean()
)

# distance to centroid
rep_rows = []
for k in sorted(measurement_df["cluster"].unique()):
    idxs = measurement_df.index[measurement_df["cluster"] == k].tolist()

```



```

if not idxs:
    continue
C = centroids.loc[k].values
dists = []
for ix in idxs:
    d = float(np.linalg.norm(Xz[ix] - C))
    dists.append((ix, d))
dists.sort(key=lambda t: t[1])
# take top N closest
N_REP = 3
for rank, (ix, d) in enumerate(dists[:N_REP], start=1):
    rep_rows.append({
        "cluster": int(k),
        "rank_in_cluster": rank,
        "session_id": measurement_df.loc[ix, "session_id"],
        "dist_to_centroid": d,
        "early_seeding": float(measurement_df.loc[ix, "early_seeding"]) if "early_seeding" in measurement_df.columns else None,
        "late_closure": float(measurement_df.loc[ix, "late_closure"]) if "late_closure" in measurement_df.columns else None,
        "delta_synthesis": float(measurement_df.loc[ix, "delta_synthesis"]) if "delta_synthesis" in measurement_df.columns else None,
        "jsd_next_overall": float(measurement_df.loc[ix, "jsd_next_overall"]) if "jsd_next_overall" in measurement_df.columns else None
    })

representative_sessions = pd.DataFrame(rep_rows).sort_values(["cluster", "rank_in_cluster"])
display(representative_sessions)

print("Next: use representative_sessions.session_id to pull transcript windows / examples for each")

```

	cluster	n_sessions
0	0	71
1	1	65
2	2	16
3	3	5

	early_seeding	early_commitment	late_synthesis	late_closure	delta_synthesis	delta_closure	delta_stress
cluster							
0	0.421317	0.243428	0.030205	0.165990	0.010673	-0.077439	0.000000
1	0.380868	0.261049	0.070224	0.243717	0.051576	-0.017333	0.100000
2	0.545777	0.051592	0.044235	0.135341	0.013953	0.083749	-0.000000
3	0.489072	0.062263	0.017423	0.568574	-0.094228	0.506311	0.000000

	early_seeding	early_commitment	late_synthesis	late_closure	delta_synthesis	delta_closure	delta_stress
cluster							
0	0.015531	0.150557	-0.505929	-0.358401	-0.299107	-0.364994	0.200000
1	-0.314306	0.297746	0.645035	0.306808	0.579312	0.001242	0.100000
2	1.030438	-1.451844	-0.102408	-0.620698	-0.228668	0.617148	-1.000000
3	0.568041	-1.362707	-0.873560	3.087019	-2.551993	3.191886	0.000000

cluster	n_sessions	most_low_features	most_high_features
0	0	71 late_synthesis (-0.51), delta_closure (-0.36),...	delta_stress_mid (+0.22), early_commitment (+0.00),...
1	1	65 early_seeding (-0.31), jsd_next_beginning (-0.00),...	late_synthesis (+0.65), delta_synthesis (+0.58),...
2	2	16 delta_stress_mid (-1.63), early_commitment (-1.00),...	early_seeding (+1.03), jsd_next_beginning (+0.00),...
3	3	5 delta_synthesis (-2.55), early_commitment (-1.00),...	delta_closure (+3.19), late_closure (+3.09), j...

	cluster	rank_in_cluster	session_id	dist_to_centroid	early_seeding	late_closure	delta_synthesis
0	0	1	2022_04_07_MND_S2	1.380992	0.449541	0.175926	0.009599
1	0	2	2021_10_08_CMC_S2	1.458326	0.404255	0.159574	0.021277
2	0	3	2021_11_05_NES_S9	1.506145	0.452055	0.212389	-0.004849
3	1	1	2021_10_01_MZT_S5	1.270289	0.384615	0.240000	0.066667
4	1	2	2021_11_05_NES_S10	1.347780	0.400000	0.205357	0.051429
5	1	3	2021_06_11_SLU_S7	1.362299	0.414286	0.277108	0.043718
6	2	1	2021_04_22_MND_S8	1.269729	0.454545	0.155172	0.021421
7	2	2	2021_09_30_MZT_S2	2.682963	0.378378	0.055556	0.018519
8	2	3	2021_11_04_NES_S2	2.906673	0.641026	0.189474	0.031579
9	3	1	2020_11_05_NES_S3	3.246542	0.454545	0.600000	-0.090909
10	3	2	2020_11_06_NES_S1	5.528188	0.714286	0.157895	-0.090226
11	3	3	2020_11_05_NES_S5	5.664494	0.200000	0.714286	-0.100000

Next: use `representative_sessions.session_id` to pull transcript windows / examples for each cluster.

```
In [17]: import os, json, re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

# =====
```

```

# 0) LOCATE + LOAD *ALL* OUTCOME JSONs (SCIALOG-wide)
# =====
# Update this root if yours differs.
DATA_ROOTS = [
    "/Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis"
    # fallback for this environment if you want to test with the single attached file:
    "/mnt/data",
]

def find_outcome_jsons(roots):
    paths = []
    for root in roots:
        if not os.path.exists(root):
            continue
        for dirpath, _, filenames in os.walk(root):
            for fn in filenames:
                if fn.endswith("_outcome.json"):
                    paths.append(os.path.join(dirpath, fn))
    return sorted(paths)

outcome_paths = find_outcome_jsons(DATA_ROOTS)
if not outcome_paths:
    raise FileNotFoundError(
        "No *_outcome.json files found under:\n" + "\n".join(DATA_ROOTS)
    )

print(f"Found {len(outcome_paths)} outcome JSON files.")
for p in outcome_paths[:5]:
    print(" -", p)
if len(outcome_paths) > 5:
    print(" ...")

def parse_year_track_from_path(path: str):
    """
    Example path includes ".../data/2021MND/2021MND_outcome.json"
    We extract year=2021 and track=MND from the folder/file prefix.
    """
    base = os.path.basename(path)

```

```

# file like 2021MND_outcome.json
m = re.match(r"(\d{4})([A-Z]{3})_outcome\.json$", base)
if m:
    return int(m.group(1)), m.group(2)

# fallback: look in parent folder name like 2021MND
folder = os.path.basename(os.path.dirname(path))
m2 = re.match(r"(\d{4})([A-Z]{3})$", folder)
if m2:
    return int(m2.group(1)), m2.group(2)

return None, None

def load_all_outcomes(outcome_paths):
    rows = []
    for path in outcome_paths:
        year, track = parse_year_track_from_path(path)
        with open(path, "r") as f:
            data = json.load(f)

            if not isinstance(data, list):
                raise ValueError(f"Expected list in {path}, got {type(data)}")

            df = pd.DataFrame(data)
            if df.empty:
                continue

            if "team_id" not in df.columns or "funded_status" not in df.columns:
                raise ValueError(
                    f"{path} missing required keys. Columns: {df.columns.tolist()}"
                )

            df = df.copy()
            df["funded_status"] = df["funded_status"].astype(int)
            df["source_path"] = path
            df["year"] = year
            df["track"] = track

```

```

        # Extract numeric team number from team_id (e.g., "MND2" -> 2)
        df["team_num"] = df["team_id"].astype(str).str.extract(r"(\d+)").astype(float)
        rows.append(df)

    out = pd.concat(rows, ignore_index=True) if rows else pd.DataFrame()
    if out.empty:
        raise ValueError("Loaded outcome files but got empty teams_df.")
    if out["team_num"].isna().any():
        # you can relax this if there are weird team_id formats
        print("Warning: some team_id values did not yield a numeric team_num.")
    out["team_num"] = out["team_num"].fillna(-1).astype(int)

    # Keep only what we need (plus traceability)
    keep_cols = ["year", "track", "team_id", "team_num", "funded_status", "members", "source_path"]
    keep_cols = [c for c in keep_cols if c in out.columns]
    return out[keep_cols].drop_duplicates(subset=["year", "track", "team_id"])

teams_df = load_all_outcomes(outcome_paths)
print("teams_df shape:", teams_df.shape)
display(teams_df.head())

# =====
# 1) BUILD outcomes_df FOR ALL sessions in measurement_df
# =====
if "measurement_df" not in globals():
    raise NameError("measurement_df not found. Run the measurement-building chunk first.")

def parse_session_id(session_id: str):
    """
    Expected: YYYY_MM_DD_<TRACK>_S<NUM>
    e.g., 2021_04_22_MND_S2 -> year=2021, track=MND, team_num=2
    """
    s = str(session_id)
    m = re.match(r"(?P<year>\d{4})_\d{2}_\d{2}_(?P<track>[A-Z]{3})_S(?P<num>\d+)$", s)
    if not m:
        return None, None, None
    return int(m.group("year")), m.group("track"), int(m.group("num"))

```

```

outcomes_df = measurement_df[["session_id"]].copy()
parsed = outcomes_df["session_id"].apply(parse_session_id)
outcomes_df[["year", "track", "team_num"]] = pd.DataFrame(parsed.tolist(), index=outcomes_df.index)

bad = outcomes_df["year"].isna().sum()
if bad:
    # don't hard crash, but make it obvious what's wrong
    print(f"Warning: {bad} session_id values did not match the expected pattern.")
    display(outcomes_df[outcomes_df["year"].isna()].head())

# Left-join to outcomes by (year, track, team_num)
# This avoids relying on team_id string formatting beyond the number.
outcomes_df = outcomes_df.merge(
    teams_df[["year", "track", "team_num", "funded_status"]],
    on=["year", "track", "team_num"],
    how="left"
)

# team_formed: did we find a matching team in the outcomes file?
outcomes_df["team_formed"] = outcomes_df["funded_status"].notna().astype(int)

# proposal_success: funded_status if present, else 0
outcomes_df["proposal_success"] = outcomes_df["funded_status"].fillna(0).astype(int)

print("Outcomes coverage (means):")
print(outcomes_df[["team_formed", "proposal_success"]].mean())
display(outcomes_df.head())

# =====
# 2) MERGE MEASUREMENT + OUTCOMES
# =====
model_df = measurement_df.merge(outcomes_df[["session_id", "team_formed", "proposal_success"]], on=

print("Merged model_df shape:", model_df.shape)
print("Unique sessions:", model_df["session_id"].nunique())
print("\nOutcome base rates:")
print(model_df[["team_formed", "proposal_success"]].mean())

```

```

# =====
# 3) BUILD PREDICTOR MATRIX
# =====
base_numeric_feats = [
    "early_seeding",
    "early_commitment",
    "late_synthesis",
    "late_closure",
    "delta_synthesis",
    "delta_closure",
    "delta_stress_mid",
    "jsd_next_overall",
    "prop_facilitator_utterances",
]
base_numeric_feats = [c for c in base_numeric_feats if c in model_df.columns]
X_parts = [model_df[base_numeric_feats].copy()]

if "cluster" in model_df.columns:
    cluster_dummies = pd.get_dummies(model_df["cluster"], prefix="cluster", drop_first=True)
    X_parts.append(cluster_dummies)

X = pd.concat(X_parts, axis=1)

y_team = model_df["team_formed"].astype(int)
y_prop = model_df["proposal_success"].astype(int)

print("\nPredictor matrix shape:", X.shape)
print("Predictors:", X.columns.tolist())

# =====
# 4) OUTCOME RATES BY CLUSTER (if present) with labels/units
# =====
if "cluster" in model_df.columns:
    rates = model_df.groupby("cluster")[["team_formed", "proposal_success"]].mean().reset_index()

    plt.figure()
    plt.bar(rates["cluster"].astype(str), rates["team_formed"])

```



```

plt.xlabel("Cluster")
plt.ylabel("Rate (proportion of sessions)")
plt.ylim(0, 1)
plt.title("Team Formation Rate by Cluster")
plt.show()

plt.figure()
plt.bar(rates["cluster"].astype(str), rates["proposal_success"])
plt.xlabel("Cluster")
plt.ylabel("Rate (proportion of sessions)")
plt.ylim(0, 1)
plt.title("Proposal Funding Rate by Cluster")
plt.show()
else:
    print("\nNo 'cluster' column found, skipping cluster bar charts.")

# =====
# 5) BASELINE LOGISTIC REGRESSION (AUC) with guardrails
# =====
def eval_logreg_auc(X, y, label):
    if y.nunique() < 2:
        print(f"\n[{label}] Skipping AUC: single-class outcome.")
        return

    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
    pipe = Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(max_iter=5000, class_weight="balanced", solver="lbfgs"))
    ])

    aucs = cross_val_score(pipe, X, y, cv=cv, scoring="roc_auc")
    print(f"\n[{label}] AUCs:", np.round(aucs, 3))
    print("Mean AUC:", round(aucs.mean(), 3), "±", round(aucs.std(), 3))

eval_logreg_auc(X, y_team, "TEAM FORMATION")
eval_logreg_auc(X, y_prop, "PROPOSAL SUCCESS (FUNDED)")

```

```

# =====
# 6) COEFFICIENTS (fit on full data)
# =====
def fit_and_show_coefs(X, y, label):
    if y.nunique() < 2:
        print(f"\n[{label}] Skipping coef table: single-class outcome.")
        return

    pipe_full = Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(max_iter=5000, class_weight="balanced", solver="lbfgs"))
    ])
    pipe_full.fit(X, y)

    coef = pipe_full.named_steps["clf"].coef_.ravel()
    coef_df = (
        pd.DataFrame({"feature": X.columns, "coef": coef})
        .sort_values("coef", ascending=False)
        .reset_index(drop=True)
    )

    print(f"\n[{label}] Top positive predictors:")
    display(coef_df.head(10))

    print(f"\n[{label}] Top negative predictors:")
    display(coef_df.tail(10))

fit_and_show_coefs(X, y_prop, "PROPOSAL SUCCESS (FUNDED)")

```

Found 8 outcome JSON files.

- /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2020NES/2020NES_outcome.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2021ABI/2021ABI_outcome.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2021CMC/2021CMC_outcome.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2021MND/2021MND_outcome.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2021MZT/2021MZT_outcome.json

...

teams_df shape: (193, 7)

	year	track	team_id	team_num	funded_status	members	source_path
0	2020	NES	NES1	1	0	[Shu Hu, Jared Mason, Shaama Sharada]	/Users/maxchalekson/Northwestern University/Su...
1	2020	NES	NES2	2	0	[Venkat Viswanathan, Charles McCrory, Jesus Ve...	/Users/maxchalekson/Northwestern University/Su...
2	2020	NES	NES3	3	0	[Sen Zhang, Praveen Bollini, Will Bowman]	/Users/maxchalekson/Northwestern University/Su...
3	2020	NES	NES4	4	0	[Betar Gallant, Michael Nippe]	/Users/maxchalekson/Northwestern University/Su...
4	2020	NES	NES5	5	1	[Nanette Boyle, Shu Hu]	/Users/maxchalekson/Northwestern University/Su...

Warning: 1 session_id values did not match the expected pattern.

	session_id	year	track	team_num
156	output-2021-05-20-ABI-S1	NaN	None	NaN

Outcomes coverage (means):

team_formed 0.993631

proposal_success 0.350318

dtype: float64

	session_id	year	track	team_num	funded_status	team_formed	proposal_success
0	2020_11_05_NES_S1	2020.0	NES	1.0	0.0	1	0
1	2020_11_05_NES_S2	2020.0	NES	2.0	0.0	1	0
2	2020_11_05_NES_S3	2020.0	NES	3.0	0.0	1	0
3	2020_11_05_NES_S4	2020.0	NES	4.0	0.0	1	0
4	2020_11_05_NES_S5	2020.0	NES	5.0	1.0	1	1

Merged model_df shape: (157, 41)

Unique sessions: 157

Outcome base rates:

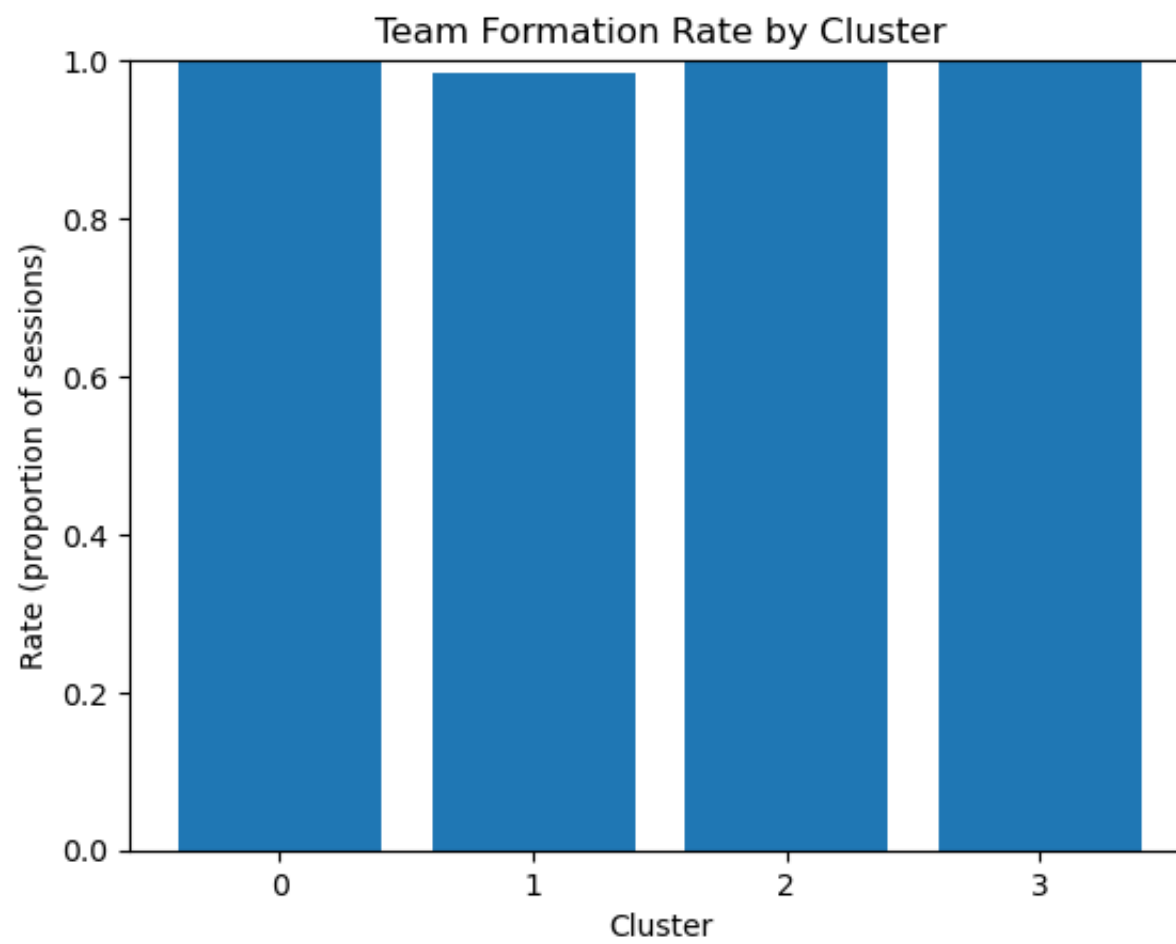
team_formed 0.993631

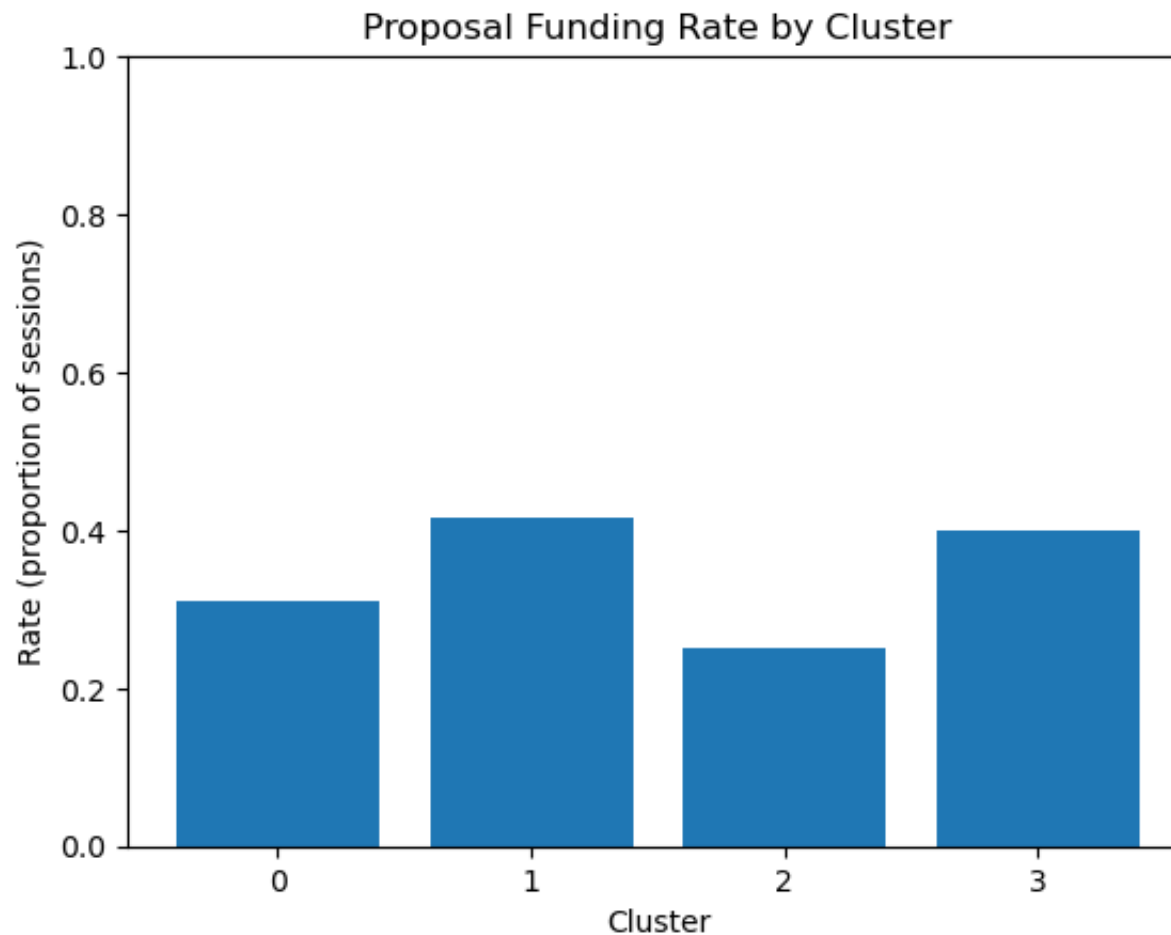
proposal_success 0.350318

dtype: float64

Predictor matrix shape: (157, 12)

Predictors: ['early_seeding', 'early_commitment', 'late_synthesis', 'late_closure', 'delta_synthesis', 'delta_closure', 'delta_stress_mid', 'jsd_next_overall', 'prop_facilitator_utterances', 'cluster_1', 'cluster_2', 'cluster_3']





[TEAM FORMATION] AUCs: [nan nan nan nan nan]

Mean AUC: nan $\pm$ nan

[PROPOSAL SUCCESS (FUNDED)] AUCs: [0.463 0.693 0.532 0.555 0.45]

Mean AUC: 0.538 $\pm$ 0.087

[PROPOSAL SUCCESS (FUNDED)] Top positive predictors:

```
/Users/maxchalekson/miniforge3/envs/gem_samp/lib/python3.11/site-packages/sklearn/model_selection/_split.py:811: UserWarning: The least populated class in y has only 1 members, which is less than n_splits=5.
```

```
warnings.warn(  
/Users/maxchalekson/miniforge3/envs/gem_samp/lib/python3.11/site-packages/sklearn/metrics/_ranking.
```

```

py:424: UndefinedMetricWarning: Only one class is present in y_true. ROC AUC score is not defined i
n that case.
    warnings.warn(
/Users/maxchalekson/miniforge3/envs/gem_samp/lib/python3.11/site-packages/sklearn/metrics/_ranking.
py:424: UndefinedMetricWarning: Only one class is present in y_true. ROC AUC score is not defined i
n that case.
    warnings.warn(
/Users/maxchalekson/miniforge3/envs/gem_samp/lib/python3.11/site-packages/sklearn/metrics/_ranking.
py:424: UndefinedMetricWarning: Only one class is present in y_true. ROC AUC score is not defined i
n that case.
    warnings.warn(
/Users/maxchalekson/miniforge3/envs/gem_samp/lib/python3.11/site-packages/sklearn/metrics/_ranking.
py:424: UndefinedMetricWarning: Only one class is present in y_true. ROC AUC score is not defined i
n that case.
    warnings.warn(
/Users/maxchalekson/miniforge3/envs/gem_samp/lib/python3.11/site-packages/sklearn/model_selection/_
validation.py:516: FitFailedWarning:
1 fits failed out of a total of 5.
The score on these train-test partitions for these parameters will be set to nan.
If these failures are not expected, you can try to debug them by setting error_score='raise'.

```

Below are more details about the failures:

```

-----
1 fits failed with the following error:
Traceback (most recent call last):
  File "/Users/maxchalekson/miniforge3/envs/gem_samp/lib/python3.11/site-packages/sklearn/model_sel
ection/_validation.py", line 859, in _fit_and_score
    estimator.fit(X_train, y_train, **fit_params)
  File "/Users/maxchalekson/miniforge3/envs/gem_samp/lib/python3.11/site-packages/sklearn/base.py",
line 1365, in wrapper
    return fit_method(estimator, *args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/Users/maxchalekson/miniforge3/envs/gem_samp/lib/python3.11/site-packages/sklearn/pipeline.
py", line 663, in fit
    self._final_estimator.fit(Xt, y, **last_step_params["fit"])
  File "/Users/maxchalekson/miniforge3/envs/gem_samp/lib/python3.11/site-packages/sklearn/base.py",
line 1365, in wrapper
    return fit_method(estimator, *args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```
File "/Users/maxchalekson/miniforge3/envs/gem_samp/lib/python3.11/site-packages/sklearn/linear_model/_logistic.py", line 1335, in fit
    raise ValueError(
ValueError: This solver needs samples of at least 2 classes in the data, but the data contains only one class: np.int64(1)

warnings.warn(some_fits_failed_message, FitFailedWarning)
```

	feature	coef
0	delta_stress_mid	0.339872
1	prop_facilitator_utterances	0.270407
2	late_closure	0.251874
3	delta_closure	0.246987
4	cluster_1	0.185090
5	early_seeding	0.071326
6	late_synthesis	0.069844
7	cluster_2	0.017336
8	jsd_next_overall	-0.077403
9	early_commitment	-0.092756

[PROPOSAL SUCCESS (FUNDED)] Top negative predictors:

	feature	coef
2	late_closure	0.251874
3	delta_closure	0.246987
4	cluster_1	0.185090
5	early_seeding	0.071326
6	late_synthesis	0.069844
7	cluster_2	0.017336
8	jsd_next_overall	-0.077403
9	early_commitment	-0.092756
10	cluster_3	-0.246980
11	delta_synthesis	-0.287045

```
In [18]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import re

from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

# =====
# 0) Robust session_id parsing (supports 2 formats)
# =====
def parse_session_id_any(session_id: str):
    """
    Supports:
    1) YYYY_MM_DD_TRK_S#
       e.g., 2021_04_22_MND_S2
```

```

    2) output-YYYY-MM-DD-TRK-S#
       e.g., output-2021-05-20-ABI-S1
Returns: (year:int, track:str, team_num:int) or (None,None,None)
"""
s = str(session_id)

m1 = re.match(r"(?P<year>\d{4})_\d{2}_\d{2}_(?P<track>[A-Z]{3})_S(?P<num>\d+)$", s)
if m1:
    return int(m1.group("year")), m1.group("track"), int(m1.group("num"))

m2 = re.match(r"output-(?P<year>\d{4})-\d{2}-\d{2}-(?P<track>[A-Z]{3})-S(?P<num>\d+)$", s)
if m2:
    return int(m2.group("year")), m2.group("track"), int(m2.group("num"))

return None, None, None

# =====
# 1) Rebuild outcomes_df using teams_df (assumes you already built teams_df)
# =====
if "measurement_df" not in globals():
    raise NameError("measurement_df not found. Run measurement-building first.")
if "teams_df" not in globals():
    raise NameError("teams_df not found. Run the outcomes-loading chunk that builds teams_df first.")

outcomes_df = measurement_df[["session_id"]].copy()
parsed = outcomes_df["session_id"].apply(parse_session_id_any)
outcomes_df[["year", "track", "team_num"]] = pd.DataFrame(parsed.tolist(), index=outcomes_df.index)

bad = outcomes_df["year"].isna().sum()
if bad:
    print(f"Warning: {bad} session_id values did not match either known pattern.")
    display(outcomes_df[outcomes_df["year"].isna()].head())

# Join by (year, track, team_num)
outcomes_df = outcomes_df.merge(
    teams_df[["year", "track", "team_num", "funded_status"]],
    on=["year", "track", "team_num"],
    how="left"

```

```

)

outcomes_df["team_formed"] = outcomes_df["funded_status"].notna().astype(int)
outcomes_df["proposal_success"] = outcomes_df["funded_status"].fillna(0).astype(int)

print("\nOutcome base rates:")
print(outcomes_df[["team_formed", "proposal_success"]].mean())
print("\nOutcome class counts:")
print("team_formed:\n", outcomes_df["team_formed"].value_counts(dropna=False))
print("\nproposal_success:\n", outcomes_df["proposal_success"].value_counts(dropna=False))
display(outcomes_df.head())

# =====
# 2) Merge to model_df
# =====
model_df = measurement_df.merge(
    outcomes_df[["session_id", "team_formed", "proposal_success"]],
    on="session_id",
    how="inner"
)

print("\nMerged model_df shape:", model_df.shape)
print("Unique sessions:", model_df["session_id"].nunique())

# =====
# 3) Predictor matrix
# =====
base_numeric_feats = [
    "early_seeding",
    "early_commitment",
    "late_synthesis",
    "late_closure",
    "delta_synthesis",
    "delta_closure",
    "delta_stress_mid",
    "jsd_next_overall",
    "prop_facilitator_utterances",

```

```

]
base_numeric_feats = [c for c in base_numeric_feats if c in model_df.columns]
X_parts = [model_df[base_numeric_feats].copy()]

if "cluster" in model_df.columns:
    X_parts.append(pd.get_dummies(model_df["cluster"], prefix="cluster", drop_first=True))

X = pd.concat(X_parts, axis=1)
y_team = model_df["team_formed"].astype(int)
y_prop = model_df["proposal_success"].astype(int)

print("\nX shape:", X.shape)
print("X columns:", X.columns.tolist())

# =====
# 4) Cluster outcome bars (only if cluster exists)
# =====
if "cluster" in model_df.columns:
    rates = model_df.groupby("cluster")[["team_formed", "proposal_success"]].mean().reset_index()

    plt.figure()
    plt.bar(rates["cluster"].astype(str), rates["team_formed"])
    plt.xlabel("Cluster")
    plt.ylabel("Rate (proportion of sessions)")
    plt.ylim(0, 1)
    plt.title("Team Formation Rate by Cluster")
    plt.show()

    plt.figure()
    plt.bar(rates["cluster"].astype(str), rates["proposal_success"])
    plt.xlabel("Cluster")
    plt.ylabel("Rate (proportion of sessions)")
    plt.ylim(0, 1)
    plt.title("Proposal Funding Rate by Cluster")
    plt.show()

# =====

```

```

# 5) CV that doesn't implode when classes are rare
# =====
def safe_n_splits(y, max_splits=5):
    # smallest class count determines how many stratified folds are even possible
    counts = y.value_counts()
    min_class = counts.min()
    # need at least 2 folds, and at least 2 samples in each class for sane CV
    return max(2, min(max_splits, int(min_class)))

def eval_logreg_auc(X, y, label):
    if y.nunique() < 2:
        print(f"\n[{label}] Skipping: single-class outcome.")
        return

    n_splits = safe_n_splits(y, max_splits=5)
    counts = y.value_counts()
    print(f"\n[{label}] class counts: {counts.to_dict()} | using n_splits={n_splits}")

    # if minority class count is 1, even n_splits=2 will fail sometimes
    if counts.min() < 2:
        print(f"[{label}] Skipping AUC: minority class has <2 samples. This is not a modeling problem")
        return

    cv = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)

    pipe = Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(max_iter=5000, class_weight="balanced", solver="lbfgs"))
    ])

    aucs = cross_val_score(pipe, X, y, cv=cv, scoring="roc_auc", error_score=np.nan)
    aucs = aucs[~np.isnan(aucs)]
    if len(aucs) == 0:
        print(f"[{label}] All folds produced NaN AUC (likely single-class folds).")
        return

    print(f"[{label}] AUCs:", np.round(aucs, 3))
    print(f"[{label}] Mean AUC:", round(aucs.mean(), 3), "±", round(aucs.std(), 3))

```

```

eval_logreg_auc(X, y_team, "TEAM FORMATION")
eval_logreg_auc(X, y_prop, "PROPOSAL SUCCESS (FUNDED)")

# =====
# 6) Coefs (proposal_success only, because team_formed is basically constant)
# =====
def fit_and_show_coefs(X, y, label):
    if y.nunique() < 2:
        print(f"\n[{label}] Skipping coef table: single-class outcome.")
        return
    if y.value_counts().min() < 2:
        print(f"\n[{label}] Skipping coef table: minority class <2 samples (unstable fit).")
        return

    pipe_full = Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(max_iter=5000, class_weight="balanced", solver="lbfgs"))
    ])
    pipe_full.fit(X, y)

    coef = pipe_full.named_steps["clf"].coef_.ravel()
    coef_df = (
        pd.DataFrame({"feature": X.columns, "coef": coef})
        .sort_values("coef", ascending=False)
        .reset_index(drop=True)
    )

    print(f"\n[{label}] Top positive predictors:")
    display(coef_df.head(12))

    print(f"\n[{label}] Top negative predictors:")
    display(coef_df.tail(12))

fit_and_show_coefs(X, y_prop, "PROPOSAL SUCCESS (FUNDED)")

```

```
Outcome base rates:
team_formed      1.000000
proposal_success  0.350318
dtype: float64
```

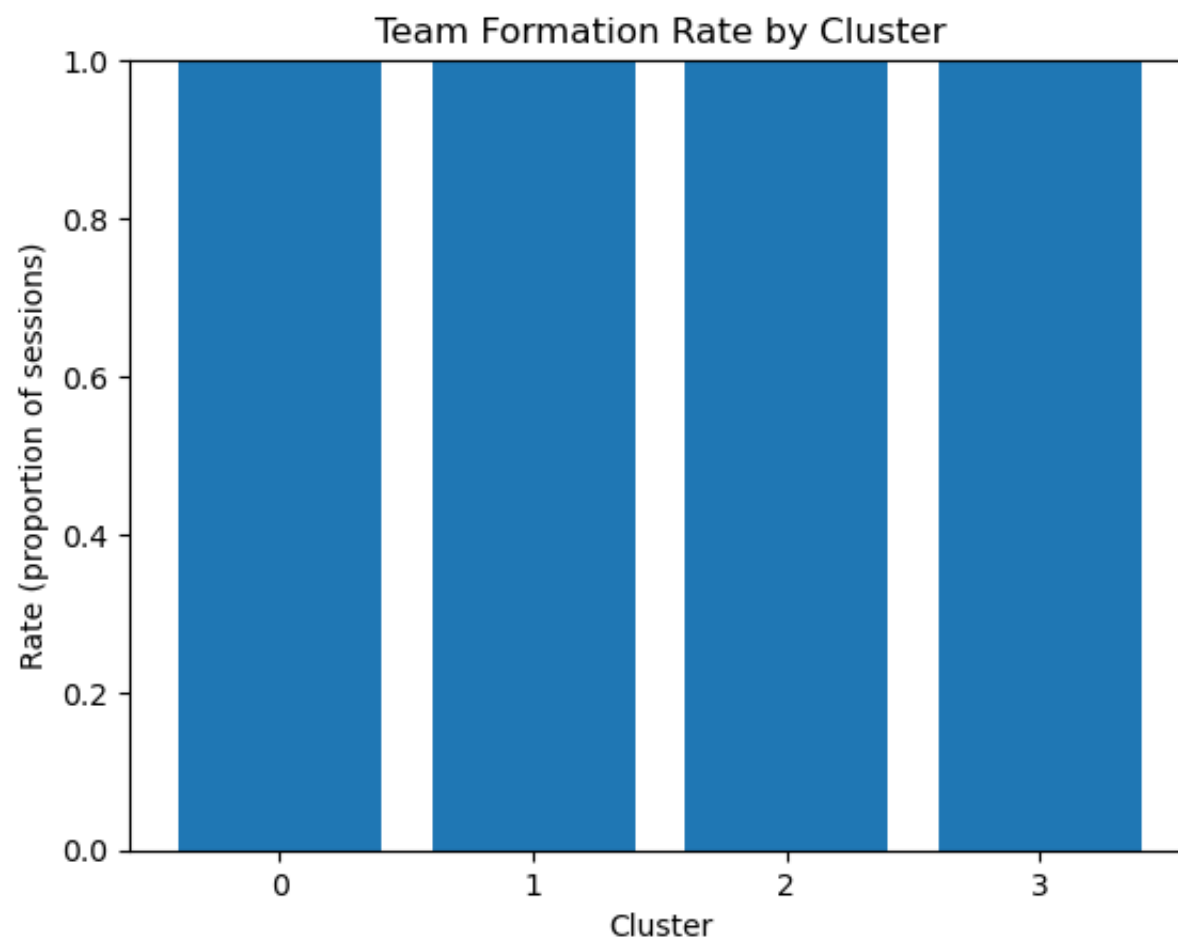
```
Outcome class counts:
team_formed:
  team_formed
1      157
Name: count, dtype: int64
```

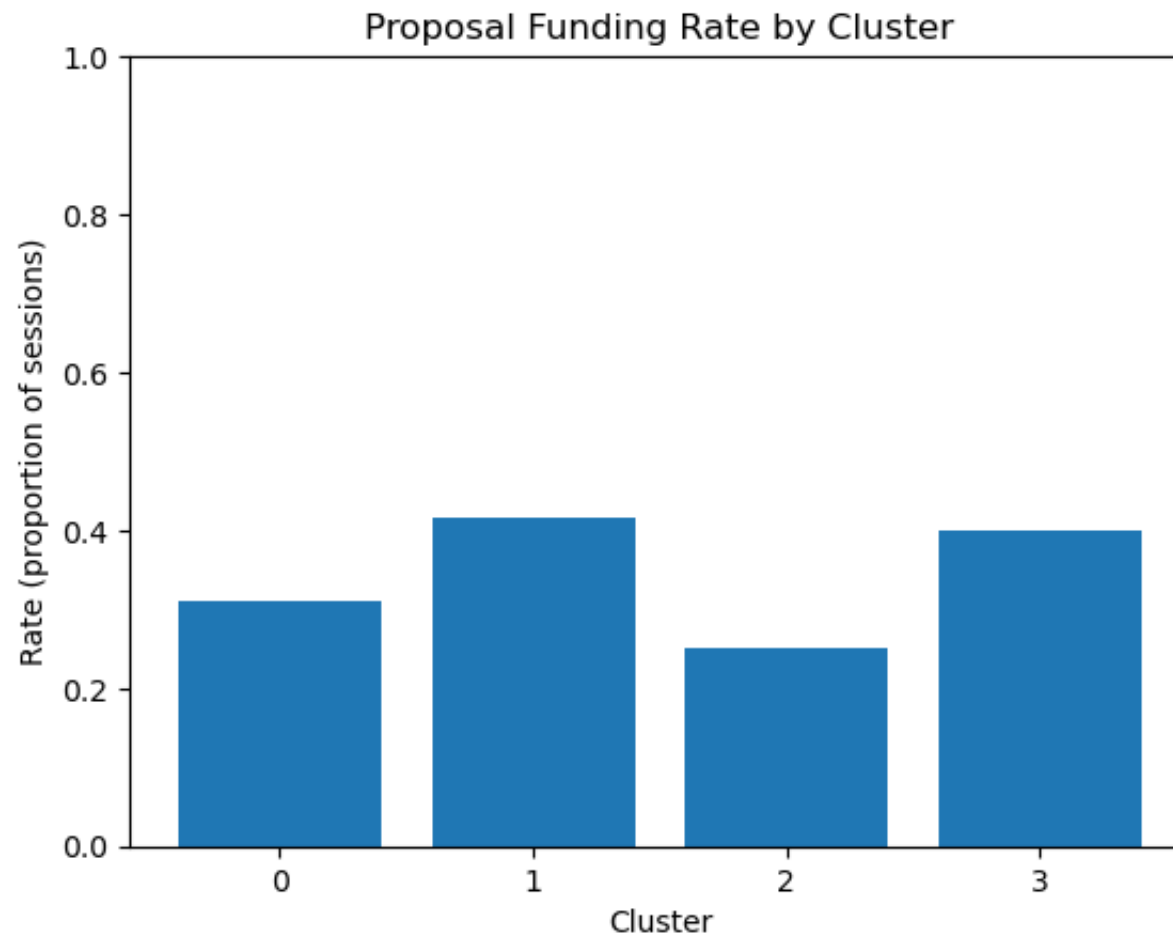
```
proposal_success:
  proposal_success
0      102
1       55
Name: count, dtype: int64
```

	session_id	year	track	team_num	funded_status	team_formed	proposal_success
0	2020_11_05_NES_S1	2020	NES	1	0	1	0
1	2020_11_05_NES_S2	2020	NES	2	0	1	0
2	2020_11_05_NES_S3	2020	NES	3	0	1	0
3	2020_11_05_NES_S4	2020	NES	4	0	1	0
4	2020_11_05_NES_S5	2020	NES	5	1	1	1

```
Merged model_df shape: (157, 41)
Unique sessions: 157
```

```
X shape: (157, 12)
X columns: ['early_seeding', 'early_commitment', 'late_synthesis', 'late_closure', 'delta_synthesis', 'delta_closure', 'delta_stress_mid', 'jsd_next_overall', 'prop_facilitator_utterances', 'cluster_1', 'cluster_2', 'cluster_3']
```





[TEAM FORMATION] Skipping: single-class outcome.

[PROPOSAL SUCCESS (FUNDED)] class counts: {0: 102, 1: 55} | using n_splits=5

[PROPOSAL SUCCESS (FUNDED)] AUCs: [0.463 0.693 0.532 0.555 0.45]

[PROPOSAL SUCCESS (FUNDED)] Mean AUC: 0.538 ± 0.087

[PROPOSAL SUCCESS (FUNDED)] Top positive predictors:

	feature	coef
0	delta_stress_mid	0.339872
1	prop_facilitator_utterances	0.270407
2	late_closure	0.251874
3	delta_closure	0.246987
4	cluster_1	0.185090
5	early_seeding	0.071326
6	late_synthesis	0.069844
7	cluster_2	0.017336
8	jsd_next_overall	-0.077403
9	early_commitment	-0.092756
10	cluster_3	-0.246980
11	delta_synthesis	-0.287045

[PROPOSAL SUCCESS (FUNDED)] Top negative predictors:

	feature	coef
0	delta_stress_mid	0.339872
1	prop_facilitator_utterances	0.270407
2	late_closure	0.251874
3	delta_closure	0.246987
4	cluster_1	0.185090
5	early_seeding	0.071326
6	late_synthesis	0.069844
7	cluster_2	0.017336
8	jsd_next_overall	-0.077403
9	early_commitment	-0.092756
10	cluster_3	-0.246980
11	delta_synthesis	-0.287045

```
In [21]: import os, json, re, glob
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

# =====
# 0) REQUIRE measurement_df
# =====
if "measurement_df" not in globals():
```

```

    raise NameError("measurement_df not found. Run the measurement-building chunk first.")

# =====
# 1) LOAD ALL OUTCOME JSON FILES -> teams_df (with year/track)
# =====
# Adjust BASE_DIR if needed
BASE_DIRS = [
    "/Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis",
    "/mnt/data", # fallback, if you ever drop outcome jsons here
]

outcome_paths = []
for base in BASE_DIRS:
    if os.path.exists(base):
        outcome_paths.extend(glob.glob(os.path.join(base, "**", "*_outcome.json"), recursive=True))

outcome_paths = sorted(set(outcome_paths))

if not outcome_paths:
    raise FileNotFoundError("No *_outcome.json files found. Check BASE_DIRS.")

print(f"Found {len(outcome_paths)} outcome JSON files.")
for p in outcome_paths[:8]:
    print(" -", p)
if len(outcome_paths) > 8:
    print(" ...")

def parse_year_track_from_path(path: str):
    """
    Expects paths like ../2021MND/2021MND_outcome.json
    Returns (year:int, track:str) or (None, None) if not parseable.
    """
    m = re.search(r"/(\d{4})([A-Z]{3})/(\d{4})([A-Z]{3})_outcome\.json$", path.replace("\\", "/"))
    if not m:
        return None, None
    year = int(m.group(1))
    track = m.group(2)
    return year, track

```

```

team_rows = []
for path in outcome_paths:
    with open(path, "r") as f:
        data = json.load(f)

        if not isinstance(data, list) or len(data) == 0:
            continue

    year, track = parse_year_track_from_path(path)

    for d in data:
        # Defensive extraction
        team_id = d.get("team_id", None)
        funded = d.get("funded_status", None)
        members = d.get("members", None)

        if team_id is None or funded is None:
            continue

        mnum = re.search(r"(\d+)$", str(team_id))
        team_num = int(mnum.group(1)) if mnum else None

        team_rows.append({
            "year": year,
            "track": track,
            "team_id": str(team_id),
            "team_num": team_num,
            "funded_status": int(funded),
            "members": members,
            "source_path": path,
        })

teams_df = pd.DataFrame(team_rows)
if teams_df.empty:
    raise ValueError("Loaded outcome JSONs but teams_df is empty. Check file structure.")

# Drop any duplicates (same team_id across sources should not happen, but just in case)
teams_df = teams_df.drop_duplicates(subset=["team_id"]).reset_index(drop=True)

```

```

print("\nteam_df shape:", team_df.shape)
display(team_df.head())

# =====
# 2) PARSE session_id -> (year, track, team_num)
# =====
# Primary pattern: 2020_11_05_NES_S1
SESSION_PATTERNS = [
    re.compile(r"^(?P<year>\d{4})_\d{2}_\d{2}_(?P<track>[A-Z]{3})_S(?P<num>\d+)$"),
    # Fallback pattern you had: output-2021-05-20-ABI-S1
    re.compile(r"^output-(?P<year>\d{4})-\d{2}-\d{2}-(?P<track>[A-Z]{3})-S(?P<num>\d+)$"),
]

def parse_session_id(session_id: str):
    s = str(session_id)
    for pat in SESSION_PATTERNS:
        m = pat.search(s)
        if m:
            return int(m.group("year")), m.group("track"), int(m.group("num"))
    return None, None, None

tmp = measurement_df[["session_id"]].copy()
tmp[["year", "track", "team_num"]] = tmp["session_id"].apply(lambda x: pd.Series(parse_session_id(x)))

bad = tmp[tmp["year"].isna() | tmp["track"].isna() | tmp["team_num"].isna()]
if len(bad) > 0:
    print(f"\nWarning: {len(bad)} session_id values did not match expected patterns.")
    display(bad.head(10))

# Build team_id
tmp["team_id"] = tmp.apply(
    lambda r: f"{r['track']}{int(r['team_num'])}" if pd.notna(r["track"]) and pd.notna(r["team_num"])
    else None,
    axis=1
)

# Map outcomes
funded_map = dict(zip(team_df["team_id"], team_df["funded_status"]))

```

```

tmp["team_formed"] = tmp["team_id"].apply(lambda t: int(t in funded_map))
tmp["proposal_success"] = tmp["team_id"].map(funded_map).fillna(0).astype(int)

outcomes_df = tmp[["session_id", "year", "track", "team_num", "team_id", "team_formed", "proposal_s

print("\nOutcomes coverage (means):")
print(outcomes_df[["team_formed", "proposal_success"]].mean())
display(outcomes_df.head())

# =====
# 3) MERGE measurement_df + outcomes_df -> model_df
# =====
model_df = measurement_df.merge(outcomes_df, on="session_id", how="inner")

print("\nMerged model_df shape:", model_df.shape)
print("Unique sessions:", model_df["session_id"].nunique())

print("\nOutcome base rates:")
print(model_df[["team_formed", "proposal_success"]].mean())

print("\nOutcome class counts:")
print("team_formed:\n", model_df["team_formed"].value_counts())
print("\nproposal_success:\n", model_df["proposal_success"].value_counts())

# =====
# 4) BUILD FEATURE MATRIX (base + optional cluster + controls)
# =====
BASE_FEATS = [
    "early_seeding",
    "early_commitment",
    "late_synthesis",
    "late_closure",
    "delta_synthesis",
    "delta_closure",
    "delta_stress_mid",
    "jsd_next_overall",
    "prop_facilitator_utterances",

```

```

]
BASE_FEATS = [c for c in BASE_FEATS if c in model_df.columns]

y = model_df["proposal_success"].astype(int)

# cluster dummies if present
X_parts = [model_df[BASE_FEATS].copy()]

if "cluster" in model_df.columns:
    cluster_dum = pd.get_dummies(model_df["cluster"], prefix="cluster", drop_first=True)
    X_parts.append(cluster_dum)

# controls
track_dum = pd.get_dummies(model_df["track"].astype(str), prefix="track", drop_first=True)
year_col = model_df[["year"]].astype(float)

X_parts.append(track_dum)
X_parts.append(year_col)

X = pd.concat(X_parts, axis=1)

print("\nX shape:", X.shape)
print("X columns:", X.columns.tolist())


# =====
# 5) CV EVAL (AUC + PR-AUC) with guardrails
# =====
def safe_n_splits(y, max_splits=5):
    return max(2, min(max_splits, int(y.value_counts().min())))

def eval_model(X, y, label):
    if y.nunique() < 2:
        print(f"\n[{label}] Skipping: single-class outcome.")
        return

    if y.value_counts().min() < 2:
        print(f"\n[{label}] Skipping: minority class <2 samples.")
        return

```



```

n_splits = safe_n_splits(y, max_splits=5)
print(f"\n[{{label}}] class counts: {y.value_counts().to_dict()} | using n_splits={{n_splits}}")

cv = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=5000, class_weight="balanced", solver="lbfgs"))
])

aucs = cross_val_score(pipe, X, y, cv=cv, scoring="roc_auc", error_score=np.nan)
pr_auc = cross_val_score(pipe, X, y, cv=cv, scoring="average_precision", error_score=np.nan)

aucs = aucs[~np.isnan(aucs)]
pr_auc = pr_auc[~np.isnan(pr_auc)]

print(f"[{{label}}] AUCs:", np.round(aucs, 3))
print(f"[{{label}}] Mean AUC: {aucs.mean():.3f} ± {aucs.std():.3f}")
print(f"[{{label}}] PR-AUC mean: {pr_auc.mean():.3f} ± {pr_auc.std():.3f}")

eval_model(X, y, "PROPOSAL SUCCESS (FUNDED)")

# =====
# 6) FULL FIT + COEFFICIENTS (printed correctly)
# =====
pipe_full = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=5000, class_weight="balanced", solver="lbfgs"))
])
pipe_full.fit(X, y)

coef = pipe_full.named_steps["clf"].coef_.ravel()
coef_df = (
    pd.DataFrame({"feature": X.columns, "coef": coef})
    .sort_values("coef", ascending=False)
    .reset_index(drop=True)
)

```

```

print("\nTop positive predictors:")
display(coef_df.head(12))

print("\nTop negative predictors:")
display(coef_df.tail(12))

# =====
# 7) INTERPRETABLE BASELINE PLOTS (by track)
# =====
track_rates = model_df.groupby("track")["proposal_success"].mean().sort_values(ascending=False)

plt.figure(figsize=(8,4))
plt.bar(track_rates.index.astype(str), track_rates.values)
plt.xlabel("Track")
plt.ylabel("Proposal funding rate (proportion)")
plt.ylim(0, 1)
plt.title("Proposal Funding Rate by Track")
plt.show()

```

Found 8 outcome JSON files.

- /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2020NES/2020NES_outcome.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2021ABI/2021ABI_outcome.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2021CMC/2021CMC_outcome.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2021MND/2021MND_outcome.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2021MZT/2021MZT_outcome.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2021NES/2021NES_outcome.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2021SLU/2021SLU_outcome.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2022MND/2022MND_outcome.json

teams_df shape: (153, 7)

	year	track	team_id	team_num	funded_status	members	source_path
0	2020	NES	NES1	1	0	[Shu Hu, Jared Mason, Shaama Sharada]	/Users/maxchalekson/Northwestern University/Su...
1	2020	NES	NES2	2	0	[Venkat Viswanathan, Charles McCrory, Jesus Ve...]	/Users/maxchalekson/Northwestern University/Su...
2	2020	NES	NES3	3	0	[Sen Zhang, Praveen Bollini, Will Bowman]	/Users/maxchalekson/Northwestern University/Su...
3	2020	NES	NES4	4	0	[Betar Gallant, Michael Nippe]	/Users/maxchalekson/Northwestern University/Su...
4	2020	NES	NES5	5	1	[Nanette Boyle, Shu Hu]	/Users/maxchalekson/Northwestern University/Su...

Outcomes coverage (means):
team_formed 1.000000
proposal_success 0.299363
dtype: float64

	session_id	year	track	team_num	team_id	team_formed	proposal_success
0	2020_11_05_NES_S1	2020	NES	1	NES1	1	0
1	2020_11_05_NES_S2	2020	NES	2	NES2	1	0
2	2020_11_05_NES_S3	2020	NES	3	NES3	1	0
3	2020_11_05_NES_S4	2020	NES	4	NES4	1	0
4	2020_11_05_NES_S5	2020	NES	5	NES5	1	1

```
Merged model_df shape: (157, 45)
Unique sessions: 157
```

```
Outcome base rates:
team_formed          1.000000
proposal_success     0.299363
dtype: float64
```

```
Outcome class counts:
team_formed:
  team_formed
1         157
Name: count, dtype: int64
```

```
proposal_success:
  proposal_success
0         110
1          47
Name: count, dtype: int64
```

```
X shape: (157, 18)
X columns: ['early_seeding', 'early_commitment', 'late_synthesis', 'late_closure', 'delta_synthesis', 'delta_closure', 'delta_stress_mid', 'jsd_next_overall', 'prop_facilitator_utterances', 'cluster_1', 'cluster_2', 'cluster_3', 'track_CMC', 'track_MND', 'track_MZT', 'track_NES', 'track_SLU', 'year']
```

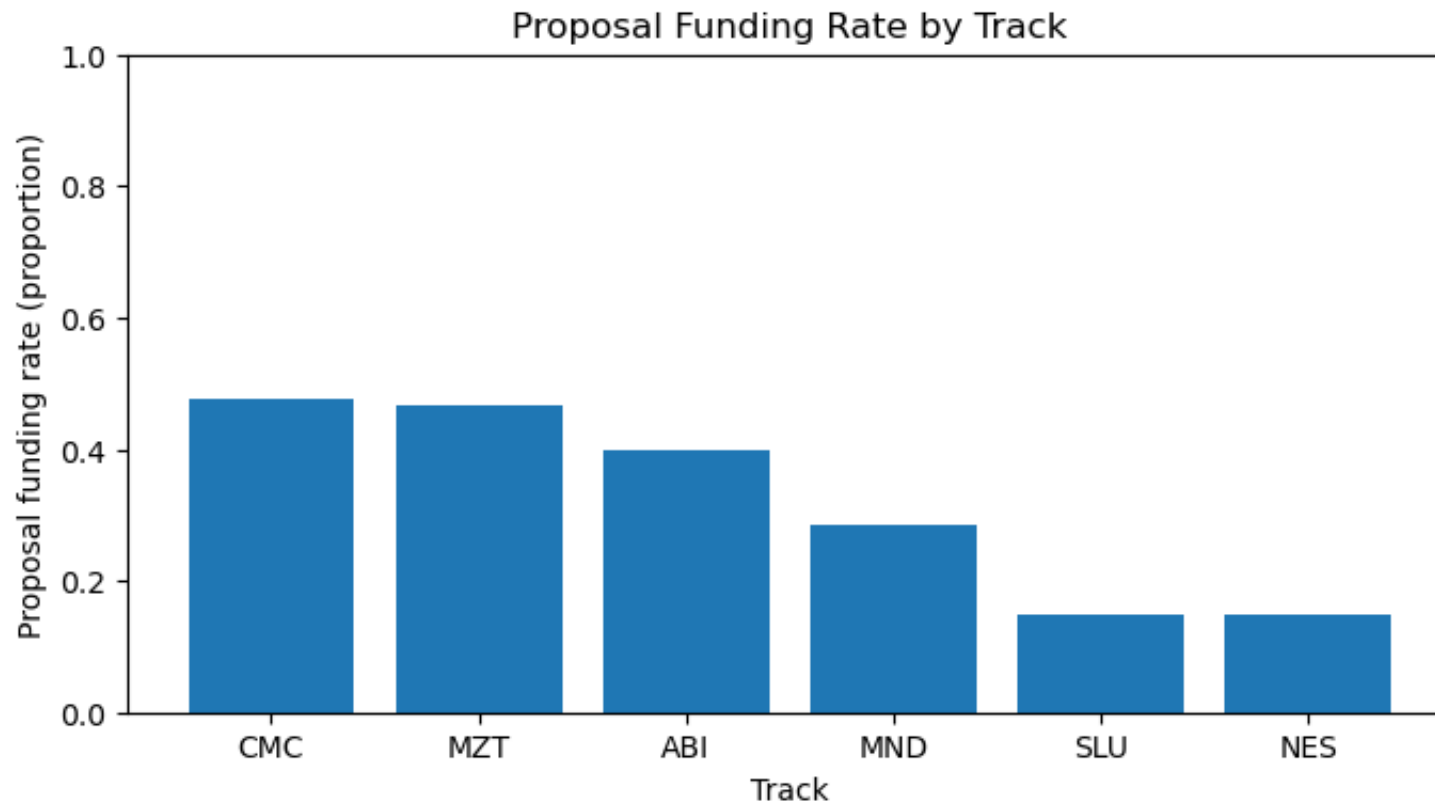
```
[PROPOSAL SUCCESS (FUNDED)] class counts: {0: 110, 1: 47} | using n_splits=5
[PROPOSAL SUCCESS (FUNDED)] AUCs: [0.591 0.595 0.662 0.788 0.475]
[PROPOSAL SUCCESS (FUNDED)] Mean AUC: 0.622 ± 0.102
[PROPOSAL SUCCESS (FUNDED)] PR-AUC mean: 0.421 ± 0.118
```

```
Top positive predictors:
```

	feature	coef
0	delta_stress_mid	0.595679
1	late_synthesis	0.277359
2	delta_closure	0.207212
3	cluster_2	0.194706
4	track_MZT	0.159622
5	late_closure	0.154876
6	track_CMC	0.153454
7	year	0.141360
8	cluster_1	0.092051
9	prop_facilitator_utterances	-0.086699
10	cluster_3	-0.115777
11	early_commitment	-0.132901

Top negative predictors:

	feature	coef
6	track_CMC	0.153454
7	year	0.141360
8	cluster_1	0.092051
9	prop_facilitator_utterances	-0.086699
10	cluster_3	-0.115777
11	early_commitment	-0.132901
12	track_MND	-0.162885
13	early_seeding	-0.229844
14	track_SLU	-0.444611
15	delta_synthesis	-0.503148
16	jsd_next_overall	-0.521746
17	track_NES	-0.551677



right idea, was using the wrong json file, updating the json file.

earlier chunks were using something like this `'/Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2021NES/2021NES_outcome.json'`

I was supposed to use something like this `'/Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2021NES/2021NES_session_outcomes.json'`

```
In [22]: import os, json, re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```

from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

# =====
# 0) LOAD *SESSION* OUTCOMES JSONs (the ones you actually want)
# =====

# Change this if your directory differs
ROOT_DIRS = [
    "/Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis",
    "/mnt/data", # fallback for attached files in this environment
]

def find_session_outcome_files(root_dirs):
    paths = []
    for root in root_dirs:
        if not os.path.isdir(root):
            continue
        for dirpath, _, filenames in os.walk(root):
            for fn in filenames:
                if fn.endswith("_session_outcomes.json"):
                    paths.append(os.path.join(dirpath, fn))
    return sorted(set(paths))

session_outcome_files = find_session_outcome_files(ROOT_DIRS)
if not session_outcome_files:
    raise FileNotFoundError(
        "No *_session_outcomes.json files found. Checked:\n" + "\n".join(ROOT_DIRS)
    )

print(f"Found {len(session_outcome_files)} session outcome JSON files.")
for p in session_outcome_files[:10]:
    print(" -", p)
if len(session_outcome_files) > 10:
    print(" ...")

# -----

```



```

# Parse session_id -> (year, track, team_num) robustly
# Supports:
#   2021_11_04_NES_S5
#   output-2021-05-20-ABI-S1
# -----
def parse_session_id(session_id: str):
    s = str(session_id)

    m1 = re.search(r"^(20\d{2})_\d{2}_\d{2}_([A-Z]{3})_S(\d+)$", s)
    if m1:
        return int(m1.group(1)), m1.group(2), int(m1.group(3))

    m2 = re.search(r"^output-(20\d{2})-\d{2}-\d{2}-([A-Z]{3})-S(\d+)$", s)
    if m2:
        return int(m2.group(1)), m2.group(2), int(m2.group(3))

    return None, None, None

# -----
# Build one big session_outcomes table across all conferences
# Each file is a dict keyed by session_id:
#   {
#       "2021_11_04_NES_S5": {"all_speakers": [...], "facilitators": [...], "teams": {...}},
#       ...
#   }
# -----
rows = []
for path in session_outcome_files:
    with open(path, "r") as f:
        d = json.load(f)

        if not isinstance(d, dict):
            print(f"Skipping unexpected format (not dict): {path}")
            continue

        for sid, info in d.items():
            if not isinstance(info, dict):
                continue

```

```

all_speakers = info.get("all_speakers", []) or []
facilitators = info.get("facilitators", []) or []
teams = info.get("teams", {}) or {}

# teams is expected dict: { "NES5": {"members":[...], "funded_status":0/1}, ... }
# We'll compute "session-relevant" teams based on intersection with speakers.
speaker_set = set([str(x).strip() for x in all_speakers])

n_teams_total = 0
n_teams_with_session_members = 0
n_funded_with_session_members = 0

members_union = set()

if isinstance(teams, dict):
    for team_id, tinfo in teams.items():
        if not isinstance(tinfo, dict):
            continue
        n_teams_total += 1

        mems = tinfo.get("members", []) or []
        mem_set = set([str(x).strip() for x in mems])

        # session-relevant team: at least one member actually appears in the session speakers
        if speaker_set and (mem_set & speaker_set):
            n_teams_with_session_members += 1
            members_union |= (mem_set & speaker_set)

            funded = int(tinfo.get("funded_status", 0) or 0)
            if funded == 1:
                n_funded_with_session_members += 1

team_formed = int(n_teams_with_session_members > 0)
proposal_success = int(n_funded_with_session_members > 0)

year, track, team_num = parse_session_id(sid)

rows.append({
    "session_id": sid,

```

```

        "year": year,
        "track": track,
        "team_num": team_num,
        "n_speakers": len(speaker_set),
        "n_facilitators": len(set([str(x).strip() for x in facilitators])),
        "n_teams_total": n_teams_total,
        "n_teams_in_session": n_teams_with_session_members,
        "n_funded_in_session": n_funded_with_session_members,
        "team_formed": team_formed,
        "proposal_success": proposal_success,
        "prop_speakers_on_team": (len(members_union) / len(speaker_set)) if len(speaker_set) else 0,
        "source_path": path,
    })

session_outcomes_df = pd.DataFrame(rows)

print("\nsession_outcomes_df shape:", session_outcomes_df.shape)
display(session_outcomes_df.head())

# =====
# 1) BUILD outcomes_df aligned to measurement_df
# =====
if "measurement_df" not in globals():
    raise NameError("measurement_df not found. Run your measurement-building chunk first.")

outcomes_df = (
    measurement_df[["session_id"]]
    .merge(
        session_outcomes_df.drop_duplicates("session_id"),
        on="session_id",
        how="left"
    )
)

# Handle sessions that didn't match the naming pattern (year/track missing)
bad = outcomes_df[outcomes_df["track"].isna()][["session_id", "year", "track", "team_num"]]
if len(bad) > 0:
    print(f"\nWarning: {len(bad)} session_id values did not match expected patterns (year/track/team_num missing)")
    display(bad.head(20))

```

```

# If a session is missing in session outcomes entirely, default to "no team formed, not funded"
outcomes_df["team_formed"] = outcomes_df["team_formed"].fillna(0).astype(int)
outcomes_df["proposal_success"] = outcomes_df["proposal_success"].fillna(0).astype(int)

# year can be useful as a numeric covariate; keep as float or int where available
# track is categorical; may be NaN for malformed IDs
print("\nOutcome base rates:")
print(outcomes_df[["team_formed", "proposal_success"]].mean())

print("\nOutcome class counts:")
for col in ["team_formed", "proposal_success"]:
    print(f"{col}:\n", outcomes_df[col].value_counts(dropna=False), "\n")

display(outcomes_df.head())

# =====
# 2) MERGE measurement + outcomes
# =====
model_df = measurement_df.merge(outcomes_df, on="session_id", how="inner")

print("\nMerged model_df shape:", model_df.shape)
print("Unique sessions:", model_df["session_id"].nunique())

# =====
# 3) BUILD X, y (with controls)
# =====
base_numeric_feats = [
    "early_seeding",
    "early_commitment",
    "late_synthesis",
    "late_closure",
    "delta_synthesis",
    "delta_closure",
    "delta_stress_mid",
    "jsd_next_overall",
    "prop_facilitator_utterances",
]
base_numeric_feats = [c for c in base_numeric_feats if c in model_df.columns]

```

```

X_parts = [model_df[base_numeric_feats].copy()]

# cluster dummies if present
if "cluster" in model_df.columns:
    X_parts.append(pd.get_dummies(model_df["cluster"], prefix="cluster", drop_first=True))

# track controls if present
if "track" in model_df.columns:
    X_parts.append(pd.get_dummies(model_df["track"], prefix="track", drop_first=True))

# year control if present and non-null
if "year" in model_df.columns:
    # keep numeric year; fill missing with median year to avoid dropping rows
    year_series = pd.to_numeric(model_df["year"], errors="coerce")
    if year_series.notna().any():
        X_parts.append(year_series.fillna(year_series.median()).rename("year").to_frame())

X = pd.concat(X_parts, axis=1).fillna(0)

y_team = model_df["team_formed"].astype(int)
y_prop = model_df["proposal_success"].astype(int)

print("\nX shape:", X.shape)
print("X columns:", X.columns.tolist())

# =====
# 4) CLEAN PLOTS WITH UNITS/LABELS
# =====
def plot_rate_by_group(df, group_col, outcome_col, title):
    if group_col not in df.columns:
        print(f"Skipping plot; missing {group_col}")
        return
    rates = df.groupby(group_col)[outcome_col].mean().sort_index()
    plt.figure(figsize=(8, 4))
    plt.bar(rates.index.astype(str), rates.values)
    plt.xlabel(group_col)
    plt.ylabel("Rate (proportion of sessions)")
    plt.ylim(0, 1)

```

```

plt.title(title)
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

plot_rate_by_group(model_df, "track", "proposal_success", "Proposal Funding Rate by Track")
if "cluster" in model_df.columns:
    plot_rate_by_group(model_df, "cluster", "proposal_success", "Proposal Funding Rate by Cluster")

# =====
# 5) MODEL EVAL (AUC + PR-AUC) with guardrails
# =====
def eval_logreg(X, y, label, n_splits=5):
    y = pd.Series(y).astype(int)

    if y.nunique() < 2:
        print(f"\n[{label}] Skipping: single-class outcome.")
        return

    # if minority class too small, reduce folds so StratifiedKFold doesn't implode
    class_counts = y.value_counts().to_dict()
    min_class = min(class_counts.values())
    use_splits = min(n_splits, max(2, min_class)) # at least 2 folds
    print(f"\n[{label}] class counts: {class_counts} | using n_splits={use_splits}")

    cv = StratifiedKFold(n_splits=use_splits, shuffle=True, random_state=42)

    pipe = Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(
            max_iter=5000,
            class_weight="balanced",
            solver="lbfgs"
        ))
    ])

    aucs = cross_val_score(pipe, X, y, cv=cv, scoring="roc_auc")
    pr_aucs = cross_val_score(pipe, X, y, cv=cv, scoring="average_precision")

```

```

print(f"[{label}] AUCs:", np.round(aucs, 3))
print(f"[{label}] Mean AUC: {aucs.mean():.3f} ± {aucs.std():.3f}")
print(f"[{label}] PR-AUC mean: {pr_auc.mean():.3f} ± {pr_auc.std():.3f}")

eval_logreg(X, y_team, "TEAM FORMATION")
eval_logreg(X, y_prop, "PROPOSAL SUCCESS (FUNDED)")

# =====
# 6) FIT FULL MODEL + COEFFICIENT TABLE (interpretability)
# =====
def fit_and_show_coefs(X, y, label, top_k=12):
    y = pd.Series(y).astype(int)
    if y.nunique() < 2:
        print(f"\n[{label}] Skipping coef table: single-class outcome.")
        return

    pipe_full = Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(
            max_iter=5000,
            class_weight="balanced",
            solver="lbfgs"
        ))
    ])
    pipe_full.fit(X, y)

    coef = pipe_full.named_steps["clf"].coef_.ravel()
    coef_df = (
        pd.DataFrame({"feature": X.columns, "coef": coef})
        .sort_values("coef", ascending=False)
        .reset_index(drop=True)
    )

    print(f"\n[{label}] Top positive predictors:")
    display(coef_df.head(top_k))

    print(f"\n[{label}] Top negative predictors:")
    display(coef_df.tail(top_k))

```

```
fit_and_show_coefs(X, y_prop, "PROPOSAL SUCCESS (FUNDED)")
```

Found 8 session outcome JSON files.

- /Users/maxchalekson/Northwestern University/Summer-2025/NIC0/NIC0 Research/gemini_data_analysis/data/2020NES/2020NES_session_outcomes.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NIC0/NIC0 Research/gemini_data_analysis/data/2021ABI/2021ABI_session_outcomes.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NIC0/NIC0 Research/gemini_data_analysis/data/2021CMC/2021CMC_session_outcomes.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NIC0/NIC0 Research/gemini_data_analysis/data/2021MND/2021MND_session_outcomes.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NIC0/NIC0 Research/gemini_data_analysis/data/2021MZT/2021MZT_session_outcomes.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NIC0/NIC0 Research/gemini_data_analysis/data/2021NES/2021NES_session_outcomes.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NIC0/NIC0 Research/gemini_data_analysis/data/2021SLU/2021SLU_session_outcomes.json
- /Users/maxchalekson/Northwestern University/Summer-2025/NIC0/NIC0 Research/gemini_data_analysis/data/2022MND/2022MND_session_outcomes.json

session_outcomes_df shape: (157, 13)

	session_id	year	track	team_num	n_speakers	n_facilitators	n_teams_total	n_teams_in_session	
0	2020_11_05_NES_S1	2020	NES	1	12	0	1	1	
1	2020_11_06_NES_S7	2020	NES	7	12	0	2	2	
2	2020_11_06_NES_S10	2020	NES	10	11	1	0	0	
3	2020_11_05_NES_S6	2020	NES	6	9	1	2	2	
4	2020_11_06_NES_S11	2020	NES	11	10	0	3	3	


```
Outcome base rates:
team_formed      0.783439
proposal_success  0.433121
dtype: float64
```

```
Outcome class counts:
team_formed:
  team_formed
1      123
0       34
Name: count, dtype: int64
```

```
proposal_success:
  proposal_success
0       89
1       68
Name: count, dtype: int64
```

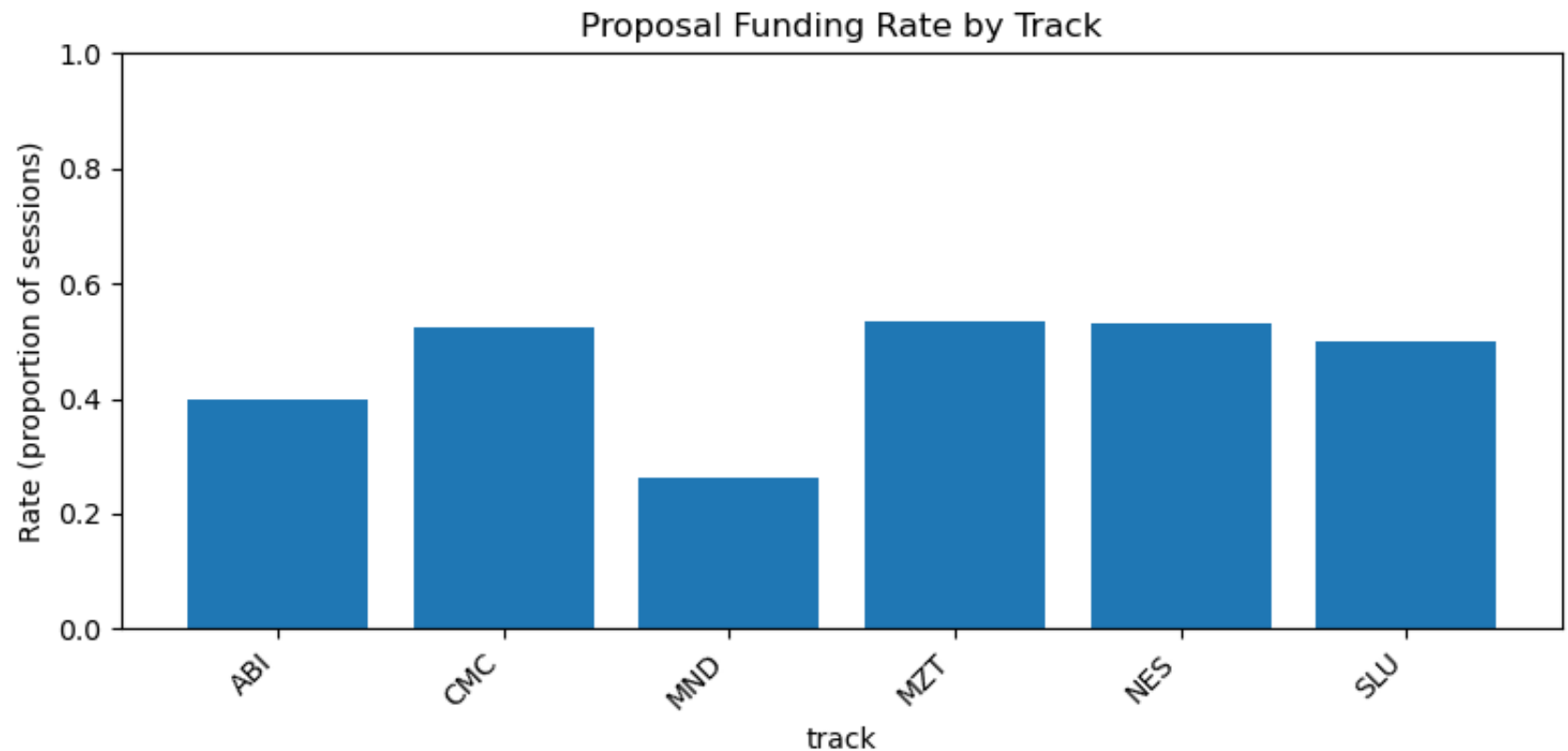
	session_id	year	track	team_num	n_speakers	n_facilitators	n_teams_total	n_teams_in_session	n
0	2020_11_05_NES_S1	2020	NES	1	12	0	1	1	
1	2020_11_05_NES_S2	2020	NES	2	11	0	3	3	
2	2020_11_05_NES_S3	2020	NES	3	10	0	3	3	
3	2020_11_05_NES_S4	2020	NES	4	9	1	1	1	
4	2020_11_05_NES_S5	2020	NES	5	11	0	1	1	

Merged model_df shape: (157, 51)

Unique sessions: 157

X shape: (157, 18)

X columns: ['early_seeding', 'early_commitment', 'late_synthesis', 'late_closure', 'delta_synthesis', 'delta_closure', 'delta_stress_mid', 'jsd_next_overall', 'prop_facilitator_utterances', 'cluster_1', 'cluster_2', 'cluster_3', 'track_CMC', 'track_MND', 'track_MZT', 'track_NES', 'track_SLU', 'year']





[TEAM FORMATION] class counts: {1: 123, 0: 34} | using n_splits=5
[TEAM FORMATION] AUCs: [0.377 0.72 0.447 0.357 0.393]
[TEAM FORMATION] Mean AUC: 0.459 ± 0.134
[TEAM FORMATION] PR-AUC mean: 0.775 ± 0.066

[PROPOSAL SUCCESS (FUNDED)] class counts: {0: 89, 1: 68} | using n_splits=5
[PROPOSAL SUCCESS (FUNDED)] AUCs: [0.829 0.687 0.645 0.568 0.542]
[PROPOSAL SUCCESS (FUNDED)] Mean AUC: 0.654 ± 0.102
[PROPOSAL SUCCESS (FUNDED)] PR-AUC mean: 0.630 ± 0.104

[PROPOSAL SUCCESS (FUNDED)] Top positive predictors:

	feature	coef
0	cluster_1	0.473611
1	late_closure	0.406888
2	prop_facilitator_utterances	0.397052
3	delta_closure	0.344290
4	delta_stress_mid	0.298072
5	track_NES	0.185922
6	early_seeding	0.175052
7	track_SLU	0.169315
8	track_MZT	0.165112
9	track_CMC	0.064440
10	late_synthesis	0.045638
11	cluster_3	0.012874

[PROPOSAL SUCCESS (FUNDED)] Top negative predictors:

	feature	coef
6	early_seeding	0.175052
7	track_SLU	0.169315
8	track_MZT	0.165112
9	track_CMC	0.064440
10	late_synthesis	0.045638
11	cluster_3	0.012874
12	track_MND	-0.053379
13	early_commitment	-0.074851
14	cluster_2	-0.198709
15	year	-0.235025
16	delta_synthesis	-0.374390
17	jsd_next_overall	-0.404093

I THINK THESE NEXT FEW CODE CHUNKS ARE THE CULMINATION OF EVERYTHING

```
In [23]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import StratifiedKFold
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    roc_auc_score, average_precision_score,
```

```

        confusion_matrix, precision_recall_curve
    )

# =====
# 7) CROSS-VALIDATED PRED PROBS PER SESSION (NO LEAKY EGO)
# =====
def cv_predict_proba(X, y, n_splits=5, random_state=42):
    y = pd.Series(y).astype(int)
    if y.nunique() < 2:
        return None

    counts = y.value_counts().to_dict()
    min_class = min(counts.values())
    use_splits = min(n_splits, max(2, min_class))

    cv = StratifiedKFold(n_splits=use_splits, shuffle=True, random_state=random_state)

    pipe = Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(
            max_iter=5000,
            class_weight="balanced",
            solver="lbfgs"
        ))
    ])

    proba = np.zeros(len(y), dtype=float)

    for train_idx, test_idx in cv.split(X, y):
        pipe.fit(X.iloc[train_idx], y.iloc[train_idx])
        proba[test_idx] = pipe.predict_proba(X.iloc[test_idx])[:, 1]

    return proba

# Funding predictions
fund_proba = cv_predict_proba(X, model_df["proposal_success"])
if fund_proba is None:
    print("Funding: single-class outcome, cannot compute CV probabilities.")
else:

```

```

model_df = model_df.copy()
model_df["p_funded_cv"] = fund_proba

y_true = model_df["proposal_success"].astype(int).values
auc = roc_auc_score(y_true, fund_proba)
prauc = average_precision_score(y_true, fund_proba)
print(f"Funding CV metrics: AUC={auc:.3f} | PR-AUC={prauc:.3f}")

# Team formation predictions (only if non-single-class)
team_proba = cv_predict_proba(X, model_df["team_formed"])
if team_proba is not None:
    model_df["p_teamformed_cv"] = team_proba

# =====
# 8) CALIBRATION / RELIABILITY PLOT (FUNDING)
# =====
def reliability_plot(y_true, p, title, n_bins=10):
    df = pd.DataFrame({"y": y_true, "p": p}).dropna()
    df["bin"] = pd.qcut(df["p"], q=n_bins, duplicates="drop")
    calib = df.groupby("bin").agg(
        mean_p=("p", "mean"),
        frac_pos=("y", "mean"),
        n=("y", "size")
    ).reset_index()

    plt.figure(figsize=(5, 5))
    plt.plot([0, 1], [0, 1])
    plt.scatter(calib["mean_p"], calib["frac_pos"])
    plt.xlabel("Mean predicted probability")
    plt.ylabel("Empirical positive rate")
    plt.title(title)
    plt.tight_layout()
    plt.show()

    return calib

if "p_funded_cv" in model_df.columns:
    calib = reliability_plot(
        model_df["proposal_success"].astype(int),

```

```

        model_df["p_funded_cv"],
        "Funding: Reliability (CV)"
    )
    display(calib)

# =====
# 9) PICK A THRESHOLD (PR CURVE) + CONFUSION MATRIX
# =====
def pick_threshold_by_f1(y_true, p):
    prec, rec, thresh = precision_recall_curve(y_true, p)
    # thresh length is (len(prec)-1)
    f1 = 2 * (prec[:-1] * rec[:-1]) / (prec[:-1] + rec[:-1] + 1e-12)
    best_i = np.argmax(f1)
    return thresh[best_i], prec[best_i], rec[best_i], f1[best_i]

if "p_funded_cv" in model_df.columns:
    y_true = model_df["proposal_success"].astype(int).values
    p = model_df["p_funded_cv"].values

    t, best_p, best_r, best_f1 = pick_threshold_by_f1(y_true, p)
    print(f"Best F1 threshold (CV): t={t:.3f} | precision={best_p:.3f} | recall={best_r:.3f} | f1={best_f1:.3f}")

    y_hat = (p >= t).astype(int)
    cm = confusion_matrix(y_true, y_hat)
    print("Confusion matrix [[TN, FP],[FN, TP]]:\n", cm)

# =====
# 10) WHERE DOES IT WORK / FAIL? (GROUPED DIAGNOSTICS)
# =====
def grouped_metrics(df, group_col, y_col, p_col):
    if group_col not in df.columns:
        return None
    out = []
    for g, sub in df.groupby(group_col):
        y = sub[y_col].astype(int).values
        p = sub[p_col].values
        if len(np.unique(y)) < 2:
            out.append({"group": g, "n": len(sub), "auc": np.nan, "prauc": np.nan, "base_rate": y.mean()})
        else:

```



```

        out.append({"group": g, "n": len(sub), "auc": roc_auc_score(y, p), "prauc": average_pre
return pd.DataFrame(out).sort_values("n", ascending=False)

if "p_funded_cv" in model_df.columns:
    print("\nFunding metrics by track:")
    display(grouped_metrics(model_df, "track", "proposal_success", "p_funded_cv"))

    if "cluster" in model_df.columns:
        print("\nFunding metrics by cluster:")
        display(grouped_metrics(model_df, "cluster", "proposal_success", "p_funded_cv"))

    if "year" in model_df.columns:
        print("\nFunding metrics by year:")
        display(grouped_metrics(model_df, "year", "proposal_success", "p_funded_cv"))

# =====
# 11) SAVE A "REVIEW TABLE" OF HARD CASES
# =====
# This is the bridge back to qualitative inspection: look at false positives / false negatives.
if "p_funded_cv" in model_df.columns:
    df_review = model_df[[
        "session_id", "track", "year", "cluster",
        "proposal_success", "p_funded_cv",
        "team_formed", "prop_facilitator_utterances"
    ]].copy()

    df_review["error"] = np.abs(df_review["proposal_success"] - (df_review["p_funded_cv"] >= 0.5)).a
    df_review = df_review.sort_values(["error", "p_funded_cv"], ascending=[False, False])

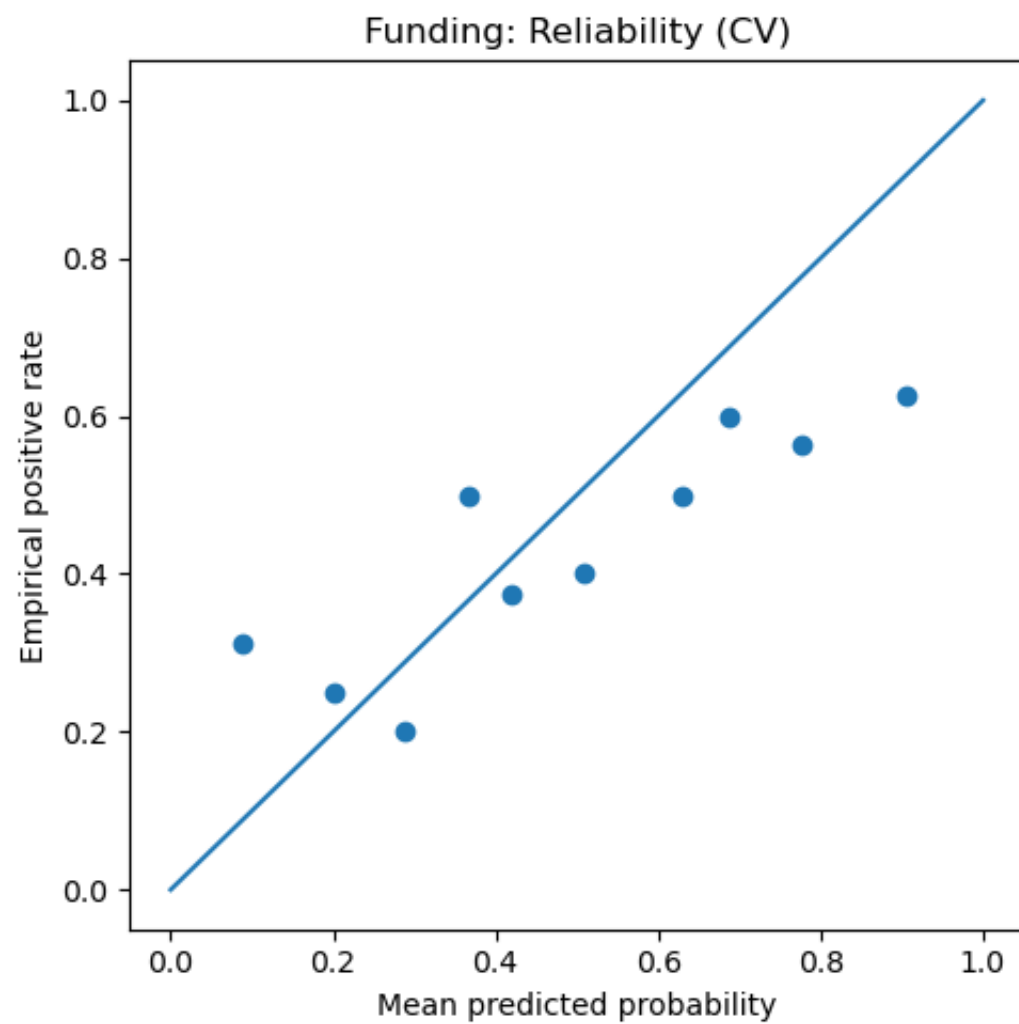
    print("\nTop 25 cases to inspect (biggest errors / highest confidence):")
    display(df_review.head(25))

```

Funding CV metrics: AUC=0.646 | PR-AUC=0.572

/var/folders/wp/cn7__9416yj3c5_vcykys8hc0000gn/T/ipykernel_60716/390545000.py:69: FutureWarning: The default of observed=False is deprecated and will be changed to True in a future version of pandas. Pass observed=False to retain current behavior or observed=True to adopt the future default and silence this warning.

```
calib = df.groupby("bin").agg(
```



	bin	mean_p	frac_pos	n
0	(0.00042, 0.169]	0.089165	0.3125	16
1	(0.169, 0.232]	0.201281	0.2500	16
2	(0.232, 0.331]	0.286966	0.2000	15
3	(0.331, 0.395]	0.367256	0.5000	16
4	(0.395, 0.469]	0.418378	0.3750	16
5	(0.469, 0.598]	0.508271	0.4000	15
6	(0.598, 0.656]	0.628499	0.5000	16
7	(0.656, 0.73]	0.688106	0.6000	15
8	(0.73, 0.824]	0.775840	0.5625	16
9	(0.824, 1.0]	0.906645	0.6250	16

Best F1 threshold (CV): t=0.334 | precision=0.514 | recall=0.824 | f1=0.633

Confusion matrix [[TN, FP],[FN, TP]]:

[[36 53]

[12 56]]

Funding metrics by track:

	group	n	auc	prauc	base_rate
2	MND	42	0.486804	0.280962	0.261905
4	NES	34	0.704861	0.778521	0.529412
0	ABI	25	0.633333	0.577501	0.400000
1	CMC	21	0.727273	0.778571	0.523810
5	SLU	20	0.540000	0.608549	0.500000
3	MZT	15	0.571429	0.581022	0.533333

Funding metrics by cluster:

	group	n	auc	prauc	base_rate
0	0	71	0.532609	0.365703	0.323944
1	1	65	0.584795	0.640166	0.584615
2	2	16	0.384615	0.212500	0.187500
3	3	5	0.250000	0.804167	0.800000

Funding metrics by year:

	group	n	auc	prauc	base_rate
1	2021	123	0.655663	0.583945	0.439024
2	2022	18	0.375000	0.303973	0.333333
0	2020	16	0.734375	0.808333	0.500000

Top 25 cases to inspect (biggest errors / highest confidence):

	session_id	track	year	cluster	proposal_success	p_funded_cv	team_formed	prop_facilitator_u
147	2022_04_08_MND_S12	MND	2022	3	0	0.996794	0	
97	2021_10_01_MZT_S8	MZT	2021	1	0	0.944502	1	
59	2021_05_21_ABI_S5	ABI	2021	1	0	0.900494	0	
130	2021_11_05_NES_S2	NES	2021	1	0	0.892249	1	
92	2021_10_01_MZT_S2	MZT	2021	1	0	0.879218	1	
70	2021_06_10_SLU_S7	SLU	2021	1	0	0.836086	0	
29	2021_04_23_MND_S14	MND	2021	0	0	0.812520	0	
37	2021_04_23_MND_S7	MND	2021	1	0	0.803345	1	
85	2021_09_30_MZT_S3	MZT	2021	1	0	0.783783	1	
119	2021_10_08_CMC_S9	CMC	2021	0	0	0.772768	1	
66	2021_06_10_SLU_S3	SLU	2021	1	0	0.757595	1	

24	2021_04_23_MND_S1	MND	2021	0	0	0.756659	1
49	2021_05_21_ABI_S10	ABI	2021	1	0	0.734601	0
118	2021_10_08_CMC_S8	CMC	2021	1	0	0.725687	1
135	2021_11_05_NES_S7	NES	2021	0	0	0.713316	1
131	2021_11_05_NES_S3	NES	2021	1	0	0.683976	1
98	2021_10_01_MZT_S9	MZT	2021	0	0	0.677463	0
129	2021_11_05_NES_S12	NES	2021	0	0	0.666100	0
72	2021_06_11_SLU_S10	SLU	2021	0	0	0.661082	1
38	2021_04_23_MND_S8	MND	2021	1	0	0.655812	0
121	2021_11_04_NES_S2	NES	2021	2	0	0.655041	1
136	2021_11_05_NES_S8	NES	2021	0	0	0.654996	0
105	2021_10_07_CMC_S7	CMC	2021	1	0	0.629131	0
137	2021_11_05_NES_S9	NES	2021	0	0	0.617241	1
48	2021_05_21_ABI_S1	ABI	2021	1	0	0.609266	0

```
In [26]: import os, json, glob, re
import numpy as np
import pandas as pd

# =====
# 0) REQUIREMENTS: need review_cases or model_df to pick sessions
# =====
if "review_cases" in globals():
    target_session_ids = sorted(review_cases["session_id"].unique())
    meta_df = review_cases.copy()
elif "model_df" in globals():
    target_session_ids = sorted(model_df["session_id"].unique())
    meta_df = model_df[["session_id"]].copy()
```

```

else:
    raise NameError("Need either review_cases or model_df in memory to know which sessions to load.")

DATA_ROOT = "/Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data

# =====
# 1) INDEX session_data JSONs: session_id -> file path
# =====
session_json_paths = glob.glob(os.path.join(DATA_ROOT, "*", "session_data", "*.json"))
if not session_json_paths:
    raise FileNotFoundError(f"No session_data JSONs found under: {DATA_ROOT}/*/session_data/*.json")

session_path_map = {os.path.splitext(os.path.basename(p))[0]: p for p in session_json_paths}

missing_paths = [sid for sid in target_session_ids if sid not in session_path_map]
if missing_paths:
    print(f"Warning: {len(missing_paths)} target sessions not found under session_data/ (showing up
    print(missing_paths[:15])

print(f"Session JSON index built: {len(session_path_map)} files")
print(f"Target sessions requested: {len(target_session_ids)} | found paths for: {len(target_session

# =====
# 2) HELPERS
# =====
def _get_utterances(payload: dict):
    """
    Your example session json uses 'all_data' as the utterance list.
    This tries a bunch of known keys + light recursion.
    """
    for key in ["all_data", "utterances", "utterance_list", "transcript", "segments", "turns"]:
        if key in payload and isinstance(payload[key], list):
            return payload[key]

    # sometimes nested
    for key in ["data", "session", "content", "payload"]:
        if key in payload and isinstance(payload[key], dict):
            out = _get_utterances(payload[key])
            if out is not None:

```

```

        return out
    return None

def _parse_time_to_sec(x):
    """
    Handles numeric seconds OR strings like:
    - 'HH:MM:SS'
    - 'MM:SS'
    - 'SS'
    """
    if x is None:
        return None
    if isinstance(x, (int, float)) and np.isfinite(x):
        return float(x)

    s = str(x).strip()
    if not s:
        return None

    # If it's like "12.34"
    try:
        return float(s)
    except Exception:
        pass

    # If it's time-like "HH:MM:SS" or "MM:SS"
    parts = s.split(":")
    try:
        parts = [float(p) for p in parts]
    except Exception:
        return None

    if len(parts) == 3:
        h, m, sec = parts
        return 3600*h + 60*m + sec
    if len(parts) == 2:
        m, sec = parts
        return 60*m + sec
    if len(parts) == 1:

```

```

        return parts[0]
    return None

def _get_time(u: dict):
    """
    Try common timing fields in your session_data json.
    Your example has start_time/end_time/timestamp as strings.
    """
    for k in ["global_timestamp_sec", "start_sec_adjusted", "start_sec", "start_time", "timestamp",
              ]:
        if k in u:
            t = _parse_time_to_sec(u.get(k))
            if t is not None:
                return t
    return None

def _get_text(u: dict):
    for tk in ["transcript", "text", "utterance", "content"]:
        v = u.get(tk)
        if isinstance(v, str):
            return v
    return None

def _extract_labels_and_phase(u: dict):
    """
    Your example: u["annotations"] is a dict:
    { "<LABEL>": {"explanation": "...", "when": "beginning"} , ... }
    Also supports list-style annotations.
    Returns: (labels:list[str], phase:str|None)
    """
    ann = u.get("annotations", None)
    phase = None
    labels = []

    if ann is None:
        return labels, phase

    # dict-of-labels
    if isinstance(ann, dict):
        for label, info in ann.items():

```



```

        if label:
            labels.append(str(label))
        if phase is None and isinstance(info, dict) and "when" in info:
            phase = info.get("when")
    return labels, phase

# list-style annotations
if isinstance(ann, list):
    for item in ann:
        if isinstance(item, str):
            labels.append(item)
        elif isinstance(item, dict):
            # try label-ish keys
            for k in ["label", "code", "name", "annotation"]:
                if k in item and item[k]:
                    labels.append(str(item[k]))
                    break
            if phase is None and "when" in item:
                phase = item.get("when")
    return labels, phase

# string
if isinstance(ann, str):
    return [ann], phase

return labels, phase

def _normalize_phase(p):
    if p is None:
        return None
    s = str(p).strip().lower()
    if "begin" in s or "early" in s:
        return "beginning"
    if "mid" in s:
        return "middle"
    if "end" in s or "late" in s:
        return "end"
    return None

```

```

# OPTIONAL: map raw annotation labels -> your 4 idea-state buckets.
# If you don't fill this, we'll just use the raw labels.
LABEL_TO_IDEASTATE = {
    # "Some label name": "Seeding",
    # "Some other label": "Stress-testing",
    # ...
}

def label_to_state(lbl: str):
    return LABEL_TO_IDEASTATE.get(lbl, None)

# =====
# 3) LOAD UTTERANCES + (MULTI-)ANNOTATIONS INTO LONG DF
# =====

rows = []
loaded = 0
skipped = 0

for sid in target_session_ids:
    p = session_path_map.get(sid)
    if not p:
        skipped += 1
        continue

    try:
        with open(p, "r") as f:
            payload = json.load(f)
    except Exception as e:
        print(f"Failed to load {sid}: {e}")
        skipped += 1
        continue

    utts = _get_utterances(payload)
    if not utts:
        # debug once in a while if you want:
        # print("No utterances found in:", sid, "top-level keys:", list(payload.keys())[:20])
        skipped += 1
        continue

    for i, u in enumerate(utts):

```

```

t = _get_time(u)
text = _get_text(u)
labels, phase = _extract_labels_and_phase(u)
phase = _normalize_phase(phase)

# If there are no labels, keep one row anyway (so timing exists)
if not labels:
    rows.append({
        "session_id": sid,
        "utt_idx": i,
        "t": t,
        "phase": phase,
        "label": None,
        "idea_state": None,
        "transcript": text,
        "source_path": p,
    })
else:
    for lbl in labels:
        rows.append({
            "session_id": sid,
            "utt_idx": i,
            "t": t,
            "phase": phase,
            "label": lbl,
            "idea_state": label_to_state(lbl),
            "transcript": text,
            "source_path": p,
        })

loaded += 1

U = pd.DataFrame(rows)

print(f"Loaded sessions: {loaded} | skipped (missing/unreadable/no-utterances): {skipped}")
print("Utterance rows:", U.shape)
if U.empty:
    raise ValueError(
        "Built 0 utterance rows. This means none of the target sessions had a recognizable utterance"
    )

```

```

        "Fix: print top-level keys from one session json and add its utterance key to _get_utterance
    )

# Fill time if missing
if U["t"].isna().all():
    print("Warning: No usable time field found. Falling back to utt_idx ordering as time.")
    U["t"] = U["utt_idx"].astype(float)
else:
    U["t"] = U["t"].fillna(U["utt_idx"].astype(float))

# =====
# 4) BUILD FLOW FEATURES
# Prefer phase=beginning/middle/end if present; else time-bin.
# =====
use_phase = U["phase"].notna().any()
print("Using phase-based flow?" , use_phase)

if use_phase:
    # Use idea_state if you mapped labels -> states; else use raw labels
    flow_key = "idea_state" if U["idea_state"].notna().any() else "label"

    tmp = U.copy()
    tmp[flow_key] = tmp[flow_key].fillna("UNLABELED")

    flow = (
        tmp.groupby(["session_id", "phase", flow_key])
        .size()
        .reset_index(name="n")
    )
    flow["phase_total"] = flow.groupby(["session_id", "phase"])["n"].transform("sum")
    flow["prop"] = flow["n"] / flow["phase_total"]

    flow_wide = (
        flow.pivot_table(index=["session_id", "phase"], columns=flow_key, values="prop", fill_value=0)
        .reset_index()
    )

    # turn phase into prefix columns: beginning_X, middle_X, end_X
    feat_frames = []

```

```

for ph in ["beginning", "middle", "end"]:
    ph_df = flow_wide[flow_wide["phase"] == ph].drop(columns=["phase"], errors="ignore").set_index("utt_idx")
    ph_df = ph_df.add_prefix(f"{ph}_")
    feat_frames.append(ph_df)

flow_feats = pd.concat(feat_frames, axis=1).reset_index()

else:
    # fallback: bin by time within session (quantiles)
    N_BINS = 5
    tmp = U.copy()
    flow_key = "idea_state" if tmp["idea_state"].notna().any() else "label"
    tmp[flow_key] = tmp[flow_key].fillna("UNLABELED")

    def add_time_bin(df):
        t = df["t"].astype(float).values
        if len(t) == 0:
            df["time_bin"] = np.nan
            return df
        qs = np.quantile(t, np.linspace(0, 1, N_BINS + 1))
        qs = np.unique(qs)
        if len(qs) <= 2:
            df["time_bin"] = 0
            return df
        # cut into bins
        df["time_bin"] = pd.cut(df["t"], bins=qs, include_lowest=True, labels=False)
        return df

    tmp = tmp.sort_values(["session_id", "t", "utt_idx"])
    tmp = tmp.groupby("session_id", group_keys=False).apply(add_time_bin)

    flow = (
        tmp.groupby(["session_id", "time_bin", flow_key])
        .size()
        .reset_index(name="n")
    )
    flow["bin_total"] = flow.groupby(["session_id", "time_bin"])["n"].transform("sum")
    flow["prop"] = flow["n"] / flow["bin_total"]

```

```

flow_wide = (
    flow.pivot_table(index=["session_id", "time_bin"], columns=flow_key, values="prop", fill_val=0)
    .reset_index()
)

EARLY_BINS = [0, 1]
LATE_BINS = [flow_wide["time_bin"].max() - 1, flow_wide["time_bin"].max()]

early = flow_wide[flow_wide["time_bin"].isin(EARLY_BINS)].groupby("session_id").mean(numeric_only=True)
late = flow_wide[flow_wide["time_bin"].isin(LATE_BINS)].groupby("session_id").mean(numeric_only=True)

flow_feats = early.join(late, how="outer").reset_index()

print("flow_feats shape:", flow_feats.shape)
display(flow_feats.head())

# =====
# 5) JOIN FLOW FEATURES BACK TO META (review_cases or model_df)
# =====
meta_plus = meta_df.merge(flow_feats, on="session_id", how="left")
print("meta_plus shape:", meta_plus.shape)
display(meta_plus.head(10))

# meta_plus is now what you inspect for FP/FN sessions:
# - if review_cases: you can compare FP vs FN flow features
# - then add selected flow columns into X for a next modeling pass

```

```

Session JSON index built: 157 files
Target sessions requested: 20 | found paths for: 20
Loaded sessions: 20 | skipped (missing/unreadable/no-utterances): 0
Utterance rows: (5559, 8)
Using phase-based flow? True
flow_feats shape: (20, 28)

```

label	session_id	beginning_Coordination and Decision Practices	beginning_Evaluation Practices	beginning_Idea Management	beginning_Information Seeking
0	2020_11_06_NES_S12	0.023256	0.093023	0.209302	0.162791
1	2020_11_06_NES_S3	0.000000	0.133333	0.300000	0.033333
2	2021_04_22_MND_S4	0.091603	0.022901	0.068702	0.083969
3	2021_04_23_MND_S11	0.140625	0.062500	0.203125	0.187500
4	2021_04_23_MND_S14	0.080000	0.026667	0.173333	0.160000

5 rows x 28 columns

meta_plus shape: (20, 83)

	session_id	beginning__Seeding	beginning__Stress-testing	beginning__Commitment/Closure	middle__Seec
0	2022_04_08_MND_S12	0.750000	0.000000	0.000000	0.000
1	2021_10_01_MZT_S8	0.382353	0.058824	0.205882	0.550
2	2021_05_21_ABI_S5	0.337500	0.087500	0.262500	0.336
3	2021_11_05_NES_S2	0.500000	0.096154	0.211538	0.305
4	2021_10_01_MZT_S2	0.380952	0.047619	0.190476	0.315
5	2021_06_10_SLU_S7	0.348315	0.033708	0.303371	0.258
6	2021_04_23_MND_S14	0.555556	0.044444	0.133333	0.628
7	2021_04_23_MND_S7	0.481481	0.074074	0.148148	0.333
8	2021_09_30_MZT_S3	0.375000	0.166667	0.291667	0.527
9	2021_10_08_CMC_S9	0.438202	0.056180	0.179775	0.500

10 rows x 83 columns

```
In [27]: import numpy as np
import pandas as pd

# =====
# 0) REQUIRE: meta_plus must exist (from the previous chunk)
# =====
if "meta_plus" not in globals():
    raise NameError("meta_plus not found. Run the previous chunk first.")

df = meta_plus.copy()

# =====
# 1) REQUIRE: we need err_type + outcome columns
# =====
```



```

needed = {"err_type", "proposal_success", "p_funded_cv"}
missing = needed - set(df.columns)
if missing:
    raise ValueError(f"meta_plus missing columns: {missing}. Make sure meta_df was review_cases (ha

# =====
# 2) Pick which feature family you want to analyze
#   A) idea-state flow columns: they have a double underscore 'beginning__'
#   B) practice-label flow columns: they look like 'beginning_Knowledge Sharing'
# =====

idea_cols = [c for c in df.columns if c.startswith(("beginning__", "middle__", "end__"))]
label_cols = [c for c in df.columns if c.startswith(("beginning_", "middle_", "end_")) and "__" not

print(f"Idea-state columns: {len(idea_cols)}")
print(f"Label-flow columns: {len(label_cols)}")

# choose one set (you can swap to label_cols if you want that view)
FLOW_COLS = label_cols # <-- change to idea_cols if you want the 4-state flow

# =====
# 3) Group summary: FP vs FN phase profiles
# =====
grp = (
    df.groupby("err_type")[FLOW_COLS]
      .mean(numeric_only=True)
      .T
      .rename_axis("feature")
      .reset_index()
)

# Pull out FP and FN means (if one type is missing, fill with 0)
fp = grp["FP"] if "FP" in grp.columns else 0.0
fn = grp["FN"] if "FN" in grp.columns else 0.0
grp["FP_minus_FN"] = fp - fn
grp["abs_delta"] = grp["FP_minus_FN"].abs()

# Big drivers
top = grp.sort_values("abs_delta", ascending=False).head(20)

```

```

print("\nTop 20 FP vs FN flow differences (higher = more FP-heavy):")
display(top)

# =====
# 4) For quick sanity: show the sessions with strongest FP confidence
# =====
fp_cases = df[df["err_type"] == "FP"].sort_values("p_funded_cv", ascending=False).head(10)
fn_cases = df[df["err_type"] == "FN"].sort_values("p_funded_cv", ascending=True).head(10)

print("\nTop FP (model most confident but actually not funded):")
display(fp_cases[["session_id", "track", "year", "cluster", "proposal_success", "p_funded_cv", "confidence"]])

print("\nTop FN (model least confident but actually funded):")
display(fn_cases[["session_id", "track", "year", "cluster", "proposal_success", "p_funded_cv", "confidence"]])

# =====
# 5) Optional: correlation with predicted probability (within review set)
# =====
corrs = (
    df[FLOW_COLS + ["p_funded_cv"]]
    .corr(numeric_only=True)["p_funded_cv"]
    .drop("p_funded_cv")
    .sort_values(ascending=False)
)

print("\nTop + correlations with p_funded_cv (review set only):")
display(corrs.head(15).reset_index().rename(columns={"index": "feature", "p_funded_cv": "corr"}))

print("\nTop - correlations with p_funded_cv (review set only):")
display(corrs.tail(15).reset_index().rename(columns={"index": "feature", "p_funded_cv": "corr"}))

```

Idea-state columns: 24

Label-flow columns: 27

Top 20 FP vs FN flow differences (higher = more FP-heavy):

err_type	feature	FN	FP	FP_minus_FN	abs_delta
18	end_Coordination and Decision Practices	0.122004	0.211637	0.089633	0.089633
14	middle_Knowledge Sharing	0.318041	0.233188	-0.084853	0.084853
23	end_Knowledge Sharing	0.257028	0.185115	-0.071913	0.071913
17	middle_Relational Climate	0.143744	0.202056	0.058312	0.058312
2	beginning_Idea Management	0.131653	0.085043	-0.046610	0.046610
10	middle_Evaluation Practices	0.062873	0.103392	0.040519	0.040519
9	middle_Coordination and Decision Practices	0.065161	0.034222	-0.030940	0.030940
0	beginning_Coordination and Decision Practices	0.087515	0.117389	0.029874	0.029874
6	beginning_None	0.048816	0.020981	-0.027834	0.027834
12	middle_Information Seeking	0.157020	0.130154	-0.026866	0.026866
26	end_Relational Climate	0.179768	0.152944	-0.026825	0.026825
16	middle_Participation Dynamics	0.044333	0.068486	0.024153	0.024153
8	beginning_Relational Climate	0.204496	0.227501	0.023005	0.023005
7	beginning_Participation Dynamics	0.089685	0.108535	0.018851	0.018851
3	beginning_Information Seeking	0.113768	0.131682	0.017915	0.017915
13	middle_Integration Practices	0.016362	0.031548	0.015185	0.015185
5	beginning_Knowledge Sharing	0.269907	0.257710	-0.012197	0.012197
22	end_Integration Practices	0.047995	0.038667	-0.009328	0.009328
15	middle_None	0.021329	0.029997	0.008668	0.008668
20	end_Idea Management	0.126173	0.133868	0.007695	0.007695

Top FP (model most confident but actually not funded):

	session_id	track	year	cluster	proposal_success	p_funded_cv	confidence	beginning_Coordinatio and Decision Practic
0	2022_04_08_MND_S12	MND	2022	3	0	0.996794	0.662794	0.00000
1	2021_10_01_MZT_S8	MZT	2021	1	0	0.944502	0.610502	0.15053
2	2021_05_21_ABI_S5	ABI	2021	1	0	0.900494	0.566494	0.1567
3	2021_11_05_NES_S2	NES	2021	1	0	0.892249	0.558249	0.10416
4	2021_10_01_MZT_S2	MZT	2021	1	0	0.879218	0.545218	0.12307
5	2021_06_10_SLU_S7	SLU	2021	1	0	0.836086	0.502086	0.20769
6	2021_04_23_MND_S14	MND	2021	0	0	0.812520	0.478520	0.08000
7	2021_04_23_MND_S7	MND	2021	1	0	0.803345	0.469345	0.10000
8	2021_09_30_MZT_S3	MZT	2021	1	0	0.783783	0.449783	0.14281
9	2021_10_08_CMC_S9	CMC	2021	0	0	0.772768	0.438768	0.10884

Top FN (model least confident but actually funded):

	session_id	track	year	cluster	proposal_success	p_funded_cv	confidence	beginning_Coordinati and Decision Practic
10	2020_11_06_NES_S12	NES	2020	2	1	0.018621	0.315379	0.0232
11	2021_04_23_MND_S11	MND	2021	0	1	0.074936	0.259064	0.1406
12	2022_04_07_MND_S6	MND	2022	0	1	0.125388	0.208612	0.0894
13	2020_11_06_NES_S3	NES	2020	2	1	0.138559	0.195441	0.0000
14	2021_04_22_MND_S4	MND	2021	1	1	0.143854	0.190146	0.0916
15	2022_04_07_MND_S4	MND	2022	0	1	0.183988	0.150012	0.1268
16	2021_09_30_MZT_S6	MZT	2021	1	1	0.199107	0.134893	0.0588
17	2021_05_20_ABI_S7	ABI	2021	0	1	0.222979	0.111021	0.1551
18	2022_04_07_MND_S5	MND	2022	1	1	0.229674	0.104326	0.0847
19	2022_04_07_MND_S1	MND	2022	0	1	0.272132	0.061868	0.1052

Top + correlations with p_funded_cv (review set only):

	feature	corr
0	end_Coordination and Decision Practices	0.485748
1	middle_Evaluation Practices	0.385648
2	beginning_Coordination and Decision Practices	0.281689
3	middle_Relational Climate	0.260698
4	middle_Participation Dynamics	0.256794
5	middle_Integration Practices	0.198179
6	beginning_Relational Climate	0.196845
7	beginning_Participation Dynamics	0.189896
8	middle_None	0.171679
9	beginning_Information Seeking	0.168617
10	end_Information Seeking	0.085899
11	end_Participation Dynamics	0.026758
12	end_Idea Management	0.023751
13	end_Evaluation Practices	-0.018077
14	end_None	-0.033951

Top – correlations with p_funded_cv (review set only):

	feature	corr
0	end_Idea Management	0.023751
1	end_Evaluation Practices	-0.018077
2	end_None	-0.033951
3	beginning_Integration Practices	-0.044943
4	middle_Idea Management	-0.060025
5	beginning_Knowledge Sharing	-0.101971
6	beginning_Evaluation Practices	-0.179035
7	end_Relational Climate	-0.185942
8	middle_Coordination and Decision Practices	-0.185952
9	end_Integration Practices	-0.197835
10	middle_Information Seeking	-0.242697
11	beginning_Idea Management	-0.438894
12	beginning_None	-0.446850
13	middle_Knowledge Sharing	-0.450920
14	end_Knowledge Sharing	-0.460703

```
In [31]: import re
import pandas as pd
import numpy as np

# =====
# REQUIREMENTS
# =====
if "U" not in globals():
    raise NameError("U not found. Run the session JSON loading chunk that creates U (utterance-level")
```

```

if "meta_plus" not in globals():
    raise NameError("meta_plus not found. Run the meta_plus merge chunk first.")

# =====
# 0) Identify columns in U
# =====
# Phase column (beginning/middle/end)
phase_col = None
for c in ["phase", "when", "segment", "part"]:
    if c in U.columns:
        phase_col = c
        break

# Label column (Coordination..., Knowledge Sharing, etc.)
label_col = None
for c in ["label", "category", "practice_label", "annotation_label", "code"]:
    if c in U.columns:
        label_col = c
        break

# Transcript column
text_col = None
for c in ["transcript", "text", "utterance", "content"]:
    if c in U.columns:
        text_col = c
        break

if phase_col is None:
    raise ValueError(f"Could not find a phase column in U. Available columns: {list(U.columns)}")
if label_col is None:
    raise ValueError(f"Could not find a label column in U. Available columns: {list(U.columns)}")
if text_col is None:
    raise ValueError(f"Could not find transcript text column in U. Available columns: {list(U.columns)}")

print(f"Using phase_col='{phase_col}' | label_col='{label_col}' | text_col='{text_col}'")

# =====
# 1) Pick sessions to inspect: top confident FP and top confident FN
# =====

```



```

df = meta_plus.copy()

fp_sids = (
    df[df["err_type"] == "FP"]
      .sort_values("p_funded_cv", ascending=False) ["session_id"]
      .head(3)
      .tolist()
)

fn_sids = (
    df[df["err_type"] == "FN"]
      .sort_values("p_funded_cv", ascending=True) ["session_id"]
      .head(3)
      .tolist()
)

inspect_sids = fp_sids + fn_sids
print("Inspect sessions:")
print("FP:", fp_sids)
print("FN:", fn_sids)

# =====
# 2) Logistics detector (regex + helper)
# =====
LOGISTICS_PAT = re.compile(
    r"\b("
    r"wait|waiting|muted|share your screen|screen share|slides?|powerpoint|ppt|"
    r"time|minutes?|seconds?|breakout|main room|report out|notes|bullet points?"
    r"kicked|go to|file"
    r")\b",
    flags=re.IGNORECASE
)

def logistics_flag(text):
    if not isinstance(text, str) or not text.strip():
        return 0
    return int(bool(LOGISTICS_PAT.search(text)))

tmp = U.copy()

```

```

tmp["is_logistics"] = tmp[text_col].apply(logistics_flag)

# =====
# 3) Compute end-coordination logistics share per session
# =====
END_PHASE_VALUE = "end"
COORD_LABEL_VALUE = "Coordination and Decision Practices"

end_coord = tmp[(tmp[phase_col] == END_PHASE_VALUE) & (tmp[label_col] == COORD_LABEL_VALUE)].copy()

logi_share = (
    end_coord.groupby("session_id")["is_logistics"]
        .mean()
        .rename("end_coord_logistics_share")
        .reset_index()
)

# Merge into meta_plus so you can slice by FP/FN easily
meta_aug = meta_plus.merge(logi_share, on="session_id", how="left")

print("\nTop sessions by end_coord_logistics_share (higher = more logistics-y coordination):")
display(
    meta_aug[["session_id", "err_type", "p_funded_cv", "proposal_success", "end_coord_logistics_share"]]
        .sort_values("end_coord_logistics_share", ascending=False)
        .head(25)
)

print("\nSummary by err_type (mean logistics share):")
display(
    meta_aug.groupby("err_type", dropna=False)["end_coord_logistics_share"]
        .agg(["count", "mean", "median"])
        .reset_index()
)

# =====
# 4) Utility: show representative utterances (with LOGI tags)
# =====
def show_snippets(session_id, phase, label, n=8):
    sub = tmp[(tmp["session_id"] == session_id) & (tmp[phase_col] == phase) & (tmp[label_col] == la

```

```

sub = sub.dropna(subset=[text_col])
print(f"\n--- {session_id} | {phase} | {label} | n={len(sub)} ---")
if len(sub) == 0:
    return

# Stable ordering if possible
sort_cols = []
if "t" in sub.columns:
    sort_cols.append("t")
if "utt_idx" in sub.columns:
    sort_cols.append("utt_idx")
if sort_cols:
    sub = sub.sort_values(sort_cols)

# evenly spaced if too many
if len(sub) > n:
    idxs = np.linspace(0, len(sub)-1, n).astype(int)
    sub = sub.iloc[idxs]

for i, row in enumerate(sub.itertuples(index=False), 1):
    txt = getattr(row, text_col)
    txt = txt.replace("\n", " ").strip()
    if len(txt) > 240:
        txt = txt[:240] + "..."
    tag = "[LOGI]" if getattr(row, "is_logistics") == 1 else " "
    print(f"{i:02d}. {tag} {txt}")

# =====
# 5) Run targeted comparisons for the inspect sessions
# =====
TARGETS = [
    ("end", "Coordination and Decision Practices"),
    ("middle", "Knowledge Sharing"),
    ("end", "Knowledge Sharing"),
]

df_by_sid = df.set_index("session_id")

for sid in inspect_sids:

```

```

print("\n" + "="*80)
print(
    "SESSION:", sid,
    "| err_type:", df_by_sid.loc[sid, "err_type"],
    "| p_funded_cv:", float(df_by_sid.loc[sid, "p_funded_cv"]),
    "| true funded:", int(df_by_sid.loc[sid, "proposal_success"]),
    "| end_coord_logistics_share:", float(meta_aug.set_index("session_id").loc[sid, "end_coord_
if sid in set(meta_aug["session_id"]) and pd.notna(meta_aug.set_index("session_id").loc[sid
else None
)
for ph, lab in TARGETS:
    show_snippets(sid, ph, lab, n=8)

```

Using phase_col='phase' | label_col='label' | text_col='transcript'

Inspect sessions:

FP: ['2022_04_08_MND_S12', '2021_10_01_MZT_S8', '2021_05_21_ABI_S5']

FN: ['2020_11_06_NES_S12', '2021_04_23_MND_S11', '2022_04_07_MND_S6']

Top sessions by end_coord_logistics_share (higher = more logistics-y coordination):

	session_id	err_type	p_funded_cv	proposal_success	end_coord_logistics_share
12	2022_04_07_MND_S6	FN	0.125388	1	0.818182
0	2022_04_08_MND_S12	FP	0.996794	0	0.666667
9	2021_10_08_CMC_S9	FP	0.772768	0	0.583333
1	2021_10_01_MZT_S8	FP	0.944502	0	0.500000
17	2021_05_20_ABI_S7	FN	0.222979	1	0.500000
8	2021_09_30_MZT_S3	FP	0.783783	0	0.411765
3	2021_11_05_NES_S2	FP	0.892249	0	0.370370
7	2021_04_23_MND_S7	FP	0.803345	0	0.370370
4	2021_10_01_MZT_S2	FP	0.879218	0	0.363636
18	2022_04_07_MND_S5	FN	0.229674	1	0.350000
19	2022_04_07_MND_S1	FN	0.272132	1	0.350000
11	2021_04_23_MND_S11	FN	0.074936	1	0.333333
15	2022_04_07_MND_S4	FN	0.183988	1	0.300000
14	2021_04_22_MND_S4	FN	0.143854	1	0.285714
6	2021_04_23_MND_S14	FP	0.812520	0	0.233333
5	2021_06_10_SLU_S7	FP	0.836086	0	0.230769
2	2021_05_21_ABI_S5	FP	0.900494	0	0.218750
13	2020_11_06_NES_S3	FN	0.138559	1	0.000000
16	2021_09_30_MZT_S6	FN	0.199107	1	0.000000
10	2020_11_06_NES_S12	FN	0.018621	1	0.000000

Summary by err_type (mean logistics share):

	err_type	count	mean	median
0	FN	10	0.293723	0.316667
1	FP	10	0.394899	0.370370

=====

SESSION: 2022_04_08_MND_S12 | err_type: FP | p_funded_cv: 0.9967944542410051 | true funded: 0 | end_coord_logistics_share: 0.6666666666666666

--- 2022_04_08_MND_S12 | end | Coordination and Decision Practices | n=3 ---

01. [LOGI] Should we wait for someone else or we are just only us?

02. I'm not sure.

03. [LOGI] This is 3.04 and we're waiting for Gianna.

--- 2022_04_08_MND_S12 | middle | Knowledge Sharing | n=0 ---

--- 2022_04_08_MND_S12 | end | Knowledge Sharing | n=1 ---

01. [LOGI] This is 3.04 and we're waiting for Gianna.

=====

SESSION: 2021_10_01_MZT_S8 | err_type: FP | p_funded_cv: 0.9445017528239149 | true funded: 0 | end_coord_logistics_share: 0.5

--- 2021_10_01_MZT_S8 | end | Coordination and Decision Practices | n=18 ---

01. [LOGI] Any any other thoughts on our topic or do we need to uh try to go to listing out some bullet points in response? It's a little soon for that. We we go to 1220, I guess we have about uh 15 minutes.

02. Okay, yeah, I got it.

03. [LOGI] Christopher, want to share your screen and so we can all see?

04. Let me pull it up then and I I can type it in while you if you guys guide me.

05. [LOGI] uh I I think we can uh we can make one big uh bullet point about um future vaccines or optimizing vaccine strategies. Um there was a lot of discussion about um uh non specific approaches, which I think also would fall under the first uh big...

06. [LOGI] Oh now now it's hard to see what the other bullet points are.

07. [LOGI] Whatever you can write down in 50 seconds.

08. [LOGI] It's going to kick us in 10 seconds. So I just want to say thanks for the conversation. This was really good.

--- 2021_10_01_MZT_S8 | middle | Knowledge Sharing | n=20 ---

01. Right if if if the idea is that we can um not only anticipate but basically um preempt viral evolution by by designing uh um vaccines into sort of the the uncharted territory of viral evolutionary space um either sequencing space or or uh e...
02. [LOGI] You know, one of the things that has to happen if you're if you have it for even a new um strain of flu that you're trying to grow grow up for live vaccine, the first thing has to be done has to be a uh you know, passage through eggs where ...
03. [LOGI] I think that we have to be carefully with with that kind of of compounds. We can generate autoimmunity, uh, you know, immune autoimmune response in in the host. Uh, no, a little we are boosting boosting the the with with different uh compon...
04. historically vaccine production is mass scale. It's not one dose, one person.
05. Um we can we can use um for example containers, right? If we have ship cargos, you know, then you can module this very successful story. Uh why why we our shipping industry um grows so rapidly ised.
06. It's been looked at.
07. Yeah, I thought the the cute nature of the infection would kind of rule out this opportunity to intervene in this way. You'd have to you know, Christoff, you you really have to know the status of the healthy individual to predict what we wh...
08. Yeah, going back to what Kurt just said, I I think the um the issue with individualized markers is that we don't really know what the what the bottleneck is. If we're talking about mRNA vaccines, um is it when when someone doesn't respond t...

--- 2021_10_01_MZT_S8 | end | Knowledge Sharing | n=16 ---

01. Technical difficulties. Uh so yes, I I think I met some of you. I'm Angela Arenas, I'm a veterinarian but I'm a vaccinologist too. So that's what I do for a living. I develop live attenuated vaccines for intracellular bacteria and we also use...
02. because we have COVID, but that makes us forget the fact that there are other uh pathogens that are affecting people. Like Lyme disease is 400,000 people just in the United States that get that disease every every year. And are we talking a...
03. Yes, uh I I think it goes back to perhaps either like a kill thing or anything combined with the adjuvants, different adjuvants, I think might be some adjuvant discovery, I think's very important. Uh and not towards like a TH2 response, whi...
04. [LOGI] Any any other thoughts on our topic or do we need to uh try to go to listing out some bullet points in response? It's a little soon for that. We we go to 1220, I guess we have about uh 15 minutes.
05. [LOGI] The file Scialog MZT meeting slides PowerPoint X.
06. [LOGI] I gave you the wrong slide number. You want to be on slide 52.
07. Yeah, it seems like I I have a read only version here. I don't know why.
08. My pleasure here. you know, a reemerging or newly emerged pathogen and the other is a pre

ventative measure and I I think these are these approaches are needed under different circumstance s. You know, is is if you're talking about highly cro...

=====

SESSION: 2021_05_21_ABI_S5 | err_type: FP | p_funded_cv: 0.900493506496639 | true funded: 0 | end_c
oord_logistics_share: 0.21875

--- 2021_05_21_ABI_S5 | end | Coordination and Decision Practices | n=32 ---

01. Um so but now I think we should just dream. You know, if we came up with the best one, um what could we do that?
02. I I think that's the benefit of this uh group. Um so when on number two, you have multimodality by using AI to fuse.
03. So you're actually talking to number two in in um the key points that um Beck has put up.
04. Shiva, we don't hear you.
05. [LOGI] Right, so you've got three minutes to uh but I think your cut point, your key points up here.
06. Okay, I saw a thumbs up, excellent.
07. [LOGI] That's a good point. But you you the fellows have additional time where and you have your break you have you have your what's it called? You have your other sessions at night where you could just you do your mini breakout or you could also ...
08. Putting our names in, so.

--- 2021_05_21_ABI_S5 | middle | Knowledge Sharing | n=46 ---

01. [LOGI] Uh, yeah, I totally second that that I like kind of almost every night I ask myself that why I'm doing what I'm doing because to me like working towards the problem statement especially from like for me developing hardware is really importa...
02. [LOGI] Yeah, I think you bring up a really good point. A lot of times we look at multimodality so that we get two different two different types of characteristics say of a patient simultaneously where they're co-registered in time and and space. H...
03. I correlated to something else there is some issue.
04. So I but it's it's um it's still slowly crossing over to um uh clinical use and that's I think a very important area and I just say one more thing and then I'm going to give it back to you is one of the features is heterogeneity. Cancer is ...
05. structural deformation has already happened and you can see on the images.
06. radiographic x-ray, we have um pet, we have um spec, we have MRI, we have ultrasound and how do we link them together because um
07. And then they do the biopsy. The biopsy failed because lack of information, lack of sample and then the neurosurgeon took the extreme case and do the complete resection.
08. There might be a cheaper alternative that potentially can be correlated in somehow with s

even Tesla Tesla but can be deployed in a countries that cannot have a seven Tesla Tesla scanner or in a region that cannot have a Tesla scanner. And y...

--- 2021_05_21_ABI_S5 | end | Knowledge Sharing | n=29 ---

01. And then there's also the um part on um uh which is related to the cost is the uh socio economic situation in which that uh imaging system uh might uh not be able to exist.

02. Yeah. That's why we we're quite a a diverse group here. I mean, there's some of us, I'm I'm an AI person. There seems to be some other folks here AI meaning merging of techniques using mathematical methods.

03. Um and there's acquisition people both at the optical uh you know, across all scales.

04. [LOGI] I think the goal in the MRI field is really to acquire a very high set of parameters in a very short time and then fuse them to get to the high number of parameters and characterize the issue. That's the that's the far goal for MRI, I thin...

05. Which is um fusion too, you know, like the AI of tech what you find at one scale related to what you find at the other, but use data reduction and AI to do it.

06. Right, all of these need to be then customized to specific anatomical parts, clinical questions. Um.

07. [LOGI] Oh, I think he pressed the button. He left the breakout. So we're 45 seconds.

08. So I'm Mary Ellen Giger from University of Chicago. I'm in radiology and medical physics and I um work in AI and medical imaging. I've done it for multiple decades um both in development, uh research, effects of different parameters as well...

=====

SESSION: 2020_11_06_NES_S12 | err_type: FN | p_funded_cv: 0.018620740362930377 | true funded: 1 | end_coord_logistics_share: 0.0

--- 2020_11_06_NES_S12 | end | Coordination and Decision Practices | n=3 ---

01. Oh. I can should I keep talking about it or should I anyone want to should I or okay.

02. No? Alright.

03. No I'm I'm not either but I have I have a tons of questions will ask later.

--- 2020_11_06_NES_S12 | middle | Knowledge Sharing | n=11 ---

01. So what I would say is because you know in biological a big issue in biological fermentat or is the local heat because the biological metabolism is exothermic. And so from engineering point of view, you think about why sometimes a big ferment...

02. because the temperature wise that's a much you need to have better control. But if you think about pressure wise, they they they are much adaptive very adaptive to that. So that's what I say I that's why I say that first of all, I wouldn't ...

03. you just bound the CO2 instead of regenerating CO2 by putting heat in to high temperature

release the gas at 120 140 degree C. What if we are doing electrochemical reaction of the bound CO
2? Maybe temperature doesn't need to be increased. S...

04. [LOGI] Sorry I had a little emergency as you probably saw but I don't I don't know if we have t
ime for just one more thing I wanted to mention I was trying to get it in edgewise but but couldn'
t. But in terms of hybrid approaches I guess when thin...

05. [LOGI] So I think one of the things we can add which we didn't call it out although we discusse
d it is the systems integration and optimization in multi-scale both in terms of time and physical
scale. That's very important when you put different t...

06. Well, I am.

07. So I don't have experience in that area but I have a lot of sort of questions and and and

08. It work, yeah.

--- 2020_11_06_NES_S12 | end | Knowledge Sharing | n=14 ---

01. [LOGI] Right. So so I had I I I could say a little bit because but before I before I tell what
my opinion, I would say I'm still a chemist by training. I got physical chemistry in PhD so I would
n't say a fully knowledgeable about chemical biology ...

02. You need to make sure which agent which program to apply for, yeah.

03. Yeah. I agree. I mean if you talk to Wilson Smith you will know that NREL they they do th
e modular approach that you have electrolyzer generate hydrogen and hydrogen fermentation has been
a very well-developed process in industry. So if if ...

04. electrochemically or thermochemically. Once we have that, we can feed that to biological
system which can upgrade to C2 plus much bigger molecule much more selectively. So interesting thin
g is people who are working on electrochemical conve...

05. No? Alright.

06. Alright good. Go ahead. Okay. So so it depends right? It depends on what kind of microbe
you're using. The the most appropriate and easier ones they make a tons of there are bacteria can m
ake 99% of acetate. Bacteria can make 99% of methane...

07. So it's already done and actually when I was a postdoc in Dan Nocera's lab at Harvard, we
we I we have the technology and we patented and now it's a startup company now in Cambridge. What i
t does is we're using a microbe that it's a soil mi...

08. [LOGI] I I agree and they they have we have they are corn fields now in Massachusetts under the
test. The company was founded 2017 and you know it takes a few seasons to get results. So we're sti
ll waiting for that. People are still waiting for th...

=====

SESSION: 2021_04_23_MND_S11 | err_type: FN | p_funded_cv: 0.07493629945580332 | true funded: 1 | en
d_coord_logistics_share: 0.3333333333333333

--- 2021_04_23_MND_S11 | end | Coordination and Decision Practices | n=21 ---

01. [LOGI] Uh over time. Yeah, the last point is how do these communication signals or how can these communication signals be monitored, hijacked or interrupted to drive desired outcomes.
02. [LOGI] Kai is the Kai you're muted. Uh you're the reporter.
03. I'm posting. Yeah, I'm posting the questions and our key message.
04. Yeah. I think we can put maybe just a keyword to the
05. [LOGI] I think questions three and the second part of the second question is the same. How to perturb and monitor.
06. Did I cover all the big points?
07. Just one more.
08. Okay, we're good.

--- 2021_04_23_MND_S11 | middle | Knowledge Sharing | n=46 ---

01. and if you think about, you know, all the different ways to transmit that signal, you know, immune cells have serotonin receptors, right? And so you know, and you have most of our immune cells are in our gut and and some circulate out. And ...
02. Yeah, so lower levels of um um HCRP.
03. I've seen people do that for the um the gut brain vagal system.
04. I I just so after actually mentioned, I just remembered a study from a couple of years ago where they gave six hydroxy dopamine to the the in mice and the mice had constipation. If I remember correctly. I'll I'll dig up the paper right now ...
05. [LOGI] Yeah, this is very stimulating. I mean, I feel yeah, it's it's more than just neuronal function that we were manipulating. Um, definitely neural will will send out the message to a lot of different systems. So we have a study it's on we hav...
06. Why it happens, we don't know, right? Because it also changes motility, right? And and so the the, you know,
07. Yeah, yeah.
08. I mean it's interesting, right? So for something so diarrhea is associated with anxiety, right? With acute anxiety. I don't know, do we know how that's mediated, right? I mean, is it does diarrhea does anxiety start in the gut, does it star...

--- 2021_04_23_MND_S11 | end | Knowledge Sharing | n=25 ---

01. I mean my graduate lab was focused on stress and GI function and so we did a lot of acute stress, chronic stress. Um and the idea was at least initially that it was chemicals released from the brain in response to stress, presumably anxiety...
02. I mean there's several different theories, one being that we have a thought, it produces an emotion and then leads to a physiological response. But there's another theory where the physiological response starts and then we try to attribute ...
03. I mean the the topic being like what are the mechanisms of communication and and interaction, right? And so timing gives you maybe some candidates for what that would be, right? Because yo

u're right. I mean if it for example, if it is a ...

04. Well, we know that meditation techniques it essentially activates the parasympathetic nervous system. Um and that's how it like has much of its beneficial effects on physiological functioning. So maybe something as quote easy as that could ...

05. I know Melanie's recording.

06. 54. Sorry, 54.

07. Okay, so that was I think most of the key points for point number one. Um point number two was about the directionality. Um so we talked a bit about brain to gut. Um the the speed, the idea of stress, uh the role of meditation or cognitive ...

08. 54.

=====

SESSION: 2022_04_07_MND_S6 | err_type: FN | p_funded_cv: 0.12538809037114626 | true funded: 1 | end_coord_logistics_share: 0.8181818181818182

--- 2022_04_07_MND_S6 | end | Coordination and Decision Practices | n=11 ---

01. [LOGI] What else? Any other any other thoughts before we go to the third bullet?

02. [LOGI] So we got a message that we have 10 minutes left. That was about a minute ago. So let's spend a few minutes on the third bullet and then we'll try to figure out what our major points are from this, okay?

03. So the third bullet is, is there a way you can make an optimal microbiome in infants and is there a way you can alter the elderly microbiome to be the optimal microbiome. Um we've we sort of mentioned a few things about that, but how do ...

04. I like the questions that we conversed on and maybe some people in the in the I see will have more have the answers, but I really liked the question about the enteric nervous system over age in age. I thought that that's a question we arr...

05. [LOGI] Um, I don't know, what other bullet points?

06. [LOGI] See you in the main room.

07. [LOGI] Welcome back everyone. Uh, is there anyone still working to get their notes into the PowerPoint?

08. [LOGI] All right, we'll give one more minute. We'll what we'll do for the report out is we'll just go to each room and ask the person doing the report out to uh give a concise summary. Uh, we want to keep that to about three minutes and then we'll...

--- 2022_04_07_MND_S6 | middle | Knowledge Sharing | n=38 ---

01. [LOGI] How are they changing over age? You know, I don't know the answer actually. I don't know the answer. I know what happens from early life but I don't know very old life. So in early life, um there our bodies have different source of macropha...

02. Well, uh actually we did the experiment. I believe there is also a paper in the nature ag

ing. They showed the they also did the immune IC on that.

03. epitope. So you I'm sure you've heard of CytoF. So then they're looking at epigenetic changes in the in the chromatin on a single cell level, like flow cytometry but using uh CytoF but for chromatin modifications. And so they indeed looked ...

04. It's young to old. Totally got it wrong.

05. And in that ramified state, they have very long processes to in order to sample their specific location in that environment and then when some sort of um invader or something like that is detected, they will actually retract processes and l...

06. And I think there's some connection with microbiome. I don't know what it is though.

07. That was something we proposed last year.

08. Uh, the global one? I I believe there are several papers out.

--- 2022_04_07_MND_S6 | end | Knowledge Sharing | n=34 ---

01. I mean one of the big issues is that we don't we don't know all like we don't have the catalog picture of metabolites and even if we did reach them, like who who's doing them? We have no clue. So I mean that there's this huge genetic like r...

02. So yeah my my major interest is nutrition but um you have to look at the ingredient list. So if it has sucrose in it, all good. That's normal sugar.

03. Um this is in mice and it was really impactful for me because they showed that in repeated uh mice um the same adaptations show up and that those adaptations can predict uh the diet in like a machine learning framework.

04. So much I don't know about the entire nervous system.

05. details there, but that is my understanding.

06. There are some links to like um immune response and susceptibility to um autoimmune disorders based on um childbirth for like seeing um cesarean or vaginal and then whether they were breastfed or not, which I find so many challenges with th...

07. [LOGI] Did you have to keep replenishing? Like was there a time point where that runs out and you have to keep um transferring? Yeah, we we we just did the transfer for for example for two months and then during the two months we married behavior,...

08. So there were microbial changes over the lifespan, which we described. Um, how these could possibly change the gut brain axis. Um, we talk about the morphology of the microglia and how some changes in the metabolites could uh come into play...

```
In [32]: import re
import pandas as pd
import numpy as np

# ---- sanity: columns ----
phase_col = "phase"
```

```

label_col = "label"
text_col = "transcript"

# =====
# 1) Two-tier logistics patterns
# =====

# STRICT: things that are almost always logistics
STRICT_PAT = re.compile(
    r"(?i)\b("
    r"you(?:'re)?\s+muted|unmute|mute|"
    r"share\s+your\s+screen|screen\s+share|stop\s+sharing|"
    r"slides?\b|slide\s+\d+|powerpoint|ppt\b|"
    r"breakout\s+room|main\s+room|report\s*out|"
    r"file\s+(?:name|is)|open\s+the\s+file|read[-\s]?only|"
    r"zoom|teams|webex|"
    r"we\s+have\s+\d+\s+(?:minutes?|seconds?)\s+left|"
    r"kicked|kick\s+us|time\s+is\s+up|wrap\s+up|"
    r"waiting\s+for\s+\w+"
    r")\b"
)

# LOOSE: your original vibe, but we'll gate it with context
LOOSE_WORDS = re.compile(
    r"(?i)\b(wait|waiting|time|minutes?|seconds?|notes|bullet\s*points?|go\s+to|file)\b"
)

# If a loose word appears, require it to be near one of these context anchors
CONTEXT_ANCHORS = re.compile(
    r"(?i)\b("
    r"breakout|main\s+room|report\s*out|slides?|powerpoint|ppt|"
    r"share\s+your\s+screen|muted|kicked|time\s+left|wrap\s+up|"
    r"we\s+have\s+\d+\s+(?:minutes?|seconds?)\s+left"
    r")\b"
)

def is_logistics_strict(text: str) -> int:
    if not isinstance(text, str) or not text.strip():
        return 0

```

```

        return int(bool(STRICT_PAT.search(text)))

def is_logistics_loose(text: str) -> int:
    """Loose match, but only count if there's also a logistics context anchor."""
    if not isinstance(text, str) or not text.strip():
        return 0
    if STRICT_PAT.search(text):
        return 1
    if LOOSE_WORDS.search(text) and CONTEXT_ANCHORS.search(text):
        return 1
    return 0

tmp = U.copy()
tmp["is_logistics_strict"] = tmp[text_col].apply(is_logistics_strict)
tmp["is_logistics_loose"] = tmp[text_col].apply(is_logistics_loose)

# =====
# 2) Compute shares for end + Coordination & Decision Practices
# =====
END_PHASE_VALUE = "end"
COORD_LABEL_VALUE = "Coordination and Decision Practices"

end_coord = tmp[(tmp[phase_col] == END_PHASE_VALUE) & (tmp[label_col] == COORD_LABEL_VALUE)].copy()

shares = (
    end_coord.groupby("session_id")[["is_logistics_strict", "is_logistics_loose"]]
        .mean()
        .rename(columns={
            "is_logistics_strict": "end_coord_logistics_share_strict",
            "is_logistics_loose": "end_coord_logistics_share_loose",
        })
        .reset_index()
)

meta_aug2 = meta_plus.merge(shares, on="session_id", how="left")

print("Top sessions by STRICT logistics share:")
display(
    meta_aug2[["session_id", "err_type", "p_funded_cv", "proposal_success",

```

```

        "end_coord_logistics_share_strict", "end_coord_logistics_share_loose"]
    .sort_values("end_coord_logistics_share_strict", ascending=False)
    .head(25)
)

print("\nSummary by err_type (STRICT vs LOOSE):")
display(
    meta_aug2.groupby("err_type", dropna=False)[
        ["end_coord_logistics_share_strict", "end_coord_logistics_share_loose"]
    ].agg(["count", "mean", "median"]).reset_index()
)

# =====
# 3) Optional: preview the strict/loose flags for a session
# =====
def preview_session_flags(session_id, n=30):
    sub = tmp[(tmp["session_id"] == session_id) &
              (tmp[phase_col] == "end") &
              (tmp[label_col] == COORD_LABEL_VALUE)].copy()
    if "utt_idx" in sub.columns:
        sub = sub.sort_values("utt_idx")
    cols = ["session_id", phase_col, label_col, "is_logistics_strict", "is_logistics_loose", text_c
    cols = [c for c in cols if c in sub.columns]
    display(sub[cols].head(n))

# example:
# preview_session_flags("2022_04_07_MND_S6")

```

Top sessions by STRICT logistics share:

	session_id	err_type	p_funded_cv	proposal_success	end_coord_logistics_share_strict	end_coord,
17	2021_05_20_ABI_S7	FN	0.222979	1	0.500000	
12	2022_04_07_MND_S6	FN	0.125388	1	0.454545	
0	2022_04_08_MND_S12	FP	0.996794	0	0.333333	
3	2021_11_05_NES_S2	FP	0.892249	0	0.333333	
9	2021_10_08_CMC_S9	FP	0.772768	0	0.250000	
18	2022_04_07_MND_S5	FN	0.229674	1	0.250000	
14	2021_04_22_MND_S4	FN	0.143854	1	0.238095	
8	2021_09_30_MZT_S3	FP	0.783783	0	0.235294	
11	2021_04_23_MND_S11	FN	0.074936	1	0.190476	
1	2021_10_01_MZT_S8	FP	0.944502	0	0.166667	
15	2022_04_07_MND_S4	FN	0.183988	1	0.150000	
7	2021_04_23_MND_S7	FP	0.803345	0	0.111111	
6	2021_04_23_MND_S14	FP	0.812520	0	0.100000	
4	2021_10_01_MZT_S2	FP	0.879218	0	0.090909	
5	2021_06_10_SLU_S7	FP	0.836086	0	0.076923	
2	2021_05_21_ABI_S5	FP	0.900494	0	0.062500	
19	2022_04_07_MND_S1	FN	0.272132	1	0.050000	
16	2021_09_30_MZT_S6	FN	0.199107	1	0.000000	
13	2020_11_06_NES_S3	FN	0.138559	1	0.000000	
10	2020_11_06_NES_S12	FN	0.018621	1	0.000000	

Summary by err_type (STRICT vs LOOSE):

	err_type	end_coord_logistics_share_strict			end_coord_logistics_share_loose		
		count	mean	median	count	mean	median
0	FN	10	0.183312	0.170238	10	0.183312	0.170238
1	FP	10	0.176007	0.138889	10	0.182257	0.145833

report wrap up feat

```
In [33]: import re
import pandas as pd

REPORTOUT_PAT = re.compile(
    r"(?i)\b("
    r"report\s*out|reporting|reporter|"
    r"bullet\s*points?|key\s+points?|"
    r"summary|summarize|"
    r"final\s+thoughts?|any\s+other\s+thoughts?|"
    r"wrap\s*up|time\s+left|minutes?\s+left|seconds?\s+left|"
    r"put\s+it\s+in|type\s+it\s+in|write\s+it\s+down|"
    r"slide\s+\d+|slides?|powerpoint|ppt|"
    r"main\s+room|breakout"
    r")\b"
)

def reportout_flag(text):
    if not isinstance(text, str) or not text.strip():
        return 0
    return int(bool(REPORTOUT_PAT.search(text)))

tmp2 = U.copy()
tmp2["is_reportout"] = tmp2["transcript"].apply(reportout_flag)

end_coord = tmp2[(tmp2["phase"]=="end") & (tmp2["label"]=="Coordination and Decision Practices")].c

reportout_share = (
```

```

        end_coord.groupby("session_id")["is_reportout"]
        .mean()
        .rename("end_reportout_share")
        .reset_index()
    )

meta_aug3 = meta_plus.merge(reportout_share, on="session_id", how="left")

print("Top sessions by end_reportout_share:")
display(
    meta_aug3[["session_id", "err_type", "p_funded_cv", "proposal_success", "end_reportout_share"]]
    .sort_values("end_reportout_share", ascending=False)
)

print("\nSummary by err_type (end_reportout_share):")
display(
    meta_aug3.groupby("err_type")["end_reportout_share"]
    .agg(["count", "mean", "median"])
    .reset_index()
)

```

Top sessions by end_reportout_share:

	session_id	err_type	p_funded_cv	proposal_success	end_reportout_share
12	2022_04_07_MND_S6	FN	0.125388	1	0.727273
17	2021_05_20_ABI_S7	FN	0.222979	1	0.583333
13	2020_11_06_NES_S3	FN	0.138559	1	0.500000
5	2021_06_10_SLU_S7	FP	0.836086	0	0.384615
2	2021_05_21_ABI_S5	FP	0.900494	0	0.375000
9	2021_10_08_CMC_S9	FP	0.772768	0	0.333333
1	2021_10_01_MZT_S8	FP	0.944502	0	0.333333
11	2021_04_23_MND_S11	FN	0.074936	1	0.333333
6	2021_04_23_MND_S14	FP	0.812520	0	0.266667
3	2021_11_05_NES_S2	FP	0.892249	0	0.259259
15	2022_04_07_MND_S4	FN	0.183988	1	0.250000
14	2021_04_22_MND_S4	FN	0.143854	1	0.238095
8	2021_09_30_MZT_S3	FP	0.783783	0	0.235294
19	2022_04_07_MND_S1	FN	0.272132	1	0.200000
18	2022_04_07_MND_S5	FN	0.229674	1	0.200000
4	2021_10_01_MZT_S2	FP	0.879218	0	0.090909
7	2021_04_23_MND_S7	FP	0.803345	0	0.074074
0	2022_04_08_MND_S12	FP	0.996794	0	0.000000
16	2021_09_30_MZT_S6	FN	0.199107	1	0.000000
10	2020_11_06_NES_S12	FN	0.018621	1	0.000000

Summary by err_type (end_reportout_share):

	err_type	count	mean	median
0	FN	10	0.303203	0.244048
1	FP	10	0.235249	0.262963

commitment narrowing

In [34]: **import** re

```

CONVERGENCE_PAT = re.compile(
    r"(?i)\b("
    r"so\s+we\s+will|so\s+we're\s+going\s+to|"
    r"the\s+main\s+idea|the\s+key\s+question|"
    r"our\s+proposal|our\s+approach|"
    r"this\s+means\s+that|which\s+means\s+that|"
    r"we\s+should\s+focus\s+on|"
    r"let's\s+focus\s+on|"
    r"the\s+takeaway|"
    r"in\s+summary|to\s+summarize"
    r")\b"
)

def convergence_flag(text):
    if not isinstance(text, str):
        return 0
    return int(bool(CONVERGENCE_PAT.search(text)))

tmp3 = U.copy()
tmp3["is_convergence"] = tmp3["transcript"].apply(convergence_flag)

end_any = tmp3[tmp3["phase"]=="end"]

conv_share = (
    end_any.groupby("session_id")["is_convergence"]
    .mean()
    .rename("end_convergence_share")
)

```

```

        .reset_index()
    )

meta_aug4 = meta_plus.merge(conv_share, on="session_id", how="left")

print("Top sessions by end_convergence_share:")
display(
    meta_aug4[["session_id", "err_type", "p_funded_cv", "proposal_success", "end_convergence_share"]]
    .sort_values("end_convergence_share", ascending=False)
)

print("\nSummary by err_type (end_convergence_share):")
display(
    meta_aug4.groupby("err_type")["end_convergence_share"]
    .agg(["count", "mean", "median"])
    .reset_index()
)

```

Top sessions by end_convergence_share:

	session_id	err_type	p_funded_cv	proposal_success	end_convergence_share
10	2020_11_06_NES_S12	FN	0.018621	1	0.040816
14	2021_04_22_MND_S4	FN	0.143854	1	0.028777
17	2021_05_20_ABI_S7	FN	0.222979	1	0.023256
3	2021_11_05_NES_S2	FP	0.892249	0	0.016260
6	2021_04_23_MND_S14	FP	0.812520	0	0.008621
11	2021_04_23_MND_S11	FN	0.074936	1	0.000000
18	2022_04_07_MND_S5	FN	0.229674	1	0.000000
16	2021_09_30_MZT_S6	FN	0.199107	1	0.000000
15	2022_04_07_MND_S4	FN	0.183988	1	0.000000
13	2020_11_06_NES_S3	FN	0.138559	1	0.000000
12	2022_04_07_MND_S6	FN	0.125388	1	0.000000
0	2022_04_08_MND_S12	FP	0.996794	0	0.000000
1	2021_10_01_MZT_S8	FP	0.944502	0	0.000000
9	2021_10_08_CMC_S9	FP	0.772768	0	0.000000
8	2021_09_30_MZT_S3	FP	0.783783	0	0.000000
7	2021_04_23_MND_S7	FP	0.803345	0	0.000000
5	2021_06_10_SLU_S7	FP	0.836086	0	0.000000
4	2021_10_01_MZT_S2	FP	0.879218	0	0.000000
2	2021_05_21_ABI_S5	FP	0.900494	0	0.000000
19	2022_04_07_MND_S1	FN	0.272132	1	0.000000

Summary by err_type (end_convergence_share):

	err_type	count	mean	median
0	FN	10	0.009285	0.0
1	FP	10	0.002488	0.0

measuring idea-space narrowing via entropy

```
In [35]: import numpy as np

def entropy(p):
    p = p[p > 0]
    return -(p * np.log2(p)).sum()

# Compute label entropy per session per phase
ent = (
    U.groupby(["session_id", "phase", "label"])
      .size()
      .rename("n")
      .reset_index()
)

ent["p"] = ent.groupby(["session_id", "phase"])["n"].transform(lambda x: x / x.sum())

ent_phase = (
    ent.groupby(["session_id", "phase"])
      .apply(lambda g: entropy(g["p"].values))
      .rename("label_entropy")
      .reset_index()
)

# Pivot for beginning vs end
ent_wide = (
    ent_phase.pivot(index="session_id", columns="phase", values="label_entropy")
      .reset_index()
)
```



```

# Delta entropy (did idea space collapse?)
if {"beginning", "end"}.issubset(ent_wide.columns):
    ent_wide["delta_entropy"] = ent_wide["end"] - ent_wide["beginning"]

meta_aug5 = meta_plus.merge(ent_wide, on="session_id", how="left")

print("Entropy change summary:")
display(
    meta_aug5.groupby("err_type")["delta_entropy"]
    .agg(["count", "mean", "median"])
    .reset_index()
)

print("\nTop sessions with strongest entropy reduction:")
display(
    meta_aug5[["session_id", "err_type", "proposal_success", "delta_entropy"]]
    .sort_values("delta_entropy")
    .head(10)
)

```

Entropy change summary:

/var/folders/wp/cn7__9416yj3c5_vcykys8hc0000gn/T/ipykernel_60716/3637143228.py:19: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include\_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.

```
.apply(lambda g: entropy(g["p"].values))
```

	err_type	count	mean	median
0	FN	10	0.104253	0.114997
1	FP	10	0.155435	0.222664

Top sessions with strongest entropy reduction:

	session_id	err_type	proposal_success	delta_entropy
13	2020_11_06_NES_S3	FN	1	-0.383526
0	2022_04_08_MND_S12	FP	0	-0.214012
9	2021_10_08_CMC_S9	FP	0	-0.029393
18	2022_04_07_MND_S5	FN	1	0.014464
10	2020_11_06_NES_S12	FN	1	0.040237
15	2022_04_07_MND_S4	FN	1	0.067967
11	2021_04_23_MND_S11	FN	1	0.080257
6	2021_04_23_MND_S14	FP	0	0.080327
16	2021_09_30_MZT_S6	FN	1	0.149738
19	2022_04_07_MND_S1	FN	1	0.158852

```
In [36]: import numpy as np
import pandas as pd

# -----
# helper: entropy
# -----
def entropy(p):
    p = np.asarray(p, dtype=float)
    p = p[p > 0]
    if p.sum() <= 0:
        return np.nan
    p = p / p.sum()
    return -np.sum(p * np.log(p))

# -----
# 1) Compute entropy per (session, phase)
# -----
if not {"session_id", "phase", "label"}.issubset(U.columns):
    raise ValueError(f"U missing required columns. Has: {list(U.columns)}")
```

```

phase_ent = (
    U.dropna(subset=["label", "phase"])
    .groupby(["session_id", "phase"])["label"]
    .value_counts(normalize=True)
    .rename("p")
    .reset_index()
    .groupby(["session_id", "phase"])["p"]
    .apply(lambda s: entropy(s.values))
    .rename("H")
    .reset_index()
)

# -----
# 2) Wide format: H_beginning, H_middle, H_end
# -----
H = phase_ent.pivot(index="session_id", columns="phase", values="H")
# make sure expected columns exist even if missing in some sessions
for ph in ["beginning", "middle", "end"]:
    if ph not in H.columns:
        H[ph] = np.nan

H = H.rename(columns={"beginning": "H_begin", "middle": "H_mid", "end": "H_end"}).reset_index()

# -----
# 3) Define "early" and "late"
#     Early = beginning; Late = end
#     (optional: you can use mean(begin,middle) as early if you want)
# -----
H["H_early"] = H["H_begin"]
H["H_late"] = H["H_end"]
H["delta_entropy"] = H["H_late"] - H["H_early"]

# -----
# 4) Merge with outcomes
# -----
m = H.merge(meta_plus[["session_id", "proposal_success", "err_type", "p_funded_cv"]], on="session_id",
# Drop sessions missing early or late entropy

```

```

m2 = m.dropna(subset=["H_early", "H_late"]).copy()

# -----
# 5) Quadrants using medians (robust, no tuning)
# -----
early_med = m2["H_early"].median()
late_med = m2["H_late"].median()

m2["early_bucket"] = np.where(m2["H_early"] >= early_med, "High early entropy", "Low early entropy")
m2["late_bucket"] = np.where(m2["H_late"] >= late_med, "High late entropy", "Low late entropy")
m2["quadrant"] = m2["early_bucket"] + " / " + m2["late_bucket"]

# -----
# 6) Funding rates by quadrant
# -----
quad_summary = (
    m2.groupby("quadrant")
    .agg(
        n=("session_id", "count"),
        funded_rate=("proposal_success", "mean"),
        mean_p_funded_cv=("p_funded_cv", "mean"),
        fp_rate=("err_type", lambda s: (s=="FP").mean()),
        fn_rate=("err_type", lambda s: (s=="FN").mean()),
    )
    .reset_index()
    .sort_values("funded_rate", ascending=False)
)

display(quad_summary)

# -----
# 7) Optional: show which sessions sit in each quadrant
# -----
quad_members = (
    m2[["session_id", "quadrant", "proposal_success", "err_type", "p_funded_cv", "H_early", "H_late", "del
    .sort_values(["quadrant", "p_funded_cv"], ascending=[True, False])
)

display(quad_members)

```

	quadrant	n	funded_rate	mean_p_funded_cv	fp_rate	fn_rate
3	Low early entropy / Low late entropy	6	0.833333	0.301153	0.166667	0.833333
1	High early entropy / Low late entropy	4	0.750000	0.301263	0.250000	0.750000
2	Low early entropy / High late entropy	4	0.250000	0.695987	0.750000	0.250000
0	High early entropy / High late entropy	6	0.166667	0.739180	0.833333	0.166667

	session_id	quadrant	proposal_success	err_type	p_funded_cv	H_early	H_late	delta_entropy
7	2021_05_21_ABI_S5	High early entropy / High late entropy	0	FP	0.900494	1.910039	2.067205	0.157166
11	2021_10_01_MZT_S2	High early entropy / High late entropy	0	FP	0.879218	1.920765	2.072277	0.151512
8	2021_06_10_SLU_S7	High early entropy / High late entropy	0	FP	0.836086	1.918788	2.051418	0.132637
4	2021_04_23_MND_S14	High early entropy / High late entropy	0	FP	0.812520	1.908560	1.964239	0.055679
9	2021_09_30_MZT_S3	High early entropy / High late entropy	0	FP	0.783783	1.912191	2.077500	0.165309

6	2021_05_20_ABI_S7	High early entropy / High late entropy	1	FN	0.222979	1.905287	2.026699	0.121412
13	2021_10_08_CMC_S9	High early entropy / Low late entropy	0	FP	0.772768	1.932272	1.911898	-0.020374
17	2022_04_07_MND_S5	High early entropy / Low late entropy	1	FN	0.229674	1.946234	1.956260	0.010026
16	2022_04_07_MND_S4	High early entropy / Low late entropy	1	FN	0.183988	1.904034	1.951145	0.047117
0	2020_11_06_NES_S12	High early entropy / Low late entropy	1	FN	0.018621	1.907605	1.935495	0.027890
12	2021_10_01_MZT_S8	Low early entropy / High late entropy	0	FP	0.944502	1.773876	1.988888	0.215012
14	2021_11_05_NES_S2	Low early entropy / High late entropy	0	FP	0.892249	1.827359	2.036943	0.209584
5	2021_04_23_MND_S7	Low early entropy / High late entropy	0	FP	0.803345	1.850054	2.009273	0.159219

entropy								
2	2021_04_22_MND_S4	Low early entropy / High late entropy	1	FN	0.143854	1.749719	2.126874	0.377155
19	2022_04_08_MND_S12	Low early entropy / Low late entropy	0	FP	0.996794	1.098612	0.950271	-0.148342
15	2022_04_07_MND_S1	Low early entropy / Low late entropy	1	FN	0.272132	1.833140	1.943248	0.110108
10	2021_09_30_MZT_S6	Low early entropy / Low late entropy	1	FN	0.199107	1.497686	1.601477	0.103795
1	2020_11_06_NES_S3	Low early entropy / Low late entropy	1	FN	0.138559	1.707973	1.442133	-0.265840
18	2022_04_07_MND_S6	Low early entropy / Low late entropy	1	FN	0.125388	1.785127	1.920473	0.135346
3	2021_04_23_MND_S11	Low early entropy / Low late entropy	1	FN	0.074936	1.899102	1.954731	0.055630

entropy

```

In [37]: import statsmodels.api as sm

# -----
# 1) Build entropy slope feature
# -----
# Map phases to numeric time
phase_map = {"beginning": 0, "middle": 1, "end": 2}

phase_ent["t"] = phase_ent["phase"].map(phase_map)
phase_ent = phase_ent.dropna(subset=["t", "H"])

# Fit slope per session
def fit_slope(g):
    if g.shape[0] < 2:
        return np.nan
    X = sm.add_constant(g["t"])
    y = g["H"]
    return sm.OLS(y, X).fit().params["t"]

slopes = (
    phase_ent.groupby("session_id")
    .apply(fit_slope)
    .rename("entropy_slope")
    .reset_index()
)

# -----
# 2) Merge with outcomes
# -----
df_slope = slopes.merge(
    meta_plus[["session_id", "proposal_success", "err_type", "p_funded_cv"]],
    on="session_id",
    how="inner"
)

# -----
# 3) Compare funded vs not
# -----
summary = (

```



```

df_slope.groupby("proposal_success")["entropy_slope"]
    .agg(["count", "mean", "median"])
    .reset_index()
)

display(summary)

# -----
# 4) Simple logistic regression
# -----
X = sm.add_constant(df_slope["entropy_slope"])
y = df_slope["proposal_success"]

logit = sm.Logit(y, X).fit(dis=False)
print(logit.summary())

```

/var/folders/wp/cn7__9416yj3c5_vcykys8hc0000gn/T/ipykernel_60716/3704662565.py:22: FutureWarning: DataFrameGroupBy.apply operated on the grouping columns. This behavior is deprecated, and in a future version of pandas the grouping columns will be excluded from the operation. Either pass `include\_groups=False` to exclude the groupings or explicitly select the grouping columns after groupby to silence this warning.

```

    .apply(fit_slope)

```

	proposal_success	count	mean	median
0	0	10	0.053870	0.077170
1	1	10	0.036131	0.039855

Logit Regression Results

=====						
Dep. Variable:	proposal_success		No. Observations:	20		
Model:	Logit		Df Residuals:	18		
Method:	MLE		Df Model:	1		
Date:	Thu, 08 Jan 2026		Pseudo R-squ.:	0.01328		
Time:	11:44:53		Log-Likelihood:	-13.679		
converged:	True		LL-Null:	-13.863		
Covariance Type:	nonrobust		LLR p-value:	0.5440		
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	0.1906	0.556	0.343	0.732	-0.899	1.280
entropy_slope	-4.2086	7.082	-0.594	0.552	-18.089	9.672
=====						

```
In [40]: import numpy as np
import pandas as pd
from scipy.stats import entropy

# -----
# 0) requirements
# -----
if "U" not in globals():
    raise NameError("U not found (utterance-level df).")
if "meta_plus" not in globals():
    raise NameError("meta_plus not found.")

# -----
# 1) detect columns in U
# -----
phase_col = next((c for c in ["phase", "when", "segment", "part"] if c in U.columns), None)
label_col = next((c for c in ["label", "category", "practice_label", "annotation_label", "code"] if c in U.columns), None)

if phase_col is None or label_col is None:
    raise ValueError(f"Need phase+label columns. Found phase_col={phase_col}, label_col={label_col}")

# -----
# 2) build per-session x phase distribution over labels
```

```

# -----
counts = (
    U.dropna(subset=[phase_col, label_col])
    .groupby(["session_id", phase_col, label_col])
    .size()
    .rename("n")
    .reset_index()
)

totals = (
    counts.groupby(["session_id", phase_col])["n"]
    .sum()
    .rename("N")
    .reset_index()
)

phase_probs_long = counts.merge(totals, on=["session_id", phase_col], how="left")
phase_probs_long["p"] = phase_probs_long["n"] / phase_probs_long["N"]

# -----
# 3) entropy per (session, phase)
# -----
def safe_entropy(pvals):
    p = np.asarray(pvals, dtype=float)
    p = p[p > 0]
    if len(p) == 0:
        return np.nan
    p = p / p.sum()
    return float(entropy(p))

H = (
    phase_probs_long
    .groupby(["session_id", phase_col])["p"]
    .apply(lambda s: safe_entropy(s.values))
    .rename("H")
    .reset_index()
    .rename(columns={phase_col: "phase"})
)

```

```

# Decide early vs late (common case: beginning vs end)
phases = set(H["phase"].unique())
if {"beginning", "end"}.issubset(phases):
    early_phase, late_phase = "beginning", "end"
elif {"early", "late"}.issubset(phases):
    early_phase, late_phase = "early", "late"
else:
    # fallback: if you only have middle/end/etc, you can adjust here
    raise ValueError(f"Can't map early/late phases from: {sorted(phases)}")

entropy_change = (
    H.pivot(index="session_id", columns="phase", values="H")
    .reset_index()
    .rename(columns={early_phase: "H_early", late_phase: "H_late"})
)

entropy_change["delta_entropy"] = entropy_change["H_late"] - entropy_change["H_early"]

print("Built entropy_change with columns:", list(entropy_change.columns))
entropy_change.head()

```

Built entropy_change with columns: ['session_id', 'H_early', 'H_late', 'middle', 'delta_entropy']

```

Out[40]:
  phase      session_id  H_early  H_late  middle  delta_entropy
0  2020_11_06_NES_S12    1.907605  1.935495  1.969160         0.027890
1  2020_11_06_NES_S3     1.707973  1.442133  1.539654        -0.265840
2  2021_04_22_MND_S4     1.749719  2.126874  1.881527         0.377155
3  2021_04_23_MND_S11    1.899102  1.954731  1.684254         0.055630
4  2021_04_23_MND_S14    1.908560  1.964239  1.796406         0.055679

```

```

In [42]: import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D

# df_final must have: session_id, H_early, H_late, proposal_success
# proposal_success should be 0/1

```

```

df_final = meta_plus.merge(
    entropy_change[["session_id", "H_early", "H_late", "delta_entropy"]],
    on="session_id",
    how="inner"
).copy()

# --- settings ---
x = df_final["H_early"].astype(float).values
y = df_final["H_late"].astype(float).values
funded = df_final["proposal_success"].astype(int).values

# axis bounds with padding
pad = 0.05
xmin, xmax = x.min() - pad, x.max() + pad
ymin, ymax = y.min() - pad, y.max() + pad
lo = min(xmin, ymin)
hi = max(xmax, ymax)

# medians for quadrant lines (optional but useful)
mx = np.median(x)
my = np.median(y)

plt.figure(figsize=(7, 7))

# Plot not funded (0) and funded (1) with different markers
mask0 = funded == 0
mask1 = funded == 1

plt.scatter(x[mask0], y[mask0], marker="o", s=90, alpha=0.85, edgecolor="k", linewidth=0.6, label="")
plt.scatter(x[mask1], y[mask1], marker="^", s=110, alpha=0.85, edgecolor="k", linewidth=0.6, label="")

# y = x reference line
plt.plot([lo, hi], [lo, hi], linestyle="--", linewidth=1.8)

# Quadrant median lines
plt.axvline(mx, linestyle=":", linewidth=1.2)
plt.axhline(my, linestyle=":", linewidth=1.2)

# Optional: label points (turn on if you want)

```

```

LABEL_POINTS = False
if LABEL_POINTS:
    for _, r in df_final.iterrows():
        plt.text(r["H_early"] + 0.01, r["H_late"] + 0.01, r["session_id"], fontsize=8, alpha=0.8)

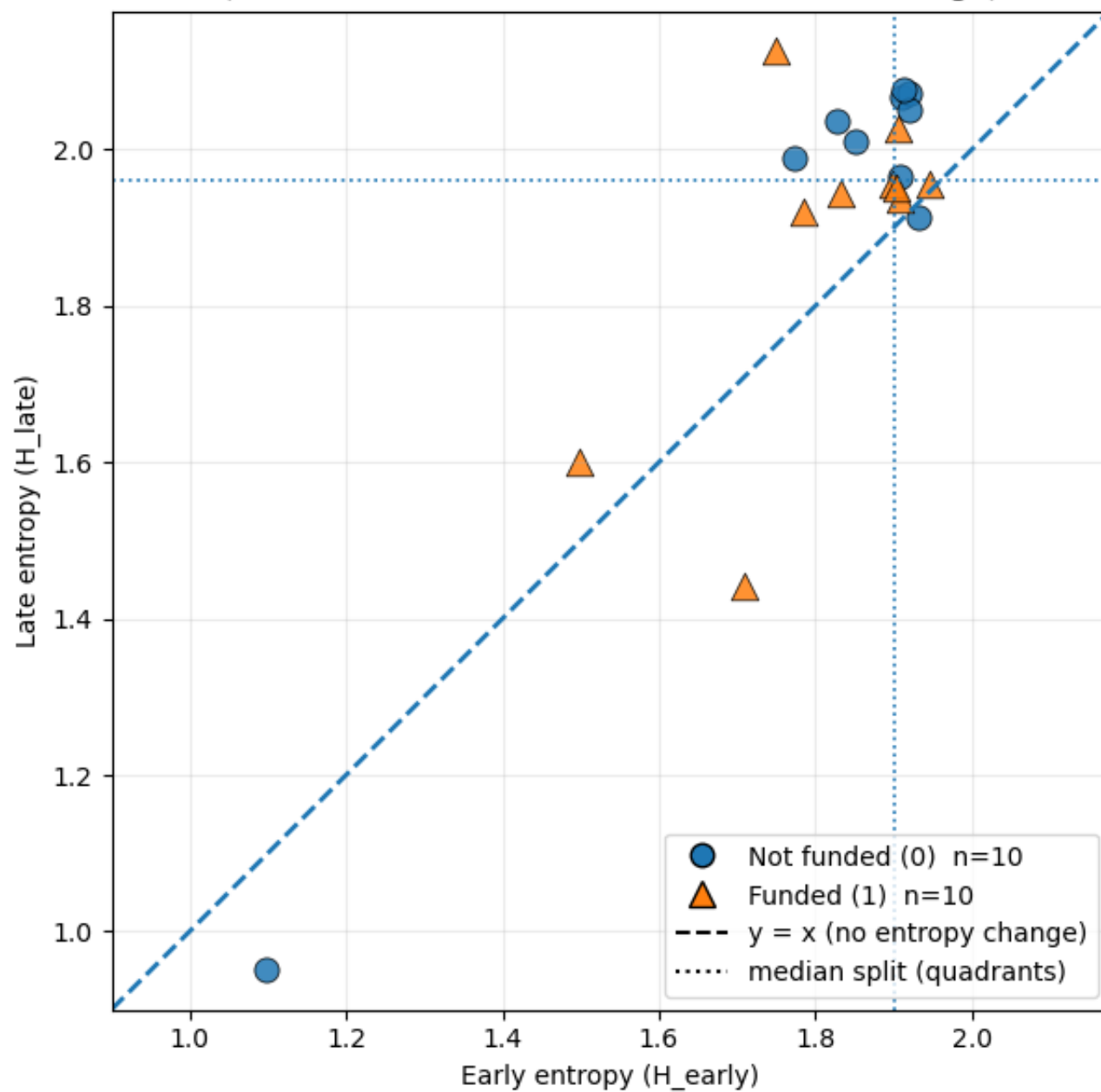
# Cosmetics
plt.xlim(lo, hi)
plt.ylim(lo, hi)
plt.gca().set_aspect("equal", adjustable="box")
plt.grid(True, alpha=0.25)
plt.xlabel("Early entropy (H_early)")
plt.ylabel("Late entropy (H_late)")
plt.title("Idea-space evolution: early vs late entropy\n(marker/color = funded, dashed line = no ch

# Legend with counts
n0 = int(mask0.sum())
n1 = int(mask1.sum())
legend_elements = [
    Line2D([0], [0], marker='o', color='w', label=f'Not funded (0) n={n0}',
           markerfacecolor='C0', markeredgecolor='k', markersize=9),
    Line2D([0], [0], marker='^', color='w', label=f'Funded (1) n={n1}',
           markerfacecolor='C1', markeredgecolor='k', markersize=10),
    Line2D([0], [0], linestyle='--', color='k', label='y = x (no entropy change)'),
    Line2D([0], [0], linestyle=':', color='k', label='median split (quadrants)'),
]
plt.legend(handles=legend_elements, loc="lower right", frameon=True)

plt.show()

```

Idea-space evolution: early vs late entropy
(marker/color = funded, dashed line = no change)



```

In [43]: import pandas as pd
import numpy as np
from scipy.stats import entropy

# =====
# 1) Compute phase-by-label probabilities for ALL sessions
# =====

# Count labels per session per phase
counts = (
    counts
    .groupby(["session_id", "phase", "label"])
    .size()
    .reset_index(name="n")
)

# Normalize to probabilities within each (session, phase)
counts["p"] = (
    counts
    .groupby(["session_id", "phase"])["n"]
    .transform(lambda x: x / x.sum())
)

# This is the canonical long table of idea distributions
phase_probs_long = counts[["session_id", "phase", "label", "p"]]

# =====
# 2) Compute entropy per session per phase
# =====

entropy_long = (
    phase_probs_long
    .groupby(["session_id", "phase"])["p"]
    .apply(lambda x: entropy(x.values))
    .reset_index(name="H")
)

# Pivot to wide format
entropy_wide = (

```



```

entropy_long
.pivot(index="session_id", columns="phase", values="H")
.reset_index()
.rename(columns={
    "beginning": "H_early",
    "middle": "H_middle",
    "end": "H_late"
})
)

# =====
# 3) Compute entropy change metrics
# =====

entropy_wide["delta_entropy"] = entropy_wide["H_late"] - entropy_wide["H_early"]
entropy_wide["entropy_slope"] = entropy_wide["delta_entropy"] # explicit name

# =====
# 4) Merge outcomes where available
# =====

entropy_all = entropy_wide.merge(
    meta_plus[["session_id", "proposal_success", "p_funded_cv"]],
    on="session_id",
    how="left" # keeps all sessions
)

print(f"Total sessions with entropy computed: {len(entropy_all)}")
entropy_all.head()

```

Total sessions with entropy computed: 20

Out[43]:

	session_id	H_early	H_late	H_middle	delta_entropy	entropy_slope	proposal_success	p_fun
0	2020_11_06_NES_S12	1.907605	1.935495	1.969160	0.027890	0.027890	1	0
1	2020_11_06_NES_S3	1.707973	1.442133	1.539654	-0.265840	-0.265840	1	0.
2	2021_04_22_MND_S4	1.749719	2.126874	1.881527	0.377155	0.377155	1	0.
3	2021_04_23_MND_S11	1.899102	1.954731	1.684254	0.055630	0.055630	1	0.
4	2021_04_23_MND_S14	1.908560	1.964239	1.796406	0.055679	0.055679	0	0.

```
In [44]: import pandas as pd
import numpy as np
from scipy.stats import entropy
import matplotlib.pyplot as plt

# =====
# REQUIREMENTS CHECK
# =====
if "U" not in globals():
    raise NameError("U not found. Run the session JSON loading chunk that creates U (utterance-level data).")

if "meta_plus" not in globals():
    raise NameError("meta_plus not found. Run the meta_plus merge chunk first.")

required_cols = {"session_id", "phase", "label"}
missing = required_cols - set(U.columns)
if missing:
    raise ValueError(f"U is missing required columns: {missing}. Available: {list(U.columns)}")

# =====
# 1) Phase-by-label probabilities for ALL sessions
# =====
counts = (
    U.groupby(["session_id", "phase", "label"])
      .size()
      .reset_index(name="n")
)
```

```

)

counts["p"] = counts.groupby(["session_id", "phase"])["n"].transform(lambda x: x / x.sum())
phase_probs_long = counts[["session_id", "phase", "label", "p"]]

# =====
# 2) Entropy per session per phase
# =====
entropy_long = (
    phase_probs_long
    .groupby(["session_id", "phase"])["p"]
    .apply(lambda x: entropy(x.values))
    .reset_index(name="H")
)

entropy_wide = (
    entropy_long
    .pivot(index="session_id", columns="phase", values="H")
    .reset_index()
)

# Rename phase columns if present
rename_map = {}
if "beginning" in entropy_wide.columns: rename_map["beginning"] = "H_early"
if "middle" in entropy_wide.columns: rename_map["middle"] = "H_middle"
if "end" in entropy_wide.columns: rename_map["end"] = "H_late"
entropy_wide = entropy_wide.rename(columns=rename_map)

# Ensure columns exist even if a phase is missing for some sessions
for col in ["H_early", "H_middle", "H_late"]:
    if col not in entropy_wide.columns:
        entropy_wide[col] = np.nan

# =====
# 3) Entropy change metrics
# =====
entropy_wide["delta_entropy"] = entropy_wide["H_late"] - entropy_wide["H_early"]
entropy_wide["entropy_slope"] = entropy_wide["delta_entropy"] # alias if you want "slope" language

```

```

# =====
# 4) Merge outcomes (keeps all sessions; outcomes may be NaN)
# =====
keep_cols = [c for c in ["session_id", "proposal_success", "p_funded_cv", "err_type"] if c in meta_
entropy_all = entropy_wide.merge(meta_plus[keep_cols], on="session_id", how="left")

print(f"Total sessions with entropy computed: {len(entropy_all)}")
print(f"Sessions with known outcome (proposal_success not null): {entropy_all['proposal_success'].r

# =====
# 5) Quick summaries
# =====
if "proposal_success" in entropy_all.columns:
    print("\nDelta entropy summary by proposal_success (known outcomes only):")
    print(
        entropy_all.dropna(subset=["proposal_success"])
            .groupby("proposal_success")["delta_entropy"]
            .agg(["count", "mean", "median"])
    )

# =====
# 6) Plot: Early vs Late entropy (all sessions; marker by funded if known)
# =====
plt.figure(figsize=(8, 7))

known = entropy_all.dropna(subset=["H_early", "H_late"])
funded = known[known["proposal_success"] == 1] if "proposal_success" in known.columns else known.i
unfunded = known[known["proposal_success"] == 0] if "proposal_success" in known.columns else known.i
unknown = known[known["proposal_success"].isna()] if "proposal_success" in known.columns else knowr

# Plot unfunded / funded / unknown (unknown is optional but helpful when scaling)
if len(unfunded) > 0:
    plt.scatter(unfunded["H_early"], unfunded["H_late"], alpha=0.6, label=f"Not funded (0)  n={len(
if len(funded) > 0:
    plt.scatter(funded["H_early"], funded["H_late"], marker="^", alpha=0.8, label=f"Funded (1)  n={
if len(unknown) > 0:
    plt.scatter(unknown["H_early"], unknown["H_late"], marker="x", alpha=0.35, label=f"Outcome unkr

# y = x line (no entropy change)

```

```

lims = [
    float(np.nanmin([known["H_early"].min(), known["H_late"].min()])),
    float(np.nanmax([known["H_early"].max(), known["H_late"].max()]))
]
plt.plot(lims, lims, "--", color="gray", label="y = x (no entropy change)")

plt.xlabel("Early entropy (H_early)")
plt.ylabel("Late entropy (H_late)")
plt.title("Idea-space evolution: early vs late entropy (all sessions)")
plt.legend(frameon=True)
plt.grid(True, alpha=0.2)
plt.tight_layout()
plt.show()

# =====
# 7) Return the main dataframe you'll use downstream
# =====
entropy_all.head()

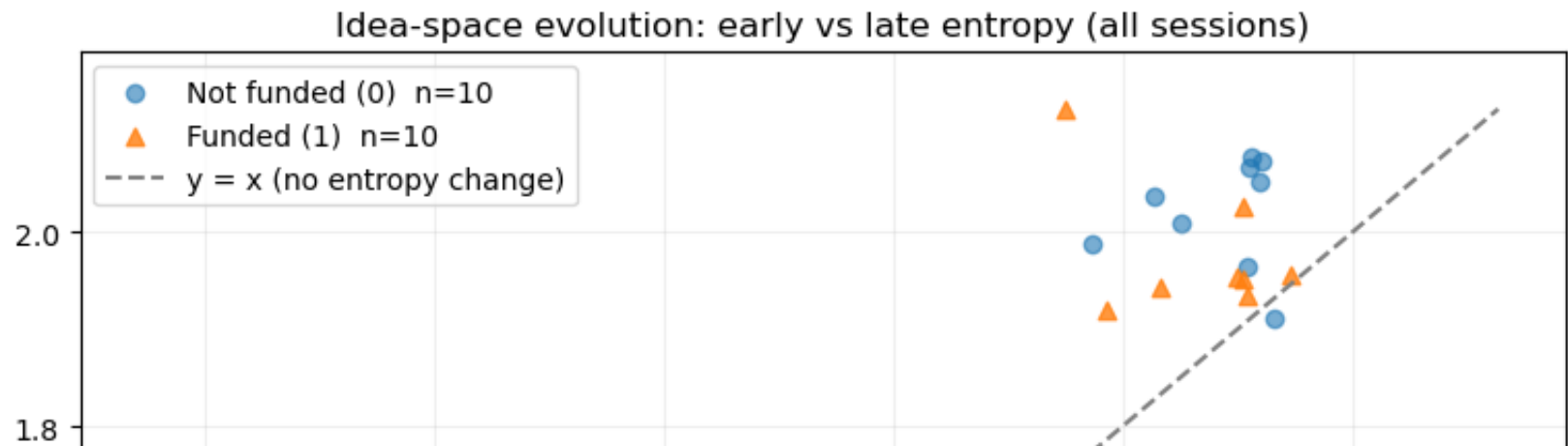
```

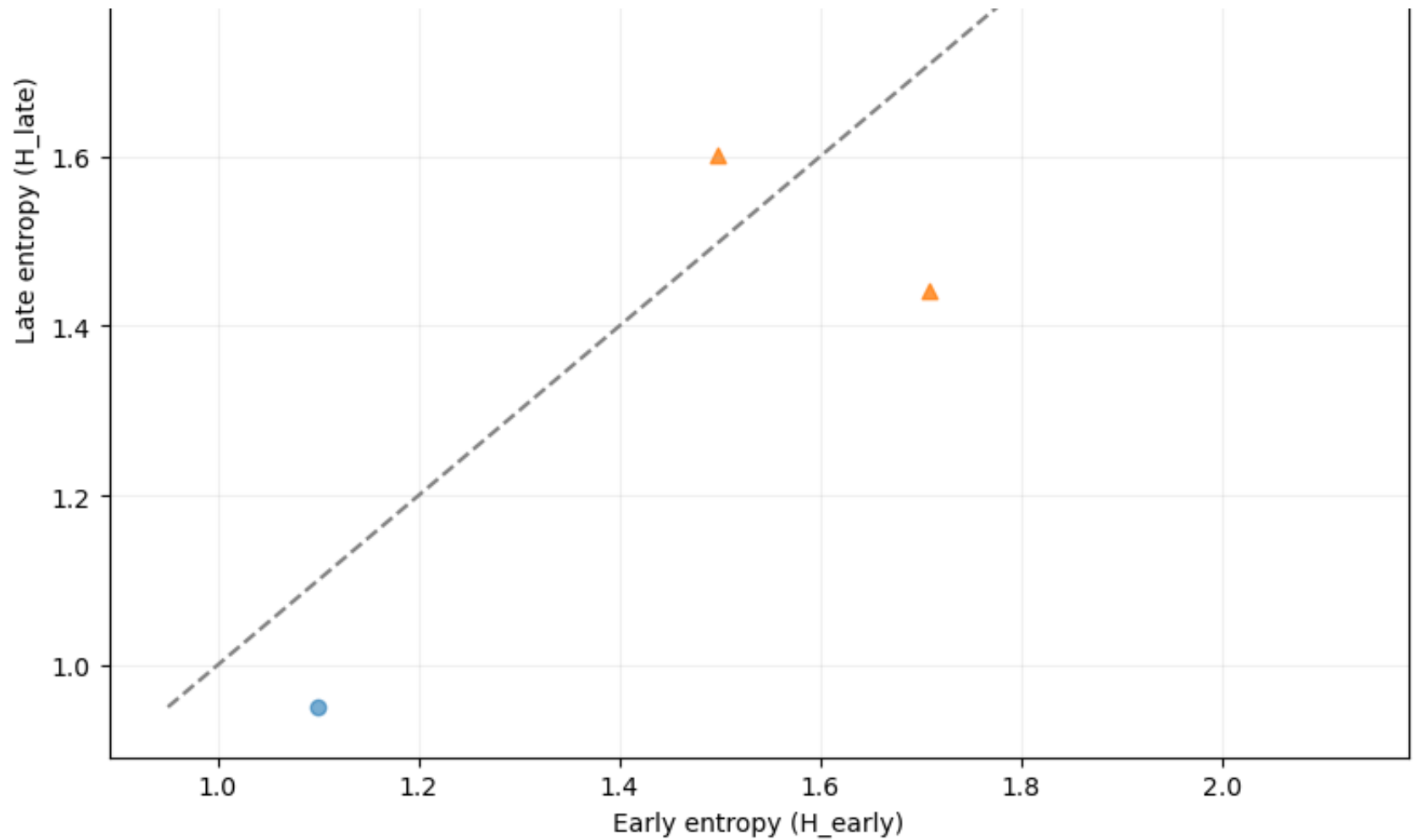
Total sessions with entropy computed: 20

Sessions with known outcome (proposal_success not null): 20

Delta entropy summary by proposal_success (known outcomes only):

	count	mean	median
proposal_success			
0	10	0.107740	0.154339
1	10	0.072263	0.079710





Out[44]:

	session_id	H_early	H_late	H_middle	delta_entropy	entropy_slope	proposal_success	p_fun
0	2020_11_06_NES_S12	1.907605	1.935495	1.969160	0.027890	0.027890	1	0
1	2020_11_06_NES_S3	1.707973	1.442133	1.539654	-0.265840	-0.265840	1	0.
2	2021_04_22_MND_S4	1.749719	2.126874	1.881527	0.377155	0.377155	1	0.
3	2021_04_23_MND_S11	1.899102	1.954731	1.684254	0.055630	0.055630	1	0.
4	2021_04_23_MND_S14	1.908560	1.964239	1.796406	0.055679	0.055679	0	0.

```

In [46]: import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# =====
# REQUIREMENTS / COLUMN DETECTION
# =====
if "U" not in globals():
    raise NameError("U not found. Run the loader that creates U (utterance-level df).")
if "meta_plus" not in globals():
    raise NameError("meta_plus not found. Run the meta_plus merge chunk first.")

U_ = U.copy()
M_ = meta_plus.copy()

# Detect columns (use your actual column names if different)
phase_col = next((c for c in ["phase", "when", "segment", "part"] if c in U_.columns), None)
label_col = next((c for c in ["label", "category", "practice_label", "annotation_label", "code"] if c in U_.columns), None)
text_col = next((c for c in ["transcript", "text", "utterance", "content"] if c in U_.columns), None)

if phase_col is None:
    raise ValueError(f"Could not find phase column. U columns: {list(U_.columns)}")
if label_col is None:
    raise ValueError(f"Could not find label column. U columns: {list(U_.columns)}")
if text_col is None:
    raise ValueError(f"Could not find transcript/text column. U columns: {list(U_.columns)}")

# Normalize phase naming if needed (optional)
# Expect: beginning/middle/end. If you have different, map here.
# Example:
# U_[phase_col] = U_[phase_col].replace({"start": "beginning", "late": "end"})

print(f"Using phase_col='{phase_col}' | label_col='{label_col}' | text_col='{text_col}'")
print(f"Total utterances in U: {len(U_):,}")
print(f"Total sessions in U: {U_['session_id'].nunique():,}")
print(f"Total sessions in meta_plus: {M_['session_id'].nunique():,}")

```

```

# =====
# 1) REGEX FLAGS: logistics/report-out/convergence-ish markers
#   (Keep these conservative; add terms as you validate)
# =====
LOGISTICS_STRICT_PAT = re.compile(
    r"\b(muted|unmute|share your screen|screen share|slides?|powerpoint|ppt|"
    r"btime\b|minutes?|seconds?|breakout|main room|report out|"
    r"wrong slide|slide \d+|read[- ]only|file)\b",
    flags=re.IGNORECASE
)

REPORTOUT_PAT = re.compile(
    r"\b(report out|report-out|summary|summarize|key points?|bullet points?|"
    r"major points?|takeaways?|we'll go to each room|concise summary)\b",
    flags=re.IGNORECASE
)

# "Convergence" here is a placeholder lexical proxy; treat it as exploratory.
CONVERGENCE_PAT = re.compile(
    r"\b(so we agree|we agree|consensus|align(ed|ment)?|settle|finalize|"
    r"wrap up|decision|decide|conclusion|takeaway|key message|"
    r"next steps|action items?)\b",
    flags=re.IGNORECASE
)

def _flag(pat, x):
    if not isinstance(x, str):
        return 0
    return int(bool(pat.search(x)))

U_["is_logistics_strict"] = U_[text_col].apply(lambda x: _flag(LOGISTICS_STRICT_PAT, x))
U_["is_reportout"]       = U_[text_col].apply(lambda x: _flag(REPORTOUT_PAT, x))
U_["is_convergence"]     = U_[text_col].apply(lambda x: _flag(CONVERGENCE_PAT, x))

# =====
# 2) MECHANISM FEATURES (session-level shares)
#   A) "end Coordination/Decision" logistics share
#   B) "end" overall report-out share
#   C) "end" overall convergence lexical share (exploratory)

```



```

# =====
END = "end"
COORD_LABEL = "Coordination and Decision Practices"

end_coord = U_[(U_[phase_col] == END) & (U_[label_col] == COORD_LABEL)].copy()
end_all = U_[U_[phase_col] == END].copy()

end_coord_logistics_share_strict = (
    end_coord.groupby("session_id")["is_logistics_strict"].mean()
    .rename("end_coord_logistics_share_strict")
    .reset_index()
)

end_reportout_share = (
    end_all.groupby("session_id")["is_reportout"].mean()
    .rename("end_reportout_share")
    .reset_index()
)

end_convergence_share = (
    end_all.groupby("session_id")["is_convergence"].mean()
    .rename("end_convergence_share")
    .reset_index()
)

# =====
# 3) ENTROPY FEATURES (phase-wise label distribution)
# Entropy over label distribution within each phase of each session.
# =====
def entropy_from_counts(counts: np.ndarray) -> float:
    counts = counts.astype(float)
    s = counts.sum()
    if s <= 0:
        return np.nan
    p = counts / s
    p = p[p > 0]
    return float(-(p * np.log(p)).sum())

# Count labels within (session, phase, label)

```

```

phase_label_counts = (
    U_.dropna(subset=["session_id", phase_col, label_col])
    .groupby(["session_id", phase_col, label_col])
    .size()
    .rename("n")
    .reset_index()
)

# Entropy per (session, phase)
H_phase = (
    phase_label_counts.groupby(["session_id", phase_col])["n"]
    .apply(lambda s: entropy_from_counts(s.values))
    .rename("H")
    .reset_index()
)

# Pivot to columns
H_pivot = H_phase.pivot(index="session_id", columns=phase_col, values="H").reset_index()

# Robustly pick early/late
# Prefer beginning/end; fall back to first/last phase alphabetically if needed.
phase_cols = [c for c in H_pivot.columns if c != "session_id"]
if "beginning" in phase_cols and "end" in phase_cols:
    H_pivot["H_early"] = H_pivot["beginning"]
    H_pivot["H_late"] = H_pivot["end"]
elif "start" in phase_cols and "end" in phase_cols:
    H_pivot["H_early"] = H_pivot["start"]
    H_pivot["H_late"] = H_pivot["end"]
else:
    # fallback: pick min/max phase column name (not ideal, but prevents crashing)
    phase_sorted = sorted(phase_cols)
    H_pivot["H_early"] = H_pivot[phase_sorted[0]]
    H_pivot["H_late"] = H_pivot[phase_sorted[-1]]
    print(f"[WARN] Couldn't find explicit beginning/end. Using phases: early='{phase_sorted[0]}' late='{phase_sorted[-1]}'")

H_pivot["delta_entropy"] = H_pivot["H_late"] - H_pivot["H_early"]

# =====
# 4) MERGE EVERYTHING INTO ONE SESSION FEATURE TABLE (ALL SESSIONS)

```

```

# =====
session_features = (
    pd.DataFrame({"session_id": sorted(U_["session_id"].unique())})
    .merge(end_coord_logistics_share_strict, on="session_id", how="left")
    .merge(end_reportout_share, on="session_id", how="left")
    .merge(end_convergence_share, on="session_id", how="left")
    .merge(H_pivot[["session_id", "H_early", "H_late", "delta_entropy"]], on="session_id", how="left")
    .merge(M_, on="session_id", how="left") # brings in proposal_success, p_funded_cv, err_type,
)

# Replace missing shares with 0 when "no utterances" for that subset
# (If a session has no end_coord rows, we can't compute end_coord_logistics_share; set 0 for conver
for c in ["end_coord_logistics_share_strict", "end_reportout_share", "end_convergence_share"]:
    if c in session_features.columns:
        session_features[c] = session_features[c].fillna(0.0)

print("\nSession_features built.")
print("Total sessions:", len(session_features))
print("Sessions with known outcome (proposal_success not null):", session_features["proposal_success"]

# =====
# 5) QUICK SUMMARIES YOU CAN SAY OUT LOUD
# =====
if "err_type" in session_features.columns:
    print("\nMean end_coord_logistics_share_strict by err_type:")
    print(session_features.groupby("err_type")["end_coord_logistics_share_strict"].agg(["count", "me

if "proposal_success" in session_features.columns:
    print("\nDelta entropy by proposal_success (known only):")
    print(session_features[session_features["proposal_success"].notna()]
            .groupby("proposal_success")["delta_entropy"].agg(["count", "mean", "median"]))

# =====
# 6) PLOTS FOR BRIAN: NOT "ONE ENTROPY GRAPH", BUT A STORY
#     Plot A: Mechanism map (logistics-coordination vs report-out)
#     Plot B: Process vs model confidence (delta entropy vs p_funded_cv)
# =====
# Helper styling
def _marker_for_funded(v):

```

```

    return "^" if v == 1 else "o"

def _edge_for_err(err):
    # edgecolor provides quick error-type cue without relying on weird colormaps
    if err == "FP":
        return "black"
    if err == "FN":
        return "gray"
    return "none"

df_plot = session_features.copy()

# Plot A
plt.figure(figsize=(8,6))
for _, r in df_plot.iterrows():
    funded = r.get("proposal_success", np.nan)
    err = r.get("err_type", None)
    x = r.get("end_coord_logistics_share_strict", np.nan)
    y = r.get("end_reportout_share", np.nan)
    if pd.isna(x) or pd.isna(y):
        continue
    plt.scatter(
        x, y,
        marker=_marker_for_funded(funded) if not pd.isna(funded) else "x",
        s=80,
        alpha=0.85,
        edgecolors=_edge_for_err(err),
        linewidths=1.2
    )

plt.xlabel("End Coordination: logistics share (strict)")
plt.ylabel("End phase: report-out share")
plt.title("End-phase coordination: logistics vs report-out\n(marker=funded, edge=error type)")
plt.grid(True, alpha=0.25)

# manual legend
from matplotlib.lines import Line2D
legend_elems = [
    Line2D([0],[0], marker="o", color="w", label="Not funded (0)", markerfacecolor="C0", markersize

```

```

Line2D([0],[0], marker="^", color="w", label="Funded (1)", markerfacecolor="C1", markersize=10,
Line2D([0],[0], marker="o", color="w", label="Edge black = FP", markerfacecolor="w", markeredge
Line2D([0],[0], marker="o", color="w", label="Edge gray = FN", markerfacecolor="w", markeredgec
]
plt.legend(handles=legend_elems, loc="best", frameon=True)
plt.show()

# Plot B (only if model prob exists)
if "p_funded_cv" in df_plot.columns:
    plt.figure(figsize=(8,6))
    for _, r in df_plot.iterrows():
        funded = r.get("proposal_success", np.nan)
        err = r.get("err_type", None)
        x = r.get("p_funded_cv", np.nan)
        y = r.get("delta_entropy", np.nan)
        if pd.isna(x) or pd.isna(y):
            continue
        plt.scatter(
            x, y,
            marker=_marker_for_funded(funded) if not pd.isna(funded) else "x",
            s=80,
            alpha=0.85,
            edgecolors=_edge_for_err(err),
            linewidths=1.2
        )
    plt.axhline(0, linestyle="--", linewidth=1)
    plt.xlabel("Model confidence (p_funded_cv)")
    plt.ylabel("Δ entropy (late - early)")
    plt.title("Process shift vs model confidence\n(marker=funded, edge=error type)")
    plt.grid(True, alpha=0.25)
    plt.legend(handles=legend_elems[:4], loc="best", frameon=True)
    plt.show()

# Keep a named object for later chunks (so you stop getting NameErrors)
entropy_change = session_features[["session_id", "H_early", "H_late", "delta_entropy"]].copy()

# Preview
display(session_features.sort_values(["proposal_success", "p_funded_cv"], ascending=[False, False]).

```

Using phase_col='phase' | label_col='label' | text_col='transcript'
Total utterances in U: 5,559
Total sessions in U: 20
Total sessions in meta_plus: 20

Session_features built.
Total sessions: 20
Sessions with known outcome (proposal_success not null): 20

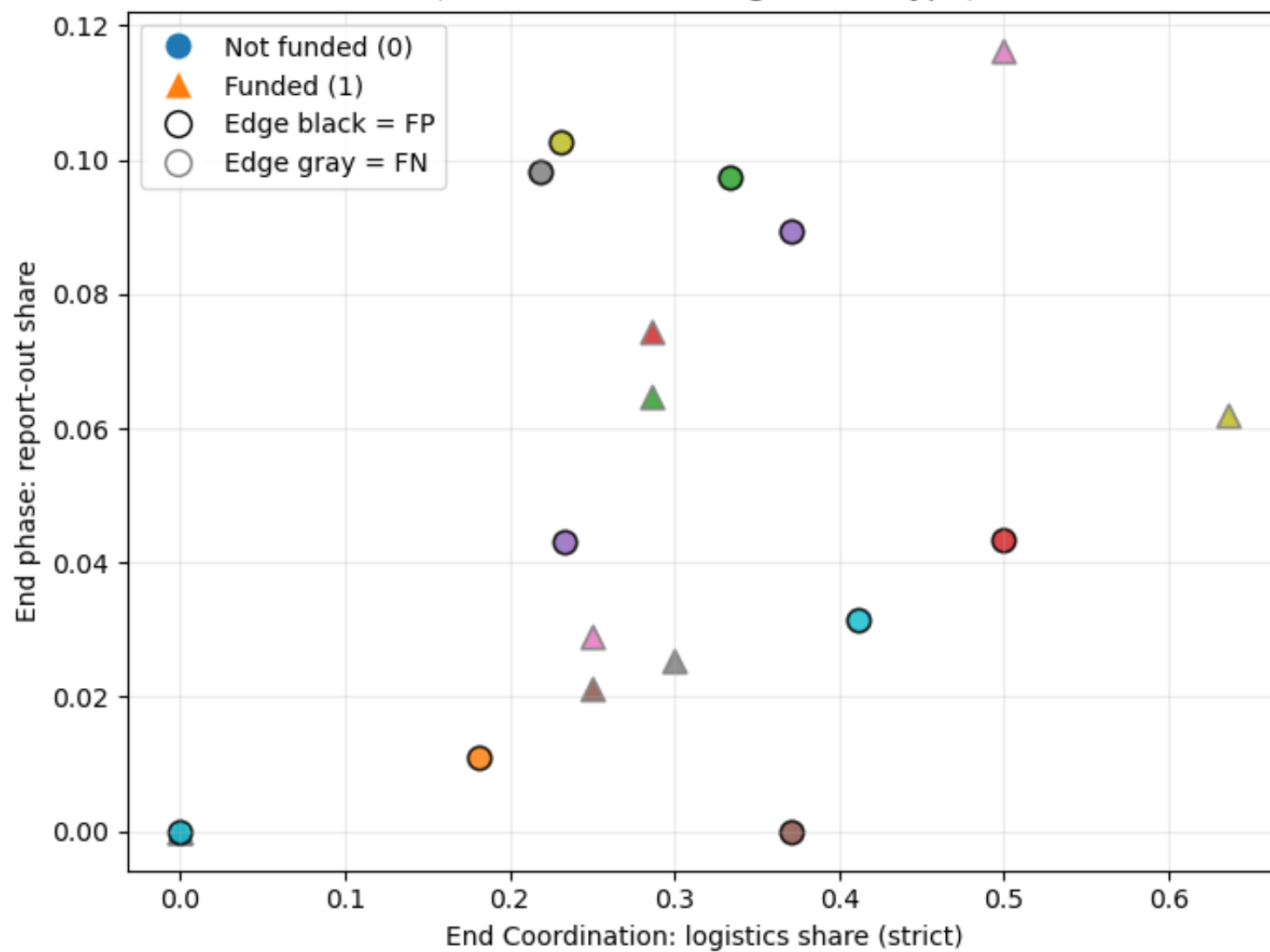
Mean end_coord_logistics_share_strict by err_type:

	count	mean	median
err_type			
FP	10	0.285051	0.283333
FN	10	0.250779	0.267857

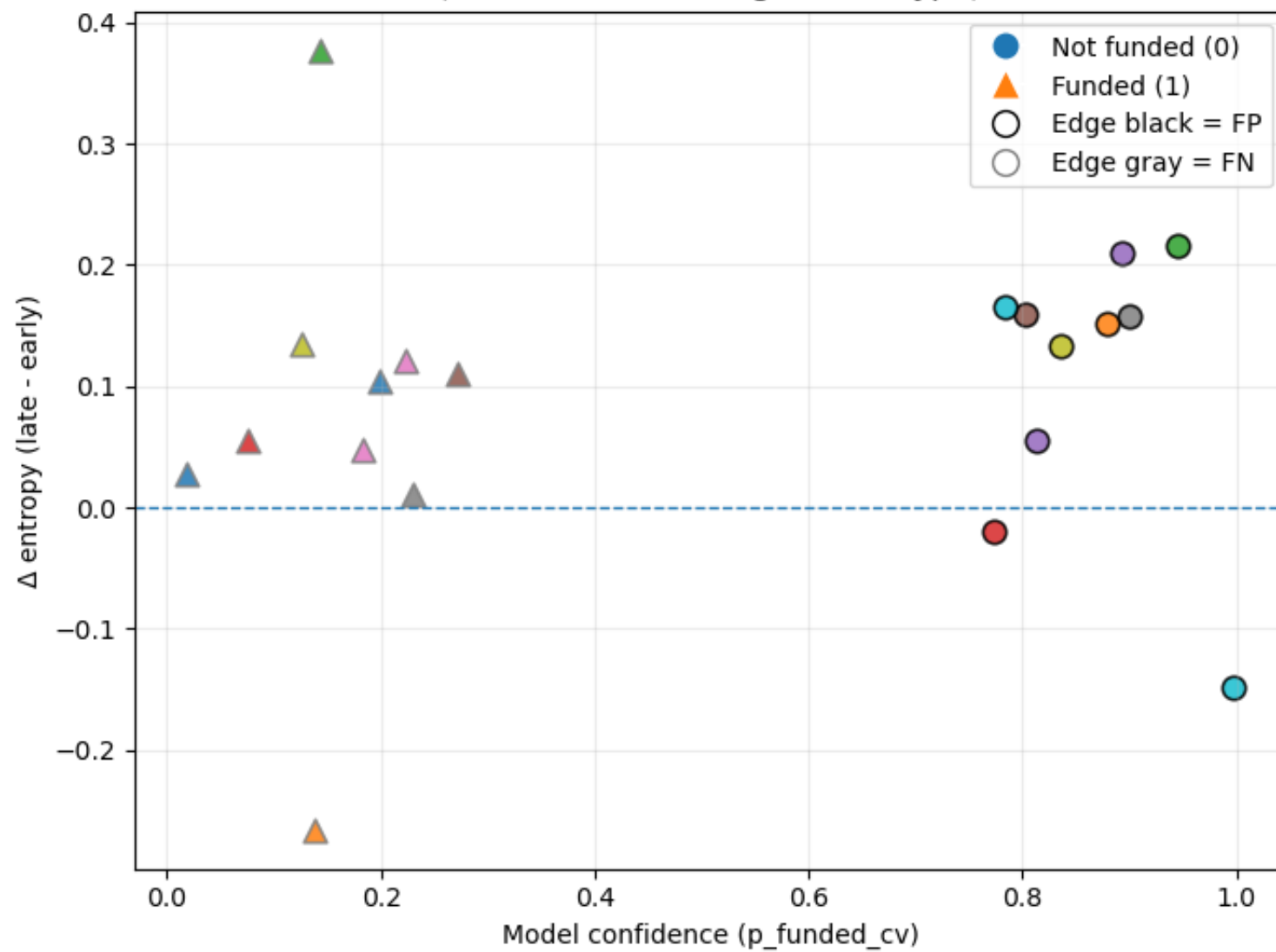
Delta entropy by proposal_success (known only):

	count	mean	median
proposal_success			
0	10	0.107740	0.154339
1	10	0.072263	0.079710

End-phase coordination: logistics vs report-out
(marker=funded, edge=error type)



Process shift vs model confidence
(marker=funded, edge=error type)



	session_id	end_coord_logistics_share_strict	end_reportout_share	end_convergence_share	H_ea
15	2022_04_07_MND_S1	0.250000	0.021277	0.000000	1.8331
17	2022_04_07_MND_S5	0.300000	0.025316	0.000000	1.9462
6	2021_05_20_ABI_S7	0.500000	0.116279	0.000000	1.9052
10	2021_09_30_MZT_S6	0.000000	0.000000	0.000000	1.4976
16	2022_04_07_MND_S4	0.250000	0.028986	0.000000	1.9040
2	2021_04_22_MND_S4	0.285714	0.064748	0.000000	1.7497
1	2020_11_06_NES_S3	0.000000	0.000000	0.000000	1.7079
18	2022_04_07_MND_S6	0.636364	0.062016	0.000000	1.7851
3	2021_04_23_MND_S11	0.285714	0.074380	0.024793	1.8991
0	2020_11_06_NES_S12	0.000000	0.000000	0.040816	1.9076
19	2022_04_08_MND_S12	0.000000	0.000000	0.000000	1.0986
12	2021_10_01_MZT_S8	0.333333	0.097345	0.000000	1.7738

12 rows × 89 columns

almost there (final graphs i hope)

```
In [47]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# =====
# REQUIREMENTS
```

```

# =====
required = ["session_features", "meta_plus"]
for r in required:
    if r not in globals():
        raise NameError(f"{r} not found. Run the chunk that builds `{r}` first.")

sf = session_features.copy()
mp = meta_plus[["session_id", "proposal_success", "p_funded_cv", "err_type"]].copy()

# =====
# 1) Map your annotation labels -> 4 "idea-state" buckets
#     (Edit this mapping to match your codebook semantics)
# =====
STATE_MAP = {
    # Idea emergence / proposing directions
    "Seeding": {
        "Seeding",
        "Information Seeking",    # often question-driven seeding
        "Idea Management",       # can include proposing/parking ideas
    },

    # Critique, constraints, challenges, evaluation pressure
    "Stress-testing": {
        "Stress-testing",
        "Evaluation Practices",
        "Information Seeking",    # sometimes belongs here; keep or remove
    },

    # Integration, combining, synthesizing ideas
    "Synthesis": {
        "Synthesis",
        "Integration Practices",
        "Knowledge Sharing",
    },

    # Decisions, commitments, wrap-up of choices (NOT logistics)
    "Closure": {
        "Commitment/Closure",
        "Coordination and Decision Practices",
    },
}

```

```

    },
}

# If you have slightly different label names in the wide table, normalize them:
# wide columns are like "beginning__Knowledge Sharing" or "end_Coordination and Decision Practices"
def parse_phase_label(col: str):
    if "__" in col:
        phase, label = col.split("__", 1)
        return phase, label
    if "_" in col:
        phase, label = col.split("_", 1)
        return phase, label
    return None, None

PHASES = ["beginning", "middle", "end"]

# Collect all (phase,label) columns in session_features
phase_label_cols = []
for c in sf.columns:
    ph, lab = parse_phase_label(c)
    if ph in PHASES and isinstance(lab, str) and len(lab) > 0:
        phase_label_cols.append((c, ph, lab))

if len(phase_label_cols) == 0:
    raise ValueError("No phase×label columns found in session_features. "
                     "Expected columns like 'beginning__Seeding' or 'end_Knowledge Sharing'.")

# =====
# 2) Build phase-wise shares for each bucket
# =====
state_phase = sf[["session_id"]].copy()

def get_cols_for(phase, labels_set):
    cols = []
    for c, ph, lab in phase_label_cols:
        if ph == phase and lab in labels_set:
            cols.append(c)
    return cols

```

```

for ph in PHASES:
    # Denominator: sum of all label shares in that phase (should be ~1, but be safe)
    denom_cols = [c for c, p, _ in phase_label_cols if p == ph]
    denom = sf[denom_cols].sum(axis=1).replace(0, np.nan)

    for state, labels_set in STATE_MAP.items():
        cols = get_cols_for(ph, labels_set)
        if len(cols) == 0:
            # If none found, fill with 0 and keep going
            state_phase[f"{ph}_{state}"] = 0.0
        else:
            state_phase[f"{ph}_{state}"] = (sf[cols].sum(axis=1) / denom).fillna(0.0)

# =====
# 3) Trajectory features (interpretable, low-dimensional)
# =====
state_phase["closure_gain"] = state_phase["end_Closure"] - state_phase["beginning_Closure"]
state_phase["synthesis_gain"] = state_phase["end_Synthesis"] - state_phase["beginning_Synthesis"]
state_phase["stress_gain"] = state_phase["end_Stress-testing"] - state_phase["beginning_Stress-t"]
state_phase["seed_drop"] = state_phase["beginning_Seeding"] - state_phase["end_Seeding"]

# Entropy features if present
for c in ["H_early", "H_late", "delta_entropy"]:
    if c in sf.columns:
        state_phase[c] = sf[c].values

# "Pruning" = negative entropy change (bigger means more pruning/convergence)
if "delta_entropy" in state_phase.columns:
    state_phase["pruning_score"] = -state_phase["delta_entropy"]
else:
    state_phase["pruning_score"] = np.nan

# Merge outcomes/model diagnostics
df_plot = state_phase.merge(mp, on="session_id", how="inner")

# =====
# 4) Brian-friendly plot: closure gain vs entropy change
#     - marker = funded
#     - edge = error type (FP black, FN gray, else light)

```

```

# =====
def edge_color(err):
    if err == "FP":
        return "black"
    if err == "FN":
        return "gray"
    return "lightgray"

fig, ax = plt.subplots(figsize=(9, 6))

# Build funded vs not funded layers
for funded_val, marker, label in [(0, "o", "Not funded (0)"), (1, "^", "Funded (1)"]]:
    sub = df_plot[df_plot["proposal_success"] == funded_val]
    if len(sub) == 0:
        continue
    ax.scatter(
        sub["closure_gain"],
        sub["delta_entropy"] if "delta_entropy" in sub.columns else np.zeros(len(sub)),
        marker=marker,
        s=90,
        edgecolors=[edge_color(e) for e in sub["err_type"]],
        linewidths=1.5,
        alpha=0.9,
        label=f"{label} n={len(sub)}",
    )

ax.axhline(0, linestyle="--", linewidth=1)
ax.axvline(0, linestyle="--", linewidth=1)
ax.set_title("Mechanism view: closure gain vs idea-space expansion ( $\Delta$  entropy)\n(marker=funded, edge=err_type)")
ax.set_xlabel("Closure gain (end_Closure - beginning_Closure)")
ax.set_ylabel(" $\Delta$  entropy (late - early)  [>0 = expands, <0 = prunes]")
ax.grid(True, alpha=0.25)
ax.legend(frameon=True)
plt.show()

# =====
# 5) Small tables: "most convergent funded" vs "most divergent not funded"
# =====
cols_show = [

```

```

    "session_id", "proposal_success", "err_type", "p_funded_cv",
    "delta_entropy", "pruning_score",
    "closure_gain", "synthesis_gain",
    "beginning_Seeding", "end_Seeding",
    "beginning_Closure", "end_Closure",
]

cols_show = [c for c in cols_show if c in df_plot.columns]

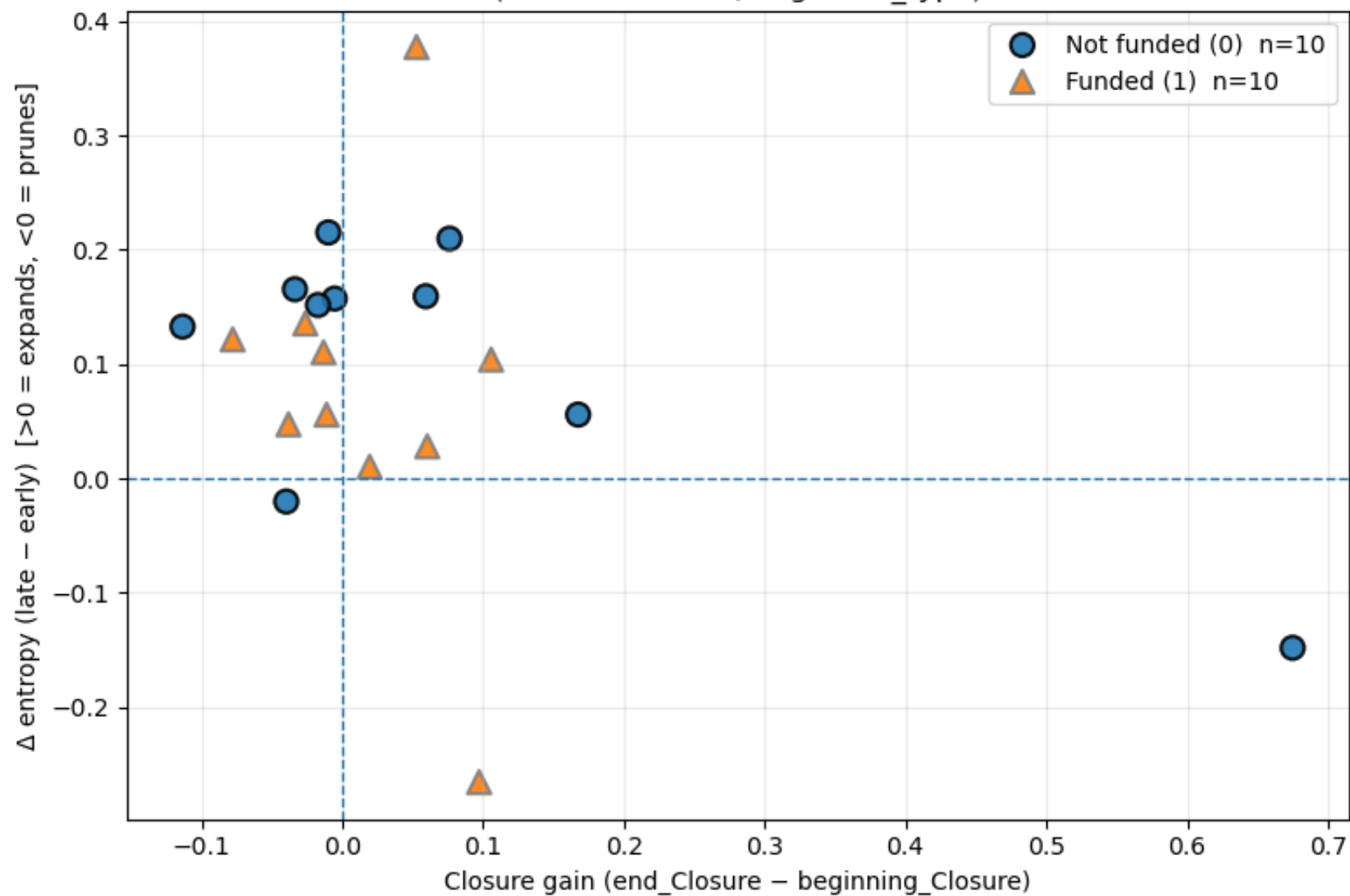
print("\nMost pruning / convergent FUNDED sessions (lowest  $\Delta$  entropy):")
display(
    df_plot[df_plot["proposal_success"] == 1]
    .sort_values("delta_entropy", ascending=True)
    [cols_show]
    .head(8)
)

print("\nMost expanding / divergent NOT-FUNDED sessions (highest  $\Delta$  entropy):")
display(
    df_plot[df_plot["proposal_success"] == 0]
    .sort_values("delta_entropy", ascending=False)
    [cols_show]
    .head(8)
)

# =====
# 6) Save for later (so you stop losing variables between cells)
# =====
idea_flow_features = df_plot.copy()
print(f"\nBuilt idea_flow_features: {idea_flow_features.shape[0]} sessions × {idea_flow_features.sh")

```

Mechanism view: closure gain vs idea-space expansion (Δ entropy)
(marker=funded, edge=err_type)



Most pruning / convergent FUNDED sessions (lowest Δ entropy):

	session_id	proposal_success	err_type	p_funded_cv	delta_entropy	pruning_score	closure_gain
1	2020_11_06_NES_S3	1	FN	0.138559	-0.265840	0.265840	0.097222
17	2022_04_07_MND_S5	1	FN	0.229674	0.010026	-0.010026	0.019500
0	2020_11_06_NES_S12	1	FN	0.018621	0.027890	-0.027890	0.060391
16	2022_04_07_MND_S4	1	FN	0.183988	0.047111	-0.047111	-0.038166
3	2021_04_23_MND_S11	1	FN	0.074936	0.055630	-0.055630	-0.011103
10	2021_09_30_MZT_S6	1	FN	0.199107	0.103791	-0.103791	0.105773
15	2022_04_07_MND_S1	1	FN	0.272132	0.110108	-0.110108	-0.013258
6	2021_05_20_ABI_S7	1	FN	0.222979	0.121412	-0.121412	-0.077776

Most expanding / divergent NOT-FUNDED sessions (highest Δ entropy):

	session_id	proposal_success	err_type	p_funded_cv	delta_entropy	pruning_score	closure_gain
12	2021_10_01_MZT_S8	0	FP	0.944502	0.215012	-0.215012	-0.009824
14	2021_11_05_NES_S2	0	FP	0.892249	0.209584	-0.209584	0.076193
9	2021_09_30_MZT_S3	0	FP	0.783783	0.165309	-0.165309	-0.033584
5	2021_04_23_MND_S7	0	FP	0.803345	0.159219	-0.159219	0.059357
7	2021_05_21_ABI_S5	0	FP	0.900494	0.157166	-0.157166	-0.005354
11	2021_10_01_MZT_S2	0	FP	0.879218	0.151512	-0.151512	-0.017219
8	2021_06_10_SLU_S7	0	FP	0.836086	0.132631	-0.132631	-0.113415
4	2021_04_23_MND_S14	0	FP	0.812520	0.055679	-0.055679	0.167561

Built idea_flow_features: 20 sessions × 24 columns

```
In [48]: # =====
# NEXT STEP: Phase-wise idea-state trajectories
```



```
# =====

import matplotlib.pyplot as plt

states = ["Seeding", "Stress-testing", "Synthesis", "Closure"]
phases = ["beginning", "middle", "end"]

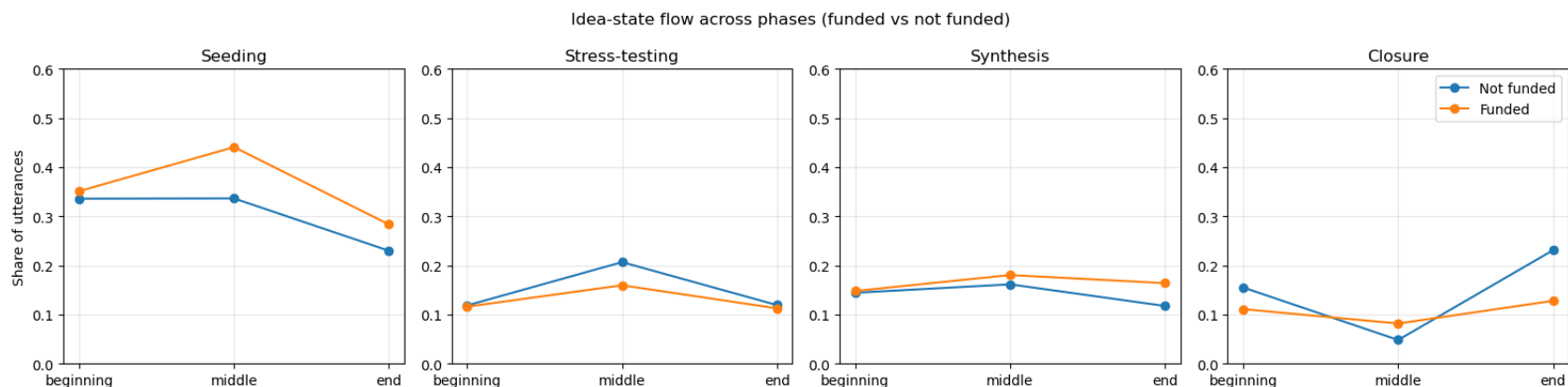
traj = (
    idea_flow_features
    .groupby("proposal_success")
    [[f"{p}_{s}" for s in states for p in phases]]
    .mean()
    .reset_index()
)

fig, axes = plt.subplots(1, len(states), figsize=(16, 4), sharey=False)

for i, state in enumerate(states):
    ax = axes[i]
    for funded, label in [(0, "Not funded"), (1, "Funded")]:
        row = traj[traj["proposal_success"] == funded]
        y = [row[f"{p}_{state}"].values[0] for p in phases]
        ax.plot(phases, y, marker="o", label=label)

    ax.set_title(state)
    ax.set_ylim(0, 0.6)
    ax.grid(alpha=0.3)

axes[0].set_ylabel("Share of utterances")
axes[-1].legend()
plt.suptitle("Idea-state flow across phases (funded vs not funded)")
plt.tight_layout()
plt.show()
```



```
In [49]: import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# =====
# 0) Pick the widest utterance table available
# =====

if "U_full" in globals() and isinstance(U_full, pd.DataFrame) and len(U_full) > len(U):
    U_use = U_full.copy()
    print("Using U_full (larger) for analysis.")
else:
    U_use = U.copy()
    print("Using U for analysis.")

# Required columns
need_cols = {"session_id"}
missing = need_cols - set(U_use.columns)
if missing:
    raise ValueError(f"U is missing required columns: {missing}. Available: {list(U_use.columns)}")

# Phase / label / text column detection (robust)
phase_col = next((c for c in ["phase", "when", "segment", "part"] if c in U_use.columns), None)
label_col = next((c for c in ["label", "category", "practice_label", "annotation_label", "code"] if c in U_use.columns), None)
text_col = next((c for c in ["transcript", "text", "utterance", "content"] if c in U_use.columns), None)
idx_col = next((c for c in ["utt_idx", "utterance_idx", "turn_idx", "idx"] if c in U_use.columns), None)
```

```

if phase_col is None or label_col is None or text_col is None:
    raise ValueError(
        f"Could not find phase/label/text columns.\n"
        f"phase_col={phase_col}, label_col={label_col}, text_col={text_col}\n"
        f"Available columns: {list(U_use.columns)}"
    )

print(f"Using phase_col='{phase_col}' | label_col='{label_col}' | text_col='{text_col}' | idx_col='")
print(f"Total utterances in U_use: {len(U_use):,}")
print(f"Total sessions in U_use: {U_use['session_id'].nunique():,}")

# Normalize phase values (just in case)
U_use[phase_col] = U_use[phase_col].astype(str).str.lower().str.strip()
phase_map = {"begin": "beginning", "start": "beginning", "early": "beginning",
             "mid": "middle", "middle": "middle",
             "end": "end", "late": "end"}
U_use[phase_col] = U_use[phase_col].replace(phase_map)

valid_phases = {"beginning", "middle", "end"}
U_use = U_use[U_use[phase_col].isin(valid_phases)].copy()

# =====
# 1) Meta/outcomes table (optional)
# =====
meta = None
if "meta_plus" in globals() and isinstance(meta_plus, pd.DataFrame) and "session_id" in meta_plus.columns:
    meta = meta_plus.copy()
else:
    meta = pd.DataFrame({"session_id": sorted(U_use["session_id"].unique())})

# Ensure columns exist (fill if absent)
for c in ["proposal_success", "p_funded_cv", "err_type"]:
    if c not in meta.columns:
        meta[c] = np.nan

# =====
# 2) "Logistics-y coordination" + "report-out" flags
# =====
LOGISTICS_STRICT = re.compile(

```

```

r"\b(muted|unmute|share (your )?screen|screen share|slides?|powerpoint|ppt|"
r"breakout|main room|report out|time|minutes?|seconds?|kicked|go to slide|"
r"notes|bullet points?|file|link|zoom)\b",
flags=re.IGNORECASE
)

REPORTOUT_PAT = re.compile(
    r"\b(report\s*out|reporting\s*out|reporter|spokesperson|share\s*back|"
    r"give a (quick|brief|concise) (summary|report)|wrap\s*up|key points)\b",
    flags=re.IGNORECASE
)

def flag_pat(x, pat):
    if not isinstance(x, str):
        return 0
    return int(bool(pat.search(x)))

U_use["_is_logistics_strict"] = U_use[text_col].apply(lambda x: flag_pat(x, LOGISTICS_STRICT))
U_use["_is_reportout"] = U_use[text_col].apply(lambda x: flag_pat(x, REPORTOUT_PAT))

# Logistics share computed ONLY inside end-phase Coordination/Decision label (as you intended)
END_COORD_LABEL = "Coordination and Decision Practices"

end_coord = U_use[(U_use[phase_col] == "end") & (U_use[label_col] == END_COORD_LABEL)].copy()

logi_share = (
    end_coord.groupby("session_id")["_is_logistics_strict"]
    .mean()
    .rename("end_coord_logistics_share_strict")
    .reset_index()
)

# Report-out share computed inside end-phase (all labels) OR restrict to Coordination label
end_phase = U_use[U_use[phase_col] == "end"].copy()
reportout_share = (
    end_phase.groupby("session_id")["_is_reportout"]
    .mean()
    .rename("end_reportout_share")
    .reset_index()
)

```

```

)

# =====
# 3) Idea-state mapping (your STATE_MAP, but applied)
# =====
STATE_MAP = {
  "Seeding": {
    "Seeding",
    "Information Seeking",
    "Idea Management",
  },
  "Stress-testing": {
    "Stress-testing",
    "Evaluation Practices",
    "Information Seeking", # you can delete this later if it muddies things
  },
  "Synthesis": {
    "Synthesis",
    "Integration Practices",
    "Knowledge Sharing",
  },
  "Closure": {
    "Commitment/Closure",
    "Coordination and Decision Practices",
  }
}

# Map label -> state (first match wins, else "Other")
label_to_state = {}
for st, labs in STATE_MAP.items():
  for lab in labs:
    label_to_state[lab] = st

U_use["_state"] = U_use[label_col].map(label_to_state).fillna("Other")

# =====
# 4) Phase-wise STATE shares per session + gains
# =====
# Share of utterances in each state per (session, phase)

```

```

state_share = (
    U_use.groupby(["session_id", phase_col, "_state"])
        .size()
        .rename("n")
        .reset_index()
)

totals = (
    U_use.groupby(["session_id", phase_col])
        .size()
        .rename("n_total")
        .reset_index()
)

state_share = state_share.merge(totals, on=["session_id", phase_col], how="left")
state_share["share"] = state_share["n"] / state_share["n_total"]

# Pivot to columns like: beginning_Seeding, middle_Closure, ...
state_wide = (
    state_share.assign(col=lambda d: d[phase_col] + "_" + d["_state"])
        .pivot_table(index="session_id", columns="col", values="share", fill_value=0.0)
        .reset_index()
)

# Convenience getters
def col_or_zero(df, col):
    return df[col] if col in df.columns else 0.0

# Closure gain = end_Closure - beginning_Closure
state_wide["closure_gain"] = col_or_zero(state_wide, "end_Closure") - col_or_zero(state_wide, "begin_Closure")
state_wide["synthesis_gain"] = col_or_zero(state_wide, "end_Synthesis") - col_or_zero(state_wide, "begin_Synthesis")

# =====
# 5) Entropy over ORIGINAL LABEL distribution per phase (not state)
# =====
# label share per session-phase
lab_share = (
    U_use.groupby(["session_id", phase_col, label_col])
        .size()

```

```

        .rename("n")
        .reset_index()
        .merge(totals, on=["session_id", phase_col], how="left")
    )
    lab_share["p"] = lab_share["n"] / lab_share["n_total"]

def entropy(p):
    p = np.asarray(p, dtype=float)
    p = p[p > 0]
    return float(-(p * np.log(p)).sum())

H = (
    lab_share.groupby(["session_id", phase_col])["p"]
    .apply(lambda s: entropy(s.values))
    .rename("H")
    .reset_index()
)

H_wide = H.pivot_table(index="session_id", columns=phase_col, values="H").reset_index()
# Standardize column names
for ph in ["beginning", "middle", "end"]:
    if ph not in H_wide.columns:
        H_wide[ph] = np.nan
H_wide = H_wide.rename(columns={"beginning": "H_early", "middle": "H_middle", "end": "H_late"})
H_wide["delta_entropy"] = H_wide["H_late"] - H_wide["H_early"]

# =====
# 6) Assemble session_features
# =====
session_features = (
    pd.DataFrame({"session_id": sorted(U_use["session_id"].unique())})
    .merge(meta[["session_id", "proposal_success", "p_funded_cv", "err_type"]], on="session_id", how="left")
    .merge(logi_share, on="session_id", how="left")
    .merge(reportout_share, on="session_id", how="left")
    .merge(H_wide[["session_id", "H_early", "H_middle", "H_late", "delta_entropy"]], on="session_id", how="left")
    .merge(state_wide, on="session_id", how="left")
)

# Fill missing shares with 0 where appropriate

```

```

for c in ["end_coord_logistics_share_strict", "end_reportout_share"]:
    if c in session_features.columns:
        session_features[c] = session_features[c].fillna(0.0)

print("\nSession_features built.")
print(f"Total sessions: {len(session_features):,}")
print(f"Sessions with known outcome (proposal_success not null): {session_features['proposal_success']

# =====
# 7) Slide-ready plots (only for sessions with known outcome)
# =====
sf_known = session_features[session_features["proposal_success"].notna()].copy()
sf_known["proposal_success"] = sf_known["proposal_success"].astype(int)

# --- Plot A: Idea-state flow across phases (funded vs not)
def mean_state_share(sf, state):
    return {
        "beginning": sf.get(f"beginning_{state}", pd.Series(0, index=sf.index)).mean(),
        "middle":     sf.get(f"middle_{state}", pd.Series(0, index=sf.index)).mean(),
        "end":        sf.get(f"end_{state}", pd.Series(0, index=sf.index)).mean(),
    }

states_to_plot = ["Seeding", "Stress-testing", "Synthesis", "Closure"]
phases = ["beginning", "middle", "end"]

fig = plt.figure(figsize=(16, 3.6))
for i, st in enumerate(states_to_plot, 1):
    ax = plt.subplot(1, 4, i)
    a0 = mean_state_share(sf_known[sf_known["proposal_success"]==0], st)
    a1 = mean_state_share(sf_known[sf_known["proposal_success"]==1], st)

    ax.plot(phases, [a0[p] for p in phases], marker="o", label="Not funded")
    ax.plot(phases, [a1[p] for p in phases], marker="o", label="Funded")
    ax.set_title(st)
    ax.set_ylim(0, 0.6)
    ax.grid(True, alpha=0.25)
    if i == 1:
        ax.set_ylabel("Mean share of utterances")
    if i == 4:

```



```

        ax.legend(loc="upper right")

plt.suptitle("Idea-state flow across phases (funded vs not funded)")
plt.tight_layout()
plt.show()

# --- Plot B: Mechanism view (closure gain vs  $\Delta$  entropy), marker=funded, edge=err_type
def edge_color(err):
    if err == "FP":
        return "black"
    if err == "FN":
        return "gray"
    return "none"

fig = plt.figure(figsize=(10,5))
ax = plt.gca()

for yval, marker, label in [(0, "o", "Not funded"), (1, "^", "Funded")]:
    sub = sf_known[sf_known["proposal_success"]==yval]
    ax.scatter(
        sub["closure_gain"], sub["delta_entropy"],
        marker=marker, s=110,
        edgecolors=sub["err_type"].map(edge_color),
        linewidths=1.5,
        label=f"{label} (n={len(sub)})",
        alpha=0.9
    )

ax.axhline(0, linestyle="--", linewidth=1)
ax.axvline(0, linestyle="--", linewidth=1)
ax.set_title("Mechanism view: closure gain vs idea-space expansion ( $\Delta$  entropy)\n(marker=funded, edge=err_type)")
ax.set_xlabel("Closure gain (end_Closure - beginning_Closure)")
ax.set_ylabel(" $\Delta$  entropy (late - early)  [>0 expands, <0 prunes]")
ax.grid(True, alpha=0.25)
ax.legend(loc="best")
plt.tight_layout()
plt.show()

# =====

```

```
# 8) Quick sanity summaries to print for your notes
# =====
print("\nΔ entropy by proposal_success (known only):")
print(sf_known.groupby("proposal_success")["delta_entropy"].agg(["count", "mean", "median"]).to_string())

print("\nEnd coordination logistics share (strict) by err_type (known only):")
print(sf_known.groupby("err_type")["end_coord_logistics_share_strict"].agg(["count", "mean", "median"]).to_string())

# Keep these around for the next steps
session_features_all = session_features
sf_known_outcomes = sf_known
```

Using U for analysis.

Using phase_col='phase' | label_col='label' | text_col='transcript' | idx_col='utt_idx'

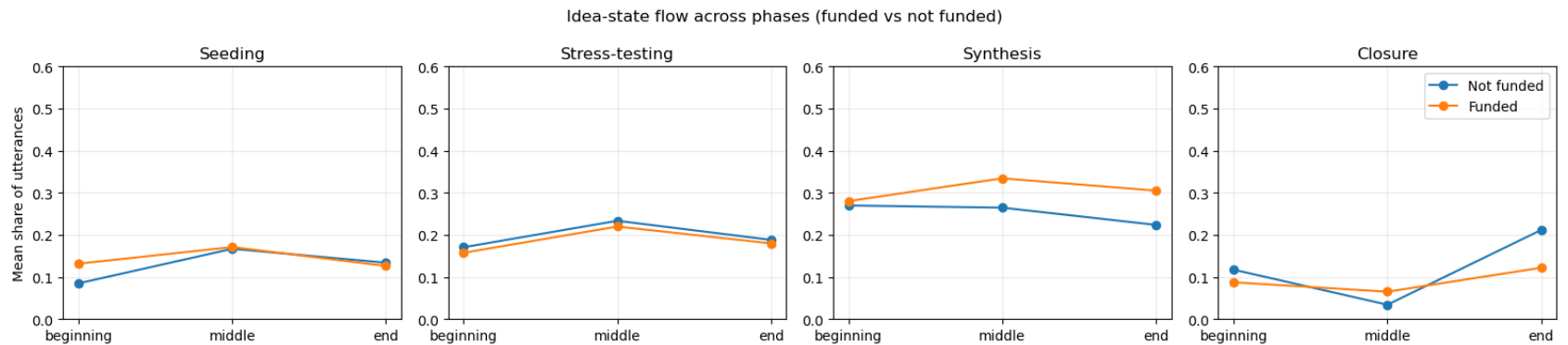
Total utterances in U_use: 5,559

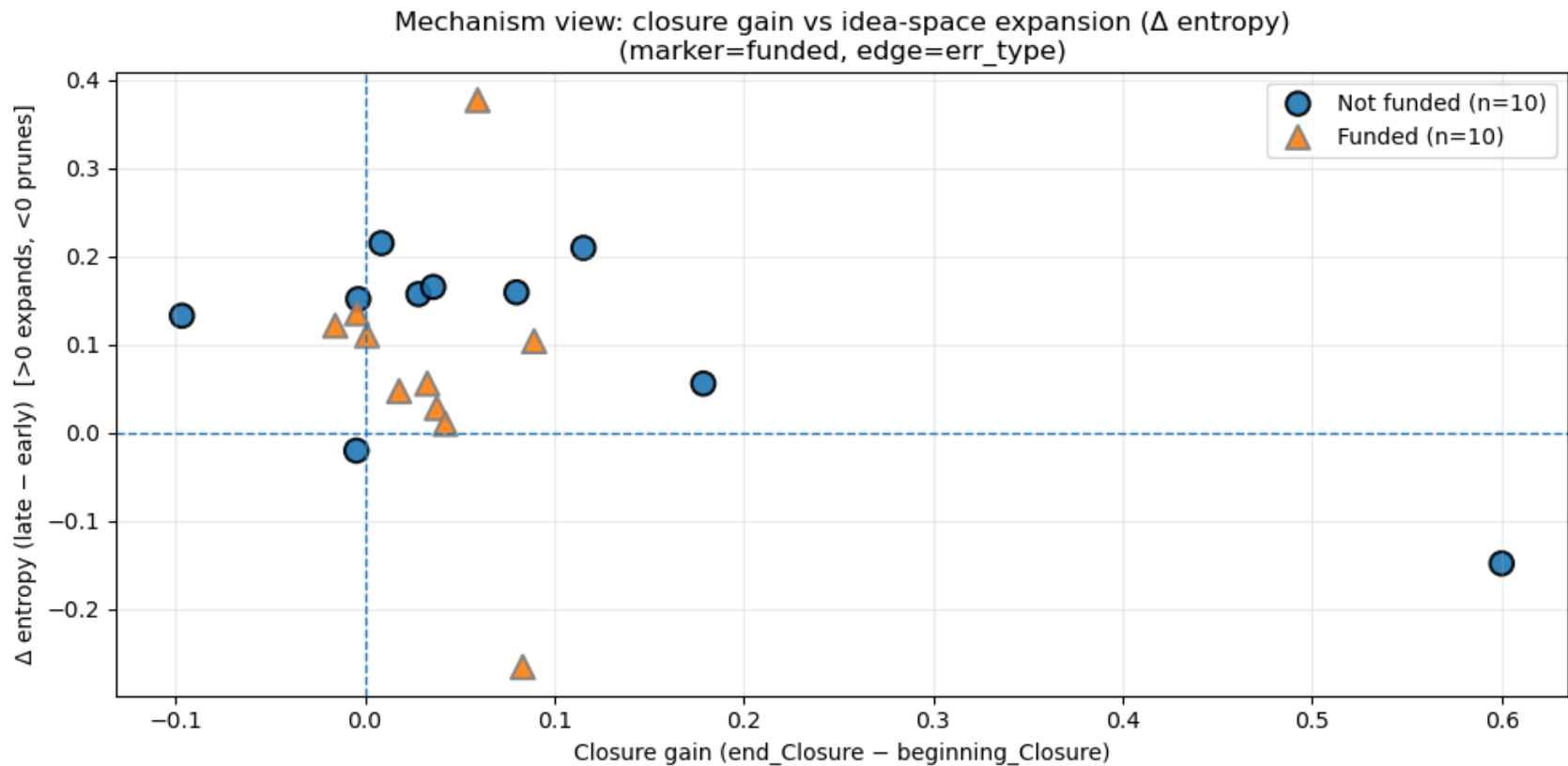
Total sessions in U_use: 20

Session_features built.

Total sessions: 20

Sessions with known outcome (proposal_success not null): 20





Δ entropy by proposal_success (known only):

	count	mean	median
proposal_success			
0	10	0.107740	0.154339
1	10	0.072263	0.079710

End coordination logistics share (strict) by err_type (known only):

	count	mean	median
err_type			
FN	10	0.289394	0.333333
FP	10	0.331936	0.367003

try rebuilding it - missing most of the seshs cause i think i pulled the 18 from 2021NES

```

In [52]: import os, glob, json
import pandas as pd
import numpy as np

# =====
# 0) SET THIS PATH
# =====
BASE_DIR = "/Users/maxchalekson/Northwestern University/Summer-2025/NIC0/NIC0 Research/gemini_data_"

# -----
# Helpers
# -----
def safe_read_json(path):
    with open(path, "r") as f:
        return json.load(f)

def session_id_from_path(p):
    return os.path.splitext(os.path.basename(p))[0]

def infer_conference_from_path(p):
    # ../data/2021CMC/session_data/2021_10_07_CMC_S4.json -> 2021CMC
    parts = os.path.normpath(p).split(os.sep)
    # find the folder right under BASE_DIR
    try:
        i = parts.index(os.path.basename(BASE_DIR))
        return parts[i + 1]
    except Exception:
        # fallback: parent of session_data
        return os.path.basename(os.path.dirname(os.path.dirname(p)))

# =====
# 1) LOAD ALL SESSION DATA JSONS -> U_full (long)
# =====
session_jsons = glob.glob(os.path.join(BASE_DIR, "*", "session_data", "*.json"))
print(f"Found {len(session_jsons)} session JSONs on disk (session_data/*.json).")

rows = []
for fp in session_jsons:

```

```

sid = session_id_from_path(fp)
conf = infer_conference_from_path(fp)

d = safe_read_json(fp)
utter_list = None

# Your examples use `all_data`
if isinstance(d, dict) and "all_data" in d and isinstance(d["all_data"], list):
    utter_list = d["all_data"]
# Some variants might use `utterances`
elif isinstance(d, dict) and "utterances" in d and isinstance(d["utterances"], list):
    utter_list = d["utterances"]
else:
    continue

for i, u in enumerate(utter_list):
    if not isinstance(u, dict):
        continue

    phase = u.get("when") # beginning/middle/end in your example
    text = u.get("transcript")
    speaker = u.get("speaker")

    ann = u.get("annotations", None)

    # explode multi-label annotations:
    # annotations is typically a dict: {label: {score, explanation, when, ...}, ...}
    if isinstance(ann, dict) and len(ann) > 0:
        for label, payload in ann.items():
            score = None
            if isinstance(payload, dict):
                score = payload.get("score", None)
                # sometimes "when" also exists inside payload; prefer explicit phase if missing
                if phase is None:
                    phase = payload.get("when", None)

            rows.append({
                "conference": conf,
                "session_id": sid,

```

```

        "utt_idx": i,
        "phase": phase,
        "label": label,
        "score": score,
        "speaker": speaker,
        "transcript": text,
    })
else:
    # no annotations: keep a placeholder so phase distributions still make sense
    rows.append({
        "conference": conf,
        "session_id": sid,
        "utt_idx": i,
        "phase": phase,
        "label": "None",
        "score": None,
        "speaker": speaker,
        "transcript": text,
    })

U_full = pd.DataFrame(rows)

print("\nU_full rebuilt.")
print("Total rows (label-level):", len(U_full))
print("Total sessions:", U_full["session_id"].nunique() if len(U_full) else 0)
print("Phase value counts:\n", U_full["phase"].value_counts(dropna=False).head(10))

# quick structural sanity checks
if len(U_full) == 0:
    raise RuntimeError(
        "U_full is empty. That means your loader didn't find `all_data`/`utterances` lists.\n"
        "Double-check BASE_DIR and confirm your session JSONs contain `all_data`."
    )

# =====
# 2) LOAD ALL OUTCOME FILES -> meta_plus
# =====
outcome_jsons = glob.glob(os.path.join(BASE_DIR, "*", "*_session_outcomes.json"))
print(f"\nFound {len(outcome_jsons)} outcome JSONs on disk (*_session_outcomes.json).")

```

```

meta_rows = []
for ofp in outcome_jsons:
    conf = os.path.basename(os.path.dirname(ofp))
    d = safe_read_json(ofp)

    # common pattern: dict keyed by session_id
    if isinstance(d, dict):
        for sid, payload in d.items():
            if not isinstance(payload, dict):
                continue
            # keep only what we need; add more keys if your file has them
            meta_rows.append({
                "conference": conf,
                "session_id": sid,
                "proposal_success": payload.get("proposal_success", payload.get("funded", payload.get("p_funded_cv", payload.get("p_funded", None))),
                "p_funded_cv": payload.get("p_funded_cv", payload.get("p_funded", None)),
            })
    # some pattern: list of dicts with session_id
    elif isinstance(d, list):
        for payload in d:
            if not isinstance(payload, dict):
                continue
            sid = payload.get("session_id")
            if sid is None:
                continue
            meta_rows.append({
                "conference": conf,
                "session_id": sid,
                "proposal_success": payload.get("proposal_success", payload.get("funded", payload.get("p_funded_cv", payload.get("p_funded", None))),
                "p_funded_cv": payload.get("p_funded_cv", payload.get("p_funded", None)),
            })

meta_plus = pd.DataFrame(meta_rows).drop_duplicates(subset=["session_id"])
print("\nmeta_plus rebuilt.")
print("Total sessions in meta_plus:", len(meta_plus))
print("Known outcomes:", meta_plus["proposal_success"].notna().sum())

# normalize outcome to 0/1 if it came in as strings/bools

```

```

def normalize_outcome(x):
    if pd.isna(x): return np.nan
    if isinstance(x, (bool, np.bool_)): return int(x)
    if isinstance(x, (int, np.integer)): return int(x)
    s = str(x).strip().lower()
    if s in {"1", "true", "funded", "yes", "y"}: return 1
    if s in {"0", "false", "not funded", "no", "n"}: return 0
    return np.nan

meta_plus["proposal_success"] = meta_plus["proposal_success"].apply(normalize_outcome)

# =====
# 3) MERGE: keep only sessions that exist in BOTH transcript data and outcomes
# =====
sessions_in_U = set(U_full["session_id"].unique())
sessions_in_meta = set(meta_plus["session_id"].unique())

print("\nOverlap check:")
print("sessions_in_U:", len(sessions_in_U))
print("sessions_in_meta:", len(sessions_in_meta))
print("overlap:", len(sessions_in_U & sessions_in_meta))

meta_plus = meta_plus[meta_plus["session_id"].isin(sessions_in_U)].copy()
print("meta_plus after overlap filter:", len(meta_plus))
print("Known outcomes after filter:", meta_plus["proposal_success"].notna().sum())

# If you want the working analysis subset:
U_use = U_full[U_full["session_id"].isin(meta_plus["session_id"])].copy()
print("\nU_use for analysis:")
print("Total rows:", len(U_use))
print("Total sessions:", U_use["session_id"].nunique())

```


Found 157 session JSONs on disk (session_data/*.json).

U_full rebuilt.

Total rows (label-level): 48987

Total sessions: 157

Phase value counts:

phase	
end	18212
beginning	16239
middle	14536

Name: count, dtype: int64

Found 8 outcome JSONs on disk (*_session_outcomes.json).

meta_plus rebuilt.

Total sessions in meta_plus: 157

Known outcomes: 0

Overlap check:

sessions_in_U: 157

sessions_in_meta: 157

overlap: 157

meta_plus after overlap filter: 157

Known outcomes after filter: 0

U_use for analysis:

Total rows: 48987

Total sessions: 157

```
In [53]: # =====
# INSPECT OUTCOME JSON SCHEMA
# =====
import json
from pprint import pprint

sample_fp = outcome_jsons[0]
print("Inspecting:", sample_fp)

with open(sample_fp, "r") as f:
```

```
d = json.load(f)

print("\nTop-level type:", type(d))

if isinstance(d, dict):
    # grab first session entry
    first_sid = next(iter(d.keys()))
    print("\nSample session_id:", first_sid)
    print("Keys inside session payload:")
    pprint(d[first_sid].keys())

    print("\nFull payload preview:")
    pprint(d[first_sid])

elif isinstance(d, list):
    print("\nList length:", len(d))
    print("\nKeys in first element:")
    pprint(d[0].keys())

    print("\nFull payload preview:")
    pprint(d[0])
```

Inspecting: /Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis/data/2021ABI/2021ABI_session_outcomes.json

Top-level type: <class 'dict'>

Sample session_id: 2021_05_20_ABI_S5

Keys inside session payload:

dict_keys(['all_speakers', 'facilitators', 'teams'])

Full payload preview:

```
{'all_speakers': ['Ed McCleskey CZI',
                  'Kevin Cash',
                  'Lisa Poulikakos',
                  'Wei Min',
                  'Silvia Ronco',
                  'Aniruddha Ray',
                  'Sapun Parekh',
                  'Joyoni Dey',
                  'Huanyu Cheng',
                  'Lingyan Shi',
                  'Ed McCleskey',
                  'Mini Das'],
 'facilitators': ['Wei Min'],
 'teams': {'ABI18': {'funded_status': 0,
                    'members': ['Sapun Parekh', 'Joyoni Dey', 'Lingyan Shi']}}}
```

```
In [56]: # =====
# SCHEMA-ROBUST OUTCOME PARSING → SESSION-LEVEL MERGE
# =====

import json
import pandas as pd
import numpy as np

# -----
# Parse team-level outcomes from heterogeneous JSON schemas
# -----

rows = []
```

```

bad_sessions = 0
unknown_schema = 0

for fp in outcome_jsons:
    with open(fp, "r") as f:
        d = json.load(f)

        # CASE 1: dict keyed by session_id
        if isinstance(d, dict):
            iterable = d.items()

        # CASE 2: list of session records
        elif isinstance(d, list):
            iterable = []
            for item in d:
                if not isinstance(item, dict):
                    continue
                sid = item.get("session_id")
                if sid is None:
                    continue
                iterable.append((sid, item))

        else:
            unknown_schema += 1
            continue

        # Parse each session payload
        for session_id, payload in iterable:
            if not isinstance(payload, dict):
                bad_sessions += 1
                continue

            teams = payload.get("teams")
            if not isinstance(teams, dict) or len(teams) == 0:
                bad_sessions += 1
                continue

            for team_id, team_info in teams.items():
                if not isinstance(team_info, dict):

```

```

        continue

    funded = team_info.get("funded_status", None)

    try:
        funded_num = int(funded) if funded is not None else np.nan
    except Exception:
        funded_num = np.nan

    rows.append({
        "session_id": session_id,
        "team_id": team_id,
        "funded_status": funded_num,
        "outcomes_file": fp,
    })

teams_df = pd.DataFrame(rows)

print("Team-level outcomes parsed:")
print("  rows:", len(teams_df))
print("  sessions:", teams_df["session_id"].nunique())
print("  bad_sessions skipped:", bad_sessions)
print("  unknown schema files:", unknown_schema)
print("\nFunded_status counts:")
print(teams_df["funded_status"].value_counts(dropna=False))

# -----
# Aggregate to session-level outcomes
# -----

session_outcomes = (
    teams_df.dropna(subset=["funded_status"])
    .groupby("session_id")
    .agg(
        proposal_success=("funded_status", lambda x: int((x == 1).any())),
        funded_rate=("funded_status", "mean"),
        n_teams=("funded_status", "size"),
    )
    .reset_index()

```

```

)

print("\nSession-level outcomes built:")
print("  sessions:", len(session_outcomes))
print(session_outcomes["proposal_success"].value_counts())

# -----
# Merge outcomes into meta_plus
# -----

meta_plus = meta_plus.drop(
    columns=[c for c in ["proposal_success", "funded_rate", "n_teams"] if c in meta_plus.columns],
    errors="ignore"
)

meta_plus = meta_plus.merge(session_outcomes, on="session_id", how="left")

print("\nmeta_plus after merge:")
print("  total sessions:", len(meta_plus))
print("  known outcomes:", meta_plus["proposal_success"].notna().sum())
print(meta_plus["proposal_success"].value_counts(dropna=False))

```

```
Team-level outcomes parsed:
  rows: 233
  sessions: 123
  bad_sessions skipped: 50
  unknown schema files: 0
```

```
Funded_status counts:
```

```
funded_status
```

```
0    150
```

```
1     83
```

```
Name: count, dtype: int64
```

```
Session-level outcomes built:
```

```
  sessions: 123
```

```
proposal_success
```

```
1     68
```

```
0     55
```

```
Name: count, dtype: int64
```

```
meta_plus after merge:
```

```
  total sessions: 157
```

```
  known outcomes: 123
```

```
proposal_success
```

```
1.0    68
```

```
0.0    55
```

```
NaN     34
```

```
Name: count, dtype: int64
```

```
In [60]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# =====
# 0) CONFIG: column names
# =====
PHASE_COL = "phase"
LABEL_COL = "label"
TEXT_COL = "transcript"      # not required here, but kept for sanity
SID_COL = "session_id"
```

```

PHASE_ORDER = ["beginning", "middle", "end"]

# =====
# 1) MACRO STATE MAP (edit here)
# =====
STATE_MAP = {
    "Seeding": {"Seeding", "Information Seeking", "Idea Management"},
    "Stress-testing": {"Stress-testing", "Evaluation Practices"},
    "Synthesis": {"Synthesis", "Integration Practices", "Knowledge Sharing"},
    "Closure": {"Commitment/Closure", "Coordination and Decision Practices"},
}

# Invert map: fine_label -> macro_state
fine_to_macro = {fine: macro for macro, fine_set in STATE_MAP.items() for fine in fine_set}

# =====
# 2) Pick the dataset to use
# =====
U = U_use.copy() # or U_full.copy()

need_cols = {SID_COL, PHASE_COL, LABEL_COL}
missing = need_cols - set(U.columns)
if missing:
    raise KeyError(f"U is missing required columns: {missing}")

U[PHASE_COL] = U[PHASE_COL].astype(str).str.lower().str.strip()
U = U[U[PHASE_COL].isin(PHASE_ORDER)].copy()

U["macro_state"] = U[LABEL_COL].map(fine_to_macro)
U_macro = U.dropna(subset=["macro_state"]).copy()

print(f"Total rows in U: {len(U):,}")
print(f"Rows mapped to macro states: {len(U_macro):,}")
print("Sessions in U:", U[SID_COL].nunique())

# =====
# 3) Macro-state shares by phase, per session
# =====

```



```

counts = (
    U_macro.groupby([SID_COL, PHASE_COL, "macro_state"])
        .size()
        .rename("n")
        .reset_index()
)

totals = (
    counts.groupby([SID_COL, PHASE_COL])["n"]
        .sum()
        .rename("phase_total")
        .reset_index()
)

counts = counts.merge(totals, on=[SID_COL, PHASE_COL], how="left")
counts["share"] = counts["n"] / counts["phase_total"]

flow_wide = (
    counts.assign(col=lambda d: d[PHASE_COL] + "__" + d["macro_state"])
        .pivot_table(index=SID_COL, columns="col", values="share", aggfunc="sum")
        .fillna(0.0)
        .reset_index()
)

# Ensure all macro x phase columns exist
for ph in PHASE_ORDER:
    for ms in STATE_MAP.keys():
        c = f"{ph}__{ms}"
        if c not in flow_wide.columns:
            flow_wide[c] = 0.0

# =====
# 4) Entropy early/late +  $\Delta$  entropy (macro distribution)
# =====
def entropy_from_row(row, phase):
    p = np.array([row[f"{phase}__{ms}"] for ms in STATE_MAP.keys()], dtype=float)
    p = p[p > 0]
    if len(p) == 0:
        return np.nan

```

```

    return float(-(p * np.log(p)).sum())

entropy_df = flow_wide[[SID_COL]].copy()
entropy_df["H_early"] = flow_wide.apply(lambda r: entropy_from_row(r, "beginning"), axis=1)
entropy_df["H_middle"] = flow_wide.apply(lambda r: entropy_from_row(r, "middle"), axis=1)
entropy_df["H_late"] = flow_wide.apply(lambda r: entropy_from_row(r, "end"), axis=1)
entropy_df["delta_entropy"] = entropy_df["H_late"] - entropy_df["H_early"]

entropy_df["closure_gain"] = flow_wide["end__Closure"].values - flow_wide["beginning__Closure"].values
entropy_df["pruning_score"] = -entropy_df["delta_entropy"] # positive = pruning / convergence

# =====
# 5) Merge with meta_plus outcomes (+ reconstruct err_type)
# =====
for c in [SID_COL, "proposal_success", "p_funded_cv"]:
    if c not in meta_plus.columns:
        raise KeyError(f"meta_plus must include '{c}' (check your outcomes merge + meta schema).")

meta_plus2 = meta_plus.copy()

# Coerce to numeric safely (keeps NaN where missing)
meta_plus2["p_funded_cv"] = pd.to_numeric(meta_plus2["p_funded_cv"], errors="coerce")
meta_plus2["proposal_success"] = pd.to_numeric(meta_plus2["proposal_success"], errors="coerce")

thr = 0.5
meta_plus2["pred_funded"] = (meta_plus2["p_funded_cv"] >= thr).astype("Int64") # nullable int
meta_plus2["proposal_success_num"] = meta_plus2["proposal_success"] # alias for clarity

cond_tp = (meta_plus2["pred_funded"] == 1) & (meta_plus2["proposal_success_num"] == 1)
cond_tn = (meta_plus2["pred_funded"] == 0) & (meta_plus2["proposal_success_num"] == 0)
cond_fp = (meta_plus2["pred_funded"] == 1) & (meta_plus2["proposal_success_num"] == 0)
cond_fn = (meta_plus2["pred_funded"] == 0) & (meta_plus2["proposal_success_num"] == 1)

# IMPORTANT FIX: default must be a STRING (not np.nan) so NumPy doesn't explode
meta_plus2["err_type"] = np.select(
    [cond_tp, cond_tn, cond_fp, cond_fn],
    ["TP", "TN", "FP", "FN"],
    default="UNK"
).astype("object")

```

```

meta_plus2.loc[meta_plus2["proposal_success_num"].isna(), "err_type"] = np.nan

# (optional) drop helper
meta_plus2 = meta_plus2.drop(columns=["proposal_success_num"])

session_features = (
    meta_plus2[[SID_COL, "proposal_success", "p_funded_cv", "err_type"]]
    .merge(flow_wide, on=SID_COL, how="left")
    .merge(entropy_df, on=SID_COL, how="left")
)

print("\nSession_features built.")
print("Total sessions:", len(session_features))
print("Known outcomes:", session_features["proposal_success"].notna().sum())
print("err_type counts (known only):")
print(session_features.dropna(subset=["proposal_success"])["err_type"].value_counts())

known = session_features.dropna(subset=["proposal_success"]).copy()
known["proposal_success"] = known["proposal_success"].astype(int)

# =====
# 6) Entropy quadrants (median split on known outcomes)
# =====
H_early_med = known["H_early"].median()
H_late_med = known["H_late"].median()

def quadrant(row):
    e = row["H_early"]
    l = row["H_late"]
    e_lab = "High" if e >= H_early_med else "Low"
    l_lab = "High" if l >= H_late_med else "Low"
    return f"{e_lab} early / {l_lab} late"

known["entropy_quadrant"] = known.apply(quadrant, axis=1)

quad_summary = (
    known.groupby("entropy_quadrant")
    .agg(
        n=("proposal_success", "size"),

```

```

        funded_rate=("proposal_success", "mean"),
        mean_delta=("delta_entropy", "mean"),
        mean_closure_gain=("closure_gain", "mean"),
    )
    .reset_index()
    .sort_values("funded_rate", ascending=False)
)

print("\nEntropy quadrant summary (known outcomes only):")
print(quad_summary)

print("\n $\Delta$  entropy by proposal_success (known only):")
print(known.groupby("proposal_success")["delta_entropy"].agg(["count", "mean", "median"]))

print("\nEnd Closure share by proposal_success (known only):")
if "end__Closure" in known.columns:
    print(known.groupby("proposal_success")["end__Closure"].agg(["count", "mean", "median"]))

# =====
# 7) PLOTS
# =====

# Plot A: Macro-state flow across phases (funded vs not)
macro_states = list(STATE_MAP.keys())

plt.figure(figsize=(12, 3.5))
for i, ms in enumerate(macro_states):
    y0 = [known.loc[known["proposal_success"]==0, f"{ph}__{ms}"].mean() for ph in PHASE_ORDER]
    y1 = [known.loc[known["proposal_success"]==1, f"{ph}__{ms}"].mean() for ph in PHASE_ORDER]

    plt.subplot(1, 4, i+1)
    plt.plot(PHASE_ORDER, y0, marker="o", label="Not funded")
    plt.plot(PHASE_ORDER, y1, marker="o", label="Funded")
    plt.ylim(0, 0.6)
    plt.title(ms)
    if i == 0:
        plt.ylabel("Mean share of utterances")
    if i == 3:
        plt.legend(loc="upper right")

```

```

plt.suptitle("Idea-state flow across phases (macro states; known outcomes only)", y=1.05)
plt.tight_layout()
plt.show()

# Plot B: Early vs late entropy scatter
plt.figure(figsize=(6.5, 5.5))
for val, marker, lab in [(0, "o", "Not funded"), (1, "^", "Funded")]:
    sub = known[known["proposal_success"]==val]
    plt.scatter(sub["H_early"], sub["H_late"], marker=marker, label=f"{lab} (n={len(sub)})", alpha=

mn = np.nanmin(pd.concat([known["H_early"], known["H_late"]]))
mx = np.nanmax(pd.concat([known["H_early"], known["H_late"]]))
plt.plot([mn, mx], [mn, mx], linestyle="--", linewidth=2, label="y = x (no change)")
plt.axvline(H_early_med, linestyle=":", linewidth=2, label="early median")
plt.axhline(H_late_med, linestyle=":", linewidth=2, label="late median")

plt.xlabel("Early entropy (H_early)")
plt.ylabel("Late entropy (H_late)")
plt.title("Idea-space evolution: early vs late entropy (known outcomes)")
plt.legend()
plt.tight_layout()
plt.show()

# Plot C: Funding rate by entropy quadrant
plt.figure(figsize=(7.8, 4.5))
plt.bar(quad_summary["entropy_quadrant"], quad_summary["funded_rate"])
plt.xticks(rotation=25, ha="right")
plt.ylim(0, 1)
plt.ylabel("Funding rate")
plt.title("Funding rate by entropy regime (known outcomes)")
plt.tight_layout()
plt.show()

# Plot D: Closure gain vs  $\Delta$  entropy (mechanism view)
plt.figure(figsize=(7.5, 5))
for val, marker, lab in [(0, "o", "Not funded"), (1, "^", "Funded")]:
    sub = known[known["proposal_success"]==val]
    plt.scatter(sub["closure_gain"], sub["delta_entropy"], marker=marker, label=f"{lab} (n={len(sub)})", alpha=

```

```
plt.axhline(0, linestyle="--", linewidth=2)
plt.axvline(0, linestyle="--", linewidth=2)
plt.xlabel("Closure gain (end__Closure - beginning__Closure)")
plt.ylabel("Δ entropy (late - early)")
plt.title("Mechanism view: closure gain vs idea-space expansion (known outcomes)")
plt.legend()
plt.tight_layout()
plt.show()
```

Total rows in U: 48,987
Rows mapped to macro states: 34,616
Sessions in U: 157

Session_features built.
Total sessions: 157
Known outcomes: 123
err_type counts (known only):
err_type
FN 68
TN 55
Name: count, dtype: int64

Entropy quadrant summary (known outcomes only):

	entropy_quadrant	n	funded_rate	mean_delta	mean_closure_gain
2	Low early / High late	30	0.633333	0.181193	-0.017840
1	High early / Low late	30	0.600000	-0.047936	-0.058862
0	High early / High late	32	0.562500	0.045599	-0.023244
3	Low early / Low late	31	0.419355	0.080425	-0.047588

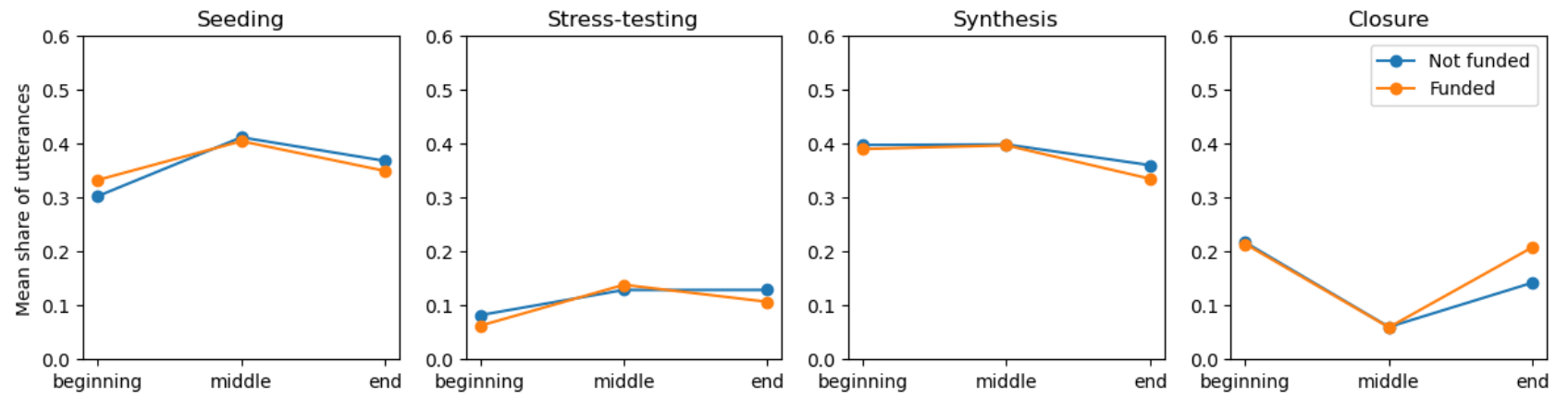
Δ entropy by proposal_success (known only):

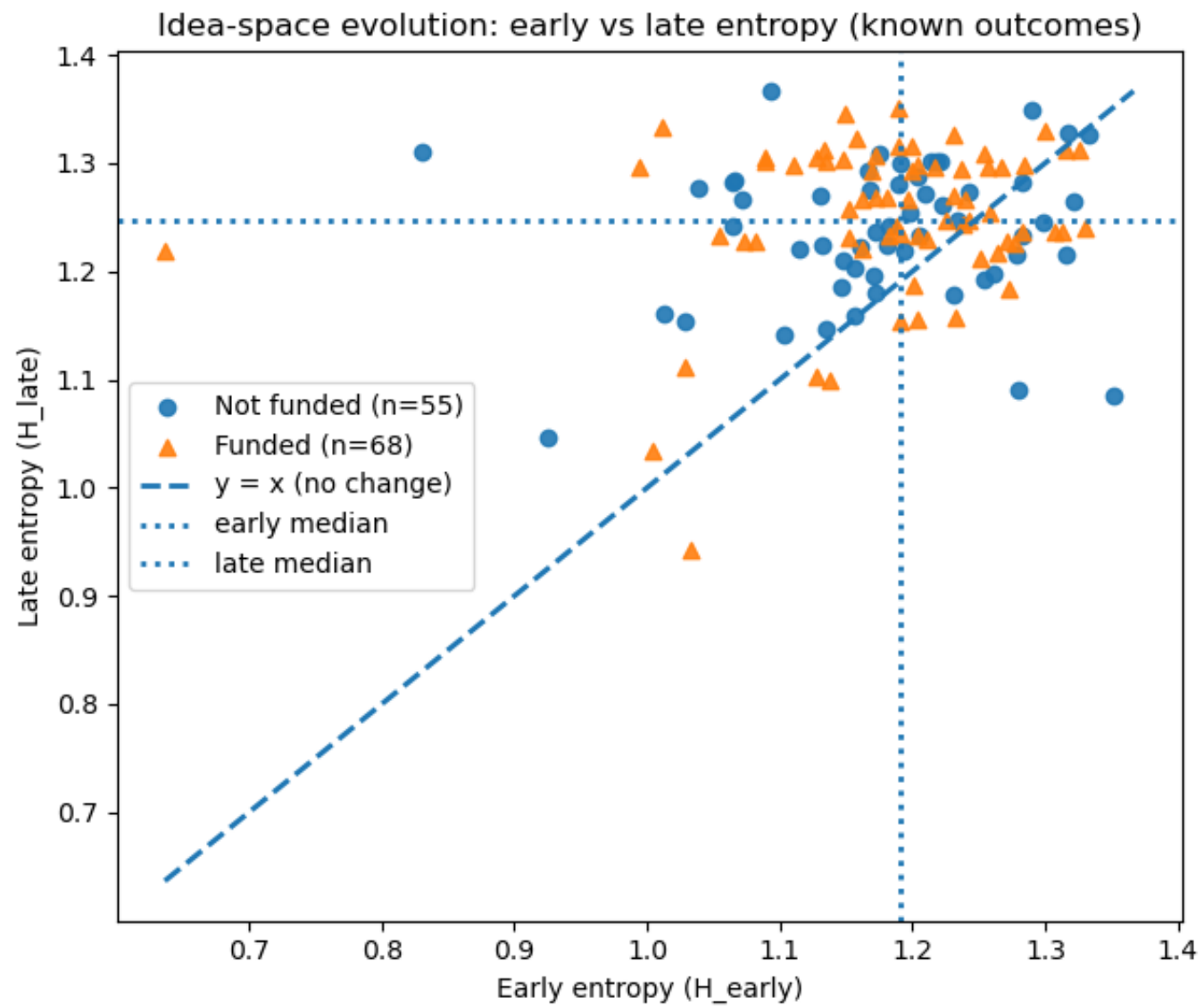
	count	mean	median
proposal_success			
0	55	0.059188	0.059296
1	68	0.069040	0.054615

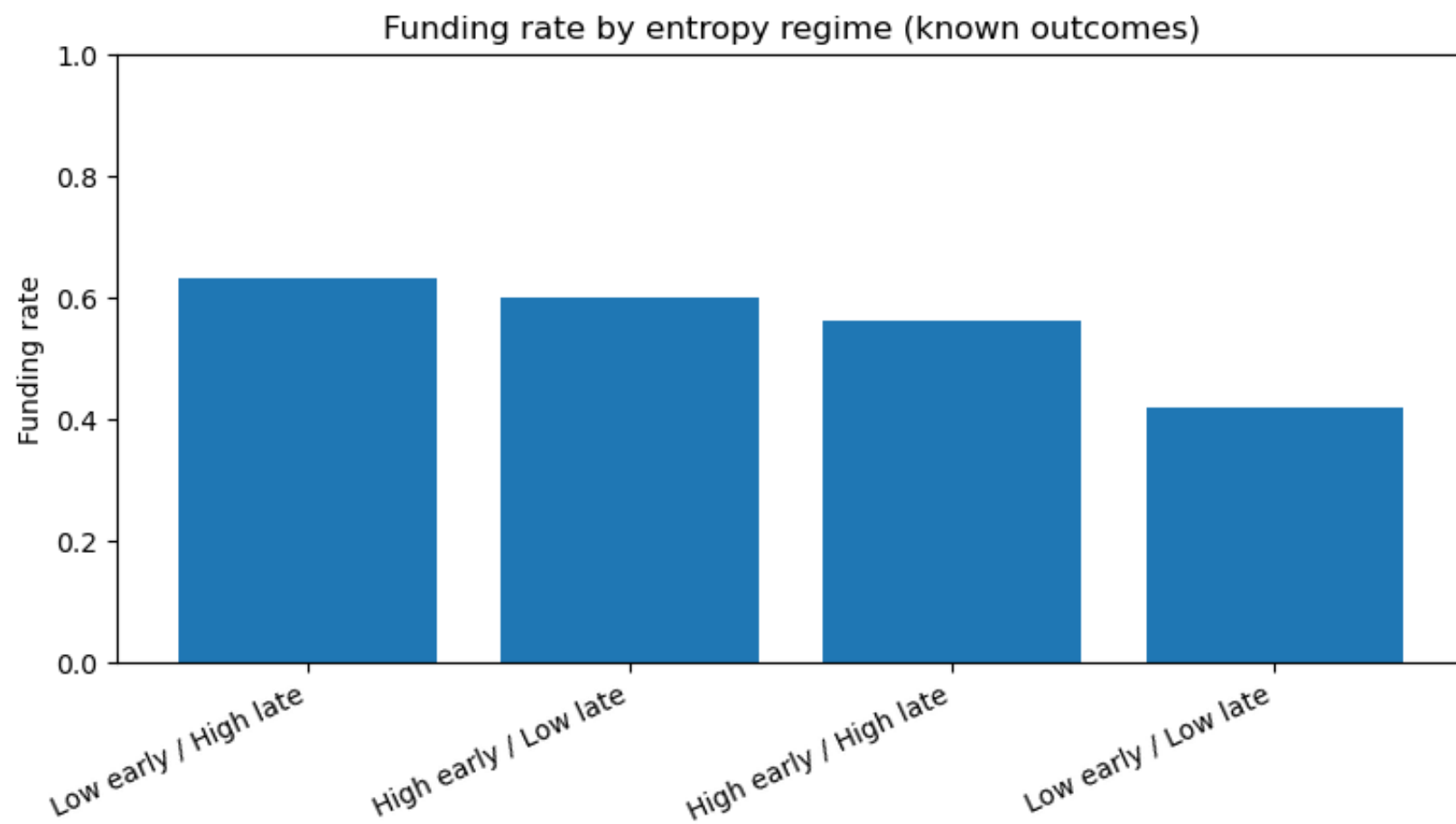
End Closure share by proposal_success (known only):

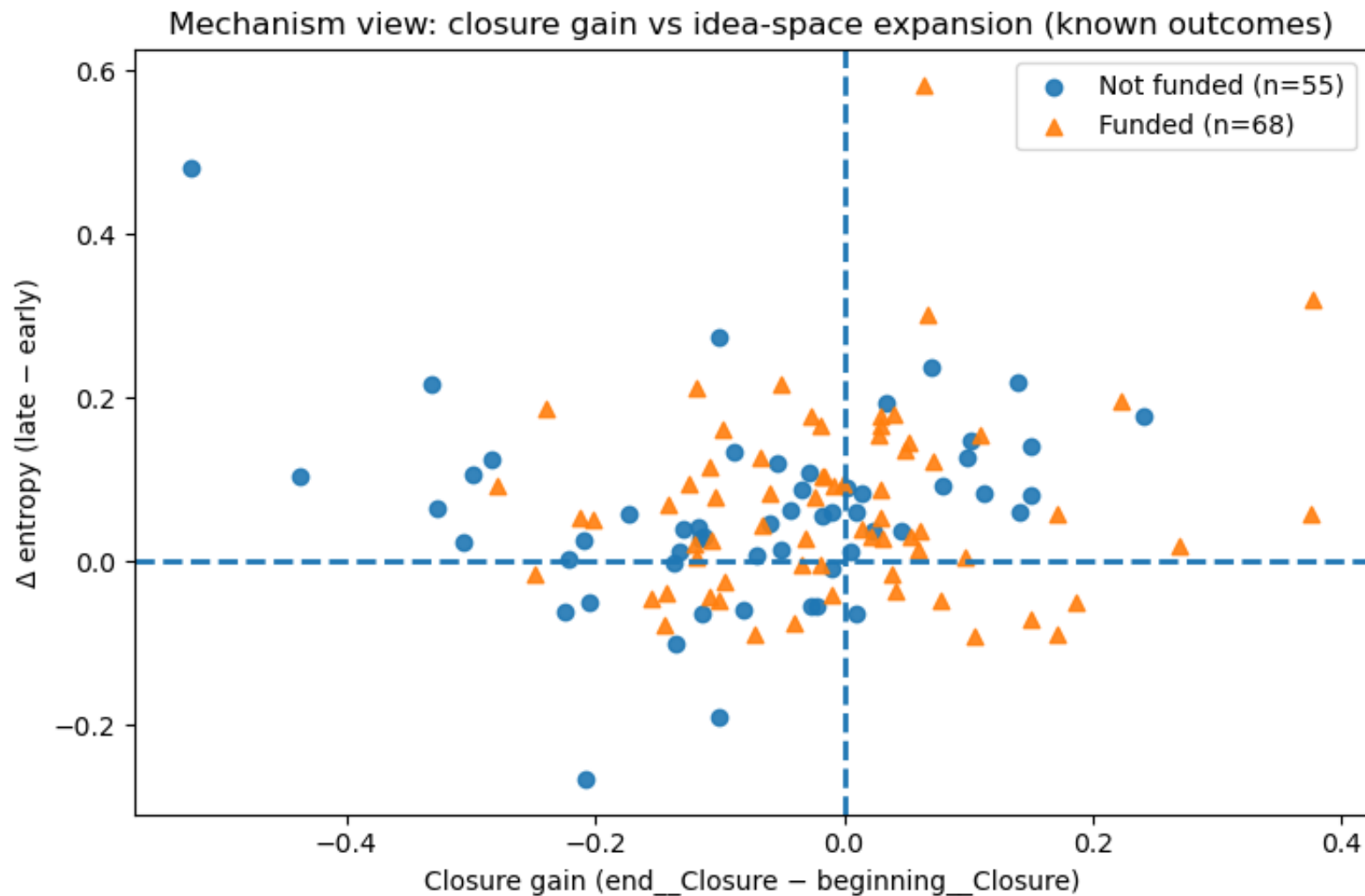
	count	mean	median
proposal_success			
0	55	0.142241	0.139241
1	68	0.208124	0.200000

Idea-state flow across phases (macro states; known outcomes only)









```
In [62]: import pandas as pd
import numpy as np

print("meta_plus2 shape:", meta_plus2.shape)
print("Columns containing 'fund' or 'success':",
      [c for c in meta_plus2.columns if ("fund" in c.lower()) or ("success" in c.lower())])

for c in ["proposal_success", "p_funded_cv"]:
    print(f"\n--- {c} ---")
```

```

if c not in meta_plus2.columns:
    print("MISSING COLUMN")
else:
    print("non-null:", meta_plus2[c].notna().sum(), "/", len(meta_plus2))
    print("dtype:", meta_plus2[c].dtype)
    print("head:", meta_plus2[c].head(5).tolist())

# Show how many rows have BOTH non-null
if ("proposal_success" in meta_plus2.columns) and ("p_funded_cv" in meta_plus2.columns):
    both = meta_plus2["proposal_success"].notna() & meta_plus2["p_funded_cv"].notna()
    print("\nRows with BOTH proposal_success and p_funded_cv non-null:", int(both.sum()))
    print("Example rows (if any):")
    print(meta_plus2.loc[both, ["session_id", "proposal_success", "p_funded_cv"]].head(10))

```

meta_plus2 shape: (157, 8)

Columns containing 'fund' or 'success': ['p_funded_cv', 'proposal_success', 'funded_rate', 'pred_funded']

```

--- proposal_success ---
non-null: 123 / 157
dtype: float64
head: [0.0, 1.0, nan, nan, 0.0]

```

```

--- p_funded_cv ---
non-null: 0 / 157
dtype: float64
head: [nan, nan, nan, nan, nan]

```

```

Rows with BOTH proposal_success and p_funded_cv non-null: 0
Example rows (if any):
Empty DataFrame
Columns: [session_id, proposal_success, p_funded_cv]
Index: []

```

```

In [63]: import numpy as np
import pandas as pd

# 1) Find a usable probability column (you might have a different name)
prob_candidates = [
    "p_funded_cv",          # what you want

```

```

    "p_funded",          # common alt
    "p_funded_mean",    # common alt
    "p_funded_proba",   # common alt
    "pred_proba",       # common alt
    "prob_funded",      # common alt
]

print("Existing columns:", meta_plus2.columns.tolist())

# 2) If p_funded_cv is all-NaN, try to recover it from an alternative column
if "p_funded_cv" in meta_plus2.columns and meta_plus2["p_funded_cv"].notna().sum() == 0:
    found = None
    for c in prob_candidates:
        if c in meta_plus2.columns and meta_plus2[c].notna().sum() > 0:
            found = c
            break

    if found is None:
        # Heuristic: search any column that looks like probabilities and isn't all NaN
        heur = []
        for c in meta_plus2.columns:
            if any(k in c.lower() for k in ["prob", "proba", "p_"]) and meta_plus2[c].notna().sum() > 0:
                heur.append(c)

        raise ValueError(
            "p_funded_cv is all NaN and I couldn't find any alternative probability column.\n"
            f"Heuristic candidates with non-null values: {heur}\n"
            "Fix: locate the dataframe where your model outputs were created (the one that had p_funded_cv)\n"
            "and merge it back into meta_plus2 on session_id."
        )

    print(f"Recovering p_funded_cv from: {found}")
    meta_plus2["p_funded_cv"] = pd.to_numeric(meta_plus2[found], errors="coerce")

# 3) Sanity check
print("\nAfter recovery attempt:")
print("p_funded_cv non-null:", meta_plus2["p_funded_cv"].notna().sum(), "/", len(meta_plus2))
print(meta_plus2["p_funded_cv"].describe())

```

```

# 4) Now your threshold diagnostics will work
known_meta = meta_plus2.dropna(subset=["proposal_success", "p_funded_cv"]).copy()
known_meta["proposal_success"] = known_meta["proposal_success"].astype(int)

print("\nKnown rows (have outcomes + model probs):", len(known_meta))

def confusion_at(thr):
    pred = (known_meta["p_funded_cv"] >= thr).astype(int).values
    y = known_meta["proposal_success"].values
    tp = int(((pred==1)&(y==1)).sum())
    tn = int(((pred==0)&(y==0)).sum())
    fp = int(((pred==1)&(y==0)).sum())
    fn = int(((pred==0)&(y==1)).sum())
    return pd.Series({"thr":thr, "TP":tp, "FP":fp, "TN":tn, "FN":fn})

grid = pd.DataFrame([confusion_at(t) for t in np.linspace(0.1, 0.9, 9)])
print("\nConfusion table across thresholds:")
print(grid)

```

Existing columns: ['conference', 'session_id', 'p_funded_cv', 'proposal_success', 'funded_rate', 'n_teams', 'pred_funded', 'err_type']

```

-----
ValueError                                Traceback (most recent call last)
Cell In[63], line 31
    28         if any(k in c.lower() for k in ["prob", "proba", "p_"]) and meta_plus2[c].notna().sum() > 0:
    29             heur.append(c)
----> 31         raise ValueError(
    32             "p_funded_cv is all NaN and I couldn't find any alternative probability column.\n"
    33             f"Heuristic candidates with non-null values: {heur}\n"
    34             "Fix: locate the dataframe where your model outputs were created (the one that had p_funded_cv before) "
    35             "and merge it back into meta_plus2 on session_id."
    36         )
    38 print(f"Recovering p_funded_cv from: {found}")
    39 meta_plus2["p_funded_cv"] = pd.to_numeric(meta_plus2[found], errors="coerce")

ValueError: p_funded_cv is all NaN and I couldn't find any alternative probability column.
Heuristic candidates with non-null values: []
Fix: locate the dataframe where your model outputs were created (the one that had p_funded_cv before) and merge it back into meta_plus2 on session_id.

```

```

In [64]: import os
import pandas as pd
import numpy as np

ROOT = "/Users/maxchalekson/Northwestern University/Summer-2025/NICO/NICO Research/gemini_data_analysis"
# If your outputs live elsewhere, change ROOT.

TARGET_KEYS = ["p_funded_cv", "p_funded", "prob", "proba", "pred_proba", "funded_prob"]
OK_EXT = (".csv", ".parquet", ".pkl", ".pickle")

hits = []

def safe_read_head(fp, n=50):
    ext = os.path.splitext(fp)[1].lower()
    try:
        if ext == ".csv":
            return pd.read_csv(fp, nrows=n)
        if ext == ".parquet":

```

```

        return pd.read_parquet(fp)
    if ext in (".pkl", ".pickle"):
        return pd.read_pickle(fp)
except Exception:
    return None
return None

print("Scanning for prediction files under:")
print(ROOT)

for dirpath, _, filenames in os.walk(ROOT):
    for fn in filenames:
        if not fn.lower().endswith(OK_EXT):
            continue
        fp = os.path.join(dirpath, fn)
        df = safe_read_head(fp)
        if df is None or not isinstance(df, pd.DataFrame) or df.empty:
            continue

        cols = [c for c in df.columns]
        has_sid = "session_id" in cols
        if not has_sid:
            continue

        # Look for any probability-ish column
        prob_cols = [c for c in cols if any(k in c.lower() for k in TARGET_KEYS)]
        if len(prob_cols) == 0:
            continue

        # Extra filter: must have at least one non-null value in a prob col
        nonnull_prob_cols = [c for c in prob_cols if pd.to_numeric(df[c], errors="coerce").notna()]
        if len(nonnull_prob_cols) == 0:
            continue

        hits.append((fp, nonnull_prob_cols, cols[:30]))

print(f"\nFound {len(hits)} candidate file(s) with session_id + non-null prob columns.\n")

for i, (fp, prob_cols, preview_cols) in enumerate(hits[:20], 1):

```



```

print(f"[{i}] {fp}")
print("    prob cols:", prob_cols)
print("    preview cols:", preview_cols)
print()

# If we found exactly one good candidate, auto-load + merge it.
if len(hits) == 1:
    fp, prob_cols, _ = hits[0]
    df_pred = safe_read_head(fp, n=10**9) # load full
    # pick best prob col (prefer p_funded_cv)
    best = "p_funded_cv" if "p_funded_cv" in df_pred.columns else prob_cols[0]
    df_pred = df_pred[["session_id", best]].copy()
    df_pred[best] = pd.to_numeric(df_pred[best], errors="coerce")

    print(f"Auto-merging '{best}' from:\n{fp}\n")
    meta_plus2 = meta_plus2.drop(columns=["p_funded_cv"], errors="ignore").merge(
        df_pred.rename(columns={best: "p_funded_cv"}),
        on="session_id",
        how="left"
    )

    print("After merge:")
    print("p_funded_cv non-null:", meta_plus2["p_funded_cv"].notna().sum(), "/", len(meta_plus2))

```

Scanning for prediction files under:

/Users/maxchalekson/Northwestern University/Summer-2025/NIC0/NIC0 Research/gemini_data_analysis

Found 0 candidate file(s) with session_id + non-null prob columns.

```

In [65]: unknown_sessions = (
    meta_plus
    .loc[meta_plus["proposal_success"].isna(), "session_id"]
    .unique()
)

print(f"Unknown-outcome sessions: {len(unknown_sessions)}")
unknown_sessions[:10]

```

Unknown-outcome sessions: 34

```
Out[65]: array(['2021_05_21_ABI_S16', '2021_05_21_ABI_S1', '2021_05_21_ABI_S6',  
               '2021_05_21_ABI_S10', '2021_05_20_ABI_S1', '2021_05_21_ABI_S5',  
               'output-2021-05-20-ABI-S1', '2021_06_11_SLU_S5',  
               '2021_06_10_SLU_S7', '2021_06_11_SLU_S2'], dtype=object)
```

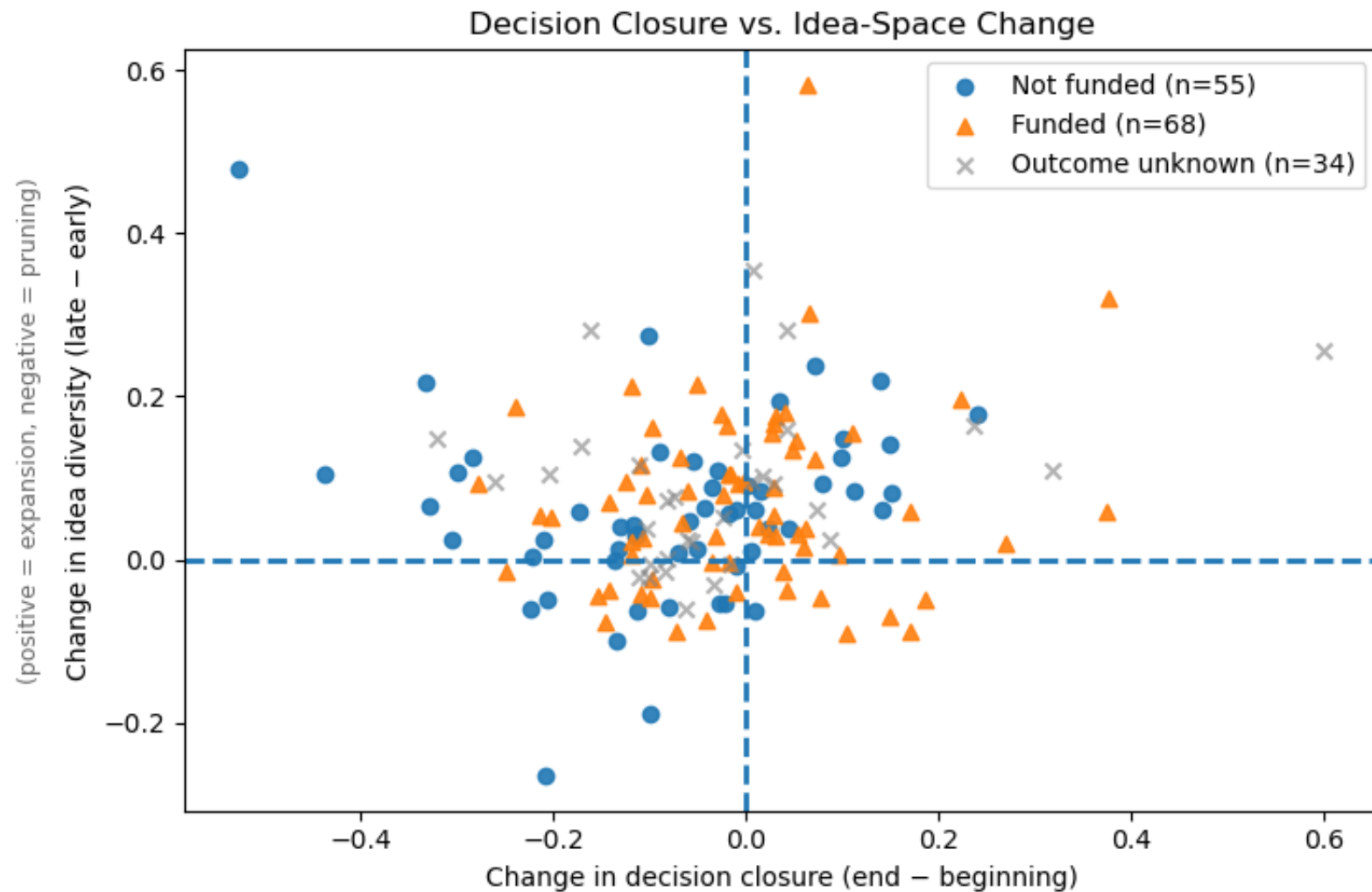
graph for update slide

```
In [69]: fig, ax = plt.subplots(figsize=(7.5, 5))  
  
# Known outcomes  
for val, marker, lab in [(0, "o", "Not funded"), (1, "^", "Funded")]:  
    sub = session_features[session_features["proposal_success"] == val]  
    ax.scatter(  
        sub["closure_gain"],  
        sub["delta_entropy"],  
        marker=marker,  
        alpha=0.9,  
        label=f"{lab} (n={len(sub)})"  
    )  
  
# Unknown outcomes  
unknown = session_features[session_features["proposal_success"].isna()]  
ax.scatter(  
    unknown["closure_gain"],  
    unknown["delta_entropy"],  
    marker="x",  
    color="gray",  
    alpha=0.6,  
    label=f"Outcome unknown (n={len(unknown)})"  
)  
  
# Reference lines  
ax.axhline(0, linestyle="--", linewidth=2)  
ax.axvline(0, linestyle="--", linewidth=2)  
  
# Labels  
ax.set_xlabel("Change in decision closure (end - beginning)")
```

```
ax.set_ylabel("Change in idea diversity (late - early)")

# Small explanatory subtitle under y-axis
ax.text(
    -0.13, 0.5,
    "(positive = expansion, negative = pruning)",
    transform=ax.transAxes,
    rotation=90,
    va="center",
    ha="center",
    fontsize=9,
    color="dimgray"
)

ax.set_title("Decision Closure vs. Idea-Space Change")
ax.legend()
plt.tight_layout()
plt.show()
```



confirming for every

```
In [70]: from scipy.stats import mannwhitneyu

# Only sessions with known outcomes
known = session_features.dropna(subset=["proposal_success"]).copy()
known["proposal_success"] = known["proposal_success"].astype(int)
```

```

funded = known[known["proposal_success"] == 1]
not_funded = known[known["proposal_success"] == 0]

def mw_test(col):
    x = funded[col].dropna()
    y = not_funded[col].dropna()
    stat, p = mannwhitneyu(x, y, alternative="two-sided")
    return {
        "funded_mean": x.mean(),
        "not_funded_mean": y.mean(),
        "U": stat,
        "p_value": p,
        "n_funded": len(x),
        "n_not_funded": len(y),
    }

results = pd.DataFrame.from_dict({
    "Δ entropy (late – early)": mw_test("delta_entropy"),
    "Closure gain (end – beginning)": mw_test("closure_gain"),
}, orient="index")

results

```

```
Out[70]:
```

	funded_mean	not_funded_mean	U	p_value	n_funded	n_not_funded
Δ entropy (late – early)	0.069040	0.059188	1899.0	0.884732	68	55
Closure gain (end – beginning)	-0.005988	-0.074780	2312.0	0.024716	68	55

```

In [71]: from scipy.stats import spearmanr
import pandas as pd

# --- Assumes you already have ---
# session_features with:
# - H_late
# - proposal_success
# - end__Synthesis / end__Seeding / etc.
# - end__Knowledge Sharing OR a proxy for late knowledge sharing

```

```

# If you have a macro version (recommended):
late_ks = session_features["end__Synthesis"] # includes Knowledge Sharing
late_entropy = session_features["H_late"]

mask = late_ks.notna() & late_entropy.notna()

rho, p = spearmanr(late_ks[mask], late_entropy[mask])

print("Late knowledge-related behavior vs late entropy")
print(f"Spearman rho = {rho:.3f}, p = {p:.4f}")

```

Late knowledge-related behavior vs late entropy
Spearman rho = -0.331, p = 0.0000

In [72]: **from** scipy.stats **import** mannwhitneyu, spearmanr

```

# -----
# 1) Whole-meeting knowledge sharing
# -----

# Count all knowledge-sharing utterances per session
ks_counts = (
    U_macro[U_macro["macro_state"] == "Synthesis"]
    .groupby("session_id")
    .size()
    .rename("ks_total")
    .reset_index()
)

# Merge into session_features
sf = session_features.merge(ks_counts, on="session_id", how="left")
sf["ks_total"] = sf["ks_total"].fillna(0)

known = sf.dropna(subset=["proposal_success"]).copy()
known["proposal_success"] = known["proposal_success"].astype(int)

# -----
# 2) Funded vs not-funded comparison
# -----

```

```

ks_funded = known.loc[known["proposal_success"] == 1, "ks_total"]
ks_not    = known.loc[known["proposal_success"] == 0, "ks_total"]

u_stat, p_val = mannwhitneyu(ks_funded, ks_not, alternative="two-sided")

print("Whole-meeting knowledge sharing:")
print(f"Funded mean: {ks_funded.mean():.2f}")
print(f"Not funded mean: {ks_not.mean():.2f}")
print(f"Mann-Whitney U p-value: {p_val:.4f}")

# -----
# 3) Relation to entropy (sanity check)
# -----

rho, p_rho = spearmanr(
    known["ks_total"],
    known["H_late"]
)

print("\nRelation to late entropy:")
print(f"Spearman rho = {rho:.3f}, p = {p_rho:.4f}")

```

Whole-meeting knowledge sharing:
 Funded mean: 85.88
 Not funded mean: 84.85
 Mann-Whitney U p-value: 0.6915

Relation to late entropy:
 Spearman rho = -0.034, p = 0.7113

In [74]: **import** pandas **as** pd

```

# pick one session to inspect
sid = U_use["session_id"].iloc[0]
print(f"Inspecting session: {sid}")

# focus on a single ambiguous code
CODE = "Information Seeking"

```

```

example = (
    U_use[
        (U_use["session_id"] == sid) &
        (U_use["label"] == CODE)
    ]
    .groupby("phase")
    .size()
    .rename("count")
    .reset_index()
)

print(f"\n'{CODE}' counts by phase for session {sid}:")
print(example)

```

Inspecting session: 2021_05_20_ABI_S5

'Information Seeking' counts by phase for session 2021_05_20_ABI_S5:

	phase	count
0	beginning	8
1	end	29
2	middle	14